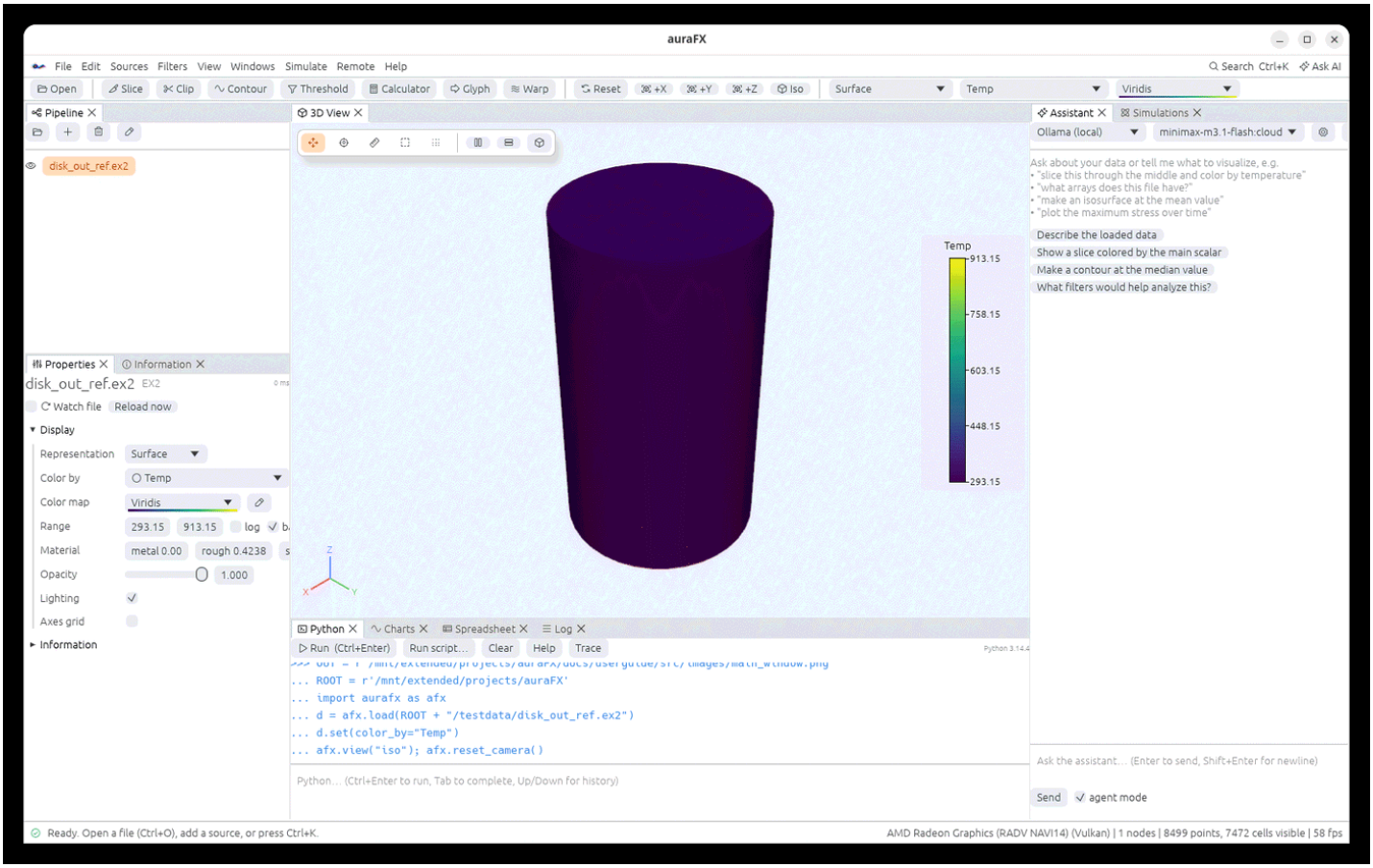
auraFX User Guide

auraFX is a scientific visualization, analysis and simulation workbench. It opens the files your solvers write (Exodus, VTK, OpenFOAM, LAMMPS, CGNS, EnSight, NetCDF and many more), shows them in a 3D view drawn by your GPU, lets you cut, contour, threshold, trace and measure them with the usual filters, and adds three things next to the 3D view:

- a **Python console** with a one-line-per-idea API (`afx.load("run.e").slice().color_by("T")`),
- an **AI assistant** that sees your pipeline and writes runnable code for you,
- a **simulation runner** that launches solvers locally, over SSH or through Slurm and opens the results when they finish.

This guide walks through every feature. Each chapter says where a feature is in the window, how to call it from Python, and what its limits are.



How this guide is organised

Part	What it covers
Basics	opening files, the pipeline, filters, coloring, views, selection, time and animation
Rendering	lighting, shadows, ambient occlusion, ray tracing, materials, volumes, point clouds
Scripting	the Python API, the AI assistant, writing plugins
Data	every file format read and written, exporting pictures and scenes, running simulations, working on remote machines
Reference	settings and keyboard shortcuts

Formats of this guide

The guide is kept as Markdown in the repository (`docs/userguide/src`), published as an online book, and printed to a PDF (`docs/userguide/auraFX-user-guide.pdf`). All three come from the same source files, so they always say the same thing.

Getting started

Installing

Prebuilt packages are attached to each release on GitHub (`github.com/aashiquear/auraFX/releases`). A one-line installer downloads the right one and puts the app in your path:

```
curl -fsSL
https://raw.githubusercontent.com/aashiquear/auraFX/main/scripts/install.sh | sh
# Linux, macOS
irm https://raw.githubusercontent.com/aashiquear/auraFX/main/scripts/install.ps1 |
iex          # Windows PowerShell
```

System	Package	Notes
Linux (x86_64, aarch64)	`auraFX-<version>-<arch>.AppImage`, `auraFX-<version>-linux-<arch>.tar.gz`	the AppImage runs from anywhere (`chmod +x` first); the installer unpacks the tarball into `~/.local` and adds a menu entry (`--prefix DIR` for another place, `--uninstall` to remove); needs a Vulkan driver (Mesa or the vendor's)
macOS 11 or newer (Apple silicon, Intel)	`auraFX-<version>-macos-<arch>.dmg`	drag the app to Applications, or use the installer line; the app is signed ad hoc, so the first start is **right-click › Open**
Windows 10/11 (x64)	`auraFX-<version>-windows-x64-setup.exe`, `auraFX-<version>-windows-x64.zip`	the installer adds a Start-menu entry and file associations; the zip is portable (unpack anywhere and run `aurafx.exe`)
Android 8 or newer	`auraFX-<version>-android.apk`	a viewer build (files, filters, rendering; no Python console, no simulations); allow the install from the browser or file manager
iOS / iPadOS 15 or newer	`auraFX-<version>-ios-unsigned.ipa`	a viewer build; sign and install it with Xcode, AltStore or Sideloadly using your own Apple ID

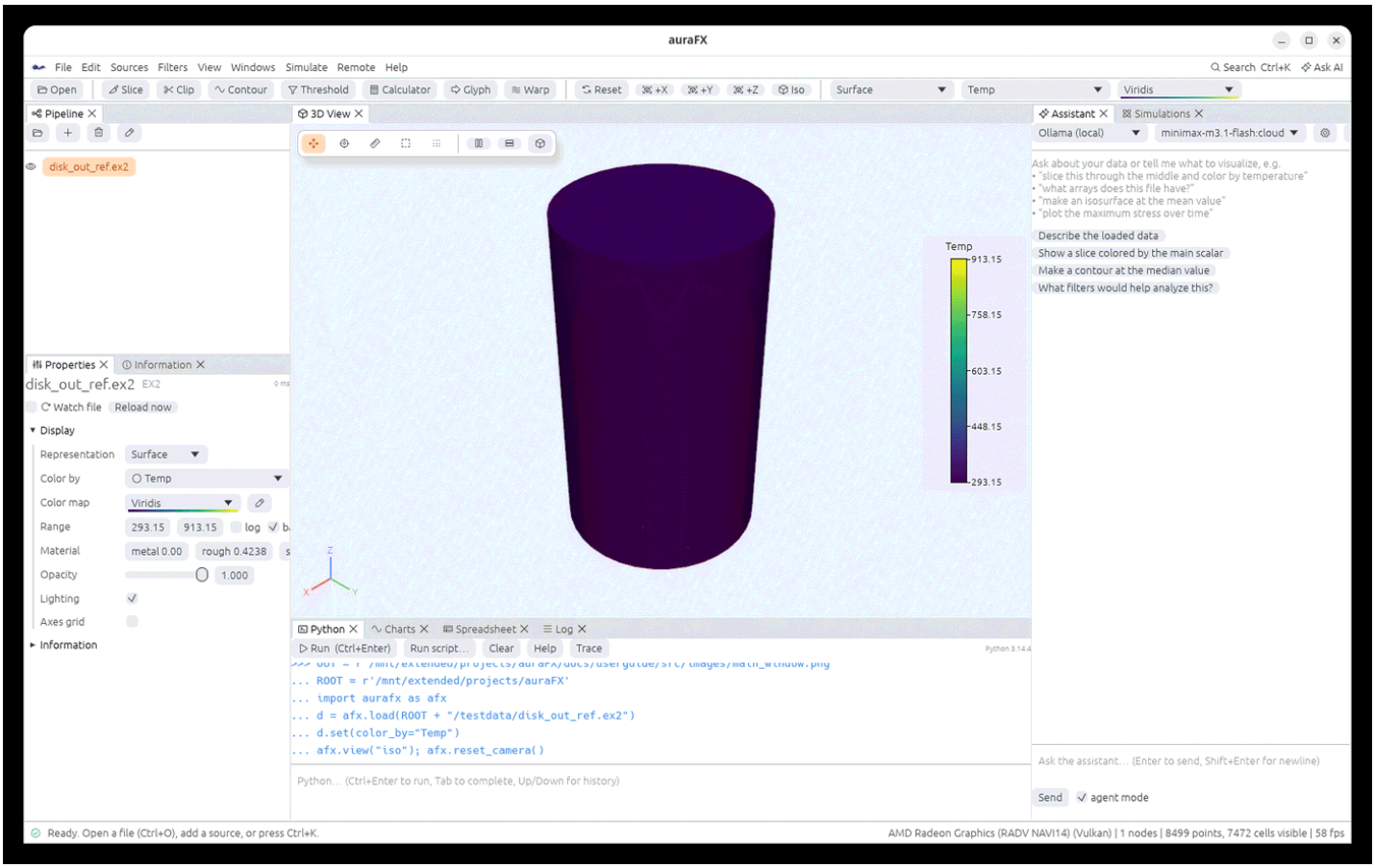
Desktop packages bundle Python 3.12 with NumPy, so the Python console works without setup; the packages carry `aurafx-server` as well, for reading data on another machine (see Data on

another machine: **File › Open on Server… › Install server** copies it to a host over SSH). On Linux the installer also sets up the `.desktop` entry (`scripts/install-desktop.sh` does only that, for a build from source).

To build from source you need Rust 1.85 or newer, Python 3.10+ with development headers, and a Vulkan (Linux, Windows) or Metal (macOS) GPU driver:

```
cargo build --release
./target/release/aurafx                    # start
./target/release/aurafx run.e              # open a file
./target/release/aurafx --script setup.py # run a Python script at startup
```

The window



The window has five areas; every panel can be closed, reopened from the **Windows** menu, and dragged to another place.

Area	Purpose
Pipeline (top left)	the files you opened and the filters below them; click to select, click the eye to show or hide

Area	Purpose
Properties / **Information** (bottom left)	the selected object's parameters and display settings; the Information tab shows arrays, ranges, bounds, cell types and time steps
3D View (middle)	the picture; drag to rotate, right-drag or wheel to zoom, middle-drag (or Shift-drag) to pan, double-click to fit
Python, **Charts**, **Spreadsheet**, **Log** (bottom)	the console, plots, the data as a table, messages
Assistant, **Simulations** (right)	the AI chat and the job runner

The toolbar has the most used filters (Slice, Clip, Contour, Threshold, Calculator, Glyph, Warp), the camera buttons (Reset, +X, +Y, +Z, Iso) and, for the selected object, its representation, the array it is colored by and the color map. The status bar at the bottom names the GPU in use and counts the visible points and cells.

Opening a file

- **File › Open…** (Ctrl+O), or drop a file onto the window, or start the app with the file name.
- **File › Open Recent** keeps the last files; the three sample datasets at the bottom of that menu (a heated disk, a wavelet volume, a sphere) are good for trying things out.
- Numbered files (`run_0001.vtu` , `run_0002.vtu` , …), `.pvd` collections, Exodus and OpenFOAM cases open as time series: the time toolbar appears under the main toolbar.

When a file opens, the object is colored by its first array and the camera fits the data. The file is watched: if a solver rewrites it, auraFX reloads it (**Watch file** in Properties).

```
d = afx.load("run.e")              # Python does the same
d.set(color_by="Temp")
afx.view("iso"); afx.reset_camera()
```

Applying a filter

Select an object, then pick a filter from the toolbar, the **Filters** menu, or the command palette (**Ctrl+K**, type part of its name). The filter runs at once with sensible defaults (a slice through the center, a contour at the middle value) and appears below its input in the Pipeline. Change its

parameters in Properties; they apply as you type. **Re-run** forces a recomputation and **Reset to defaults** restores the defaults.

When a filter is added, every object above it in the Pipeline is **hidden**, so the view shows the new result alone; click an object's eye to show it again. Filters whose result is a table or a chart (Histogram, Plot Over Line, the statistics) leave their input shown.

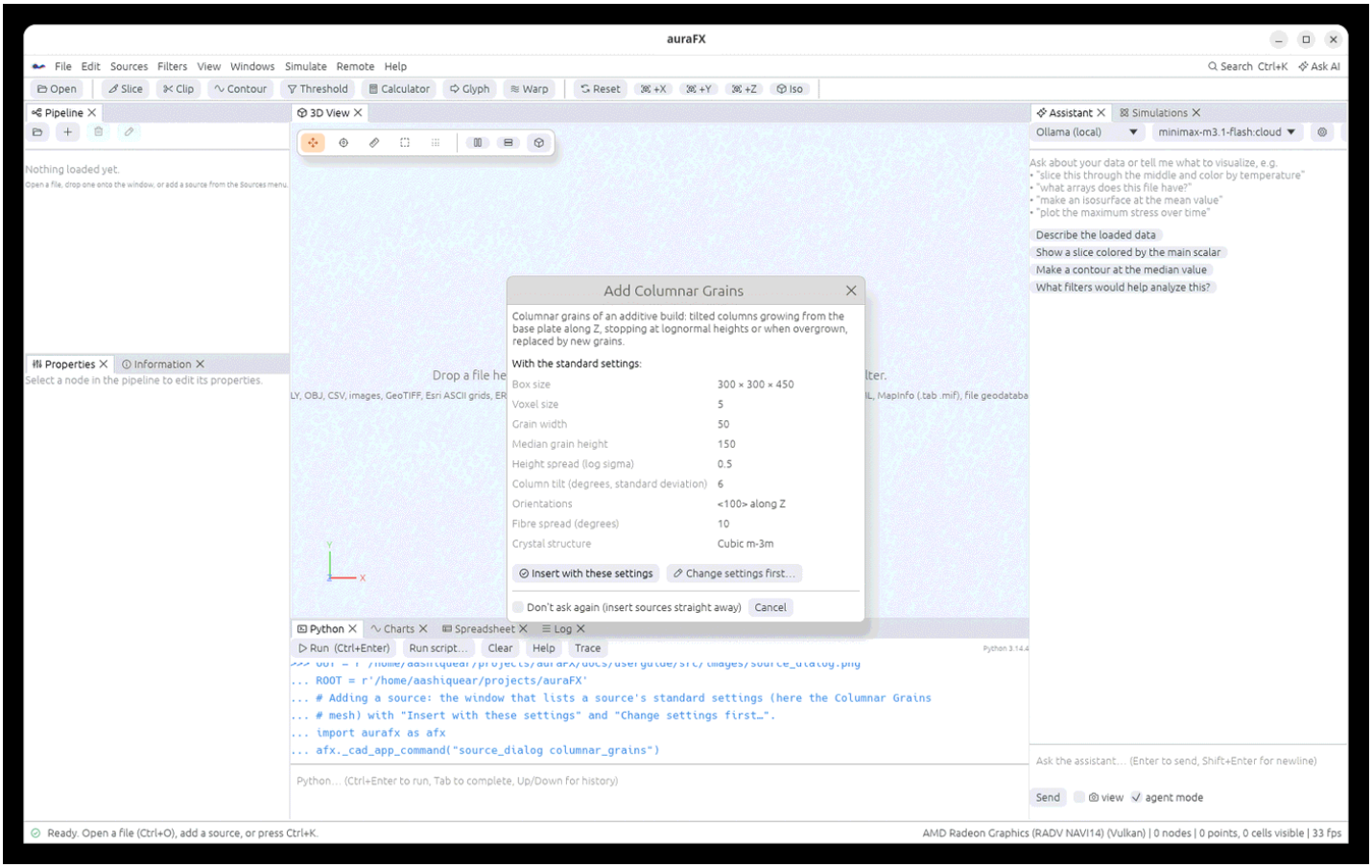
Filters can be stacked: a Contour of a Clip of a file. Delete an object with the trash button or the Delete key; the objects below it go too.

Adding a source

The **Sources** menu makes data without a file: shapes, synthetic fields, meshes, CAD parts. Picking one opens a small window that lists its standard settings (a sphere's radius and resolution, a mesh's sizes, a CAD Model's 40 × 30 × 8 mm plate) with two choices:

- **Insert with these settings** adds it as it is.
- **Change settings first…** shows its basic settings to edit before it is added (for a CAD Model, the starter part, a template or the plate with its width, depth and thickness). The other settings are in Properties afterwards.

Tick **Don't ask again** to insert sources straight away; **Settings › Ask before adding a source** turns the window back on. Sources added from Python (`afx.sphere(radius=2)`) never ask.



Saving your work

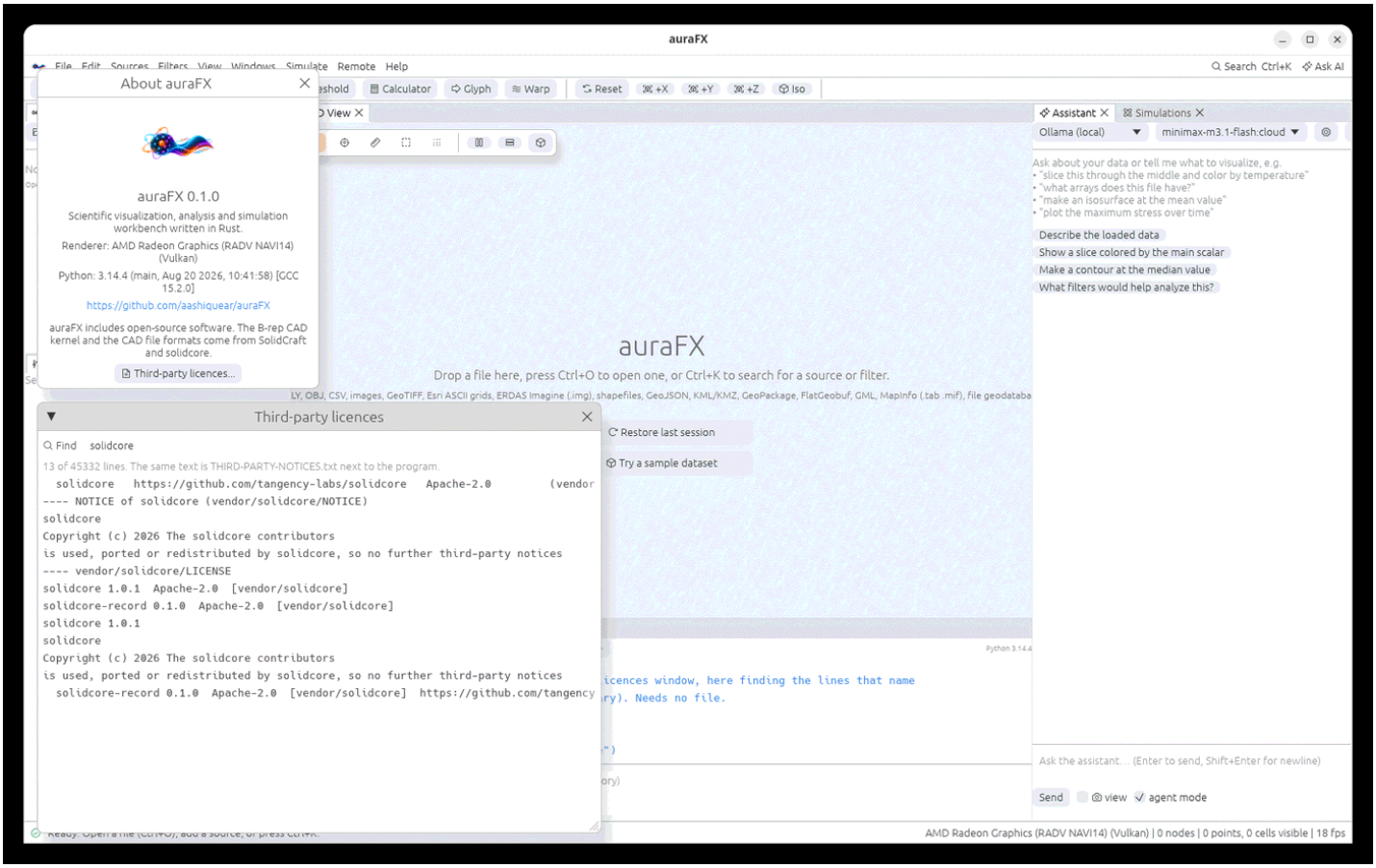
- **File › Save State** (Ctrl+S) writes an `.afx` file holding the pipeline, every parameter, the display settings, the camera and any keyframes. **File › Open** reopens it, including objects that live on a remote server.
- On the next start, **Restore last session** in the empty view brings back the last state.
- Right-click an object › **Save data…** writes its output to a file (see File formats).
- **Edit › Undo / Redo** cover every change to the pipeline and display settings.

Getting help

- **Help › Keyboard shortcuts** and **Help › Python API cheat sheet** open inside the app.
- The command palette (Ctrl+K) finds filters, sources and actions by name.
- Menus with many entries (the Sources menu, a category of filters) scroll with the mouse wheel when they are taller than the window. Each filter appears once, under its category; the Pipeline's right-click **Add filter** has the same categories.
- The AI assistant answers questions about the loaded data and the app (see AI assistant).

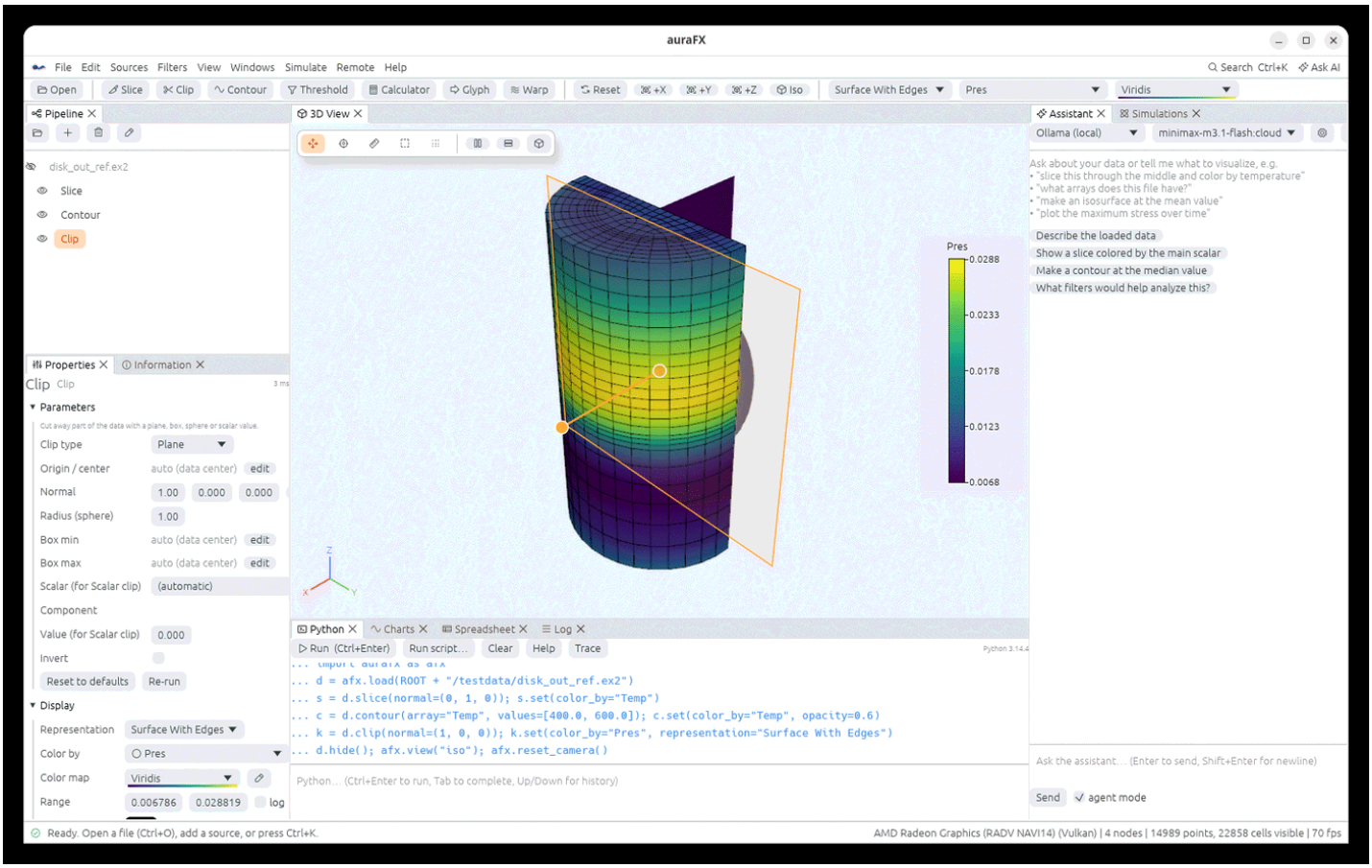
Open-source software in auraFX

Help › About auraFX shows the version, the graphics card in use and the Python version. auraFX includes open-source software: the CAD environment's B-rep kernel and its CAD file formats come from the SolidCraft and solidcore projects (see the README's Credits), and the program is built from several hundred open-source Rust libraries. **Third-party licences…** in the About window lists each of them with its licence, followed by the copyright notices and licence texts they ask to be passed on. Type in **Find** to show only the lines with a library's name or a licence (for example `Apache-2.0`). The same text is the file `THIRD-PARTY-NOTICES.txt` next to the program in every package.



Filters

A filter takes the output of the object above it and makes a new object. Select an input, choose the filter, adjust it in Properties. Every filter is also a method in Python (`node.slice(...)`), and the assistant knows all of them.



Common filters (toolbar)

Filter	What it does	Main parameters
Slice	the data on a plane or a sphere, with the values interpolated onto it	type, origin, normal (or radius); the plane can be dragged in the view
Clip	keeps one side of a plane, the inside/outside of a box or sphere, or the cells above/below a scalar value; **crinkle** keeps whole cells	type, origin, normal, box corners, radius, scalar and value, invert

Filter	What it does	Main parameters
Contour	isosurfaces (or isolines on 2-D data) at one or more values	array, values; computes normals
Threshold	the cells whose values lie in a range	array, min, max, all/any point
Calculator	a new array from an expression over other arrays (`sqrt(u*u+v*v)` , `mag(V)` , vectors with `iHat` …)	expression, result name, point or cell
Glyph	arrows, cones, spheres, cubes or lines at the points, scaled and oriented by arrays	shape, orientation and scale arrays, factor, maximum glyphs
Warp By Vector / Scalar	moves points by a vector array (displacements) or along the normal by a scalar	array, factor
Stream Tracer	streamlines of a vector field from a line or a point cloud of seeds; **Tube** makes them solid	vectors, seed type, seed line or cloud, integration direction and length
Plot Over Line	samples the data along a line and plots it in Charts	the two end points (draggable)
Probe Location	the values at one point, shown in Information and the Spreadsheet	location (draggable)

Analysis

Resample To Image, Resample With Dataset, Histogram, Descriptive Statistics, Integrate Variables (areas, volumes and integrals of every array), Gradient (with divergence, vorticity and Q-criterion), Cell Size, Mesh Quality, Elevation, Cell Centers, Connectivity, Find Data, Extract Selection, Temporal Statistics (mean, min, max and standard deviation over all time steps), Temporal Interpolator, Temporal Shift Scale, Temporal Array Operator, and the particle filters (Particle Tracer, Particle Path, Streak Line) that follow particles through a time-varying flow. Texture Map to Plane / Cylinder / Sphere, Perlin Noise and Spatio Temporal Harmonics are described below.

Geometry

Extract Surface, Extract Edges, Feature Edges, Outline, Transform, Reflect, Shrink, Clean, Surface Normals, Smooth, Triangulate, Tetrahedralize, Tessellate (curved cells into straight pieces), Mask Points, Merge Blocks, Extract Block, Extract Subset (a sub-volume of an image), Append Datasets / Geometry / Attributes, Cell Data to Point Data and back, Extract Component, Rename Array, Pass Arrays, and OpenFOAM Boundary Patches.

Sources

Sources adds objects without a file: Sphere, Cone, Cylinder, Box, Plane, Line, Point Cloud, Wavelet (a test volume), Spatio Temporal Harmonics (a test volume of waves with its own time steps), AMR Gaussian Pulse (a small AMR dataset), Hyper Tree Grid and Random Hyper Tree Grid (hyper tree grids), Cell Grid (one DG cell with test fields) and Unstructured Cell Types (one of every cell type, for tests).

Textures and made-up data

Showing an image on a surface

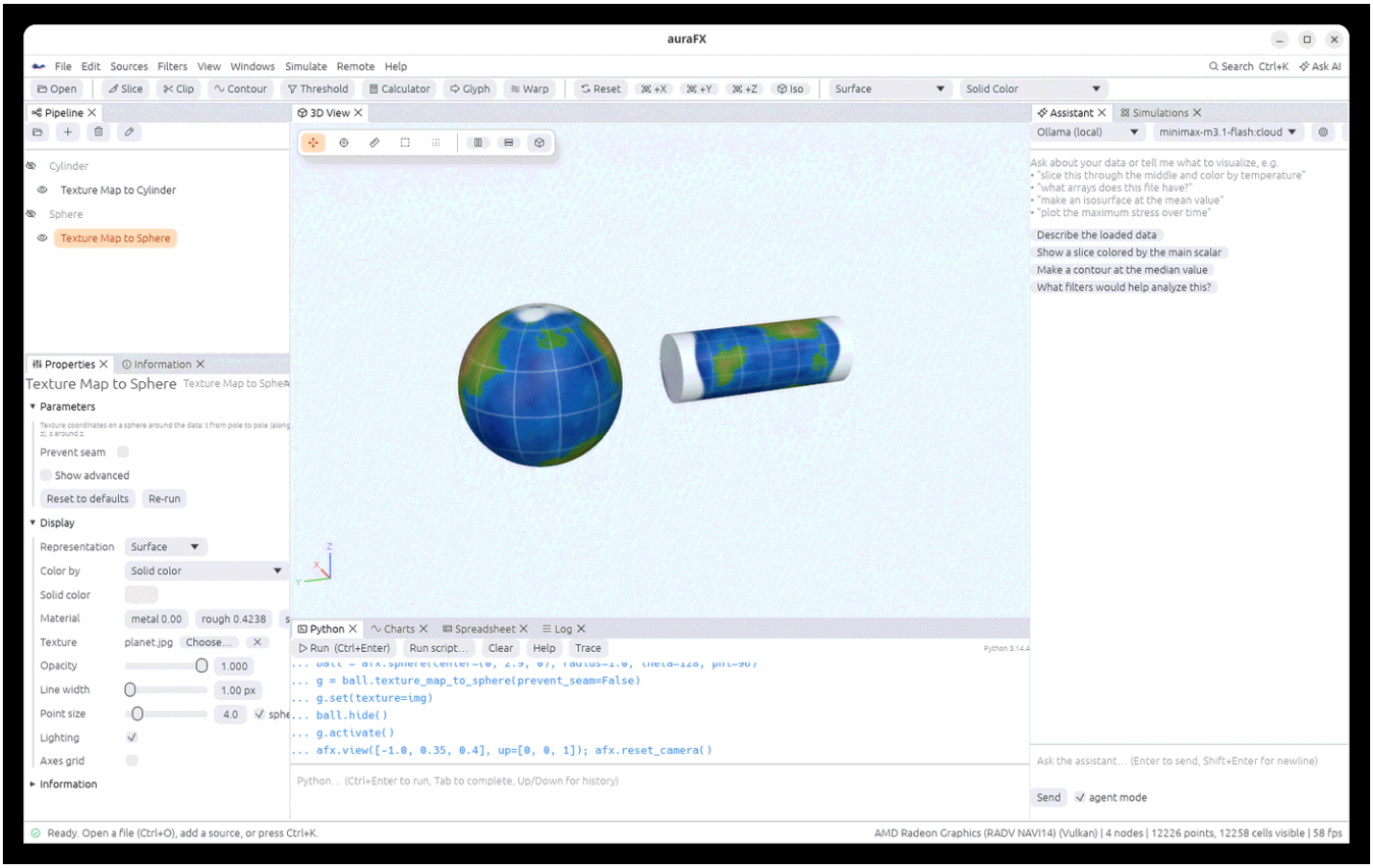
An image (PNG or JPEG) can be wrapped onto a surface, the way a world map goes onto a globe. The surface needs **texture coordinates**, which say where each point falls in the image. Some files and the Plane source come with them. For anything else, add one of these filters (**Filters › Texture Map to Plane / Cylinder / Sphere**):

Filter	How the image is laid on	Parameters
Texture Map to Plane	projected straight onto the surface from a plane, like a slide projector	**Automatic** fits the plane to the points and stretches the image over the bounds. Turn it off and give **Origin** (image corner), **Point 1** (the image's right edge) and **Point 2** (its top edge).
Texture Map to Cylinder	rolled around an axis: up the axis is up the image, around it is across	**Automatic** puts the axis along the object's longest side. Turn it off and give **Point 1** and **Point 2**, the axis ends.

Filter	How the image is laid on	Parameters
Texture Map to Sphere	wrapped around a center: the image's top at the +z pole, its bottom at the −z pole	**Center** (under Show advanced; empty = the middle of the points)

Prevent seam (cylinder and sphere, on by default) runs the image across half the way round and back mirrored. That avoids a visible join, but shows the image twice. Turn it off to wrap the image once, as for a world map. A thin seam then shows where the edges meet.

Then pick the image in **Properties › Display › Texture › Choose…** (× removes it). The row appears when the object has texture coordinates. The image replaces the array colors and the solid color; lighting and the material still apply. Coordinates outside 0 to 1 repeat the image.




```
ball = afx.sphere(radius=1, theta=128, phi=96)
globe = ball.texture_map_to_sphere(prevent_seam=False)
globe.set(texture="/path/to/world.jpg")        # texture=None removes it
p = d.texture_map_to_plane(automatic=False, origin=(0, 0, 0), point1=(10, 0, 0),
point2=(0, 5, 0))
c = d.texture_map_to_cylinder(point1=(0, 0, -1), point2=(0, 0, 1),
automatic=False)
```

The new array is called **Texture Coordinates**. The coordinates can also be colored by like any other array.

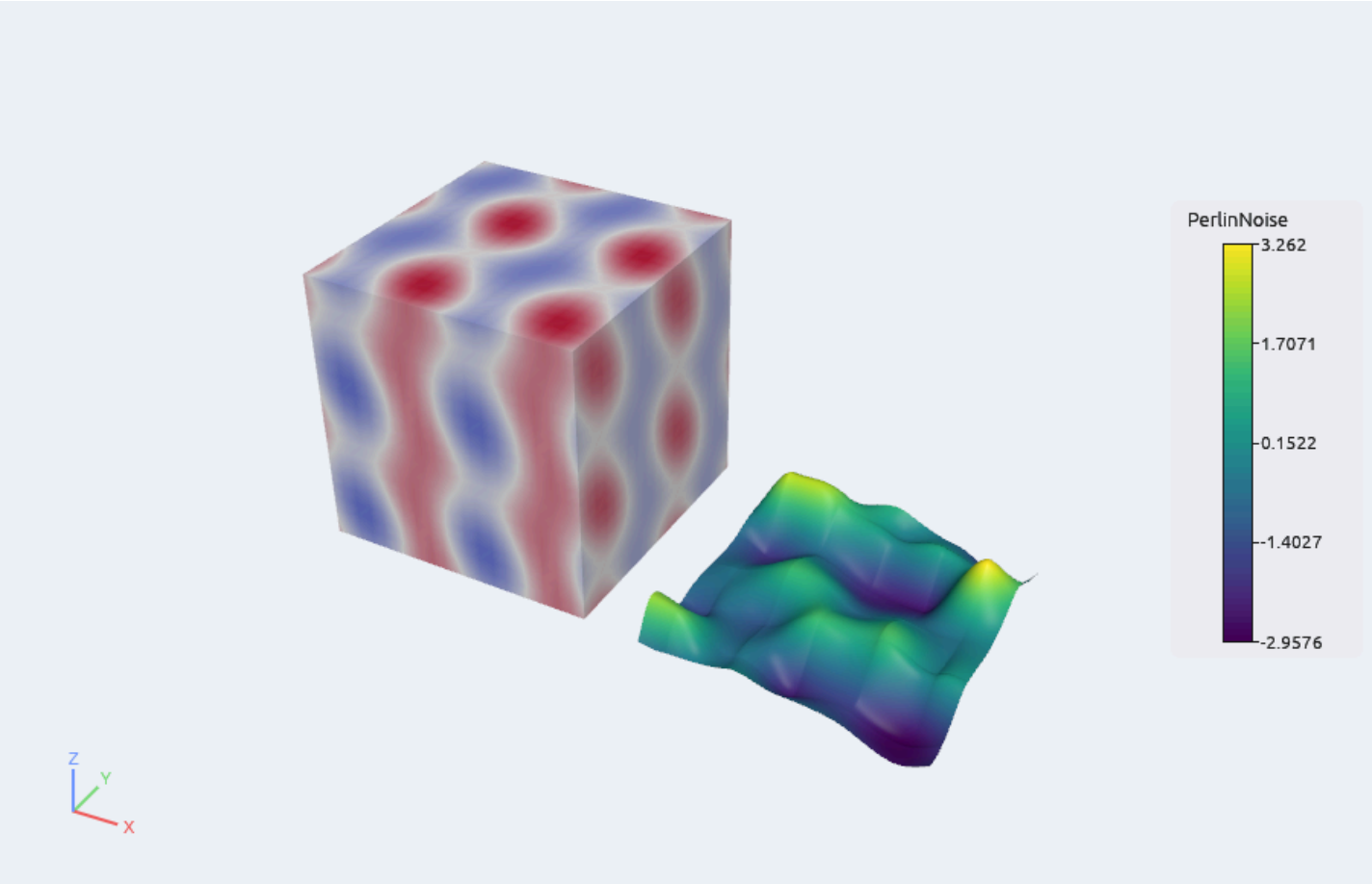
Limits:

- The image is shown on surfaces (Surface and Surface With Edges) and on lines. Points and volumes don't show it.
- Ray tracing, scene export and pictures drawn on a server show the solid color instead of the image.
- The coordinates are stored at half precision for drawing (about 1/2000 of the image width), which only matters for images repeated many times.

Made-up data for tests and demos

Filter or source	What it makes	Parameters
Perlin Noise (filter)	a smooth random-looking value at every point (array **PerlinNoise**), for test data or rough-looking surfaces	**Frequency** (bumps per unit along x, y, z), **Phase** (shifts the pattern), **Amplitude**, result name
Spatio Temporal Harmonics (filter)	a sum of travelling sine waves at every point (array **SpatioTemporalHarmonics**)	**Harmonics**: 6 numbers per wave (amplitude, temporal frequency, wave vector x y z, phase). The value is the sum of amplitude × sin(frequency × time + wave vector · point + phase).
Spatio Temporal Harmonics (source, **Sources** menu)	an image volume with those waves and **20 time steps of its own** (one period), so it animates without a file	**Whole extent** (x min, x max, y min, y max, z min, z max; spacing 1), **Harmonics**, **Time steps**

The source's time steps join the time toolbar like a file's. The filter follows the time of its input. On data without time steps it uses time 0. The default frequencies round π/5 to 0.6283.




```
h = afx.spatio_temporal_harmonics_grid()     # waves in a 21×21×21 volume, 20 time steps
h.set(harmonics=[2, 1, 0.3, 0, 0, 0])         # one wave along x
afx.time(5)
n = afx.plane(xres=200, yres=200).perlin_noise(frequency=(4, 4, 4), amplitude=0.1)
n.warp_by_scalar(array="PerlinNoise")
```

Programmable filters

Programmable Filter, **Programmable Source** and **Python Calculator** run Python that is stored in the pipeline and in state files. The filter's code gets `data` (points and arrays as NumPy) and returns it changed, or a new dataset built with `afx.make_*`. For code you want to keep across sessions, write a plugin.

Filters with several inputs

Append Datasets, Append Geometry, Append Attributes, Resample With Dataset and Cell Grid Probe take several inputs. Ctrl+click the inputs in the Pipeline, then choose the filter; the

Properties panel gets an **Inputs** section where the order can be changed.

```
afx.apply_multi("append_datasets", [a, b, c])
r = probe_mesh.resample_with_dataset(source)
```

Parameters that say “auto”

Many parameters start as **auto (data center)** or similar: the value is taken from the data each time it runs (the center of the bounds, the middle of a range, 5 % of the size). Press **edit** to type a value instead. Auto values follow the data through time steps and across pieces of a parallel server.

Speed

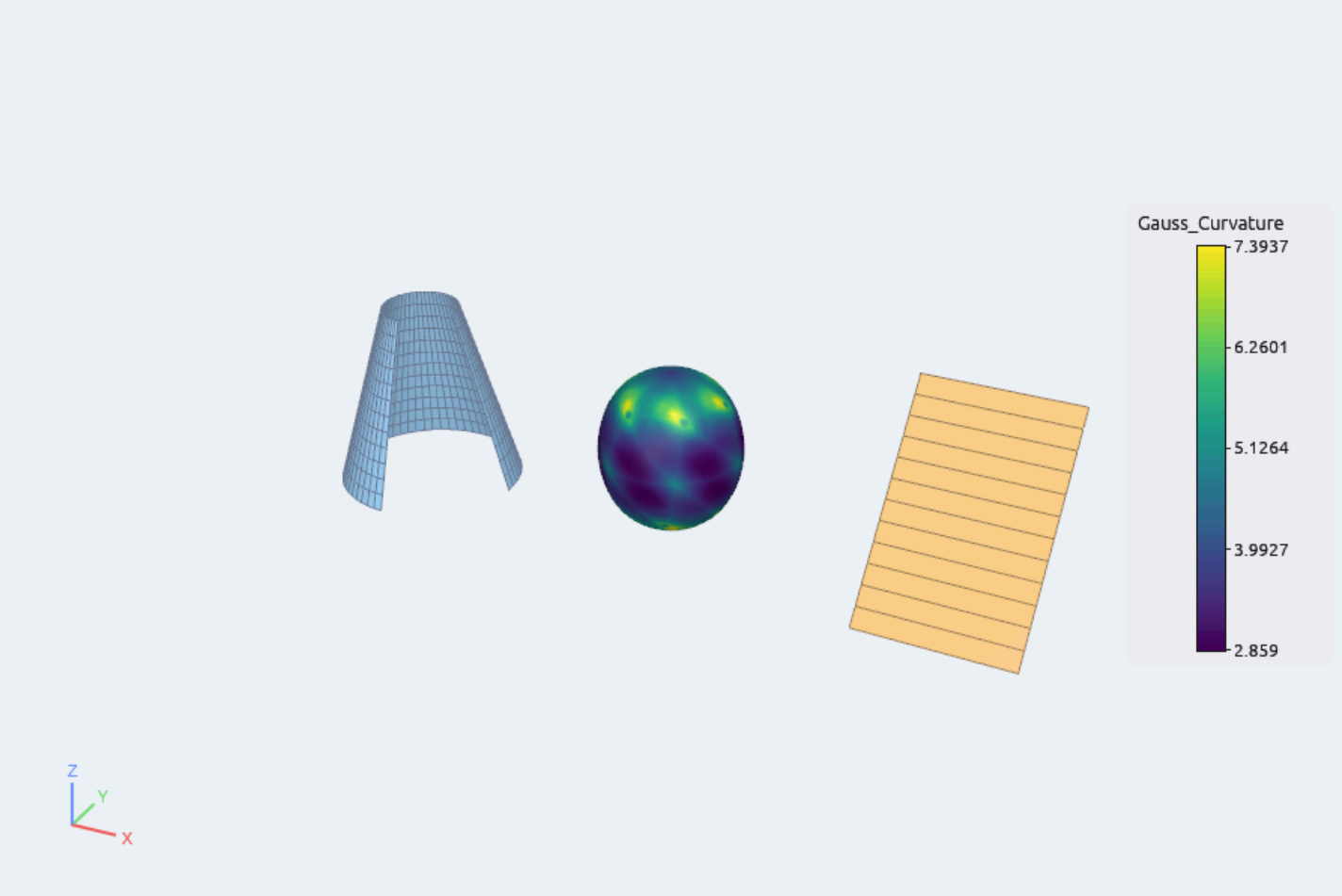
Slice, Contour, Clip, Threshold, Extract Surface, Stream Tracer, Gradient and Integrate Variables run on every core and give exactly the same result on any number of threads. A slice of 4 million cells takes about 80 ms. The thread count can be limited in **Settings › Filter threads** (0 = all cores) or with `AURAFX_THREADS`.

Surface operations

Eight filters work on surfaces and lines (**Filters › Geometry**, **Data Analysis**):

- **Reverse Sense** turns a surface inside out: every polygon’s vertex order is reversed, and by default the sign of its normals arrays too.
- **Linear Extrusion** sweeps lines into strips of quads along a *Vector* (times a *Scale factor*), and polygons into closed prisms (with their ends capped, optionally).
- **Rotational Extrusion** sweeps lines (and the boundary of polygons) round an *Axis* through a point by an *Angle*, in *Steps*: a surface of revolution, with *Translation* along the axis for a screw and *Change of radius per turn* for a spiral.
- **Ribbon** makes a strip of a given *Width* along every polyline, facing a chosen direction.
- **Subdivide** splits every triangle into four, *Levels* times. **Linear** keeps the shape; **Loop** rounds a coarse mesh toward a smooth surface (the corners pull in).
- **Decimate** removes a *Reduction* fraction of a surface’s triangles by collapsing the edges that change its shape least (quadric error), refusing collapses that fold a triangle over; the open edges stay by default. A sphere at half its triangles keeps its volume within 2 %.

- **Curvature** gives the *Gaussian* (1/length²) or *Mean* (1/length, positive where the surface is convex) curvature of a triangle surface at its points, as the point array `Gauss_Curvature` or `Mean_Curvature`. On a sphere of radius r they are 1/r² and 1/r.
- **Delaunay 2D** triangulates points seen from above so that no point lies inside another triangle's circle; an *Alpha* radius drops long thin triangles and leaves holes and concavities.



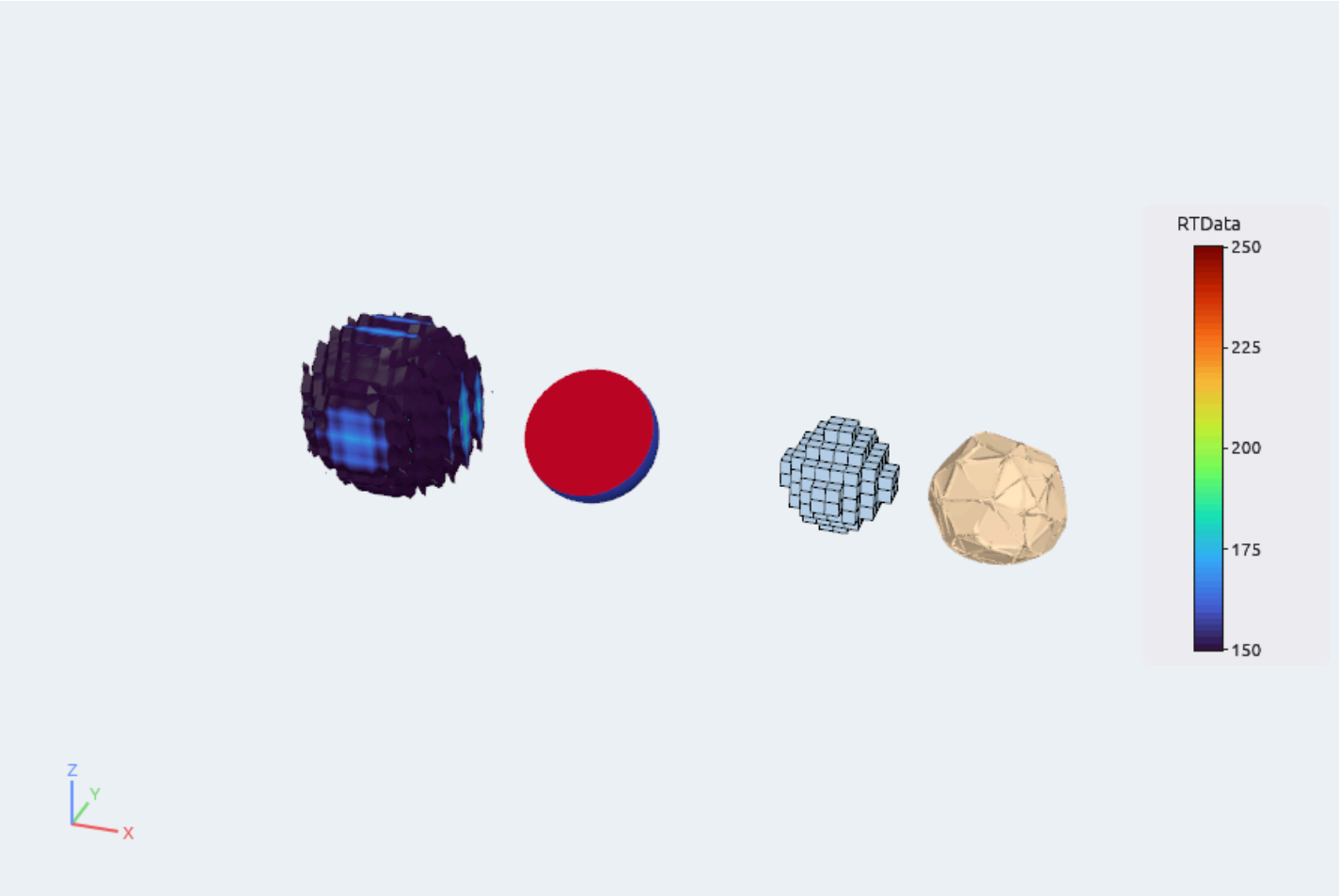

```
vase = afx.line(point1=(0.6, 0, 0), point2=(0.25, 0,
1.2)).rotational_extrusion(angle=270)
ball = afx.sphere(theta=6, phi=5).subdivide(scheme="Loop",
levels=3).curvature(kind="Gaussian")
sheet = line.linear_extrusion(vector=(1, 0, 0))
tin = points.delaunay_2d(alpha=0.5)
light = sphere_surface.decimate(reduction=0.5)
```

Extracting parts and making tetrahedra

- **Iso Volume** (**Filters › Data Analysis**) keeps the part where a point scalar lies between a *Lower* and an *Upper* value. It cuts through the cells where the scalar reaches those values,

so the result's outer faces are the two value surfaces, not cell faces. Of a field that grows linearly along x, the part between a quarter and three quarters is exactly half the box.

- **Extract Cells By Region** keeps **whole** cells (nothing is cut) inside a *Box*, a *Sphere* or a half-space (the side a *Plane*'s normal points to). *All points inside* keeps the cells entirely in the region; *Crossing the boundary too* also keeps those the region's surface passes through. *Outside instead* keeps the rest.
- **Extract Cells By Type** keeps whole cells by type: *Types* lists VTK names (`Hexahedron` , `Tetra` , `Wedge` , `Pyramid` , `Triangle` , `Quad` , `Line` , `Vertex` , quadratic types…) or dimensions (`3D` , `2D` , `1D` , `0D`), separated by commas; *All but these* inverts it.
- **Clip Closed Surface** (**Filters › Geometry**) cuts a closed surface with a plane, keeps the side its *Normal* points to, and closes the cut with a flat cap, so the result is still a closed (watertight) surface: its volume can be measured, it can be meshed. A cut through a ring leaves the hole open in the cap. The cap's triangles have the cell array `Cap` = 1.
- **Delaunay 3D** joins a dataset's points into tetrahedra so that no point lies inside the sphere through another tetrahedron's corners, filling the points' convex hull (a regular grid of points is handled too). An *Alpha* radius drops the tetrahedra whose sphere is larger, leaving concavities and gaps between separate clusters.

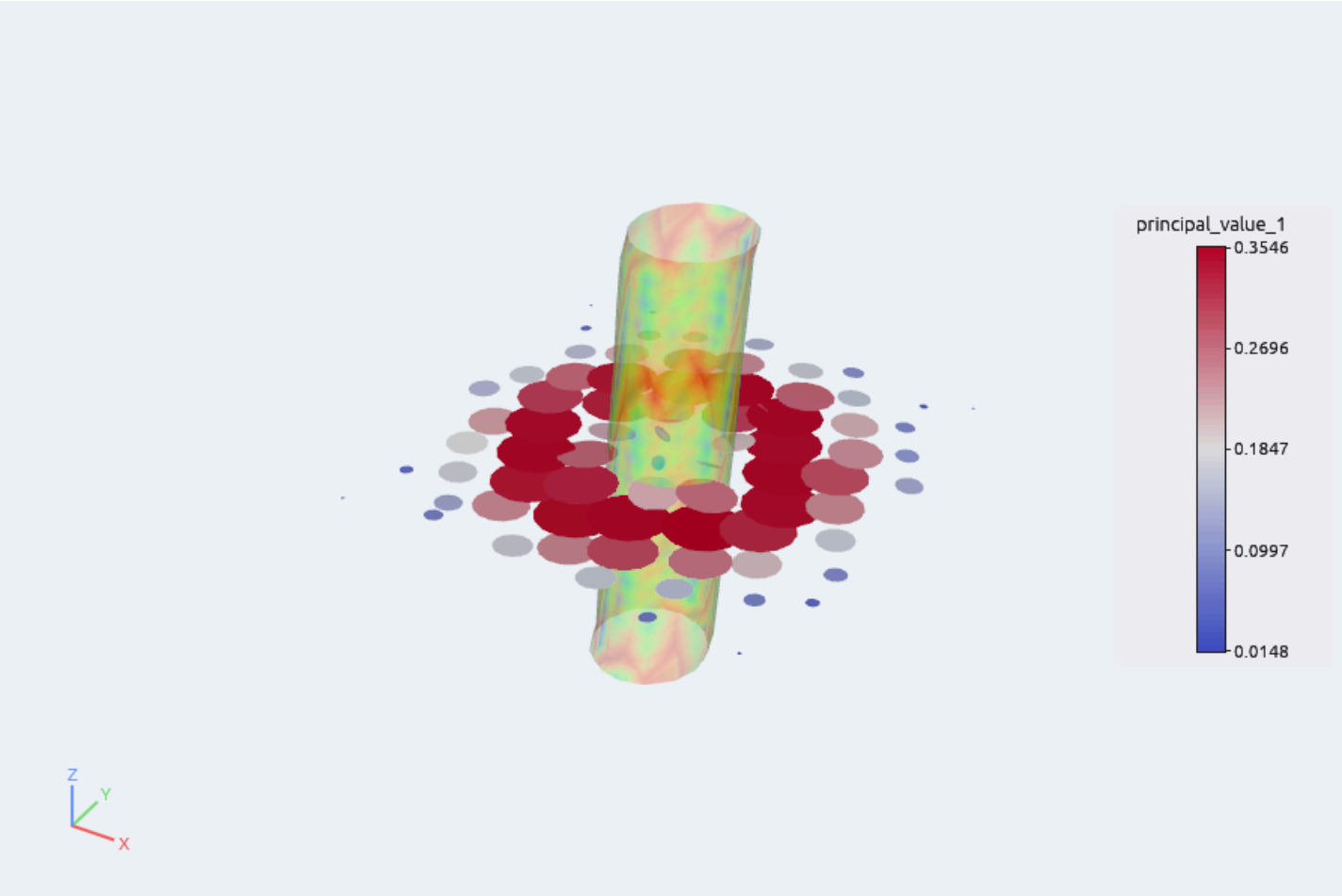


```
band = data.iso_volume(array="T", lower=300, upper=400)
core = mesh.extract_cells_by_region(region="Sphere", origin=(0, 0, 0), radius=2,
mode="Crossing the boundary too")
hexes = mesh.extract_cells_by_type(types="Hexahedron")
half = closed_surface.clip_closed_surface(origin=(0, 0, 0), normal=(0, 0, 1))
tets = points.delaunay_3d(alpha=0.0)
```

Tensors, derivatives and tables

Fields of vectors and tensors (**Filters › Data Analysis**):

- **Compute Derivatives** of a point vector field (a velocity, a displacement) adds its gradient `<name>_gradient` (9 components, dv_i/dx_j), the strain rate `<name>_strain_rate` (6 components), the vorticity `<name>_vorticity` (the curl), the divergence, and two vortex criteria: `Q_criterion` (positive where rotation beats strain; an isosurface of a positive value outlines vortex cores) and `lambda2` (negative inside a core). For a rigid rotation the vorticity is twice the angular velocity and the strain is zero.
- **Tensor Principal Invariants** of a symmetric tensor array (6 components XX YY ZZ XY YZ XZ, or 9 row by row): the principal values (largest first), the three principal directions, and the invariants I1, I2, I3 and J2, J3 of the deviator.
- **Yield Criteria** of a stress tensor array: `von_Mises`, `Tresca` (σ1 − σ3), `max_principal`, `pressure` (minus the mean stress) and `triaxiality` (mean over von Mises). A uniaxial stress σ gives von Mises and Tresca σ; a pure shear τ gives √3 τ and 2τ.
- **Tensor Glyph** draws an ellipsoid at points (every n-th), its axes along the principal directions and as long as the principal values times a scale, colored by the largest one.
- **Surface Vectors** makes a vector field on a surface tangent to it (or keeps its normal part); **Surface Flow** gives the flux ∫ v·n dA through a surface and its area, as a one-row table.



Tables and bookkeeping arrays:

- **Table To Points** turns the rows of a table (a CSV file) into points: the columns named x, y, z, or chosen ones (or the first three numeric columns) give the positions, every column becomes point data; *2D* puts z at 0. **Table To Structured Grid** does the same into a structured grid of given dimensions (x fastest).
- **Transpose Table** swaps rows and columns (numeric columns; the column `Column` names the old columns). **Table FFT** gives the discrete Fourier transform of every numeric column, as `<name>_fft` (real and imaginary) and `<name>_magnitude`, for any number of rows, and a `Frequency` column when you give the time step.
- **Merge Vector Components** joins three scalar arrays into a vector. **Point And Cell Ids** adds `PointIds` and `CellIds`; **Generate Global Ids** adds ids unique over all the blocks of a multiblock (coinciding points share one); **Coordinates** adds the positions as an array.

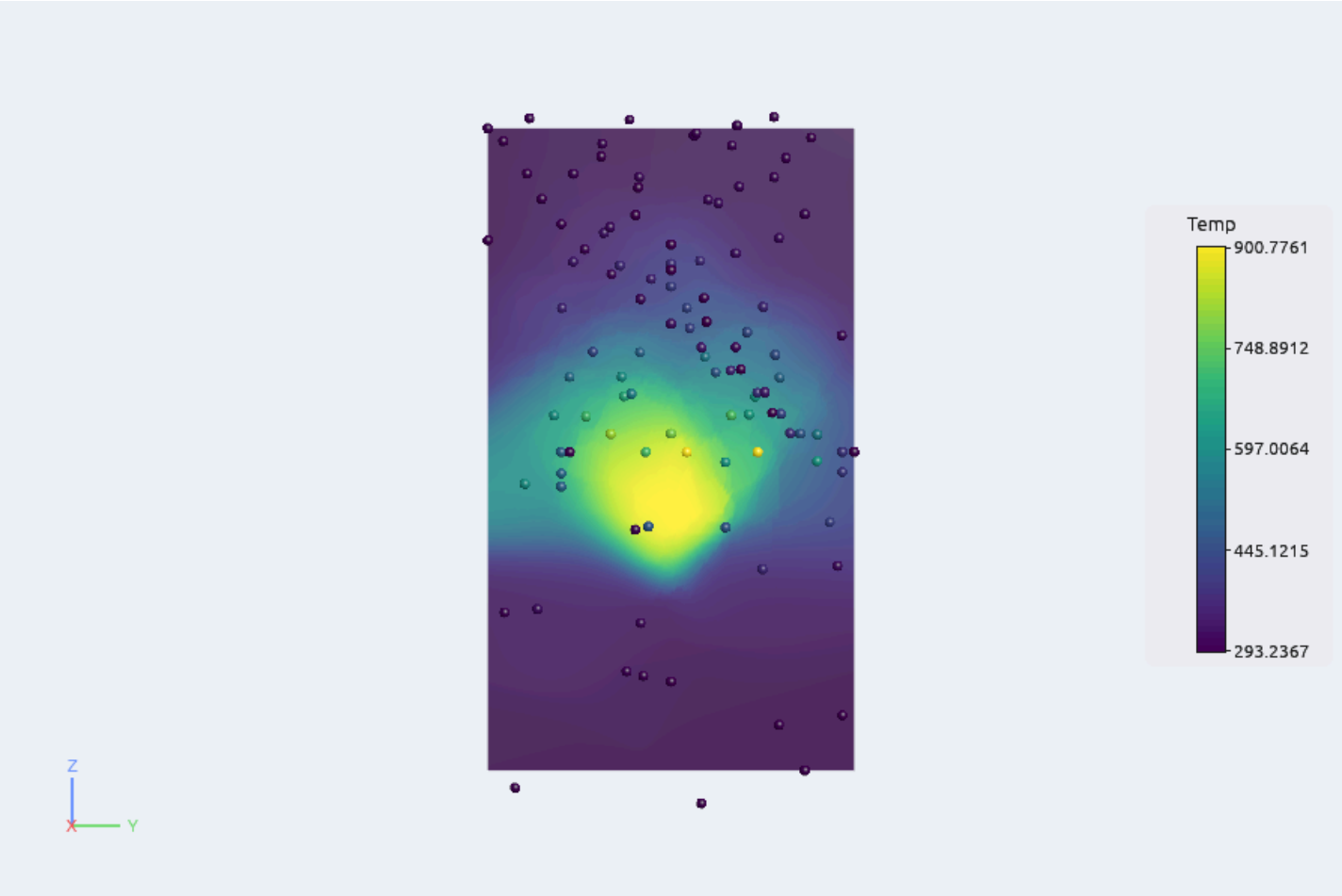
```
d = flow.compute_derivatives(array="velocity")      # velocity_vorticity,
Q_criterion, lambda2, ...
cores = d.contour(array="Q_criterion", values=[0.3])
s = result.yield_criteria(array="stress")            # von_Mises, Tresca,
max_principal, pressure, triaxiality
p = result.tensor_principal_invariants(array="stress")
g = slice_.tensor_glyph(array="stress", scale=0.01, stride=10)
pts = afx.load("probe.csv").table_to_points(x="X", y="Y", z="Z")
spec = afx.load("signal.csv").table_fft(dt=0.001)
```

Scattered points onto grids and lines

Point Interpolator (**Filters › Data Analysis**) carries the point arrays of scattered points (particles, probe measurements, a point cloud) onto other points. Each new value is a weighted mean of the nearest points:

Kernel	Weight of a neighbour at distance d
Shepard	$1 / d^p$ (the power p, 2 by default)
Gaussian	$\exp(-(s \cdot d/R)^2)$, with the sharpness s and the search radius R
Linear	the same for every neighbour (an average)
Nearest point	the closest point's value only
SPH cubic, **SPH quintic**	a smoothed-particle kernel of smoothing length h (support 2h); with *density* (and *mass*) arrays each particle counts with its volume, mass / density, otherwise the weights are scaled to add up to one

The neighbours are the **N closest** points or those within a **radius** (0 picks one from the points' spacing). Where no point is near, the result is a value of your choice or the nearest point's. Without a second input the values go onto a **Volume** (a grid over the points' bounds), a **Plane** across them or a **Line**; with a second input (Properties › Destination), onto that data's points, keeping its cells and arrays. `vtkValidPointMask` is 0 where nothing was near. A linear field sampled by 20 000 random points comes back within 2 % (Shepard, Gaussian) and SPH on a lattice of particles within 1 %.



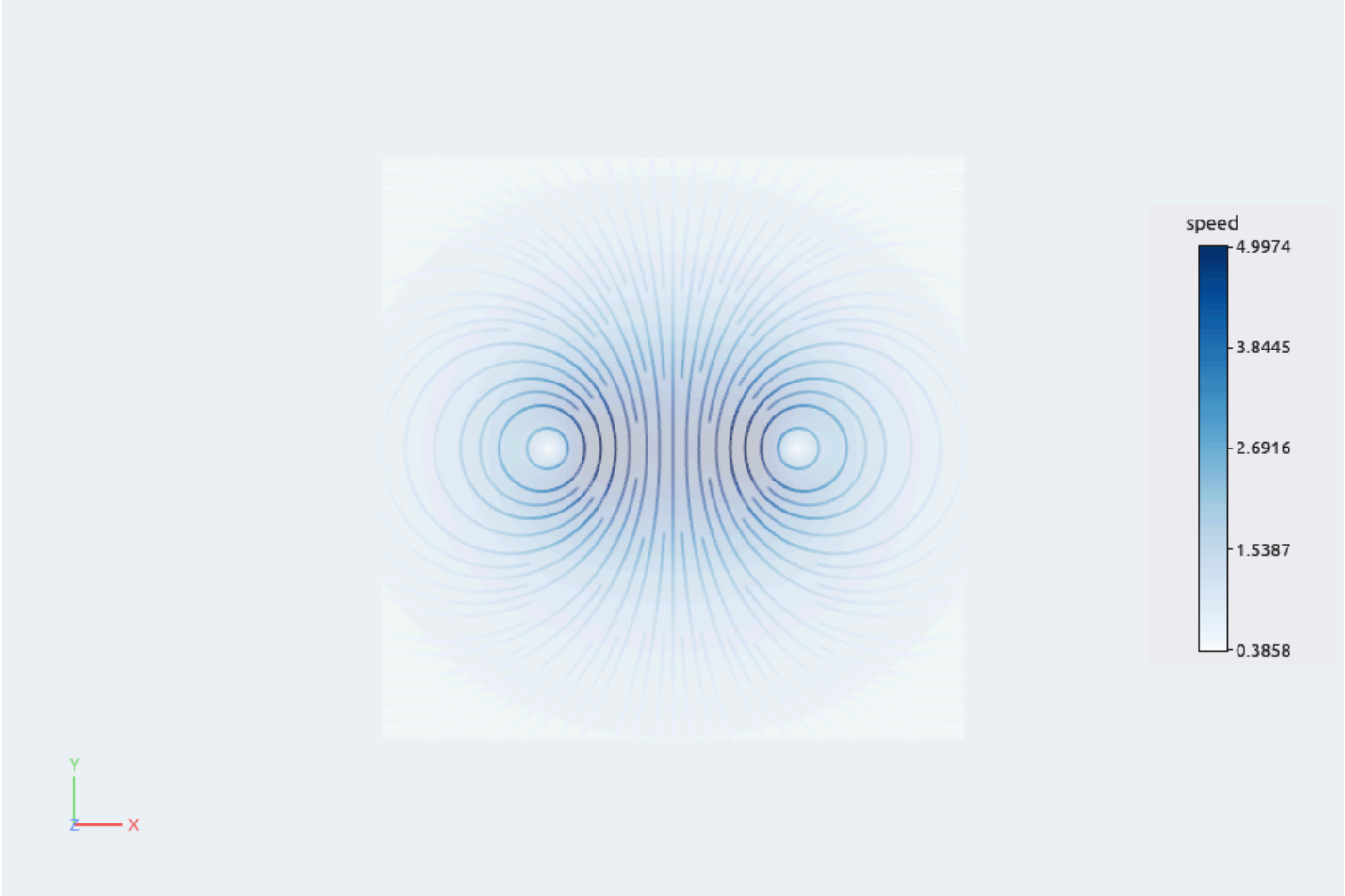
Lines to plot (all in **Filters › Data Analysis**, shown in the Charts panel):

- **Resample To Line** samples a straight line at evenly spaced points, **at the cell boundaries** (where the line enters and leaves each cell) or **at the cell centres** (once in the middle of each cell it crosses), with `arc_length`.
- **Plot On Sorted Lines** joins line segments (from Slice, Contour on a surface, Feature Edges) into curves ordered from one end to the other, each with its `arc_length`, one block per curve.
- **Plot On Intersection Curves** cuts a surface (a volume's outer surface) with a plane and gives the curves, ordered, with the surface's arrays: a profile around a section of a body.

A chart of data with several blocks draws one line per block (each against its own x array), named `block/array`.

```
pts = afx.load("probes.csv").table_to_points()
grid = pts.point_interpolator(kernel="Gaussian", output="Plane", axis="Z", nx=200,
ny=200)
onto = pts.point_interpolator(mesh, kernel="Shepard", n_closest=12)   # onto
another mesh
sph = particles.point_interpolator(kernel="SPH cubic", smoothing_length=0.02,
density="rho", mass="m")
prof = body.plot_on_intersection_curves(origin=(0, 0, 0.5), normal=(0, 0, 1))
cells = mesh.resample_to_line(point1=(0, 0, 0), point2=(1, 0, 0), sampling="Cell
boundaries")
```

Evenly Spaced Streamlines 2D fills the XY plane with streamlines of a vector field, evenly spread: each new line starts at the *separation* from the lines already there and stops where it comes closer than a fraction of it (or closes on itself), until no gap is left. There are no seeds to place; the field's z component is not used.



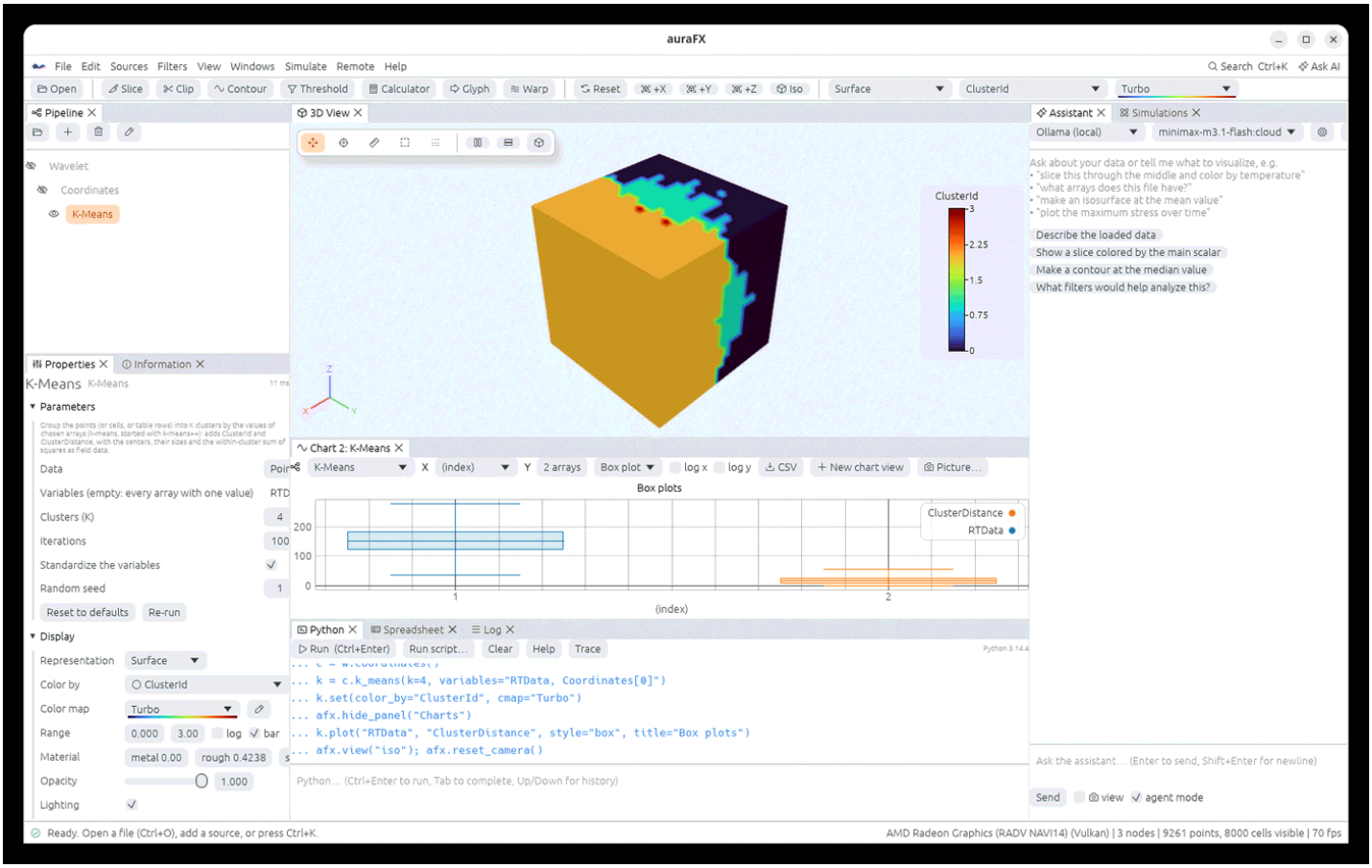

```
s = flow.evenly_spaced_streamlines_2d(vectors="V", separation=0.05)
```

Statistics

In **Filters › Data Analysis**; *variables* is a list of array names (`T, p`; `V` is a vector's magnitude, `V[1]` its second component), empty for every array with one value; *data* chooses the point or cell arrays (table columns for tables).

- **K-Means** groups the points, cells or rows into K clusters by those values (k-means++ start, repeated until nothing moves; *standardize* divides each variable by its spread first). It adds `ClusterId` and `ClusterDistance`, and field data with each cluster's centre, size and the within-cluster sum of squares.
- **Principal Component Analysis** adds the projections `PCA 0`, `PCA 1` … (largest variance first) and gives as field data the eigenvalues, the fraction of the variance each explains and the components.
- **Multicorrelative Statistics**: the means, covariance matrix and (Pearson) correlation matrix as a table.
- **Contingency Statistics** of two arrays: how often each pair of values occurs (real values are put in bins), with joint and conditional probabilities; chi-square, its degrees of freedom and the mutual information as field data.
- **Compute Quartiles**: minimum, Q1, median, Q3, maximum, interquartile range and whisker ends per variable. The chart style **Box plot** draws a box of every plotted array's values.
- **Random Attributes** adds a random array (uniform between two values, scalars or vectors).
- **Kernel Smoothing** gives a smooth estimate of an array's distribution (a density that integrates to one) for the Charts panel; **Density Map** puts the joint density of two arrays on an image, with the levels holding 50, 90 and 99 % of the samples.

Three clusters of 200 points each are found exactly; PCA of a cloud stretched along (1, 1) gives its two variances within 3 %.



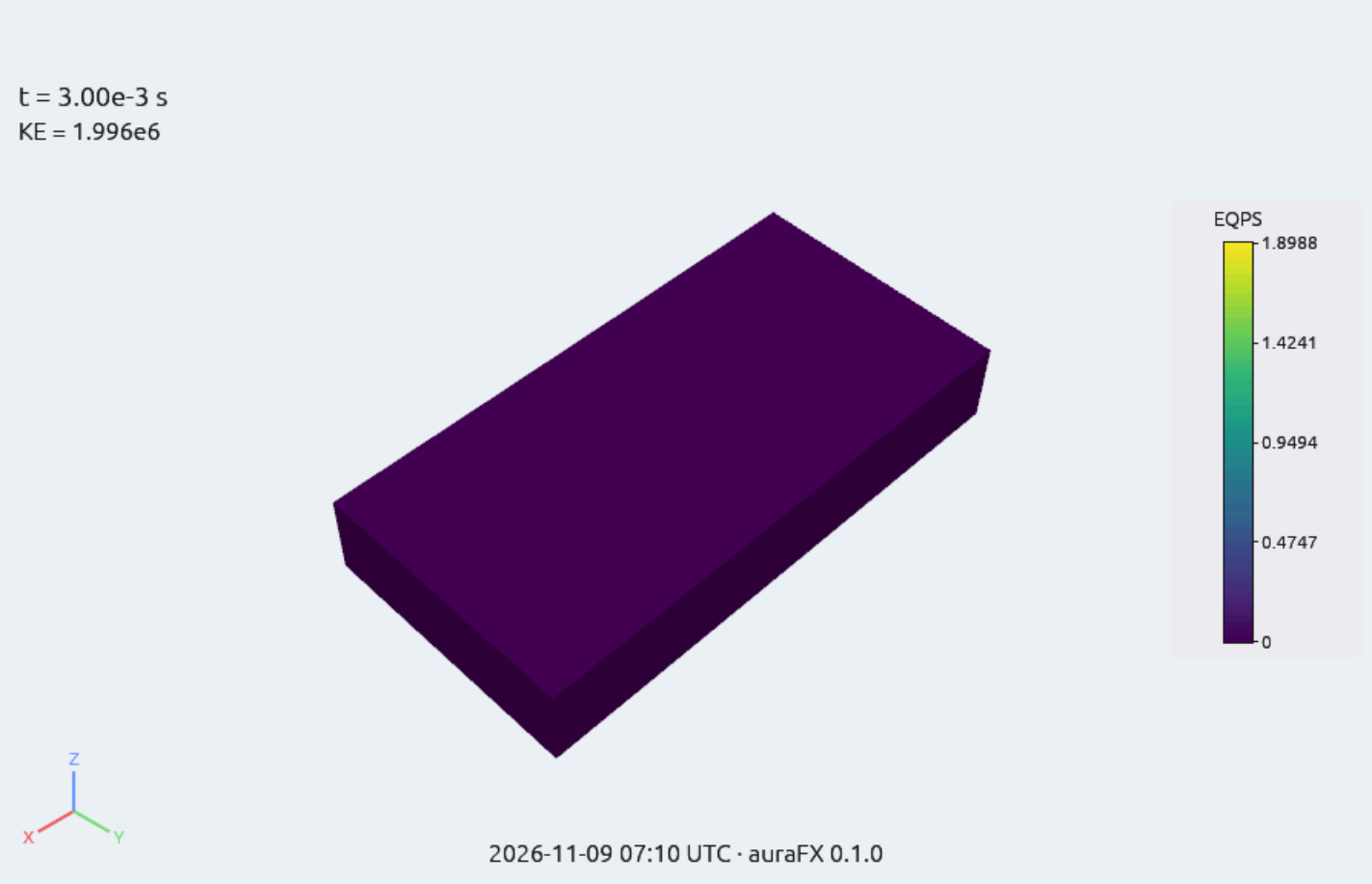

```
k = w.k_means(k=4, variables="RTData, Coordinates[0]")
p = w.principal_component_analysis(variables="RTData, Coordinates[0],
Coordinates[1]", keep=2)
c = afx.chart_view(w.compute_quartiles(), y=["Q1"])
w.plot("RTData", style="box")
```

Text in the 3D view

The **Annotation** filters write a line of text that every 3D view showing them draws in a corner of your choice, at a size of your choice (hide the filter to hide the text):

- **Annotate Time Filter**: the time, in a format such as `Time: {:.3} s` (`{}` is the value, `{:.3}` three decimals, `{:.2e}` an exponent, `{:.4g}` four significant digits), scaled and shifted if you like.
- **Annotate Global Data**: a global (field) value, such as an energy; one value per time step shows the current step's.
- **Annotate Attribute Data**: the value of an array at one point or cell, following the time steps.
- **Annotate Selection**: on the output of Extract Selection, how many points are selected and the first ones with their numbers and values.

- **Environment Annotation**: the date and time, the user, the machine, the operating system, the auraFX version.
- **Python Annotation**: the result of a Python expression; the point and cell arrays are NumPy arrays by name (the blocks of a multi-block joined), with `points`, `inputs`, `t` (the time) and `step`, e.g. `f'max T = {T.max():.1f} K'`.

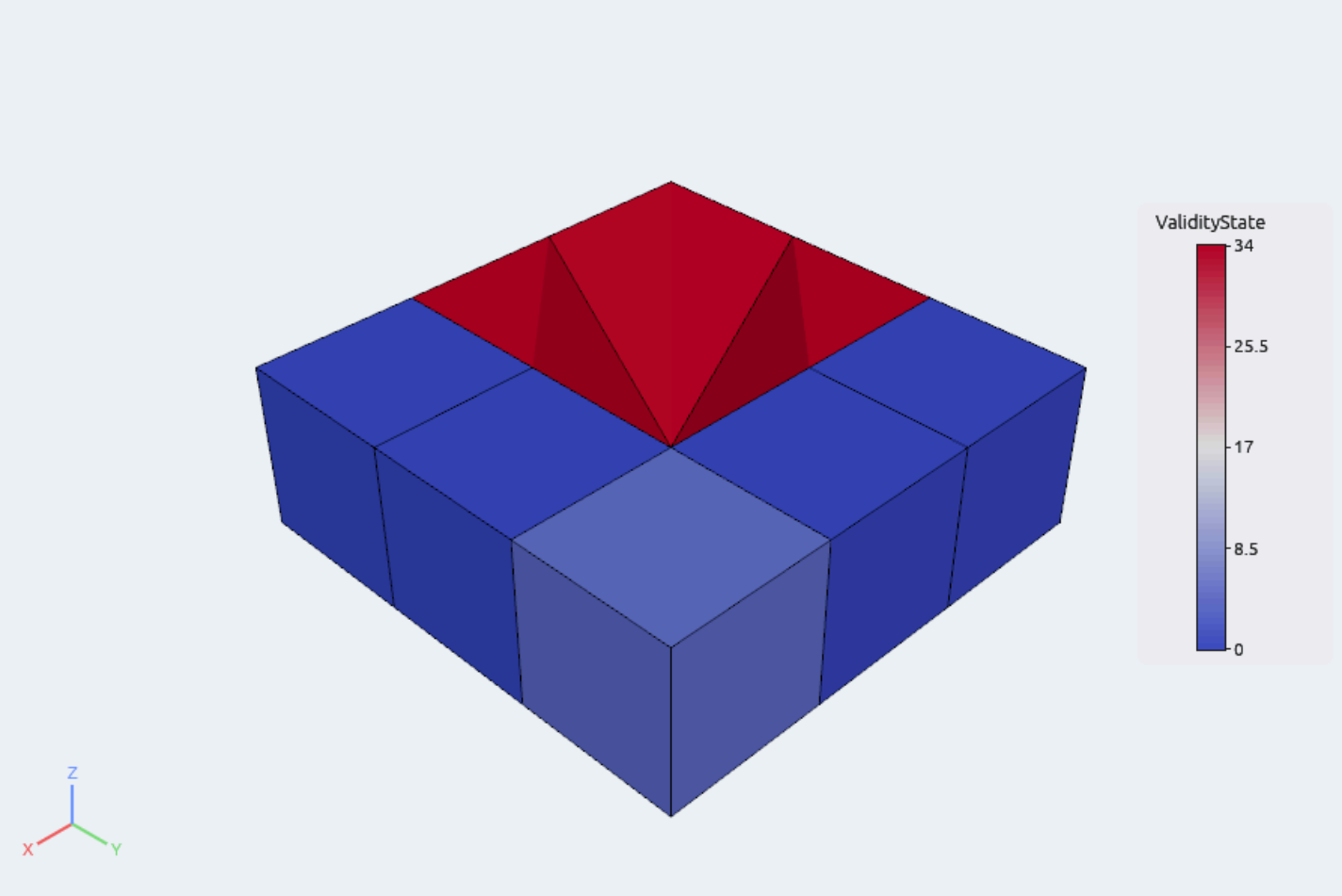



```
c.annotate_time_filter(format="t = {:.2e} s", corner="upper right")
c.annotate_global_data(array="KE", format="kinetic energy {:.3g} J")
c.python_annotation(expression="f'max EQPS {EQPS.max():.3f}'", corner="lower
left", size=20)
```

Checking meshes

- **Validate Cells** checks every cell: `ValidityState` is 0 for a good cell, otherwise the sum of 1 (wrong number of points), 2 (degenerate: no volume, area or length), 4 (inverted, turned inside out), 8 (a face that is not flat), 16 (a polygon that is not convex), 32 (repeated points); field data counts each problem. Threshold on `ValidityState` ≥ 1 to see the bad cells.

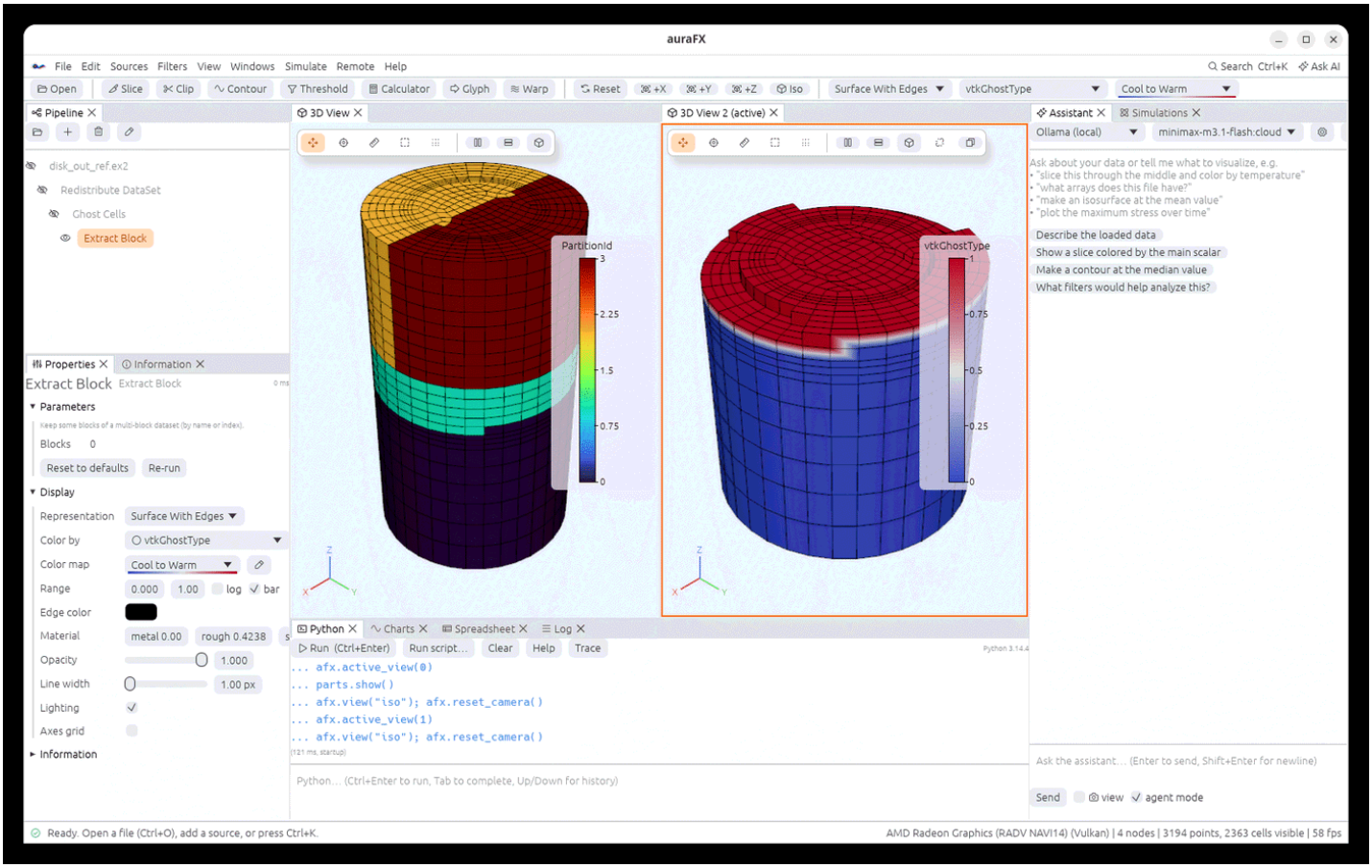
- **Overlapping Cells Detector** counts, for each cell, the other cells that share volume with it (or area, for surface cells in one plane): `NumberOfOverlapsPerCell`. Cells that only touch do not count.



Partitions and ghost cells

Partitioned data is a multi-block whose blocks are the partitions: a `.pvtu`, `.pvtp`, `.pvts`, `.pvtr` or `.pvti` file opened with the reader option **Pieces › Partitions** (one block per piece file), or the output of Redistribute DataSet.

- **Redistribute DataSet** splits a mesh into partitions of nearly equal cell counts, cutting it in halves along its longest side again and again (64 cells into 3 partitions: 21, 21 and 22).
- **Ghost Cells** gives each partition layers of the neighbouring partitions' cells that touch it, marked in `vtkGhostType` (1 for a ghost cell, and for a point only ghost cells use), so that filters that look at neighbours agree at partition borders.
- **Remove Ghost Information** drops the ghost cells and the ghost arrays again.




```
parts = mesh.redistribute_dataset(partitions=8, ghost_layers=1)
case = afx.load("run.pvtu"); case.set(pieces="Partitions")
g = case.ghost_cells(layers=2)
```

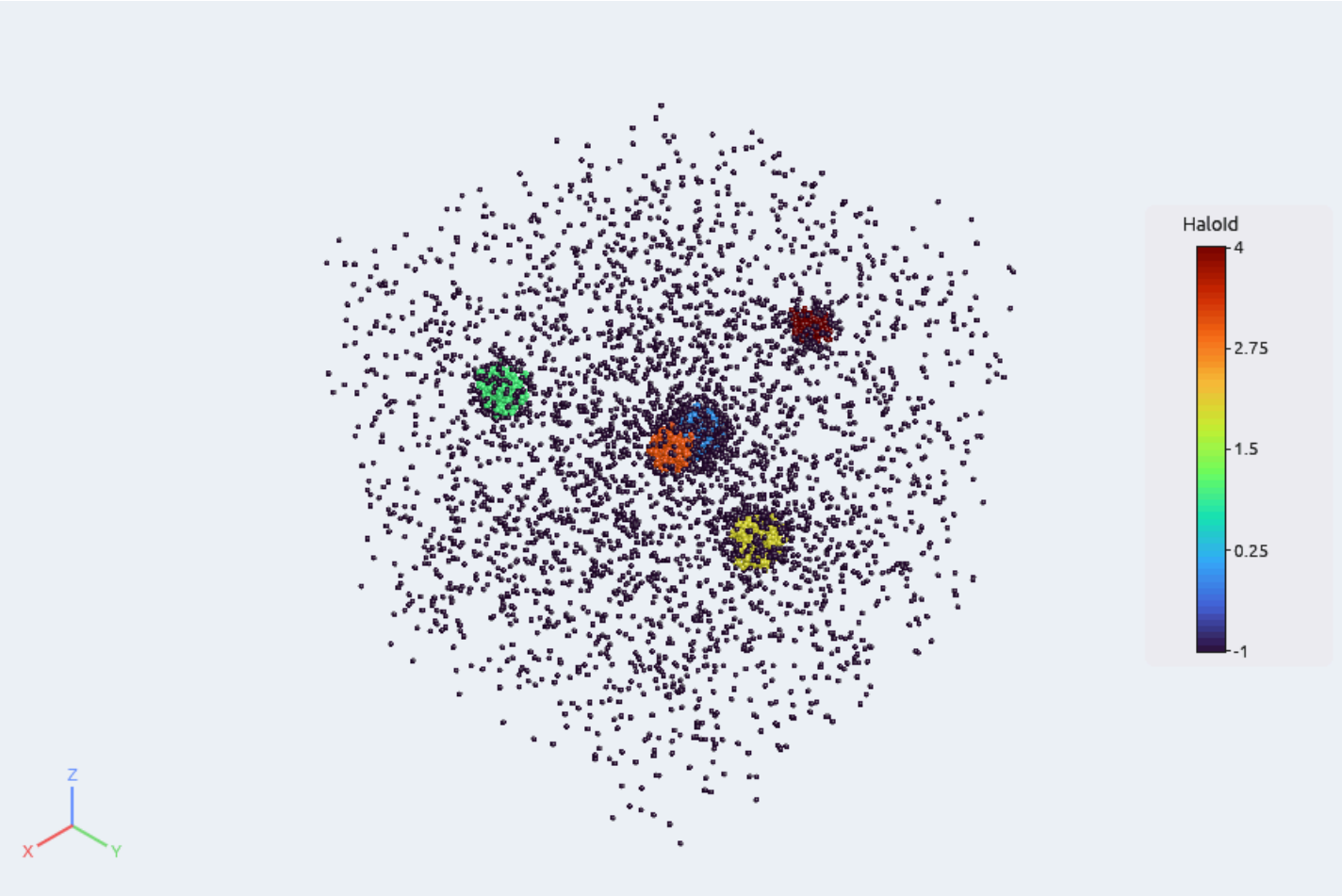
Volume fractions and fragments

- **Material Interface** rebuilds the interfaces between materials from cell arrays of volume fractions (one per material, 0 to 1): in every mixed cell a plane facing down the fraction's gradient, placed so that the cell's material side holds exactly its fraction. One block of interface polygons per material (lines on surface meshes) with `VolumeFraction`, and the material's volume as field data. For a ball sampled on a 24³ grid the interface area is within 4 % of the sphere's and every interface point within a third of a cell of it.
- **Fragment Integration** finds the fragments of a material: cells whose fraction (or any cell array) is at least a threshold, joined through shared faces (across the levels of AMR data, through touching boxes). The table gives each fragment's cell count, volume (fraction × cell volume), centroid and the volume integrals of the other cell arrays; *Output › Cells* gives the cells with `FragmentId`. Two balls come back as two fragments with their volumes within 1 %.

- **AMR Dual Clip** clips AMR data by a cell array without cracks between levels: the levels' cell centres are joined into one stitched mesh and the part above a value is kept.

Halos and shape measures

- **Halo Finder** finds halos in particles: particles closer than a *linking length* (a fraction of the mean spacing, 0.2 by default) are friends, and friends of friends form one halo. It adds `HaloId` (−1 outside halos) or gives a table of halos with their count, mass, centre of mass, mean velocity, and `RadiusSOD` / `MassSOD`, the radius and mass inside which the mean density is the *overdensity* (200) times the box's mean density.
- **Minkowski Functionals** measures the region where an array is above a threshold, from its closed boundary surface: volume, surface area, integrated mean curvature, Euler characteristic (1 for a ball, 0 for a torus) and the normalised W0 to W3. A ball sampled on a 61^3 grid comes within 1 % (volume), 2 % (area) and 3 % (mean curvature) of the exact values.



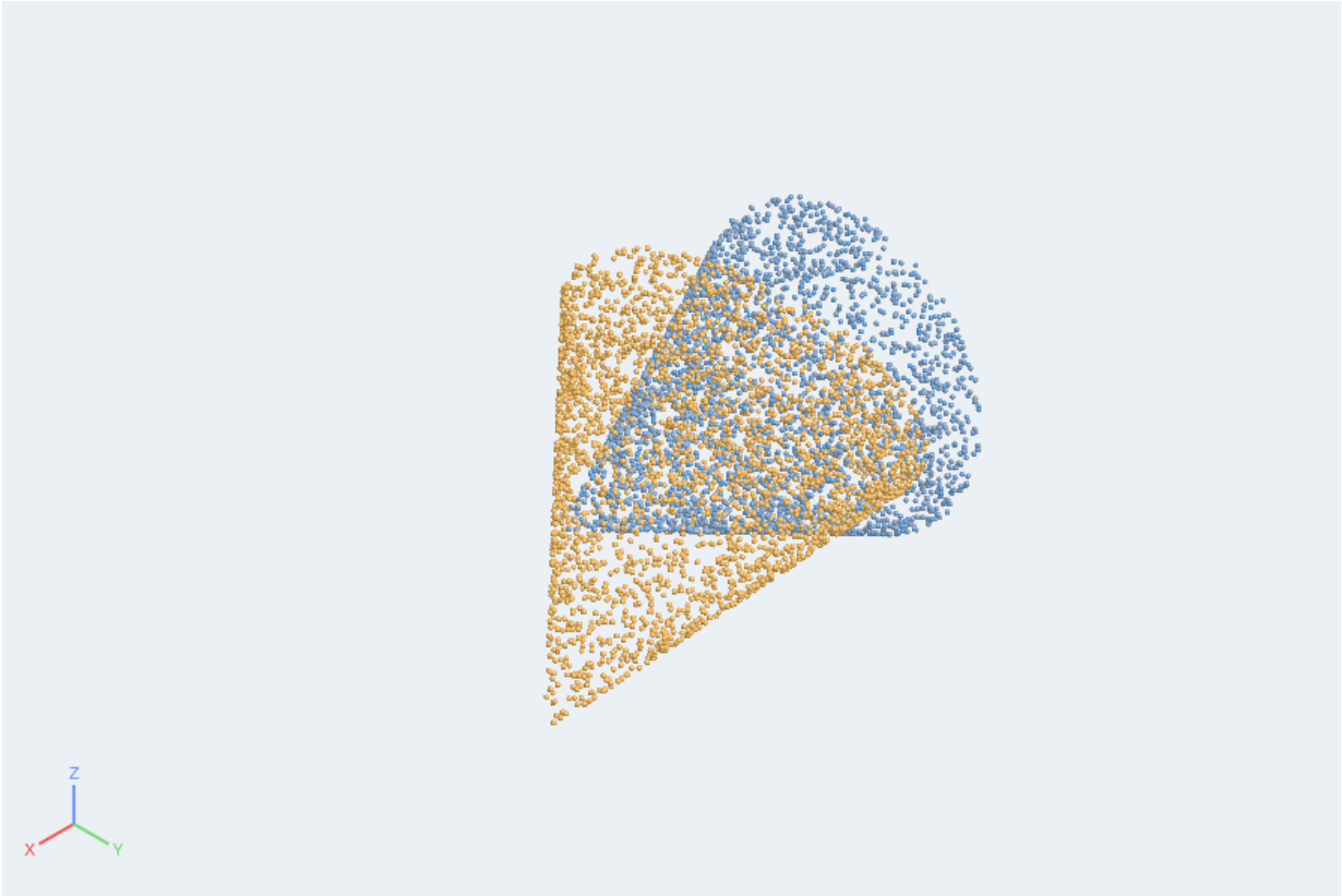

```
h = particles.halo_finder(linking=0.2, min_members=20, mass="mass", velocity="v")
table = particles.halo_finder(output="Halos")
m = volume.minkowski_functionals(array="porosity", threshold=0.5)
```

Point sets

In **Filters › Geometry**:

- **Point Sample Triangle Geometry** spreads a number of points over a surface at random (seeded), each triangle getting points in proportion to its area.
- **Crop Points** keeps the points inside a box, with their arrays.
- **Iterative Closest Point** moves a point set (the first input) rigidly onto another (the second, chosen in Properties): each round pairs every point with its nearest target point and finds the rotation and translation that bring the pairs closest, until it stops improving. The output is the moved points, with the 4 × 4 `Transform` (row by row) and the `RMSDistance` in its field data; a cloud turned by 10° and moved comes back to within 10^{-6}. It finds the nearest fit from where the points start, so the two sets must already roughly overlap.

```
target = afx.load("part.stl").point_sample_surface(samples=3000)
moved = target.transform(rotate=[0, 20, 10], translate=[0.2, 0, 0])
aligned = afx.iterative_closest_point(moved, target, iterations=100)
```



From Python

```
afx.filters()                  # every filter with its parameters
afx.help_filter("clip")        # one filter explained
s = d.slice(normal=(0, 1, 0))
c = d.contour(array="Temp", values=[400, 600])
k = d.clip(type="Box", box_min=(-3, -3, -2), box_max=(3, 3, 4))
t = d.threshold(array="Pres", min=0.01)
s.set(origin=(0, 1, 0))        # change a parameter; the filter re-runs
s.update()                     # wait for it
```

Coloring and color maps

Color by an array

The **Color by** box in Properties (and in the toolbar) lists every point and cell array of the object. Vectors are colored by magnitude unless a **Component** is chosen. **Solid color** uses one color for the whole object. New files are colored by their first array automatically.

Range is taken from the data; type values to fix it (useful when comparing time steps or objects), press **auto** to return to the data range, tick **log** for a logarithmic scale and **bar** to show the legend in the view. Values that are not a number (NaN, fill values of NetCDF files) are drawn grey.

```
d.set(color_by="Temp", cmap="Inferno", range=(300, 900), log_scale=False)
d.set(color_by="V", component=0)        # x component of a vector
d.set(color_by=None, solid_color=(0.8, 0.2, 0.1))
```

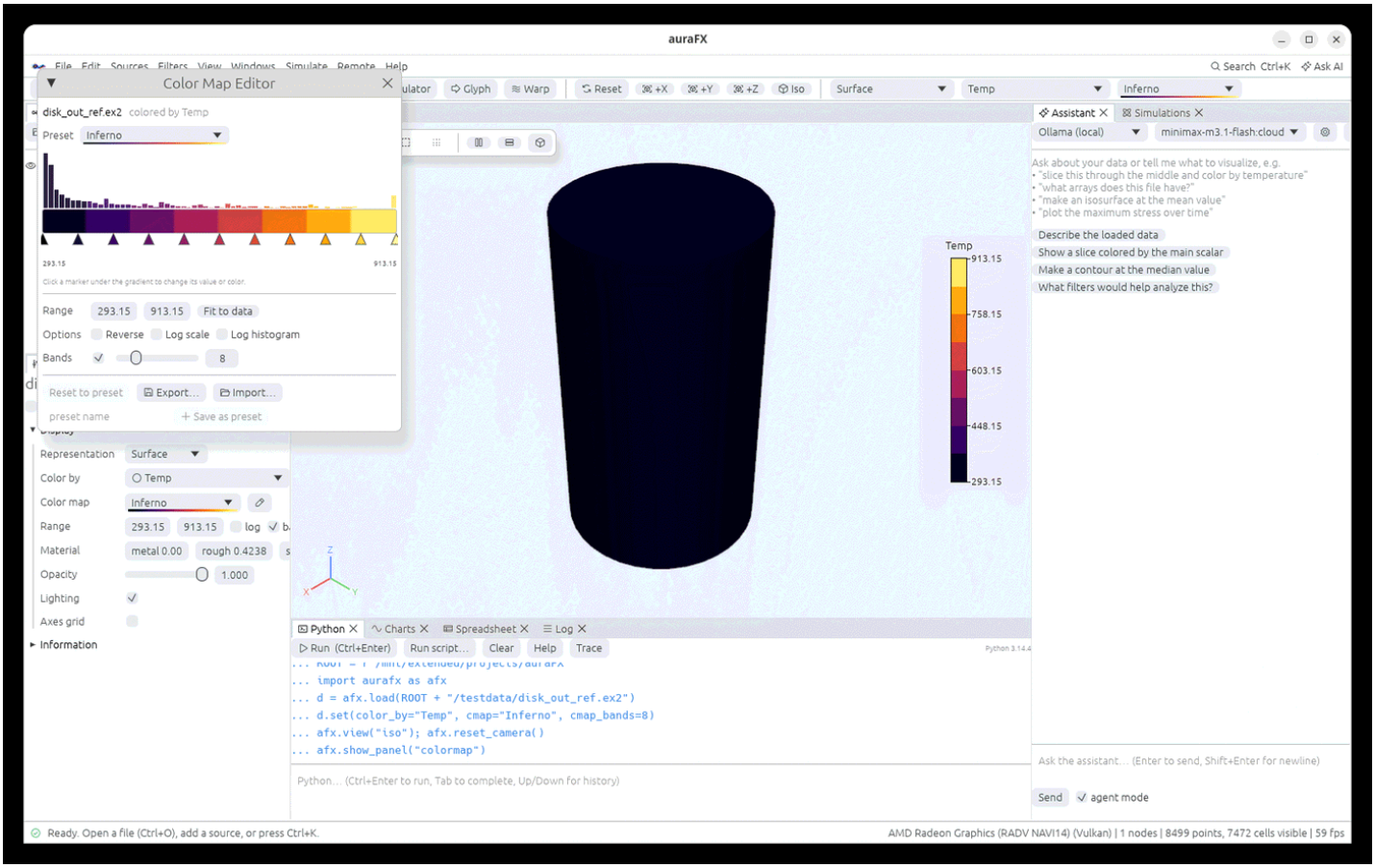
An array of 8-bit values with 3 or 4 components (red, green, blue and maybe opacity, such as the `IPFColor` array of Orientation Colors) is drawn with its own colors, without a legend. Clear **Colors: the array's own colors** under Color by, or use `d.set(direct_colors=False)`, to map it through the color map instead.

To show an image on a surface instead of colors (a map on a globe), see Textures.

Color maps

Eighteen maps ship with the app (Viridis, Inferno, Magma, Plasma, Cool to Warm, Jet, Rainbow, Grayscale, ColorBrewer sets and more); the **Color map** box shows each with a preview and marks the ones safe for color-blind readers. **Reverse** flips a map and **Bands** turns it into steps.

The color map editor



View › Color Map Editor… (or the pencil next to the map) opens the editor for the selected object:

- the histogram of the array behind the map shows where the values lie;
- drag the markers to move colors, double-click to change a color, right-click to delete one, click between markers to add one;
- **Bands**, **Reverse**, and the range fields as in Properties;
- **Save as preset** keeps your map under a name for other objects and sessions;
- **Import…** reads ParaView color map files (`.json`) as presets; **Export…** writes them.

```
afx.colormaps()                                   # names
afx.define_colormap("hot steel", [[0, 0, 0, 0], [0.5, 1, 0.3, 0], [1, 1, 1, 1]])
afx.import_colormaps("paraview_maps.json")
d.set(cmap="hot steel", cmap_reverse=True, cmap_bands=8)
d.set(cmap_points=[[0, 0, 0, 1], [1, 1, 0, 0]])    # a one-off map on this object
```

The legend

The legend (scalar bar) shows the map, its range and the array name for the selected object, or for the first object with a legend when nothing is selected. Untick **bar** to hide it. It is included in screenshots and animations, and drawn as real text in SVG exports.

Views, layouts and the window

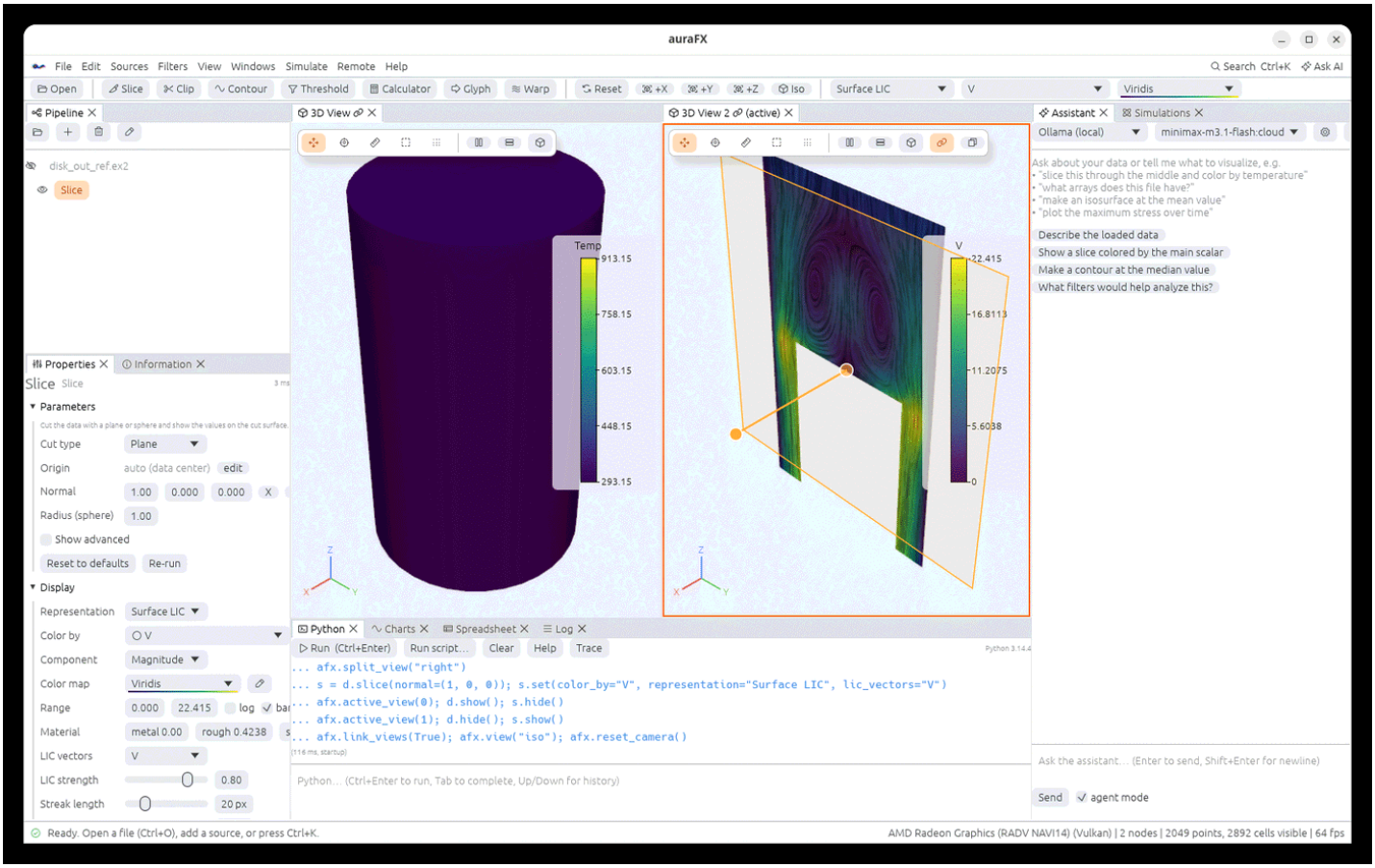
The 3D view

Action	Mouse / key
rotate	drag
pan	middle-drag, or Shift+drag
zoom	wheel, right-drag, or Ctrl+drag; pinch on a trackpad
fit the data	double-click, **R**, or the **Reset** button
look along an axis	**+X**, **+Y**, **+Z**, **Iso** in the toolbar, or **View › Look from…**
orthographic projection	**View › Orthographic projection**
orientation axes (bottom left)	**View › Orientation axes**

The small toolbar inside the view switches the mouse mode: navigate (**Esc**), pick (**P**), ruler (**M**), select cells (**S**) and select points (**D**); see Selection, picking and widgets.

```
afx.view("iso"); afx.view("+x"); afx.view([1, 0.5, 0.3])   # a direction from the data to the camera
afx.reset_camera()
```

Several views



The split buttons in the view toolbar (or **Windows › Split view right / down**, `afx.split_view("right")`) add a 3D view next to the active one. Each view has its own camera and its own list of visible objects; click a view to make it active (orange frame), then the eye icons in the Pipeline and `node.show()` / `hide()` apply to it. From Python, `afx.active_view(1)` makes a view active (0 is the first one) and returns once it is, so the lines after it act on that view:

```
afx.split_view("right")                  # returns the new view's number (1)
afx.active_view(0); d.show(); s.hide()    # left: the data
afx.active_view(1); d.hide(); s.show()    # right: the slice
```

- **Link** (the chain button, `afx.link_views(True)`) keeps the cameras of the linked views together; **Match** (`afx.match_views()`) copies the active camera to the others once.
- Right-click an object › **Send to view** moves it to another view or into a new one (`node.send_to_view(1)` or `"right"`, `keep=True` to keep it in the current view too).
- Close a view with the × on its tab; its objects stay in the pipeline.

Panels and layouts

The **Windows** menu lists every panel (Pipeline, Properties, Information, 3D views, Python, Charts, Spreadsheet, Log, Assistant, Simulations) with a tick to show or hide it, and four layout presets:

Preset	Shows
Visualize	pipeline, properties and a large 3D view
Analyze	the view with charts and the spreadsheet
Script	the view with a tall Python console
Simulate	the simulations panel, the log and the view

Reset layout returns to the default. Panels can be dragged by their tabs into new positions or into tabs of each other; the arrangement is remembered between sessions.

```
afx.layout("analyze")
afx.show_panel("Charts"); afx.hide_panel("Log")
```

Theme and scale

Settings › Theme chooses dark, light or follow-the-system; the 3D background follows the theme (and can be set per theme). **UI scale** enlarges the whole interface. From Python: `afx.set_theme("light")`.

Command palette

Ctrl+K (or Ctrl+Space, or the **Search** button) opens the palette: type a few letters of a filter, a source, a panel or an action (open, save screenshot, reset camera, undo, light theme, ask the AI…) and press Enter. It is the fastest way to reach anything in the app.

Undo, redo and state

Edit › Undo / Redo (Ctrl+Z, Ctrl+Y) cover opening files, adding and deleting filters, parameter and display changes. **File › Save State** writes everything to an `.afx` file (see Getting started); `afx.save_state("work.afx")` and `afx.load_state("work.afx")` do the same from Python.

Selection, picking and widgets

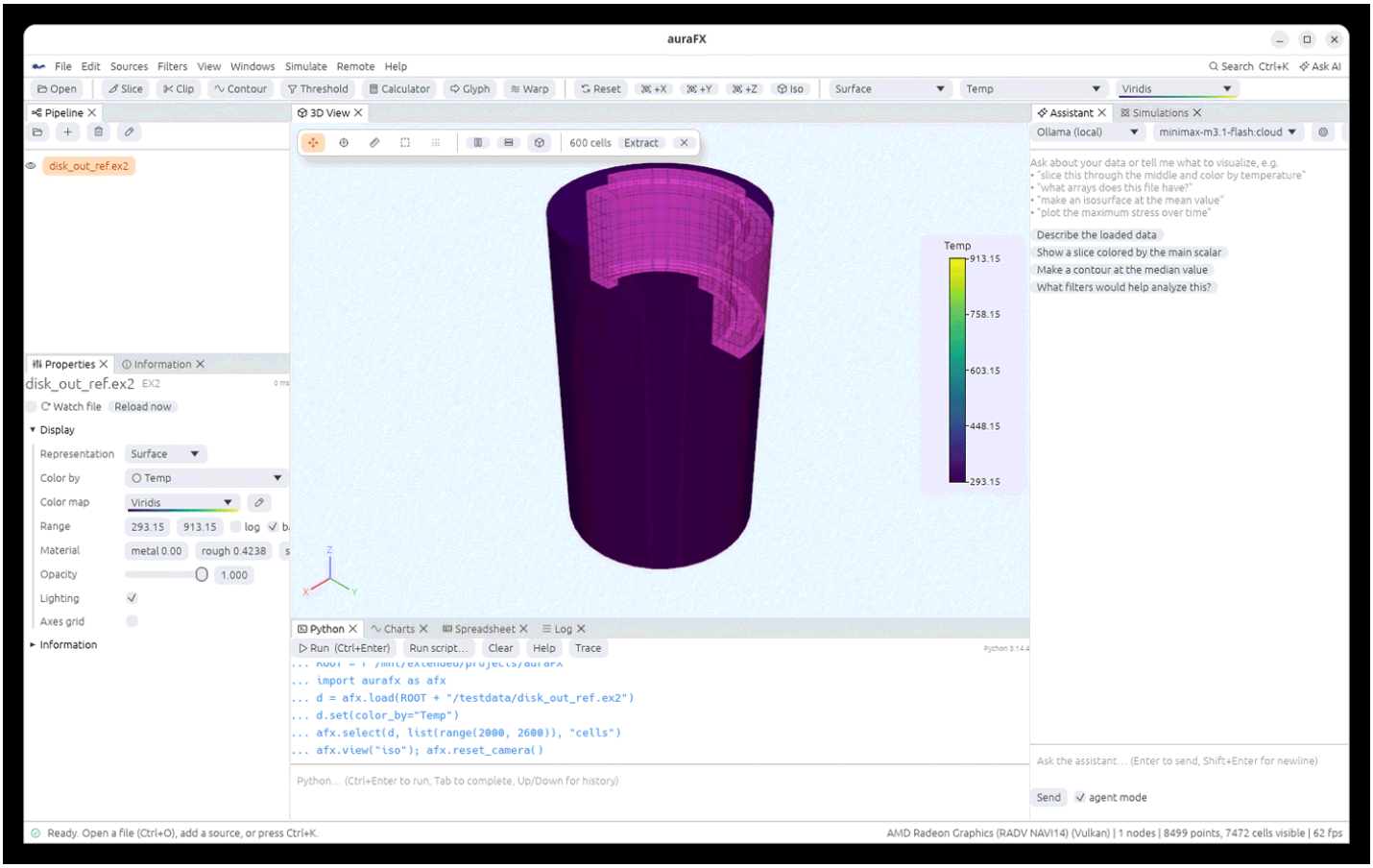
Picking values

Press **P** (or choose the pick mode in the view toolbar, or Ctrl+click in any mode) and click a surface: the point is marked and the Information panel lists every array's value there, with the object's name. `afx.probe(node, (x, y, z))` returns the same values from Python.

The ruler

Press **M**, click two points on surfaces: the distance is drawn and printed between them. Press M again (or Esc) to leave the mode.

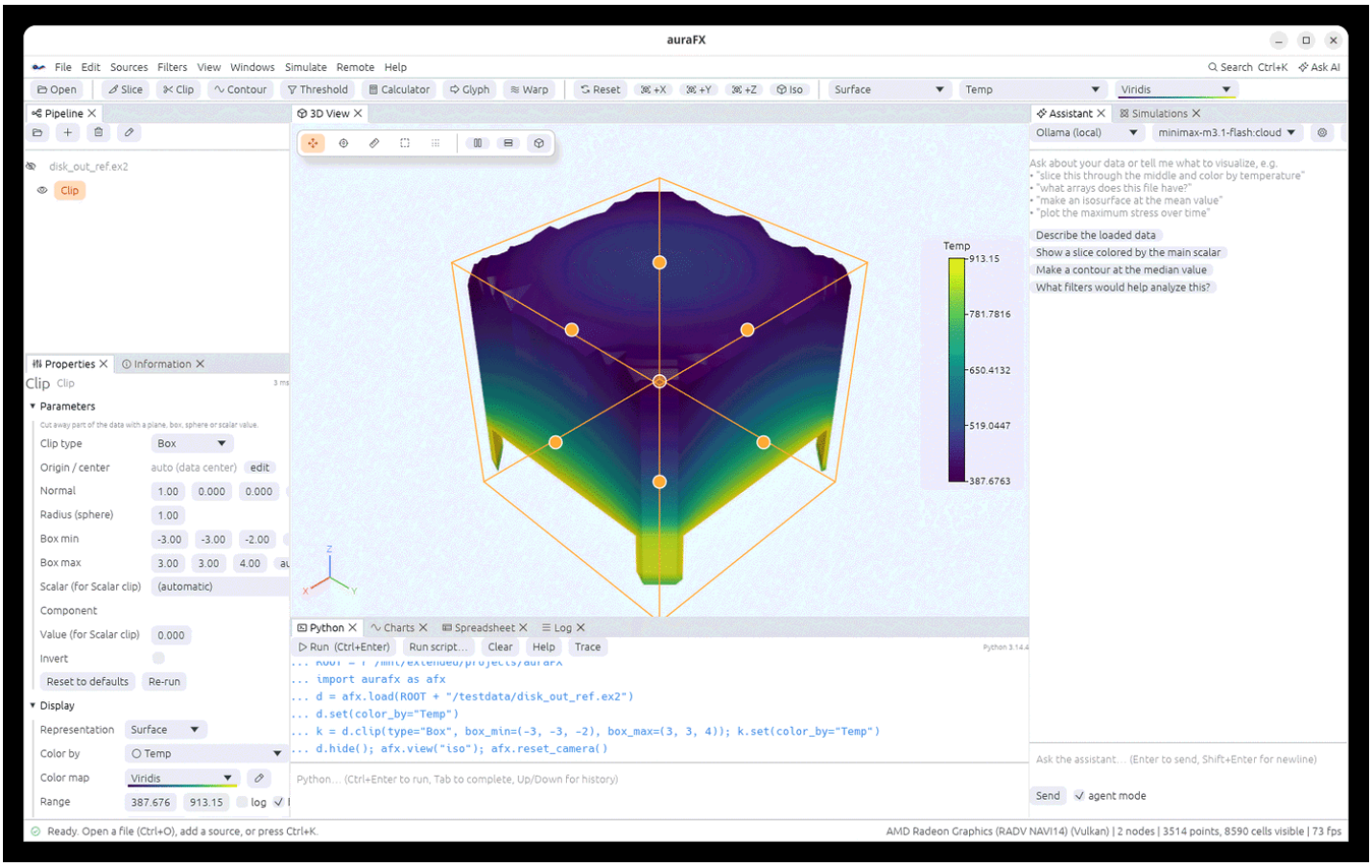
Selecting cells and points



Press **S** (cells) or **D** (points) and drag a rectangle over the view. The selected cells are drawn highlighted through the surface. **Shift** adds to the selection, **Ctrl** removes from it. By default only the cells you can see are selected; **Select through** in the view toolbar takes every cell inside the rectangle, hidden ones included.

- **Create an Extract Selection node** (the button in the view toolbar, `afx.extract_selection()`) turns the selection into a new object you can filter further.
- **Find Data** (Filters) selects by a query such as `Temp > 600` or the N largest values.
- The Spreadsheet shows the selected rows; `afx.selection()` returns them, `afx.select(node, ids, "cells")` sets them, `afx.clear_selection()` clears.

Widgets: dragging filter shapes in the view



When the selected filter has a shape, it is drawn in the view with orange handles:

Filter	Widget
Slice, Clip (plane)	a plane: drag the center along the normal, drag the tip to tilt

Filter	Widget
Clip (box), Box source	a box: drag the center to move it, a face handle to resize that side
Slice/Clip (sphere), Stream Tracer point-cloud seeds, Sphere source	a sphere: center and radius handles
Plot Over Line, Stream Tracer line seeds, particle tracers, Line source	a line: either end, or the middle to move both
Probe Location	a point

The parameters in Properties follow the drag, and typing values moves the widget. Custom lights can be aimed the same way; see Lighting.

Annotations

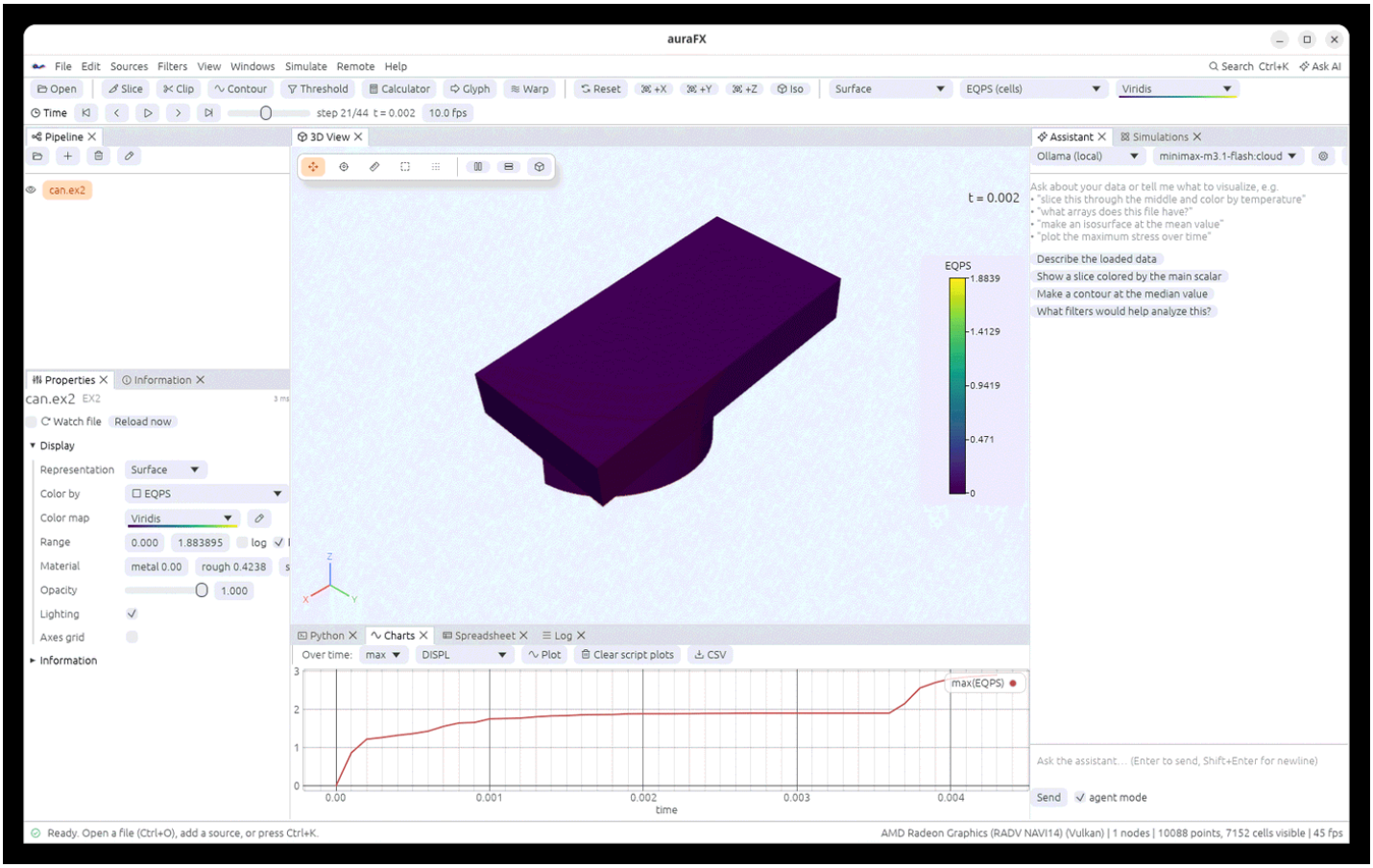
Filters › Annotation, `afx.annotate("Run 42", corner="upper left", size=18)` puts text in a corner of the view; `afx.time_label(True)` (or **View › Time label**) shows the current time. Both are drawn in screenshots and animations. `afx.clear_annotations()` removes them.

Time, charts and animation

Time steps

Files with several time steps (Exodus, `.pvd`, OpenFOAM cases, LAMMPS dumps, numbered file series, VTKHDF, EnSight, Xdmf, NetCDF, CGNS), and the Spatio Temporal Harmonics source, show the **time toolbar**: play/pause, first, previous, next, last, a slider, the step number, the time value and the playback speed. Every filter follows the time step; filters that look across time (Temporal Statistics, Particle Tracer, Temporal Interpolator…) read the steps they need.

```
afx.times()          # the time values
afx.time(20)         # go to step 20
```



Charts

The **Charts** panel plots the arrays of an object as lines, points, lines with markers or bars, with zoom, a legend and log axes:

- **Plot Over Line** (a filter) plots the arrays along a line;
- **Edit › Plot Over Time…** or `afx.over_time(node, "EQPS", stat="max")` plots a statistic (max, min, mean, sum, or the value of a global variable) over every time step;
- `afx.plot(x, y, label="T")` plots your own NumPy arrays; `afx.clear_plots()` empties the panel.

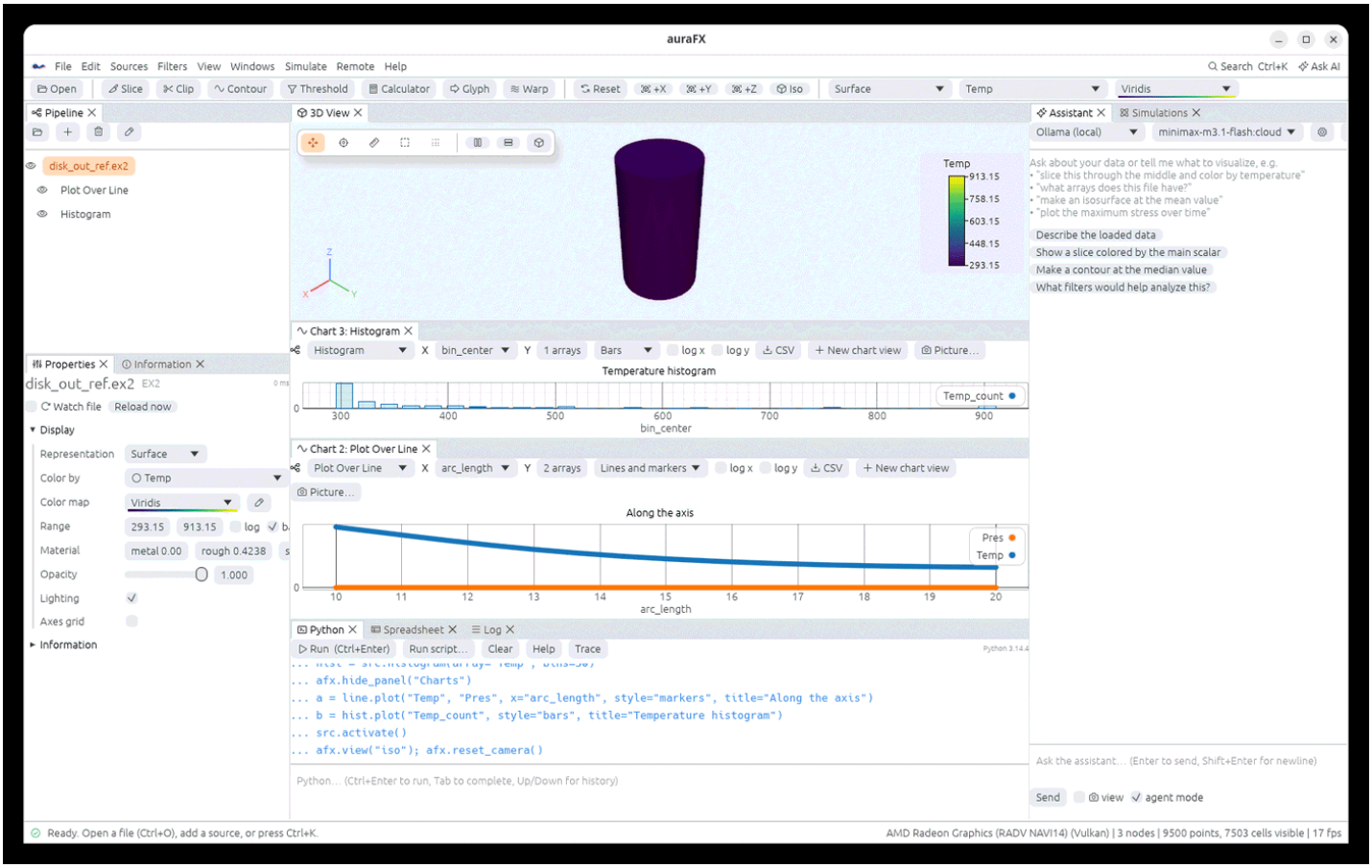
The boxes along the top of a chart choose, from left to right: the **object** it plots (*Selected object* follows whatever is selected in the Pipeline), whether a mesh's **point** or **cell** arrays are plotted, the **X** array (or the index), the **Y** arrays, the **style** (lines, points, lines and markers, bars), and **log x** / **log y** (values that are not positive are left out). **CSV** saves the plotted values; **Picture…** saves the chart as a PNG without these controls.

Several chart views

A chart view can be opened for every object you want to compare, and each keeps its own object, arrays and style while you select other things:

- **View › New Chart View** opens one for the selected object;
- right-click an object in the Pipeline › **Plot in a new chart view**;
- **New chart view** on any chart opens another one beside it.

Chart views open beside the Charts panel and can be dragged, split and stacked like the 3D views (drag a tab to an edge). Closing a chart's tab closes the view. State files (`.afx`) keep the chart views and reopen them.




```
line = src.plot_over_line(point1=[0, 0, -10], point2=[0, 0, 10])
hist = src.histogram(array="Temp", bins=30)
a = line.plot("Temp", "Pres", x="arc_length")        # a new chart view of the line
b = hist.plot("Temp_count", style="bars")            # another one, of the
histogram
afx.chart_view(view=a, log_y=True)                   # change view a later
afx.chart_view(y="V_X")                              # a view that follows the
selection
afx.chart_views()                                    # a list of dicts, one per
view
afx.plot(t, energy, label="E", view=b)               # your own line in chart view
b
afx.close_chart_view(a)
```

`node.plot(*arrays, x=..., style=..., data="points"|"cells", log_x=..., log_y=..., title=...)` and `afx.chart_view(node=None, y=..., x=..., style=..., data=..., log_x=..., log_y=..., title=..., view=None)` return the view's number (`view=0` is the Charts panel). Charts plot up to two million values per array.

The **Spreadsheet** panel shows the selected object's points, cells or field data as a table with sorting, column choice and export to CSV.

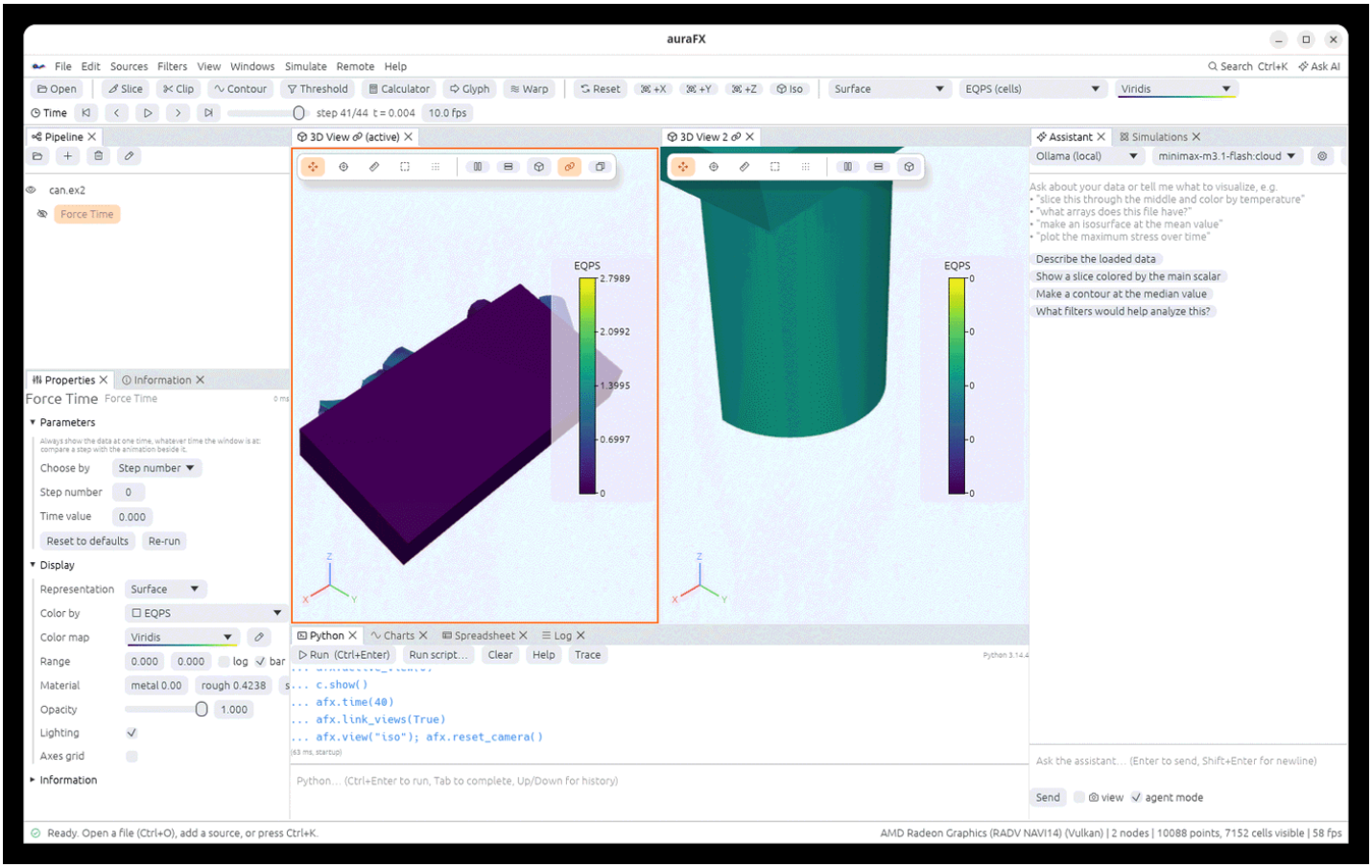
Choosing and changing time steps

In **Filters › Temporal**:

- **Extract Time Steps** keeps some steps: a list such as `0, 3, 5-8, 10:20:2` (step numbers from 0; `start:stop:stride` leaves out stop), or every N-th step from a first one.
- **Generate Time Steps** gives data a new set of evenly spaced time values (each shows the data at that time, or at the step before it); static data gets a time axis to animate filters against.
- **Force Time** always shows the data at one step (or time), whatever time the window is at: put step 0 beside the animation to compare.
- **Temporal Snap to Time Step** shows the data at one of its own steps (nearest, previous or next) when the window is at a time in between, set by other data or by Temporal Interpolator.
- **Temporal Cache** keeps the last steps shown in memory, so stepping back and forth or replaying a loop does not read and filter them again.

Particle Tracer, Particle Path and Streak Line release their seeds again every N steps with *Release again every N steps*, which makes streak lines from a continuous injection.

```
first = case.force_time(step=0)                # step 0 beside the playing data
every5 = case.extract_time_steps(stride=5)
smooth =
case.temporal_interpolator(interval=0.01).temporal_snap_to_time_step(mode="Previou
s")
```



Animation

File › Animation… records one of three things:

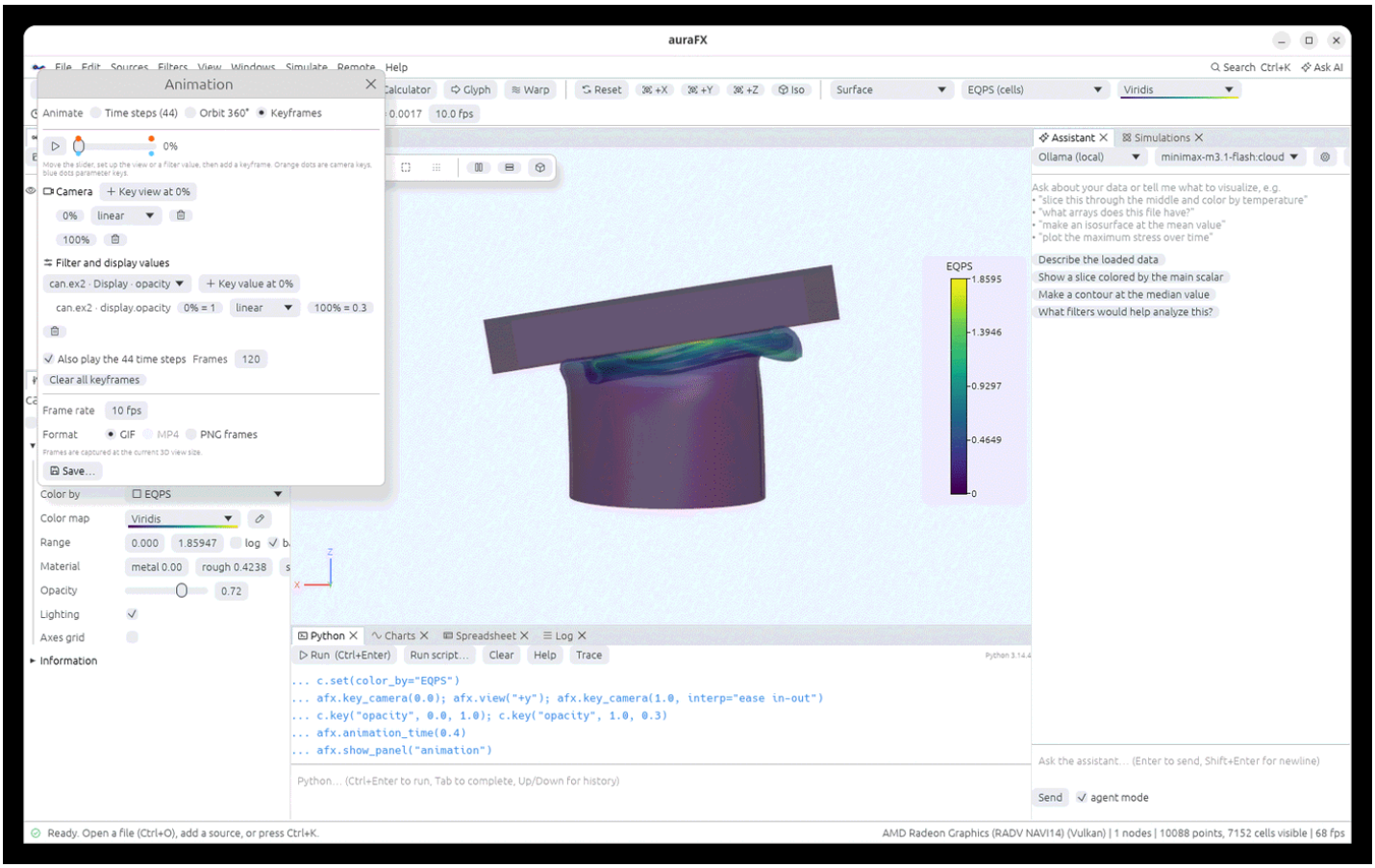
Mode	What it plays
Time steps	every time step of the data
Orbit 360°	the camera turning once around the data (choose the number of frames)
Keyframes	the keyframe timeline described below

Pick the frame rate and the format: **GIF**, **MP4** (needs `ffmpeg` on the PATH) or a folder of **PNG frames**. Frames are captured at the 3D view's size, with the legend, labels and annotations.

Record › all views records every visible 3D view and chart view in each frame, placed as they are laid out in the window: the charts follow the time steps (a chart of a Plot Over Line changes with the data) while the 3D views play. **Record › chart views** records only the charts. These frames have the window's pixels; the Animation window is hidden while they are recorded, and the charts' controls are left out.

```
afx.save_animation("run.gif", fps=10)          # time steps (or keyframes if any)
afx.save_animation("turn.mp4", orbit=72)       # a turntable
afx.save_animation("all.mp4", views="all")     # every 3D view and chart view, as
laid out
```

Keyframes



The **Keyframes** tab of the Animation window is a timeline from 0 % to 100 %:

1. Move the slider, set up the view, press **Key view at …** to record the camera.
2. Select a filter or display value in the list (any numeric filter parameter, or opacity, point size, line width, colors…), set it, press **Key value at …**.
3. Each key has an interpolation menu for the way it moves on to the next key: **linear**, **step** (hold), **ease in**, **ease out**, **ease in-out**, **exponential** (constant ratio, for zooms and scales) or **wave** (there and back, a sinusoid).
4. Press play to preview; tick **Also play the time steps** to run the data’s time along the animation; set the number of **Frames**; **Save…** records it.

The camera moves on a sphere around the focal point, so turns look even and zooms do not rush. Orange dots on the timeline are camera keys, blue dots value keys; click a key to jump to

it, right-click to delete it.

```
afx.key_camera(0.0); afx.view("+y"); afx.key_camera(1.0, interp="ease in-out")
c.key("opacity", 0.0, 1.0); c.key("opacity", 1.0, 0.3)
s.key("origin", 0.5, (0, 2, 0), interp="wave")
afx.animation(frames=120, time=True)
afx.animation_time(0.4)                          # preview a moment
afx.save_animation("keys.mp4", fps=24)
afx.clear_keyframes()
```

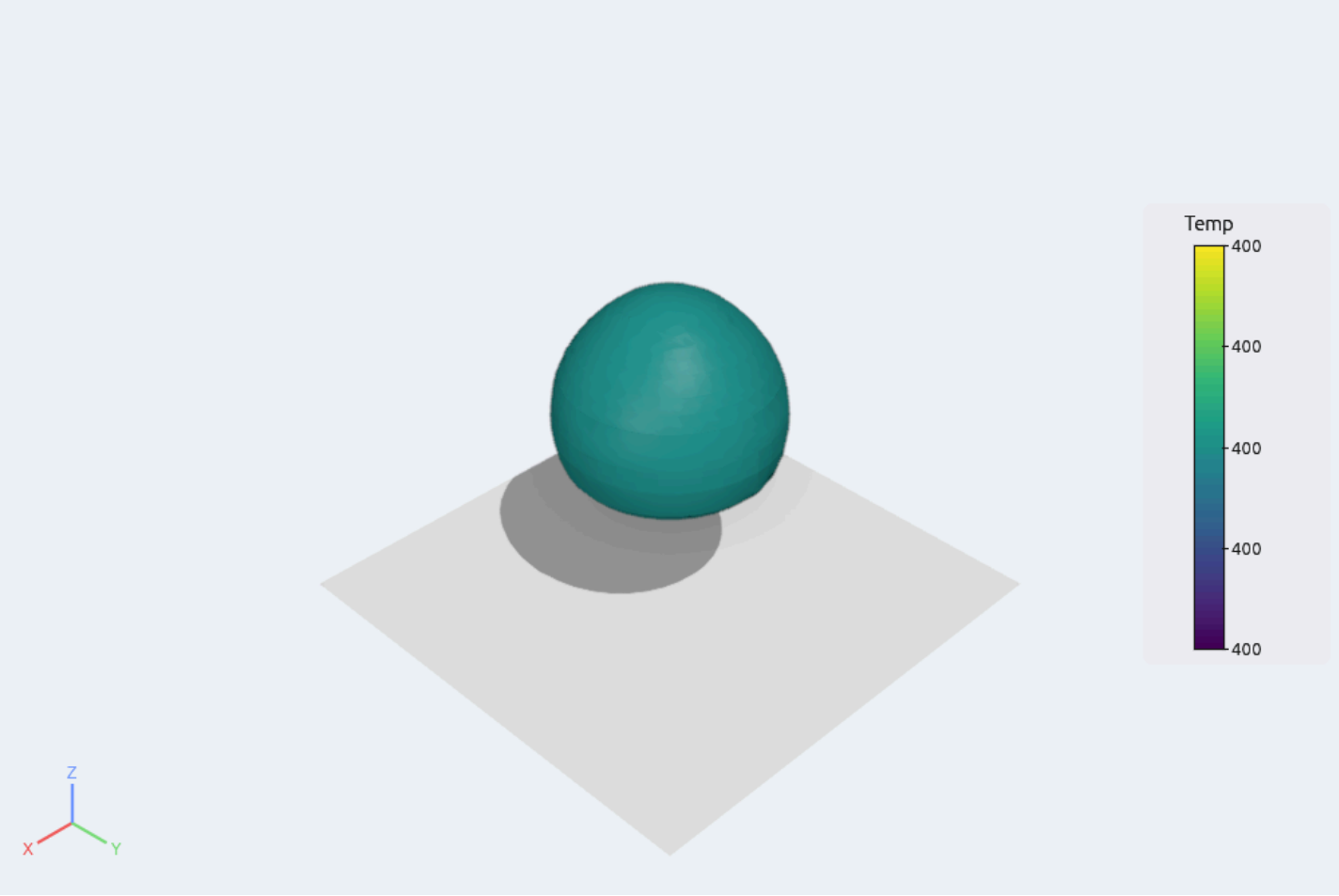
Animations driven by a function

`afx.save_animation("a.mp4", cue=f, frames=60)` calls your function `f(t)` with the animation time `t` from 0 to 1 before each frame (after the keyframes are applied), so anything can change between frames: filter values, colors, the data loaded. The call returns when the file is written.

```
def cue(t):
    c.set(values=[350 + 500 * t])
afx.save_animation("contour_sweep.gif", fps=8, cue=cue, frames=40)
```

Lighting, shadows and effects

Everything here is in **View › Lighting**, and in Python through `afx.lighting(...)`. Calling `afx.lighting()` without arguments returns the current defaults; unknown options raise an error, so typos are caught.



Lights

- **Light kit** (on by default): a key light from the upper right, a fill light from the lower left and a head light. Untick it for a single head light.
- **Custom lights**: tick it to edit the lights yourself. Each light has a color, an intensity, an azimuth and an elevation, follows the camera or stays fixed in the scene, and can cast shadows. **Add light** adds one; **ambient** sets the light from everywhere.
- **Show lights in the view** draws each custom light as a small sun around the data; drag a sun to aim its light.

```
afx.lighting(light_kit=False)
afx.lighting(lights=[{"direction": (0.5, 0.5, 1), "intensity": 0.8},
                     {"direction": (-0.7, -0.2, 0.5), "intensity": 0.3, "color":
(0.6, 0.7, 1), "camera_relative": False}],
             ambient=0.15)
```

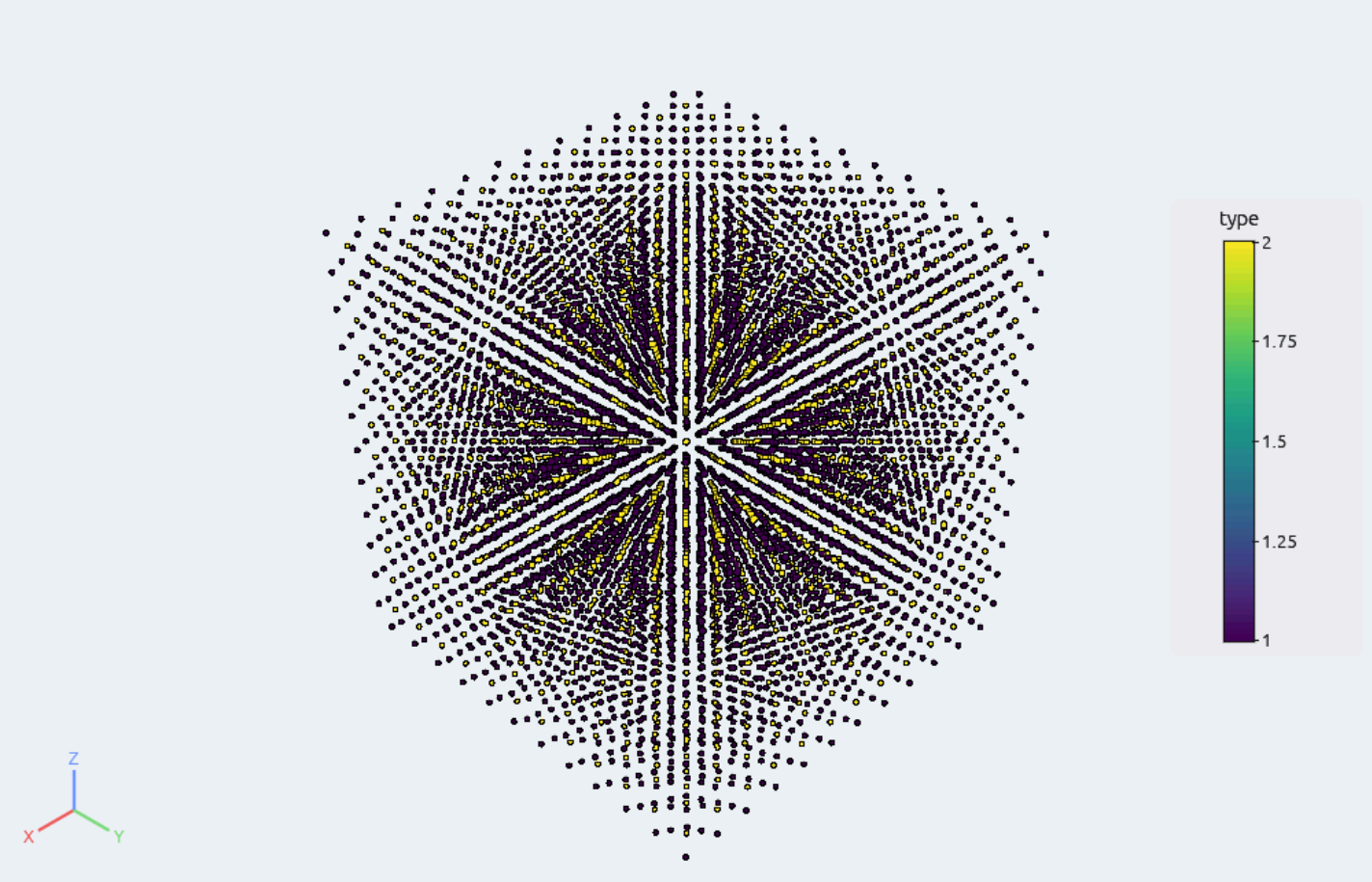
Shadows

Shadows draws the shadow of the key light (or of the first custom light with shadows ticked) on every opaque surface: a shadow map fitted to the visible objects, softened at the edges. Objects drawn by a remote server are shadowed on that machine, so they do not cast shadows on local objects.

Ambient occlusion

Ambient occlusion darkens creases, cavities and the contact between objects, which makes depth easier to read. The sliders set its strength and radius (a fraction of the data size). It works from the picture’s depth, so it costs the same whatever the mesh size.

Eye-Dome Lighting



Eye-Dome Lighting shades every pixel by how much it stands out from its neighbours in depth: outlines and depth cues for point clouds, particles and dense meshes without any normals. The slider sets the strength. `afx.lighting(edl=True)`.

Anti-aliasing, tone mapping and exposure

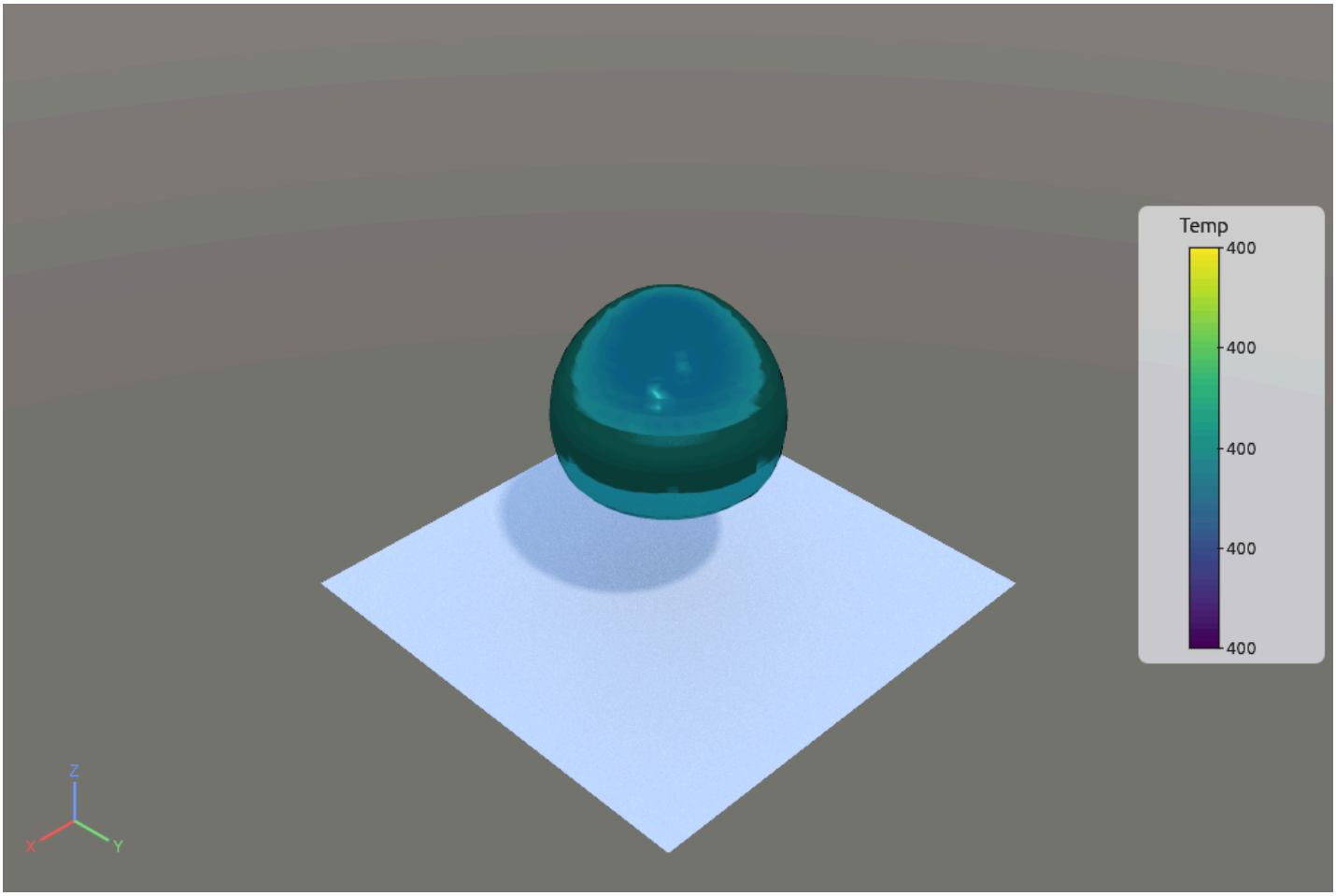
Edges are smoothed by multisampling (4 samples). **FXAA** adds a cheap post pass that smooths the remaining jagged edges of thin lines. **Tone mapping** compresses bright pictures with the **Reinhard** or **ACES filmic** curves, and **exposure** scales brightness before it.

```
afx.lighting(fxaa=True, tone_mapping="aces", exposure=1.2)
```

Transparency

Translucent objects (**Opacity** below 1 in Properties) are blended in a way that does not depend on the order they are drawn in, so several overlapping translucent surfaces look right from any angle. Translucent objects are always drawn on this computer, even when the rest of the data is drawn by a server.

Materials and environment maps



Material in Properties gives each surface a **metal** factor (0 plastic-like, 1 metal: reflections take the object's color), a **rough**ness (small: sharp highlights and mirror-like reflections; 1: matte) and a highlight strength. The defaults reproduce the plain shading.

View › Lighting › Environment lights the scene with a picture of the surroundings: the built-in **sky**, **studio** and **white** environments, or your own equirectangular `.hdr` / `.png` / `.jpg` (z up). It replaces the flat ambient light with soft directional light, adds reflections on shiny and metallic surfaces, and **as background** shows it behind the data. While an environment is on, the built-in lights are dimmed by half.

```
c.set(metallic=1.0, roughness=0.25, specular=0.5)
afx.lighting(environment="sky", environment_intensity=1.2, show_environment=True)
afx.lighting(environment="/data/hdri/studio_small.hdr")
```

Ray tracing

View › Lighting › Ray tracing path-traces the opaque surfaces: soft shadows, light from the sky or environment, and light bouncing between surfaces. It refines while the camera is still (the rasterizer draws while you move) and the status corner shows the samples done. Set the number of **samples** per pixel to stop at, the number of **bounces**, and where it runs: **auto** (the GPU when the data fits in its memory, else the CPU), **GPU** or **CPU**. Edges, points, lines, translucent objects, volumes and the legend are still drawn on top. Screenshots wait for the tracer to finish and are traced at the requested size.

```
afx.lighting(ray_tracing=True, ray_samples=256, ray_bounces=3, ray_device="auto")
afx.screenshot("traced.png", width=3840)
```

Surface LIC

The **Surface LIC** representation (Properties › Representation) draws a vector field as streaks on any surface while the surface stays colored by any array: the classic way to see a flow on a slice. Choose the vector array (**LIC vectors**), the streak **strength** and the **streak length** in pixels.

```
s.set(representation="Surface LIC", lic_vectors="V", color_by="Temp",
lic_length=30)
```

Volumes, point clouds and curved cells

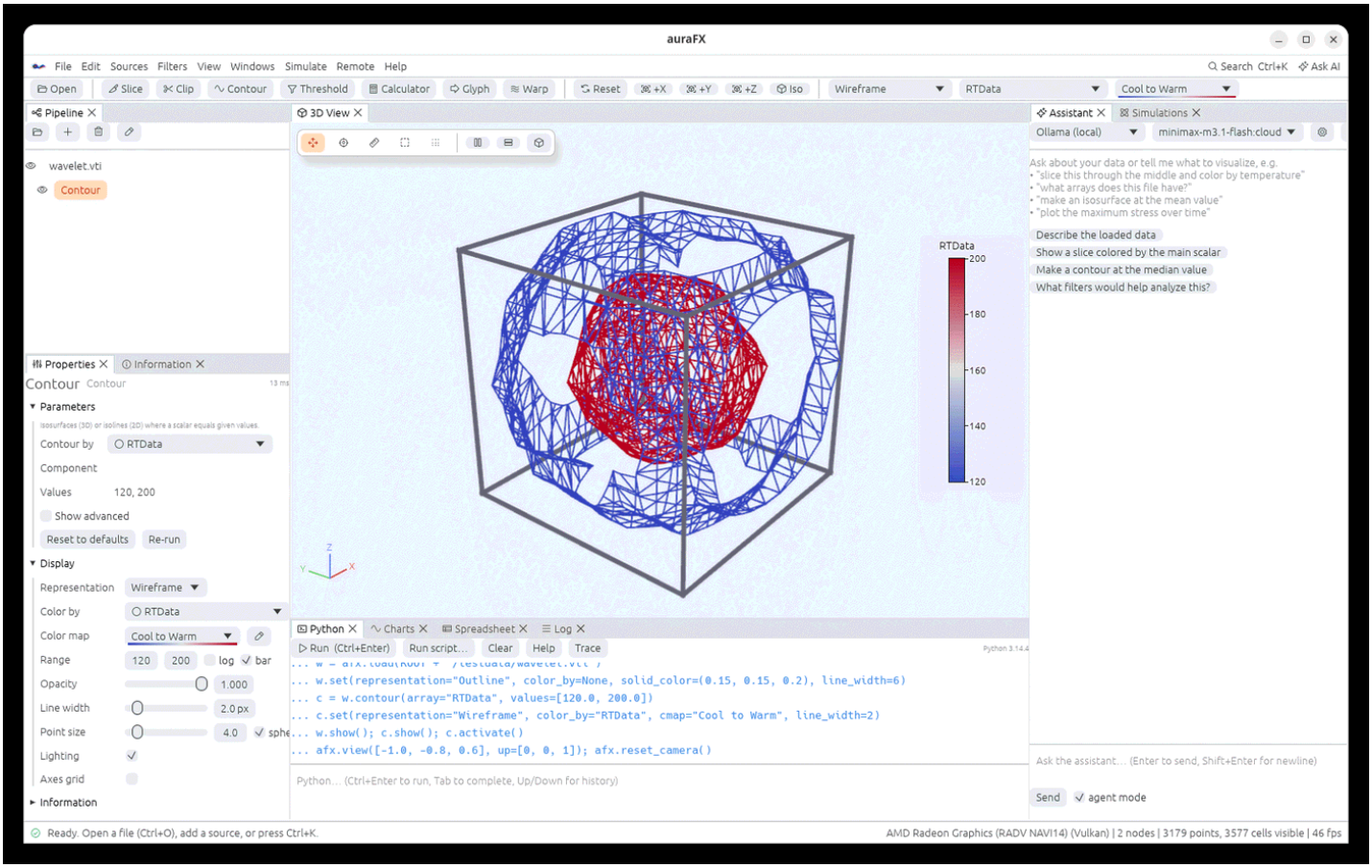
Representations

Representation in Properties (and the toolbar) chooses how an object is drawn:

Representation	Use
Surface	the outer surface, smooth-shaded (hard edges stay sharp: a 30° crease angle)
Surface With Edges	the surface with cell edges (**Edge color** in Properties, `edge_color=` in Python)
Wireframe	edges only
Points	the points, as flat dots or shaded spheres (**Point size**, **spheres**)
Outline	the bounding box (every box of AMR data)
Volume	GPU ray casting of image data (other meshes are resampled first)
Surface LIC	a vector field as streaks (see Lighting)
Molecule	atoms as spheres and bonds as cylinders (see Molecules)

Lighting can be switched off per object (flat colors, for 2-D data). **Axes grid** draws a labelled grid around the object with tick values.

Line width (Properties › Display, shown for Wireframe, Surface With Edges, Outline and line data; `line_width=` in Python) sets how many pixels wide lines, wireframes and cell edges are drawn, from 1 to 10 (any width from Python). It is a width on the screen, so lines keep their width when you zoom. Wider lines are drawn as flat bands with square ends (reaching half the width past each point), which fills the joints of bent lines. Wide lines show in screenshots and in pictures drawn on a server; picking still uses 1-pixel lines. For lines with a thickness in the scene (shaded, and growing as you zoom in), use the **Tube** filter.




```
c.set(representation="Wireframe", line_width=2)
w.set(representation="Outline", line_width=6)
```

Volume rendering

The **Volume** representation draws image data as a translucent cloud colored by the array: **Volume density** sets how opaque it is and **Transparent below** makes values under a fraction of the range invisible, which clears away the noise around a feature. Volumes are depth-merged with the surfaces around them.

```
w.set(representation="Volume", color_by="RTData", volume_opacity=2.0,
volume_threshold=0.3)
```

Point clouds and particles

LAMMPS dumps, XYZ files, PDB files and point sources open as points. Draw them as spheres sized by **Point size**, color them by type, and switch on Eye-Dome Lighting to see the structure of a dense cloud. The **Glyph** filter draws arrows or spheres scaled by an array; **Tube** gives lines a thickness.

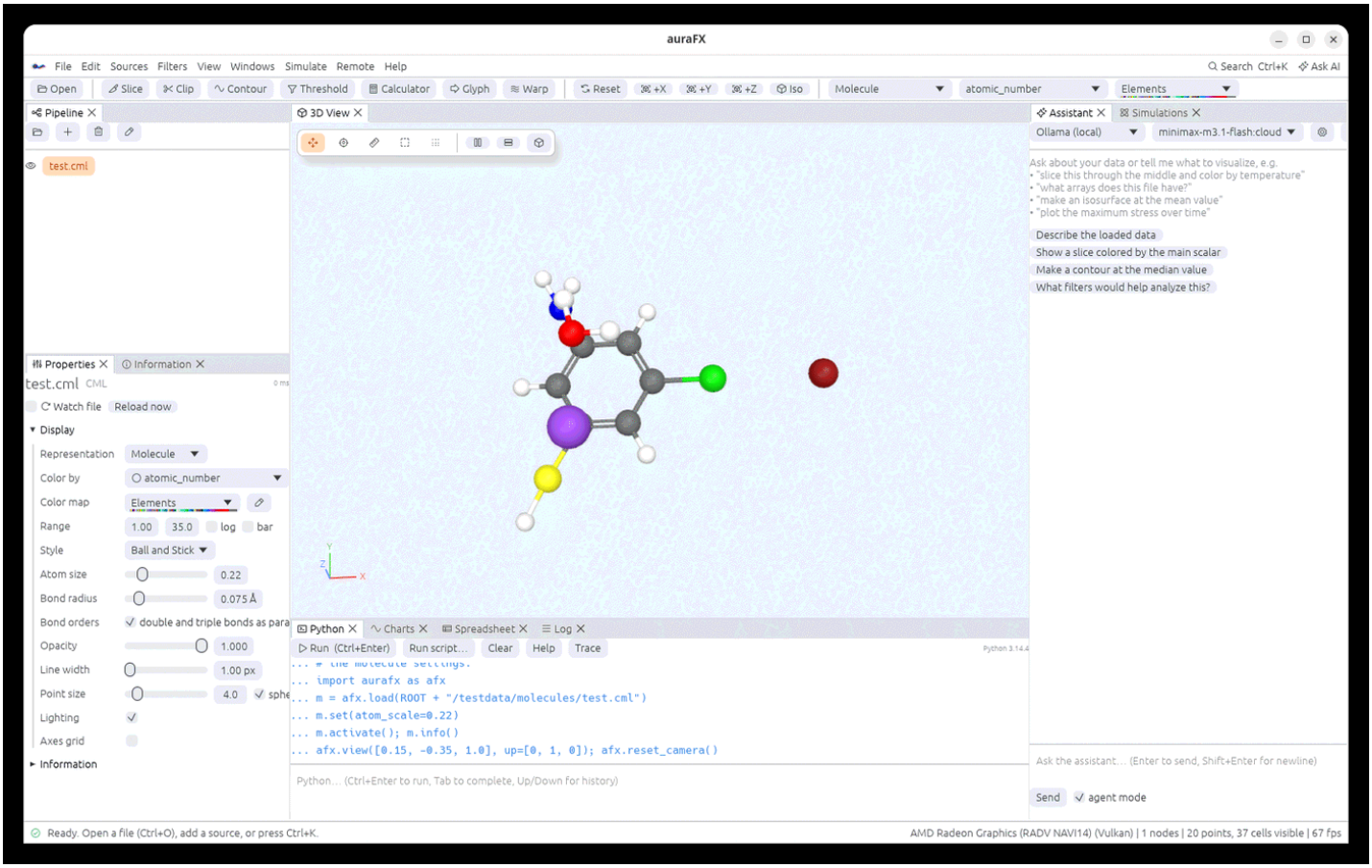
Molecules

Molecules are points (the atoms) with an atomic number array (`atomic_number` , or VTK's `Atomic Numbers`), and lines (the bonds). The PDB, XYZ, VASP and CML readers produce them (and the atoms block of a Gaussian cube file). A molecule opens in the **Molecule** representation, colored by element with the **Elements** color map (Jmol's colors). Files with more than 100 000 atoms open as points instead; switch to Molecule when you need it.

Settings (Properties › Display, or `node.set(...)`):

Setting	Python	Default	What it does
Style	`molecule_style`	Ball and Stick	**Ball and Stick**: atoms at a fraction of their van der Waals radius, bonds as thin sticks. **Van der Waals**: atoms at their full radius, no bonds (space filling). **Liquorice**: atoms and bonds as tubes of one width
Atom size	`atom_scale`	0.3	fraction of the van der Waals radius (Ball and Stick)
Bond radius	`bond_radius`	0.075 Å	radius of the sticks
Bond orders	`bond_orders`	on	double and triple bonds (a `bond_order` array, from CML files) drawn as two or three sticks

Each half of a bond takes the color of its atom. Color by any other atom array (charge, residue, B factor…) with **Color by** as usual. The radii and colors come from VTK's periodic table.

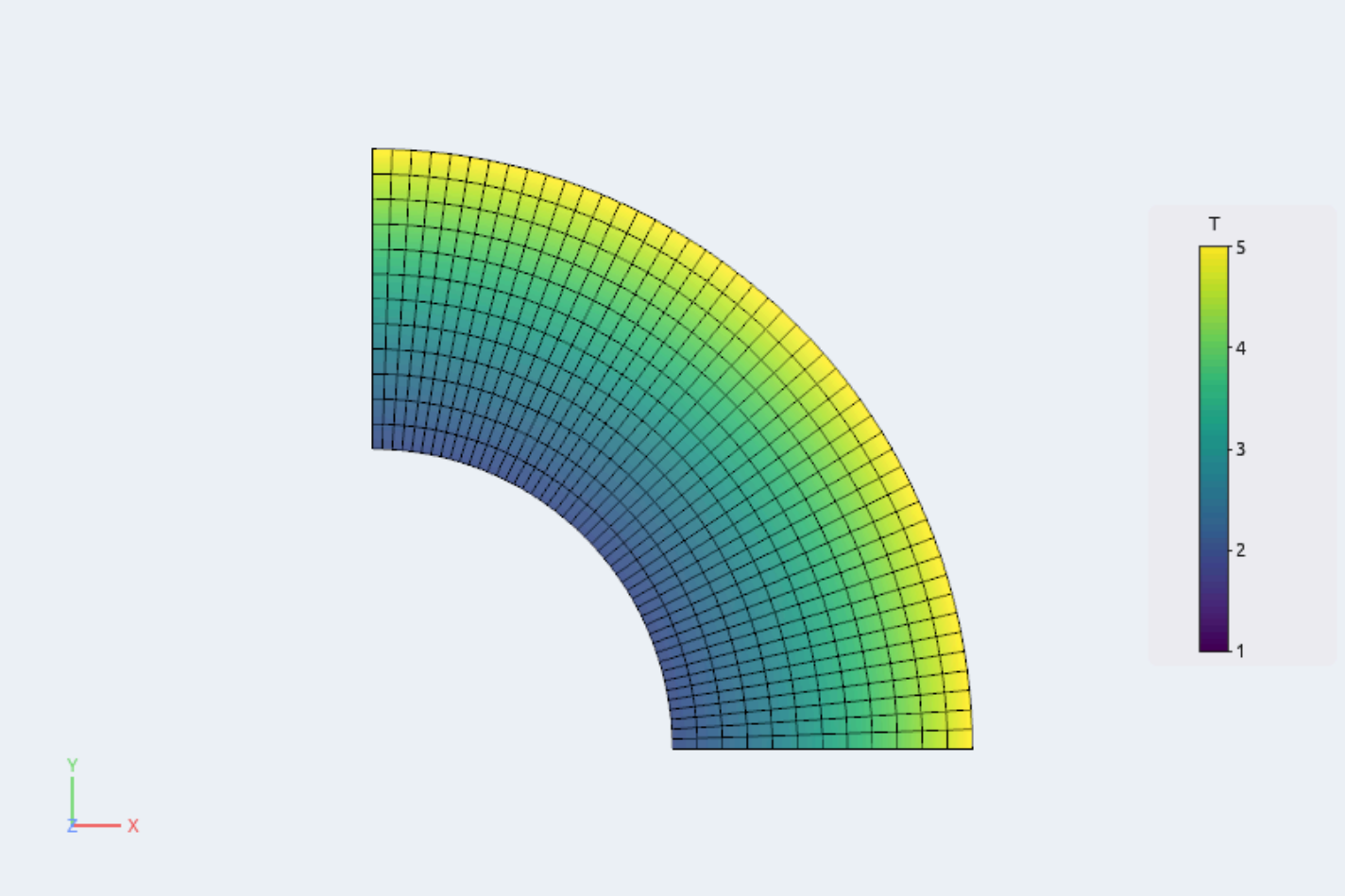



```
m = afx.load("molecule.cml")                  # Molecule representation, colored by
element
m.set(molecule_style="Van der Waals")         # or "Liquorice", "Ball and Stick"
m.set(atom_scale=0.25, bond_radius=0.1, bond_orders=False)
b = m.compute_molecule_bonds(tolerance=0.45) # bonds from distances
```

Filters › Compute Molecule Bonds finds bonds from distances: two atoms are bonded when they are closer than the sum of their covalent radii, each increased by half the **Tolerance** (0.45 Å). With **Absolute tolerance** off, each radius is multiplied by the tolerance instead. Bonds already there stay; only new ones are added. The bonds were verified on a test molecule with each tolerance setting.

Large structures (crystals, MD snapshots) draw fastest as **Points** with **spheres**. The Molecule representation builds geometry for every atom, which is fine up to a few hundred thousand atoms.

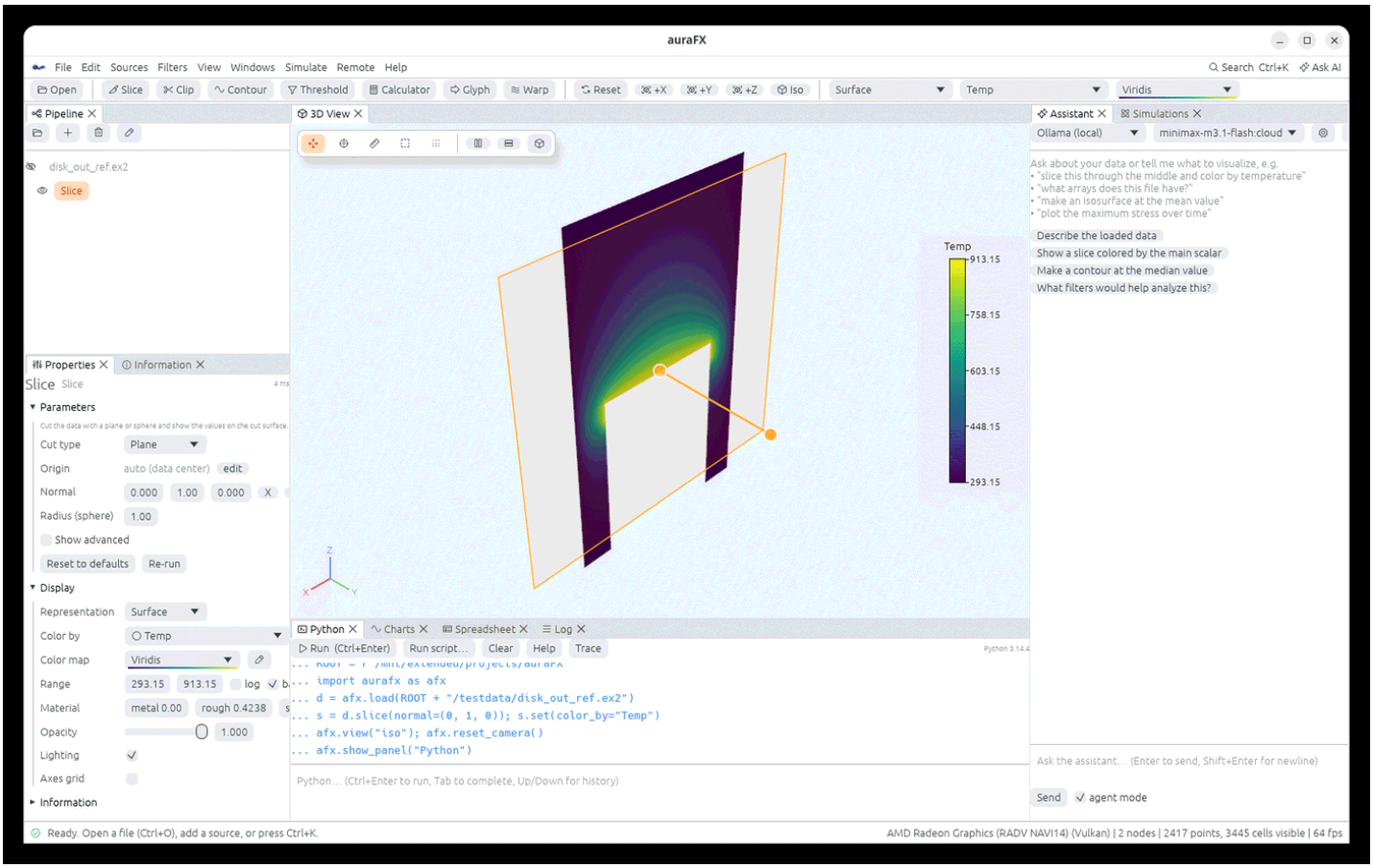
Curved cells



Quadratic cells and Lagrange or Bezier cells of any order (VTK types 68–80, with rational weights and per-cell degrees) are drawn curved. **Curved cells › subdivision** in Properties sets how finely: 0 draws straight edges between corners, 1 passes through the nodes, each further level halves the pieces. Slice, Clip, Contour, Probe and Integrate Variables follow the curved shape; the **Tessellate** filter turns curved cells into straight ones for filters that need them. The Information panel lists the cell types.

```
a.set(nonlinear_subdivision=3)
t = a.tessellate(level=2)
```

Python



auraFX embeds Python with NumPy. The **Python** panel is a console: type a line and press **Ctrl+Enter** (or **Run**), use Tab to complete, Up/Down for history, **Run script…** to run a file, **Clear** to empty the log, **Help** for the cheat sheet. **Trace** turns what you did with the mouse into the equivalent code, which you can paste into a script. Everything the UI does is reachable from the `afx` module, and `aurafx --script file.py` runs a script at startup (the app stays open afterwards).

Packages installed into the embedded interpreter with `pip` (SciPy, pandas, PyTorch…) are available in the console.

Loading and the pipeline

```
d = afx.load("run.e")                       # a Node; afx.load("slices/img_*.png")
stacks images
d = afx.load("/scratch/run.e", host="cluster")   # read on another machine (see
Data on another machine)
afx.nodes(); afx.active(); afx.get("Slice")      # find nodes
s = d.slice(normal=(0, 1, 0))               # any filter: node.<filter>(**params)
s.set(origin=(0, 1, 0), opacity=0.5)        # parameters and display settings
s.inputs(); s.activate(); s.delete()
s.update(); s.output()                      # wait; the data (an afx.Data: points,
arrays as NumPy)
s.info()                                    # arrays, ranges, bounds, counts
d.array("Temp")                             # a NumPy array; d.add_array("T_C", T -
273.15)
afx.apply_multi("append_datasets", [a, b])
afx.points_from(xyz, temperature=t)         # a node from NumPy
afx.filters(); afx.help_filter("contour")
afx.run()                                   # run whatever is pending
afx.undo(); afx.redo()
afx.save_state("work.afx"); afx.load_state("work.afx")
```

Display

```
d.set(color_by="Temp", cmap="Inferno", range=(300, 900), log_scale=True,
representation="Surface With Edges")
d.set(opacity=0.6, point_size=6, points_as_spheres=True, line_width=2,
axes_grid=True)
d.set(metallic=1, roughness=0.2, specular=0.5)           # material
d.set(representation="Volume", volume_opacity=2, volume_threshold=0.3)
d.set(representation="Surface LIC", lic_vectors="V")
d.set(nonlinear_subdivision=2)                           # curved cells
d.show(); d.hide(); d.set_name("inlet")
```

Views and camera

```
afx.view("iso"); afx.view("+x"); afx.view([1, 1, 0.5]); afx.reset_camera()
afx.split_view("right"); afx.active_view(1); d.send_to_view(1, keep=True)
afx.link_views(True); afx.match_views()
afx.layout("analyze"); afx.show_panel("Charts"); afx.hide_panel("Log");
afx.set_theme("light")
afx.annotate("Run 42", corner="upper left", size=18); afx.time_label(True);
afx.clear_annotations()
afx.lighting(shadows=True, ssao=True, edl=False, fxaa=True, tone_mapping="aces",
             environment="sky", ray_tracing=False)      # see Lighting
```

Time, charts, selection

```
afx.times(); afx.time(10)
afx.over_time(d, "EQPS", stat="max")      # returns (times, values) and plots them
afx.plot(x, y, label="T", title="Temperature"); afx.clear_plots()
afx.temporal_statistics(d)
afx.select(d, ids, "cells"); afx.selection(); afx.extract_selection();
afx.clear_selection()
afx.probe(d, (0, 0, 1))                   # values at a point
```

Pictures, animations, files

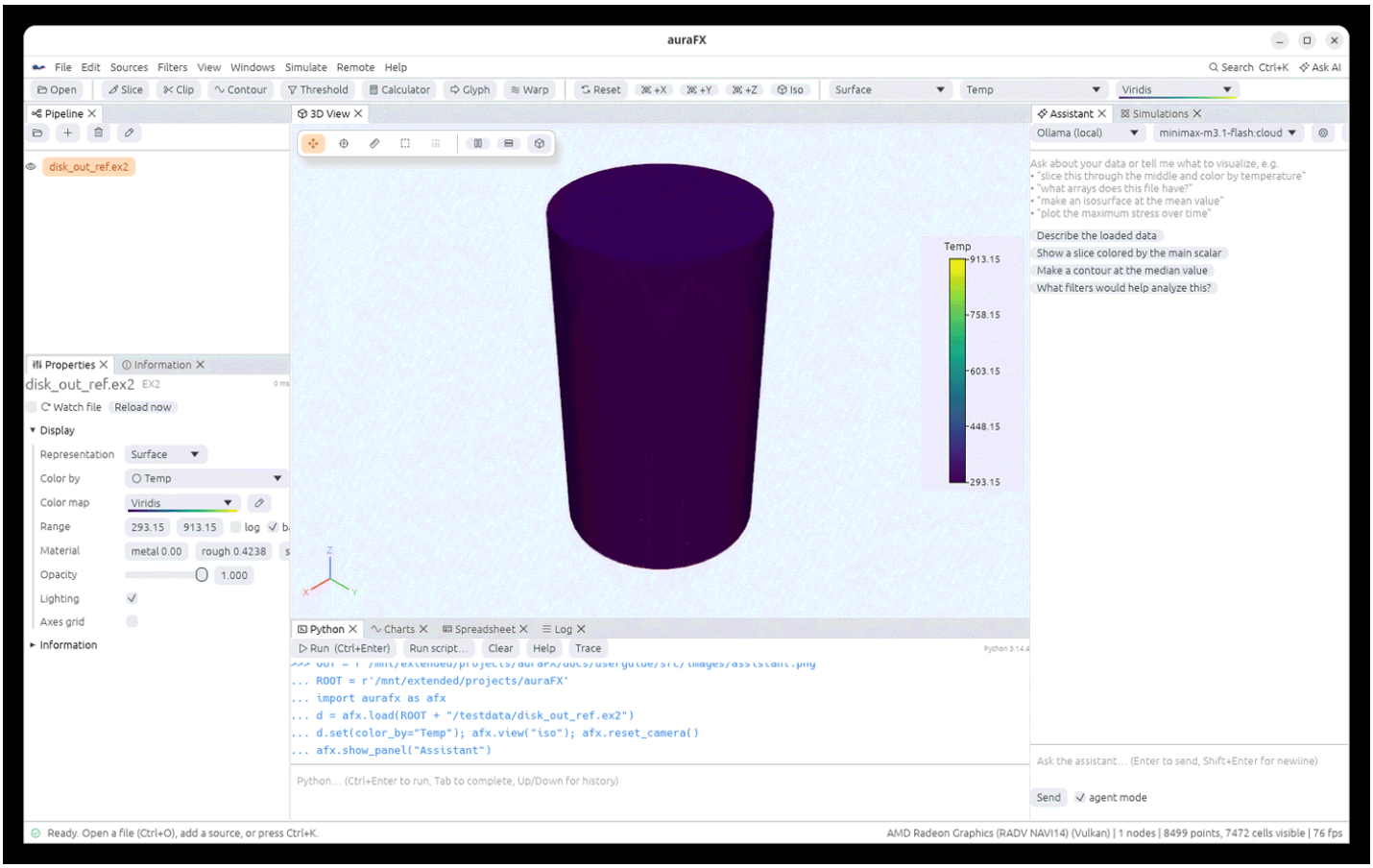
```
afx.screenshot("fig.png", width=3840, transparent=True, annotations=True)   #
waits until written
afx.screenshot("fig.svg")                 # vector picture
afx.export_scene("scene.glb")             # also .obj, .x3d, .wrl
afx.save_animation("run.mp4", fps=24); afx.save_animation("turn.gif", orbit=72)
afx.key_camera(0); afx.key_camera(1, interp="ease in-out"); d.key("opacity", 1,
0.2)
afx.animation(frames=120, time=True); afx.animation_time(0.5);
afx.clear_keyframes()
afx.save_animation("sweep.gif", cue=lambda t: c.set(values=[300 + 600 * t]),
frames=60)
d.save("out.vtu"); d.save("all.exo", all_times=True); d.save("all.vtkhdf",
all_times=True)
afx.colormaps(); afx.define_colormap(name, points);
afx.import_colormaps("maps.json")
```

Assistant, plugins, simulations, servers

```
afx.ask("plot the maximum stress over time", agent=True)
afx.plugins(); afx.load_plugin("tools.py")
afx.submit("lmp -in in.melt", host="rocm-server", workdir="~/runs", gpu="amd:0",
outputs="dump.*", auto_load=True)
afx.sweep("lmp -var T {T} -in in.melt", T=[300, 600, 900], host="cluster")
afx.jobs(); afx.job(id); afx.cancel(id); afx.gpus("rocm-server"); afx.hosts()
afx.servers(); afx.load_remote("cluster", "/scratch/run.e")
```

The **Help › Python API cheat sheet** window lists the same calls inside the app, and every function has a docstring (`help(afx.lighting)`).

AI assistant



The **Assistant** panel (right side; **Windows › Assistant**, or the **Ask AI** button) is a chat with a language model that knows the app and your data. Ask in plain words: “slice this through the middle and color by temperature”, “what arrays does this file have?”, “plot the maximum stress over time”, “why does my contour look cracked?”. The assistant answers and, when something should happen in the app, writes Python using the `afx` API.

What the assistant sees

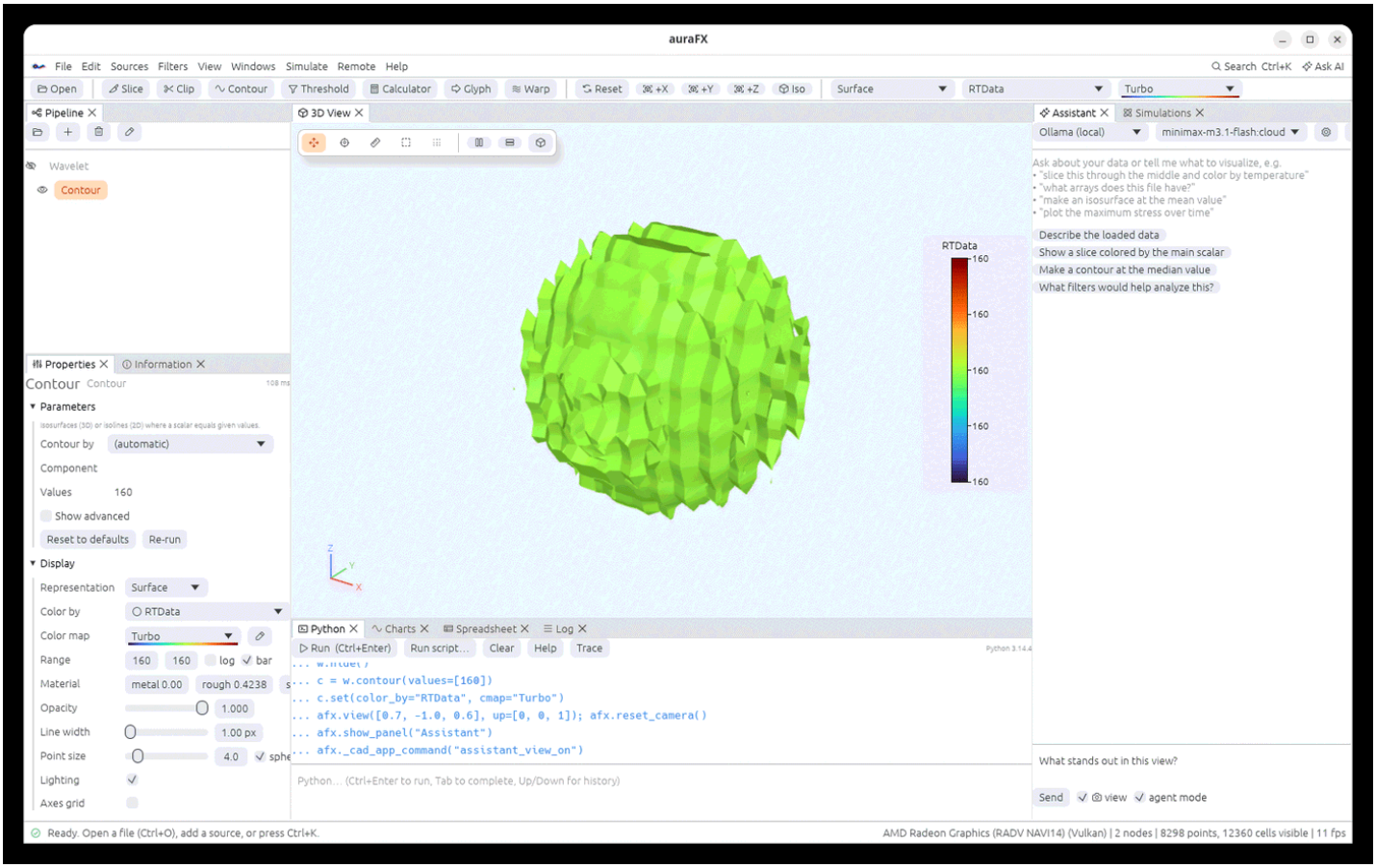
A text description of your pipeline: the objects, their filters and parameters, array names with ranges, bounds, cell types and time steps, plus the last error. It never receives the data itself. The description is rebuilt for every message.

Running the code it writes

Code appears in a block with a **Run code** button; nothing runs until you press it, unless **auto-run code** is ticked. With **agent mode** on, the assistant runs its code itself, looks at the result (values, errors, the pipeline) and keeps going until the task is done, so “find where the plate yields and plot it” becomes several steps without you in between.

```
afx.ask("make an isosurface at the mean temperature")
afx.ask("find where the plate yields and plot the max stress over time",
agent=True)
```

Showing it the view. Tick the camera box next to **Send** and a picture of the 3D view goes with the message, for models that see images (Claude, GPT-4o, and Ollama models such as llava, gemma or qwen-vl): ask “what stands out in this picture?” or “is this mesh refined enough near the hole?”. From Python: `afx.ask("describe the view", view=True)`.



Describe a filter. Filters › Describe a filter… (also in the command palette) takes a sentence about what you want to see in the selected object, such as “regions where the von Mises stress is above 300 MPa” or “the vortex cores of the flow”; the assistant adds the filters that show it (in agent mode, so it looks at the arrays and ranges first), and its steps appear in the Assistant panel.

Privacy. In the assistant's settings: **Local only** sends nothing off your network (only an Ollama server at a local or private address, and no cloud models; anything else is refused with the reason), and **Hide file paths** replaces your home folder with `~` and full paths with file names in what is sent. The data itself is never sent, only the pipeline's description (names, arrays, ranges) and the results of code the assistant runs.

A script can wait for the assistant and read what it said: `afx.assistant_status()` returns `{"busy", "agent", "messages", "last", "error"}` (an answer is arriving, agent mode is iterating, the number of messages, the last answer, the last error).

Modelling parts

The assistant knows the CAD modeler (see CAD modeling). Ask it to **start** a part ("a base plate 60 by 40 by 8 mm; name the dimensions width, depth and thickness"), to **build** it ("add a wall along the back edge, two 6 mm holes, round the corner"), and to **change** it ("make the plate 80 mm wide and the holes 8 mm"). It names dimensions, uses the part's faces by name, checks the volume against numbers it worked out, and can open the part in the CAD environment, edit the working copy, and apply it; when applying would break a node below the part, it is asked for the fixes. It also places parts side by side (assemblies) and uses the face operations (patterns on a face, draft, mirror).

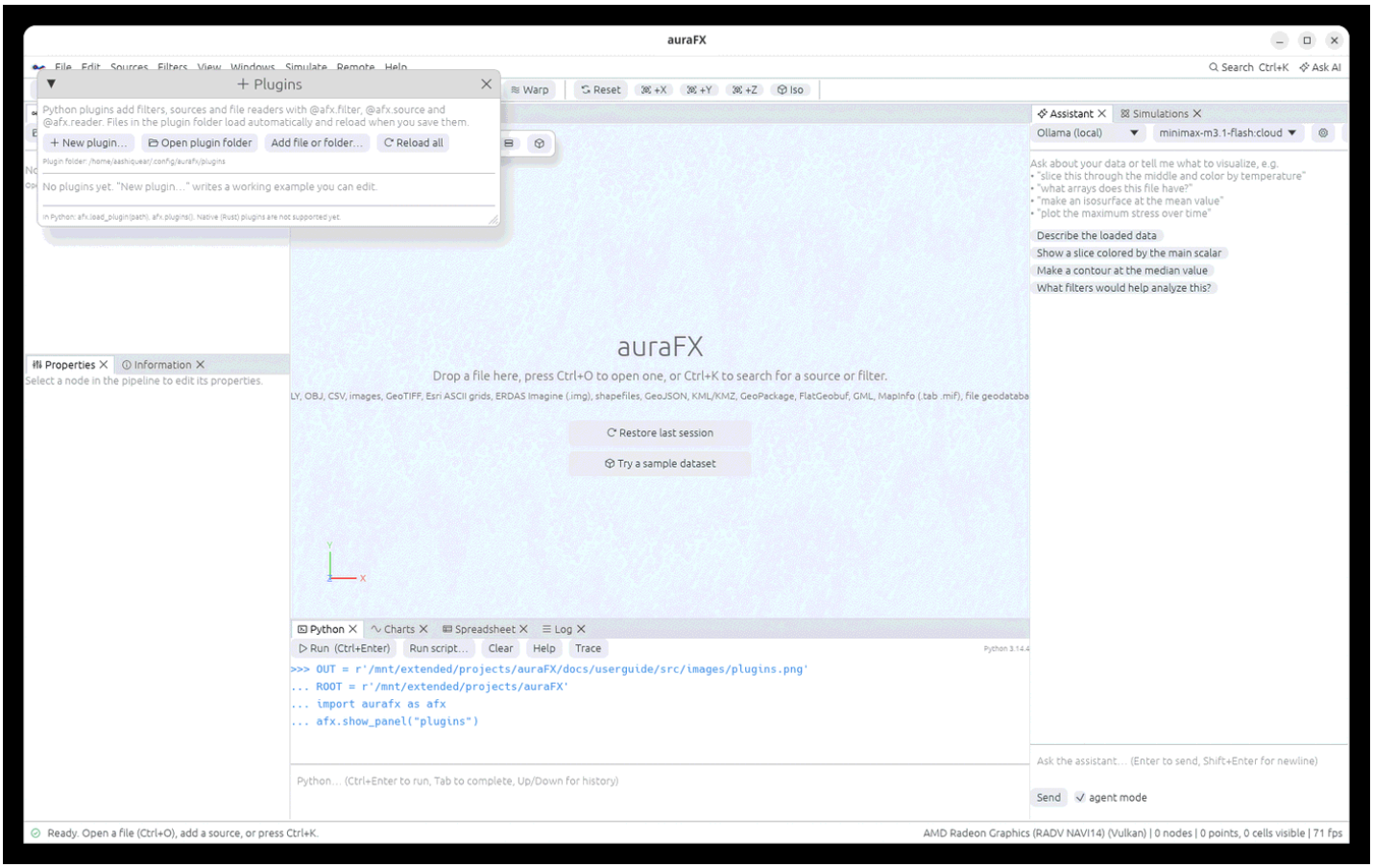
Providers

Choose the provider and model in the panel's drop-downs or in **Settings**:

Provider	Setup
Ollama (local or cloud models)	the server URL from `OLLAMA_HOST` or Settings; the model list comes from the server
Anthropic	set `ANTHROPIC_API_KEY` or paste a key in Settings
OpenAI-compatible (OpenAI, OpenRouter, Groq, vLLM, LM Studio…)	the base URL and key in Settings

Answers stream as they are produced. The suggestion buttons under the welcome text are good first questions for a new file.

Plugins



A plugin is a Python file that adds filters, sources and file readers. They appear in the Filters and Sources menus, in the command palette, in Python (`node.my_filter()` , `afx.my_source()`), in the Open dialog (readers) and to the AI assistant, like the built-in ones.

Where plugins live

- The plugin folder: **Edit › Plugins… › Open plugin folder** (`~/.config/aurafx/plugins` on Linux, the auraFX config folder on macOS and Windows). Every `.py` file in it is loaded.
- Folders listed in the `AURAFX_PLUGIN_PATH` environment variable.
- Files or folders added with **Add file or folder…** in the Plugins window.
- From a script: `afx.load_plugin("path/to/tools.py")` .

Plugins reload when you save them, and the objects that use their filters re-run. Files whose name starts with `_` are skipped (use them for shared helpers). A plugin that fails to load shows its error in the Plugins window and the Log; switch a plugin off with its checkbox. **New plugin…** writes a working example and opens it in your editor.

A filter

```
import aurafx as afx
import numpy as np

@afx.filter(category="Flow")
def speed(data, vectors=afx.ArrayParam(vector=True), name="speed"):
    """Magnitude of a vector array."""
    v = data.point_data[vectors]
    data.point_data[name] = np.linalg.norm(v, axis=1)
    return data
```

Keyword arguments become parameters in Properties; their type comes from the default: `float`, `int`, `bool`, `str`, a 3-tuple (a vector), `afx.Choice([...])`, `afx.ArrayParam(vector=False, cells=False)` (an array of the input) and `afx.Code("...")` (a text box). The docstring's first line is the description. Other decorator options: `label`, `description`, `keywords`, `id`, and `inputs=2` or `inputs=(2, None)` for filters with several inputs (the function receives a list).

`data` is an `afx.Data`, a copy of the input: `data.points` (N×3), `data.point_data["T"]`, `data.cell_data[...]`, `data.field_data[...]` (assign to add, `del` to remove), `data.n_points`, `data.n_cells`, `data.bounds`, `data.type`, `data.cells` (offsets, connectivity, VTK types), `data.blocks()`, `data.copy()`. The spellings `data.PointData.append(a, "name")` and `data.CellData` work too. Build new data with `afx.make_points`, `afx.make_polydata`, `afx.make_image`, `afx.make_grid` and `afx.make_multiblock`.

Sources and readers

```
@afx.source(category="Shapes")
def helix(turns=4.0, points=400):
    t = np.linspace(0, turns * 2 * np.pi, int(points))
    return afx.make_polydata(np.c_[np.cos(t), np.sin(t), t / 10], lines=
[list(range(len(t)))])

@afx.reader(extensions=["xyzt"], label="XYZ + temperature")
def read_xyzt(path):
    a = np.loadtxt(path, ndmin=2)
    d = afx.make_points(a[:, :3])
    d.point_data["T"] = a[:, 3]
    return d
```

Readers take precedence over the built-in readers for their extensions.

Rules

- Plugin functions run on a worker thread while the pipeline runs: do not call pipeline functions (`afx.load`, `node.slice()`, `show`…) inside them.
- Ids must be unique and cannot replace built-in filters.
- For one-off code, the **Programmable Filter**, **Programmable Source** and **Python Calculator** run Python stored in the pipeline (and in state files).
- Native (Rust) plugins are not supported yet.

Tutorials and examples

The `examples/` folder of the repository holds small, complete workflows. The simulations are plain Python with NumPy that write a VTK file every few steps; run them from the Simulations panel (template **Python pipeline**, see Running simulations) or with `afx.submit`, and auraFX opens the files when the run ends. The tutorials are auraFX scripts: `aurafx --script <file>`, or paste them into the Python console.

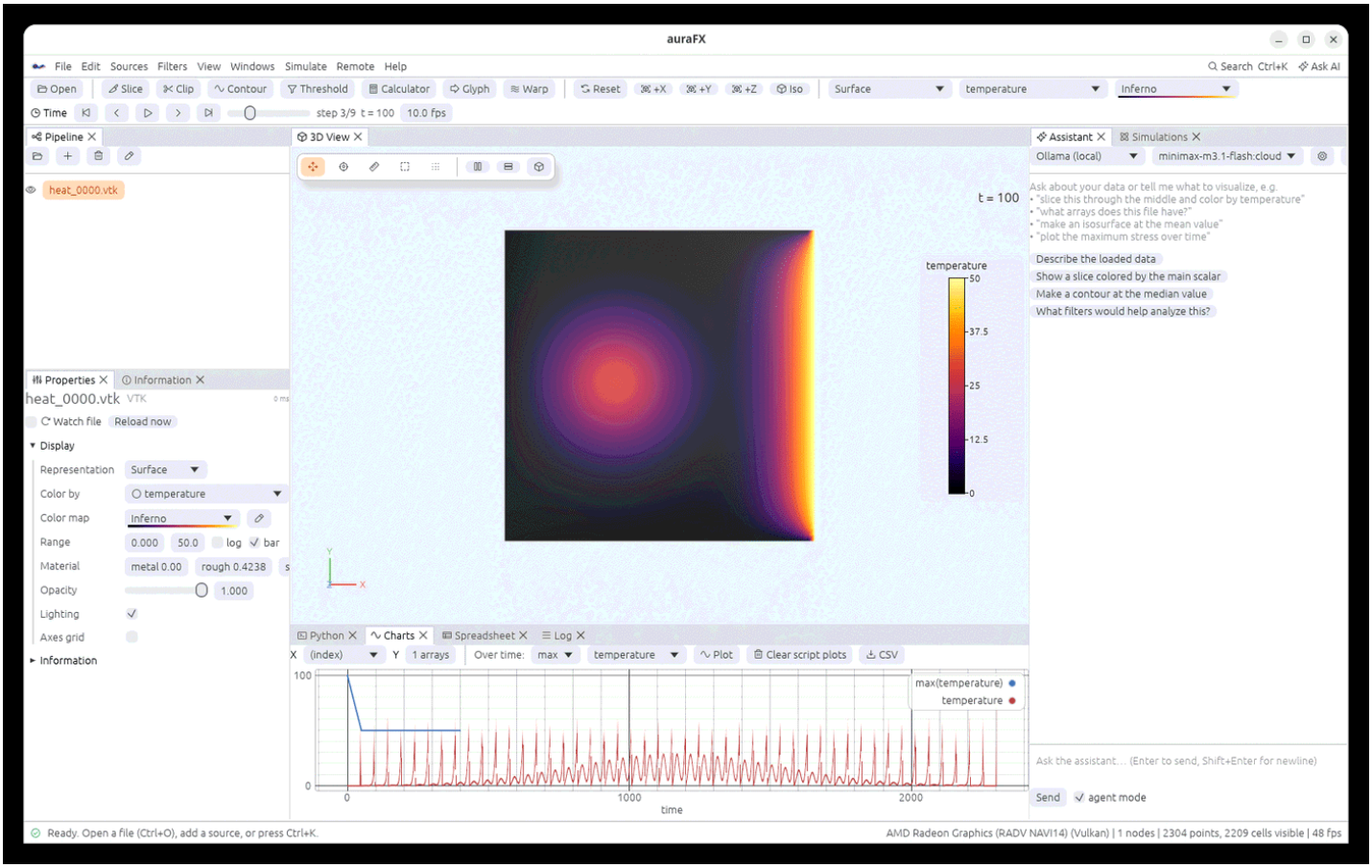
Folder	What it shows
`examples/heat2d/`	2-D heat diffusion: the simplest Python-pipeline run; five output files are included, so it opens without running anything
`examples/phase_field/`	spinodal decomposition (the Cahn-Hilliard equation, a spectral solver in NumPy): a phase-field time series in about ten seconds
`examples/grain_growth/`	grain growth with the Potts model on a 2-D lattice: grain ids over Monte Carlo sweeps, for the grain filters
`examples/polycrystal/`	geometry → hexahedral mesh → Voronoi grains → quality table → Exodus and `.tex` files → a crystal-plasticity case
`examples/ebsd/`	an EBSD volume → grains → sizes, shapes, average orientations, IPF colors, pole figures, a CSV table

A simulation that writes a time series

Open the Simulations panel, choose **Python pipeline**, set the working directory to `examples/phase_field` and the input to `cahn_hilliard.py`, set **Result files** to `ch_*.vtk` and press **Run**. The log shows each step; when the run ends the series opens, colored by the concentration. Press play in the time toolbar, or

```
afx.submit("python3 -u cahn_hilliard.py", workdir="examples/phase_field",
outputs="ch_*.vtk", auto_load=True)
afx.save_animation("spinodal.mp4", fps=12)
```

The grain-growth example works the same way (`potts.py`, result files `grains_*.vtk`); color it by `grain_id` with the Turbo color map. **Segment Features** (method Scalar, tolerance 0) on a step gives feature ids the grain filters measure (sizes, neighbors, shapes).



From a geometry to a solver case

`examples/polycrystal/polycrystal_case.py` builds a plate with a hole, meshes it with hexahedra, assigns 60 grains with one element block each, writes the mesh, the orientations and the input files of a crystal-plasticity case into `polycrystal_out/`. The chapters Geometry and meshing and Running simulations explain every step; with a solver installed, `afx.run_case` runs the case and opens the result.

An EBSD volume to statistics

`examples/ebsd/ebsd_analysis.py` segments the AM Bench window shipped in `testdata/dream3d`, cleans the grains, measures them, colors them by the inverse pole figure key, draws the pole figures in a second view and writes the grain table as `grains.csv` (Grains and crystals).

File formats

Every reader was checked against a reference implementation: the same points, cells, bounds and value ranges on the same files. The comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Readers

Format	Extensions	Notes
Exodus II	`.e .exo .ex2 .g .gen`	NetCDF-3 and NetCDF-4; blocks, side and node sets, nodal/element/global variables, time steps, displacements
VTK legacy	`.vtk`	ASCII and binary, all dataset types
VTK XML	`.vtu .vtp .vti .vtr .vts .vtt`, parallel `.pvtu …`, `.vtm`, `.pvd`	inline, appended, base64; zlib and LZ4; curved (Lagrange/Bezier) cells
AMR (adaptive mesh refinement)	VTK `.vthb .vth`, AMReX/BoxLib plotfiles (the folder, or its `Header`), FLASH and Chombo HDF5 files, Enzo (`.hierarchy`)	levels of boxes with cell data; coarse cells under finer boxes are left out; see below
Hyper tree grids	VTK `.htg` (XML version 2)	trees of refined cells; drawn as their leaves; see below
Cell grids (discontinuous Galerkin fields)	VTK `.dg`	cells whose fields are polynomials; drawn with the fields at the corners; see below
VTKHDF	`.vtkhdf .hdf`	image data, unstructured grids, poly data, partitioned collections with their assembly, time steps; chunked and gzip storage; no HDF5 library needed
OpenFOAM	`.foam`	ASCII/binary/gzip, decomposed and reconstructed cases, time directories, boundary patches

Format	Extensions	Notes
EnSight Gold	`.case .encas`	ASCII and C binary, all element types, structured blocks, time sets, wildcard file names
CGNS	`.cgns`	HDF5 and ADF storage; structured and unstructured zones, MIXED and NGON/NFACE sections, solutions, time steps
Xdmf 2/3	`.xmf .xdmf`	inline, HDF5 and raw binary data; collections; `xi:include` and XPath references (FEniCS/Dolfinx); HyperSlab and JOIN items
NetCDF (CF)	`.nc .nc4 .cdf`	classic and NetCDF-4; lon/lat grids on a sphere with cell bounds, curvilinear grids with 2-D longitude/latitude (POP, CICE, ROMS), rectilinear otherwise; time steps; packing; fill values become NaN (grey); see below
CAM-SE	`.nc` + connectivity file (found from the contents)	CESM atmosphere on its cubed sphere: lon/lat map or sphere, surface, midpoint or interface variables, one level or all; see below
MPAS	`.nc` (found from the file's contents)	MPAS ocean, atmosphere and land-ice output: cell centers joined into triangles, any vertical level or all of them, sphere or lon/lat map, time steps; see below
Tecplot	`.dat .plt .tec .tp`	ASCII (ordered and FE zones, BLOCK/POINT, cell-centered variables) and binary `.plt`; `.szplt` is not read
PLOT3D	`.xyz .x .p3d .bin` + `.q`	layout detected from the file (Fortran/C binary, byte order, precision, multi-grid, 2-D/3-D, IBLANK), ASCII
LAMMPS	`.dump .lammpstrj` `.atom` (+ `.gz`), `.data` `.lmp`	multi-frame dumps with any columns, triclinic boxes; data files with bonds
XYZ / extended XYZ	`.xyz .extxyz`	multi-frame; properties and lattice from the comment line

Format	Extensions	Notes
PDB	`.pdb` `.ent`	all atoms (hydrogens kept), MODELs as time steps, CONECT and distance bonds, residues and chains
Gaussian cube	`.cube`	the density as a volume in Å, plus the atoms with bonds
Chemical Markup Language	`.cml`	the atoms and the file's bonds with their orders (`bond_order`)
VASP	`POSCAR` , `CONTCAR` , `CHGCAR` , `LOCPOT` , `ELFCAR` , `PARCHG` (by name), `.vasp`	atoms with bonds and the lattice; volumes on the cell's axes (charge per Å³)
Images	`.png` `.jpg` `.bmp` `.tif`	as image data; multi-page TIFFs and `img_*.png` wildcards as volumes
GIS rasters	GeoTIFF `.tif` , Esri ASCII grid `.asc`	at their map coordinates, a cell per pixel, every band; no-data values become NaN (grey); see below
GIS vectors	Shapefile `.shp` , GeoJSON `.geojson` `.json` , KML `.kml` , KMZ `.kmz` , GeoPackage `.gpkg` , FlatGeobuf `.fgb` , GML `.gml`	points, lines and polygon outlines (or filled areas) with their attributes as cell data, a block per layer; see below
EBSD maps and microstructures	TSL/EDAX `.ang` , Oxford `.ctf` , `.h5ebsd` stacks, EDAX and Bruker `.h5` , `.dream3d`	crystal orientations on a square grid with phases and quality columns; voxel volumes and meshes with feature and phase arrays; see Grains and crystals
SPPARKS sites	`.spparks` , `.sites` ; lattice text dumps (`id i1 x y z`)	a site file without a `Sites` section (a lattice filling its box) and a dump of every site of a lattice open as a volume with `FeatureIds` (from `i1`); a site file with coordinates opens as points
Surfaces	`.stl` `.ply` `.obj` , `.off`	OFF also with point colors (`COFF`) and face colors
Scenes	glTF 2.0 `.gltf` `.glb` , VRML 2.0 `.wrl`	triangles, lines and points of every placed mesh (moved by its nodes or transforms), normals, vertex colors; a block per mesh; see below

Format	Extensions	Notes
Medical and microscopy volumes	NRRD `.nrrd` `.nhdr`, MetaImage `.mhd` `.mha`, MRC `.mrc` `.map` `.rec`, DICOM `.dcm` (or the folder of a series), OME-TIFF `.ome.tif`	image data at the file's spacing, origin and orientation; see below
Point clouds (lidar)	LAS `.las`, LAZ `.laz`	LAS 1.0 to 1.4, point formats 0 to 10, with intensity, returns, classification, scan angle, GPS time and colors; see below
Particles on HDF5	H5Part `.h5part`, openPMD `.h5`	time steps of particles; openPMD meshes as image data and a block per particle species; see below
Nek5000	`.nek5000` (or a data file `name0.f00001`)	spectral elements as hexahedra (quadrilaterals in 2-D), velocity, pressure, temperature, passive scalars, time steps
LS-DYNA	`d3plot` and its family files	solids, shells, beams; states with moving nodes, velocities, accelerations, stresses, plastic strain, beam resultants; see below
Nastran bulk data	`.bdf` `.nas` `.bulk`, `.dat` with GRID cards	small, large and free fields; coordinate systems; solid, shell, bar and point elements with their properties as blocks
Fluent	`.msh` `.cas`	ASCII and binary sections; cells rebuilt from their faces; cell zones as blocks, boundary zones as side sets
CSV / TSV	`.csv` `.tsv` `.txt`	delimiter and header detection; tables with x, y, z columns become points

Numbered files (`run_0001.vtu`, `frame_0001.png`, `scan_0001.las` …) open as a series when one of them is opened; a wildcard name (`img_*.png`) stacks pictures into one volume instead.

Medical and microscopy volumes

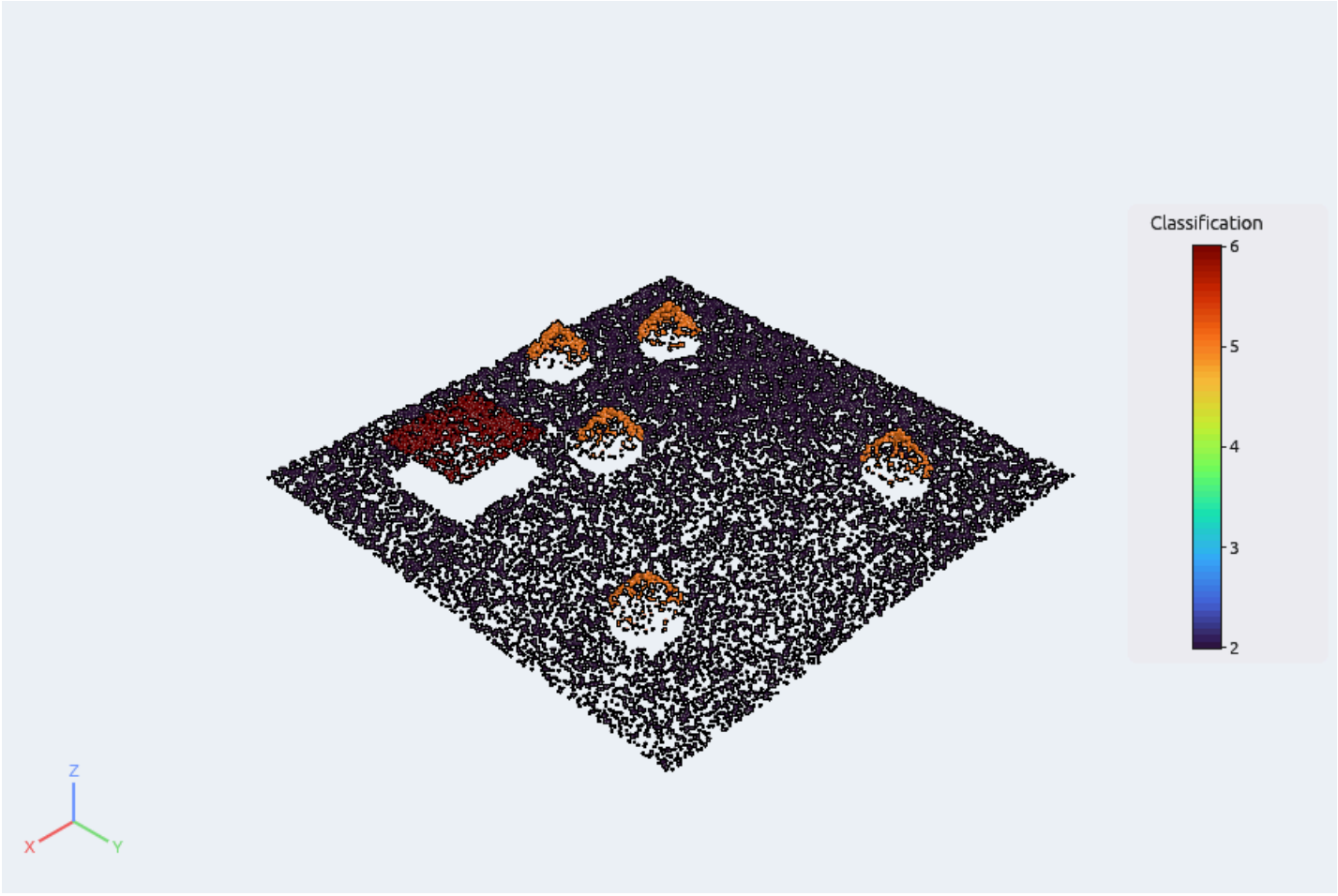
Format	Notes
NRRD (`.nrrd` , detached `.nhdr` + data)	raw, gzip and ASCII encodings, any byte order; spacing from `spacings` or `space directions` (also the orientation), origin from `space origin` ; a first axis of kind vector, RGB-color… gives components. The array is `NRRDImage`
MetaImage (`.mha` , `.mhd` + `.raw` / `.zraw`)	every element type, zlib-compressed data, `Offset` , `ElementSpacing` , `TransformMatrix` , channels. The array is `MetaImage`
MRC / CCP4 (`.mrc` , `.map` , `.rec` , `.st`)	cryo-EM maps and tomograms: modes 0 (int8), 1 (int16), 2 (float32), 6 (uint16), 12 (float16); the voxel size from the cell and sample counts; the origin; the axis order of the file (columns, rows, sections) put back to x, y, z
DICOM (`.dcm` , extension-less files, or a folder)	a file opens its whole series (the files of its folder with the same series id), ordered along the slice normal; spacing from Pixel Spacing and the distance between slices, position and orientation from the patient coordinates; values rescaled by slope and intercept (Hounsfield units for CT). A folder opens its largest series. Compressed (JPEG) files are not read: save them uncompressed
OME-TIFF (`.ome.tif`)	the planes in the order the OME-XML gives (Z, channels, times), the physical pixel sizes, one array per channel named after it, times as time steps

Axes that a file swaps or reverses (rows of a DICOM picture running down, a NRRD space direction along −x) are laid out along +x, +y, +z, so the points sit where the file puts them. An oblique orientation (axes not along x, y and z) is kept as the image's orientation, but auraFX places the points along the axes: measure in the grid's own frame for such files.

```
ct = afx.load("scans/patient7")            # a folder of DICOM slices
mri = afx.load("brain.nrrd")
cells = afx.load("cells.ome.tif")          # DAPI, GFP... arrays; time steps
```

Point clouds

LAS and **LAZ** files (lidar) open as points with the arrays `Intensity`, `ReturnNumber`, `NumberOfReturns`, `Classification` (2 ground, 5 high vegetation, 6 building…), `ScanAngle` (degrees), `UserData`, `PointSourceId`, `GpsTime` and `Color` (drawn as its own colors), at real coordinates (scaled and offset). LAZ (compressed LAS) is read too. Point Gaussian or Eye-Dome Lighting (View › Lighting) suit large clouds; Threshold on `Classification` keeps one class.



Scenes

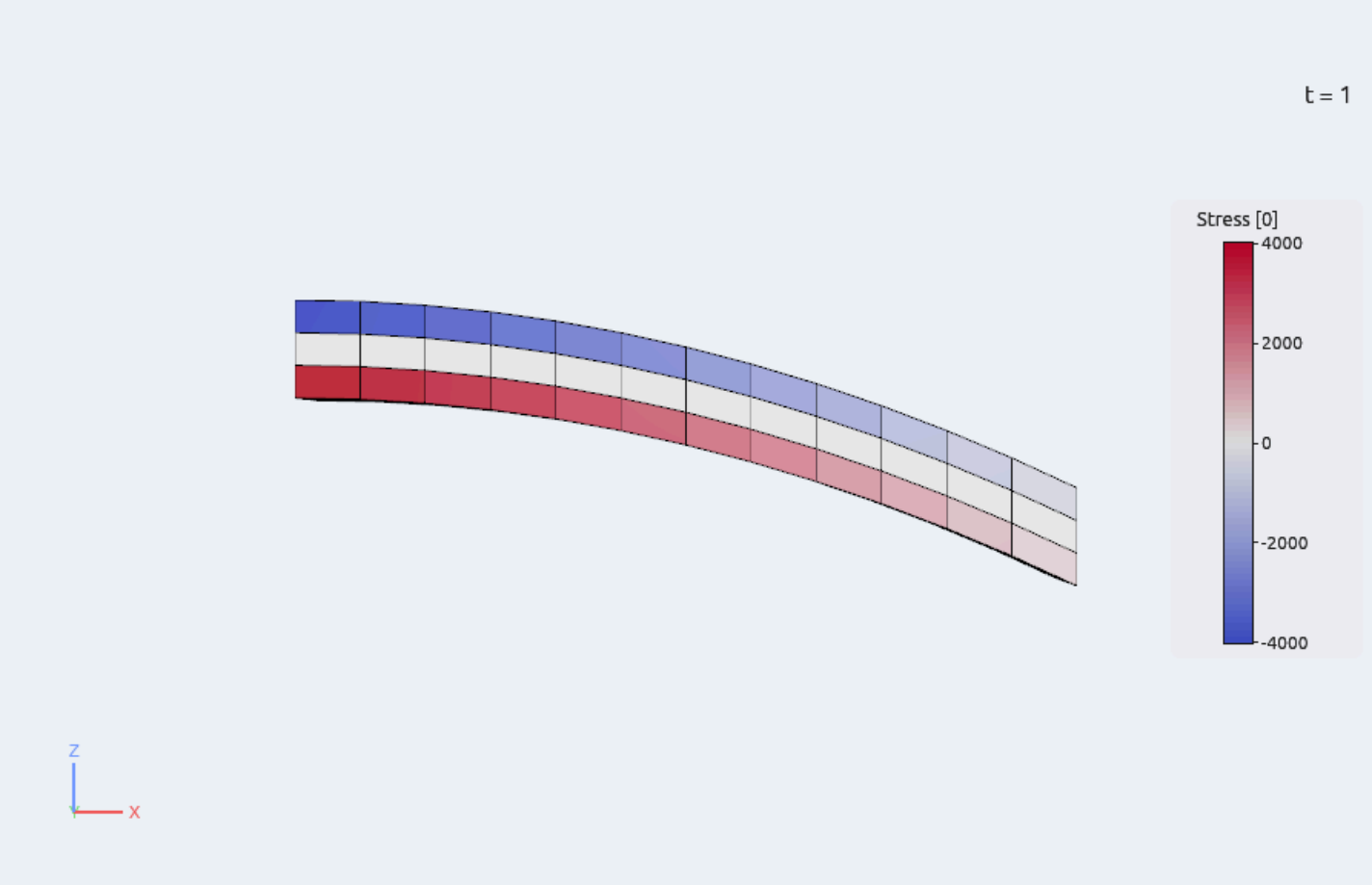
glTF 2.0 (`.gltf` with embedded or separate buffers, `.glb`) and VRML 2.0 (`.wrl` , also gzipped) files from modelling programs and web viewers open as surfaces: every mesh of the scene, moved by its node's (or transform's) translation, rotation and scale, with its normals, vertex colors (`Colors`) and texture coordinates; a block per mesh, named after its node. VRML IndexedFaceSet, IndexedLineSet and PointSet are read, with per-vertex or per-face colors; PROTO definitions are not.

Particles on HDF5

- **H5Part** files (`Step#0` , `Step#1` … groups) give one time step per group: points at `x` , `y` , `z` with every other dataset as a point array, times from each step's `TimeValue` .
- **openPMD** series (HDF5; one file holding all iterations, or one file per iteration, which opens with its siblings) give one time step per iteration at its time in seconds: the meshes as image data (grid spacing, offset and axis labels in metres, values times `unitSI`) and a block per particle species (positions plus position offsets, every record as a point array, constant records filled in).

LS-DYNA results

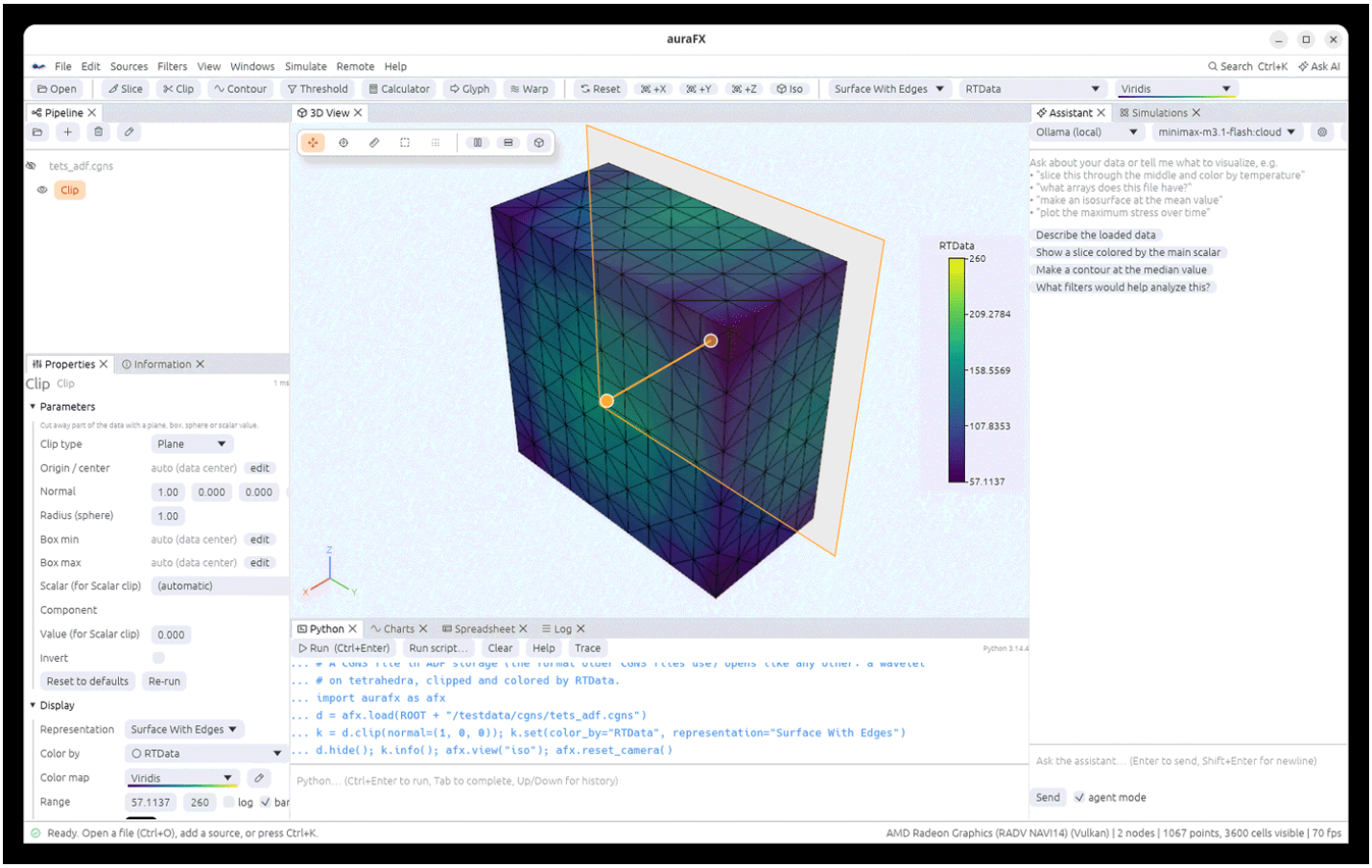
Open `d3plot` : its family files (`d3plot01` , `d3plot02` ...) are read with it. The mesh holds the solids (tetrahedra, pyramids and wedges stored as hexahedra with repeated nodes are made the cells they are), thick shells, beams and shells, with `Material` , `ElementId` (the user numbers when the file has them) and `ElementKind` (1 solid, 2 thick shell, 3 beam, 4 shell). Each state moves the nodes to their current positions; `Displacement` is the move, with `Velocity` , `Acceleration` and `Temperature` when written, `Stress` (6 components) and `EffectivePlasticStrain` of solids and shells (the middle integration point), `BeamResultants` , and the global energies as field data. Fluid, adaptive and 8-node shell models are not read.



Solver meshes

Nastran bulk data (`GRID` , `CTETRA` , `CPENTA` , `CHEXA` , `CPYRAM` with their mid-side nodes, `CTRIA3` , `CQUAD4` , `CBAR` / `CBEAM` / `CROD` , `CONM2` ; small, large and free fields with continuations; `CORD2R` , `CORD2C` and `CORD2S` coordinate systems) and Fluent meshes and cases (`.msh` , `.cas` ; ASCII and binary node, cell and face sections; the cells are rebuilt from their faces, polyhedra kept as polyhedra) open as meshes with the auraFX block and set arrays: the Nastran property or the Fluent cell zone is the `ElementBlock` , Fluent boundary zones are side sets named after the zones. Both are also written (see Writers).

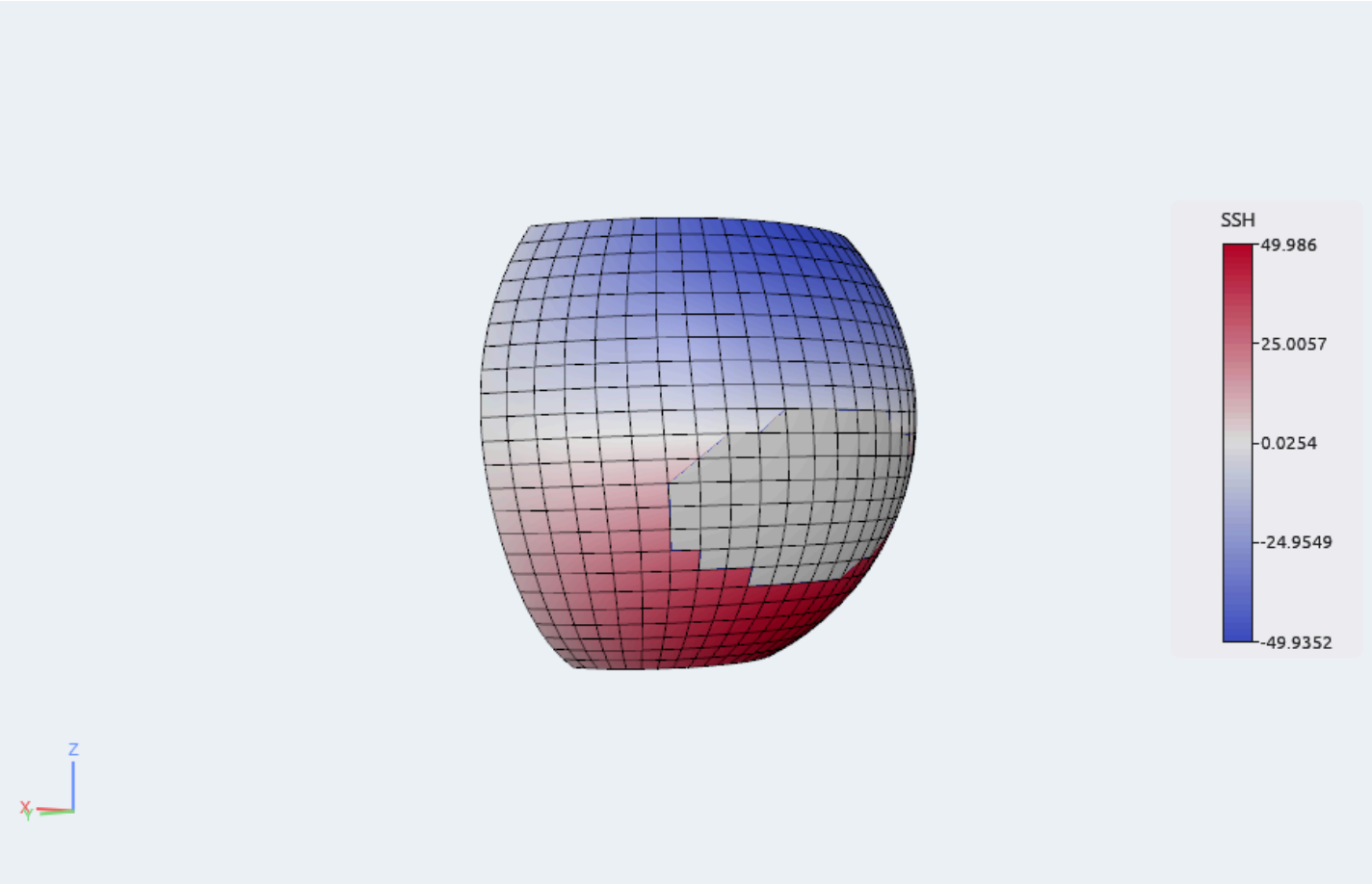
CGNS files come in two storage formats with the same `.cgns` extension: HDF5, the default since CGNS 3.4, and ADF, which older solvers and older CGNS libraries write. auraFX looks at the file and reads either one the same way, with **File › Open** or `afx.load("wing.cgns")`. Both the 64-bit ADF layout and the older 32-bit one are read, in either byte order. Links inside the file and links to other ADF files are followed. A link to an HDF5 file from an ADF file is not.



NetCDF climate and ocean files

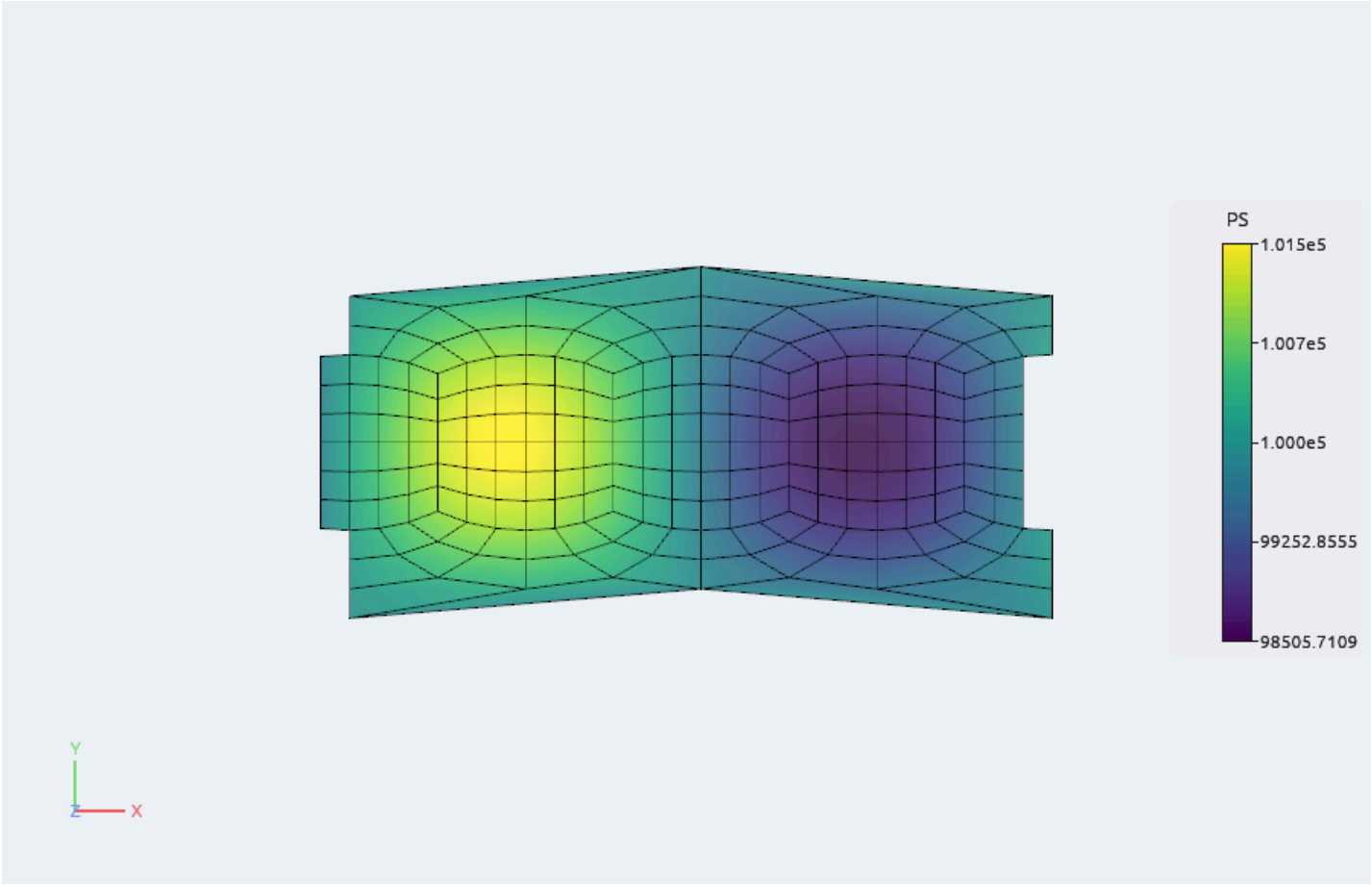
Files that follow the CF conventions are placed by their coordinates:

- Longitude and latitude axes put the grid on a sphere of radius 1. A third (vertical) axis becomes the radius. Depths marked `positive="down"` are turned around, and all levels are shifted so that none is below zero, so deeper levels lie further in (the deepest at the center).
- Ocean and sea-ice models such as POP (CESM's ocean), CICE and ROMS use index dimensions (`nlat`, `nlon`) and give each point's longitude and latitude in 2-D variables (`TLONG`, `TLAT`) named in the `coordinates` attribute. These become curvilinear grids on the sphere. The coordinate variables themselves are not shown as data.
- When the variables live on different grids (say `TEMP` on depth, latitude and longitude, and `SSH` on the surface only), **Reader options › Variables on** picks the grid. Python: `ds.set(variables="(nlat, nlon)")`. The grid with the most dimensions is read first.



CAM-SE output (the spectral-element core of CESM's atmosphere model) comes as two files: the data, with columns `ncol` at `lat` / `lon`, and a connectivity file whose `element_corners` joins the columns into quads. Open the data file. auraFX finds the connectivity file in the same folder, or you can name it under **Reader options › Connectivity file** (Python: `ds.set(connectivity="ne30np4_connectivity.nc")`). It draws the grid on a flat lon/lat map:

- Quads that cross the 0/360° seam are moved to the side where they stick out least, on copies of their points, so the map runs a little past 0 and 360.
- **Variables** picks the surface variables `[time, ncol]`, or those on midpoint (`lev`) or interface (`ilev`) levels. With **All levels** on, the default, each quad becomes a column of hexahedra, the top level highest, and the `lev` (or `ilev`) array holds each point's level. With it off, **Level** chooses one level.
- **Layout › Sphere** puts the columns on a sphere instead. There is no seam on a sphere.
- A single level lies at height 0.



MPAS output (from the ocean, atmosphere or land-ice models of the Model for Prediction Across Scales) is NetCDF too. When a `.nc` file has MPAS's mesh variables (`xCell` , `cellsOnVertex`), auraFX reads it as MPAS instead of as a CF grid:

- Each MPAS cell center becomes a point, and the cells around each MPAS vertex are joined into a triangle (or a quad, on quad meshes). On a sphere, the points lie at the planet's radius in meters.
- Variables on cells (`temperature` , `ssh` …) become point data, and variables on vertices (`relativeVorticity` …) become cell data. Time steps are numbered 0, 1, 2…, and the date of each step (`xtime`) is kept with the data.
- For other dimensions than time and position, the first index is used, except the vertical levels, which you choose with the reader options below.

Reader options

Some readers have settings for the file they read. They appear under **Properties › Reader options** when the file is selected, and from Python they are set like any other property, with `node.set(...)` . Changing one reads the file again; the choice is saved with the state file. **Reset to defaults** goes back to the file as first opened.

An MPAS file offers:

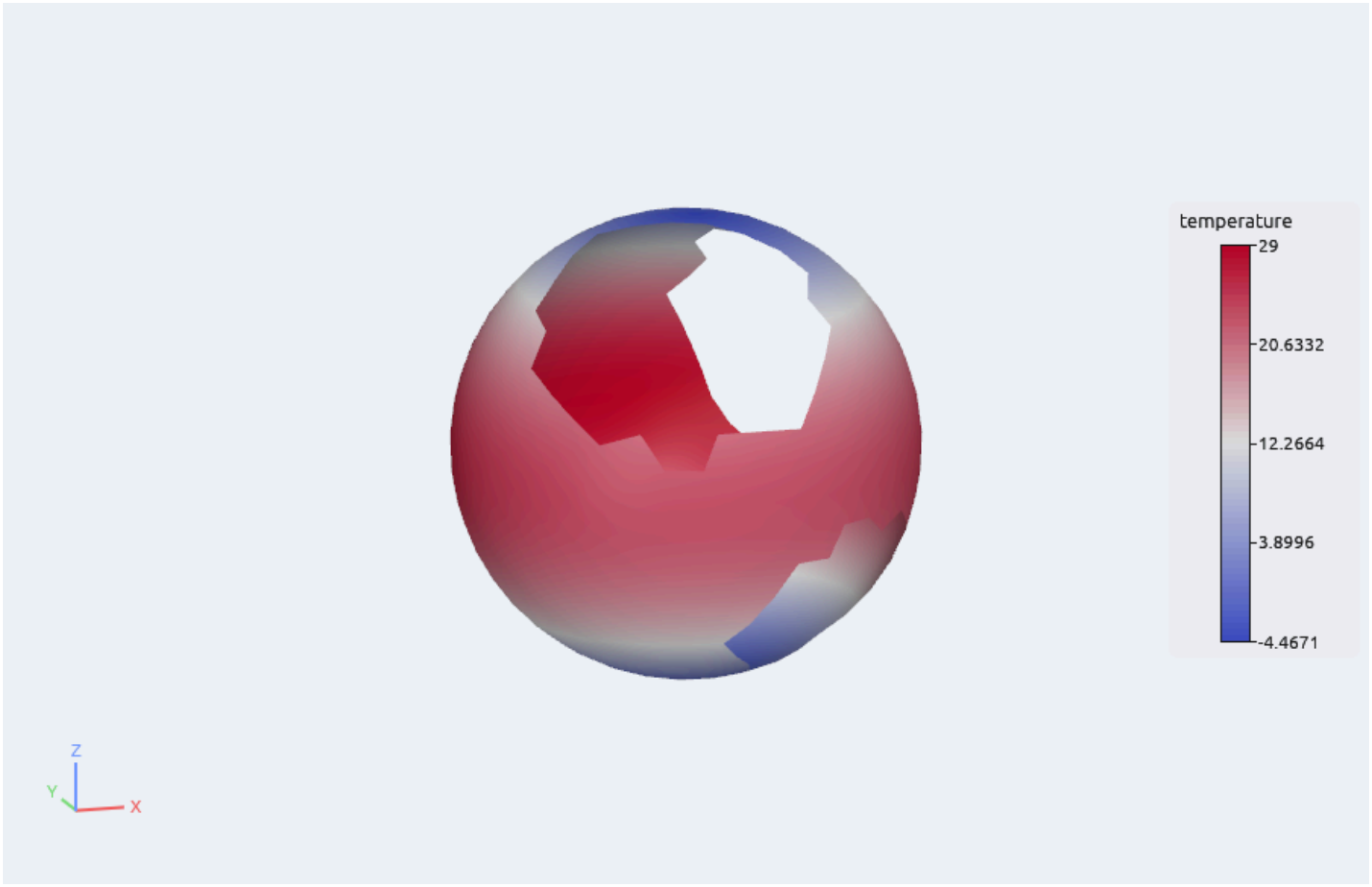
Option	Python	What it does
Vertical level	`vertical_level=2`	which level of 3-D variables is shown (0 is the top of the ocean, the bottom of the atmosphere)
All levels (multilayer)	`multilayer=True`	stacks every level: each triangle becomes a column of wedges, and points carry each level's values
Layer thickness	`layer_thickness=10000`	the distance between levels in the multilayer view, in meters on the sphere and degrees on the map; 0 picks one so that the whole stack is easy to see
Layers go up (atmosphere)	`atmosphere=True`	stacks levels outwards (or upwards on the map) instead of downwards
Layout	`layout="Lat/Lon map"`	`"Sphere"` (default) or a flat map with longitude and latitude in degrees
Map center longitude	`center_lon=0`	the longitude in the middle of the map (180 by default)

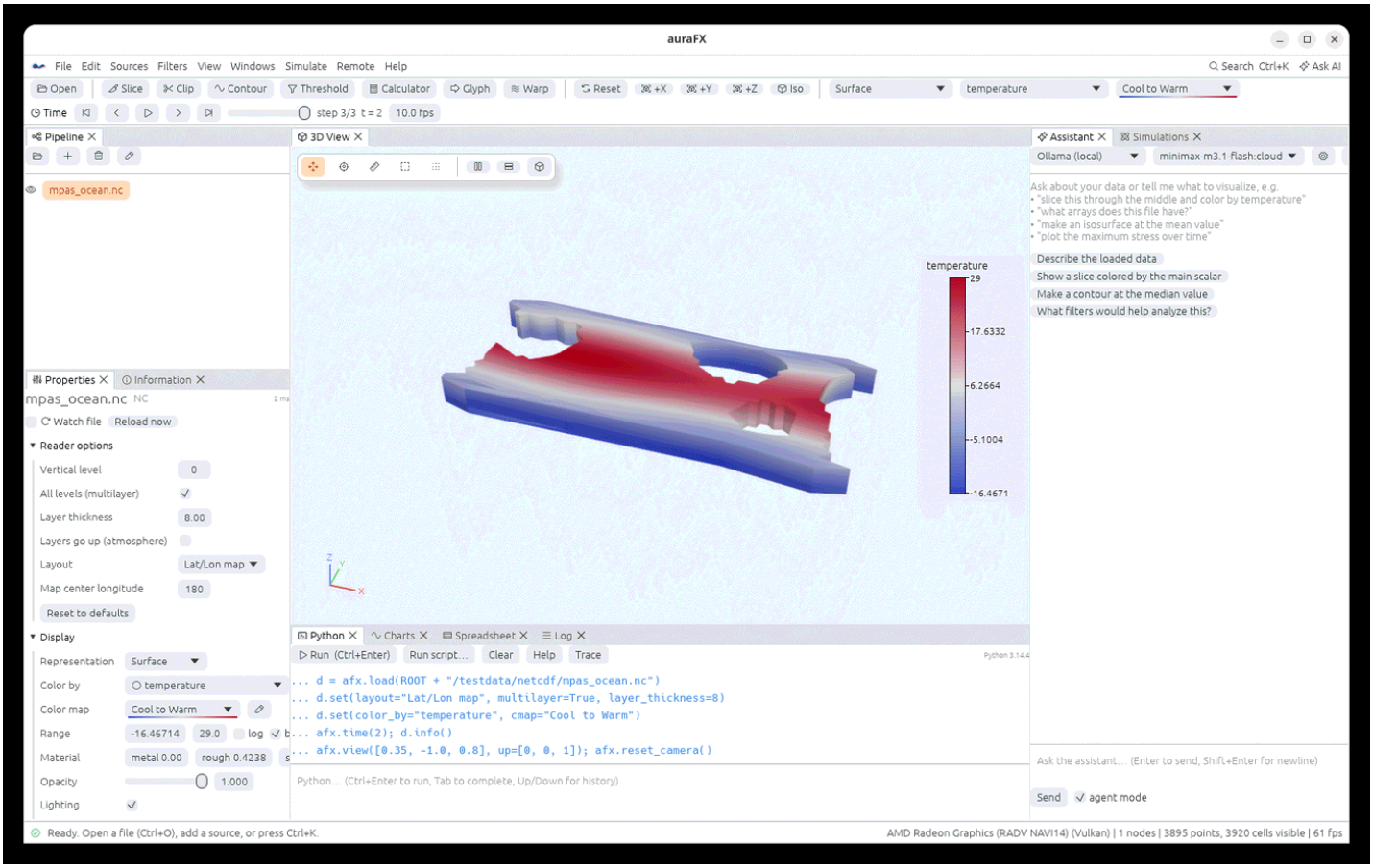
```
ocean = afx.load("output.nc")
ocean.set(vertical_level=10)                          # the 11th level down
ocean.set(layout="Lat/Lon map", multilayer=True)      # all levels, as a flat 3-D
map
```

On the map, cells that cross its left or right edge are drawn on both sides, so the map reaches a little beyond 360°.

- The triangles at a mesh boundary and, when the file has `maxLevelCell`, the triangles that touch land are left out, so land shows as holes. In the multilayer view, the levels below the sea floor are left out too, and each point carries its level's value.
- Reader options do not reach files opened on another machine through a server yet.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).


temperature
29
20.6332
12.2664
3.8996
-4.4671
Z
Y
X


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
temperature
Cool to Warm
Time
step 3/3 t = 2
10.0 fps
Pipeline
mpas_ocean.nc
3D View
temperature
29
17.6332
6.2664
-5.1004
-16.4671
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
mpas_ocean.nc
NC
Watch file
Reload now
Reader options
Vertical level
0
All levels (multilayer)
Layer thickness
8.00
Layers go up (atmosphere)
Layout
Lat/Lon map
Map center longitude
180
Reset to defaults
Display
Representation
Surface
Color by
temperature
Color map
Cool to Warm
Range
-16.46714
29.0
log
Material
metal 0.00
rough 0.4238
Opacity
1.000
Lighting
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... d = afx.load(ROOT + "/testdata/netcdf/mpas_ocean.nc")
... d.set(layout="Lat/Lon map", multilayer=True, layer_thickness=8)
... d.set(color_by="temperature", cmap="Cool to Warm")
... afx.time(2); d.info()
... afx.view([0.35, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 3895 points, 3920 cells visible | 61 fps

GIS rasters

A GeoTIFF is a TIFF that also records where it lies on the map. auraFX notices these tags and reads the file as a map raster instead of as a picture. Esri ASCII grids (`.asc`), a common text format for elevation models, and ERDAS Imagine files (`.img` , common in remote sensing) are read the same way. Open them with **File › Open** or `afx.load("dem.tif")` .

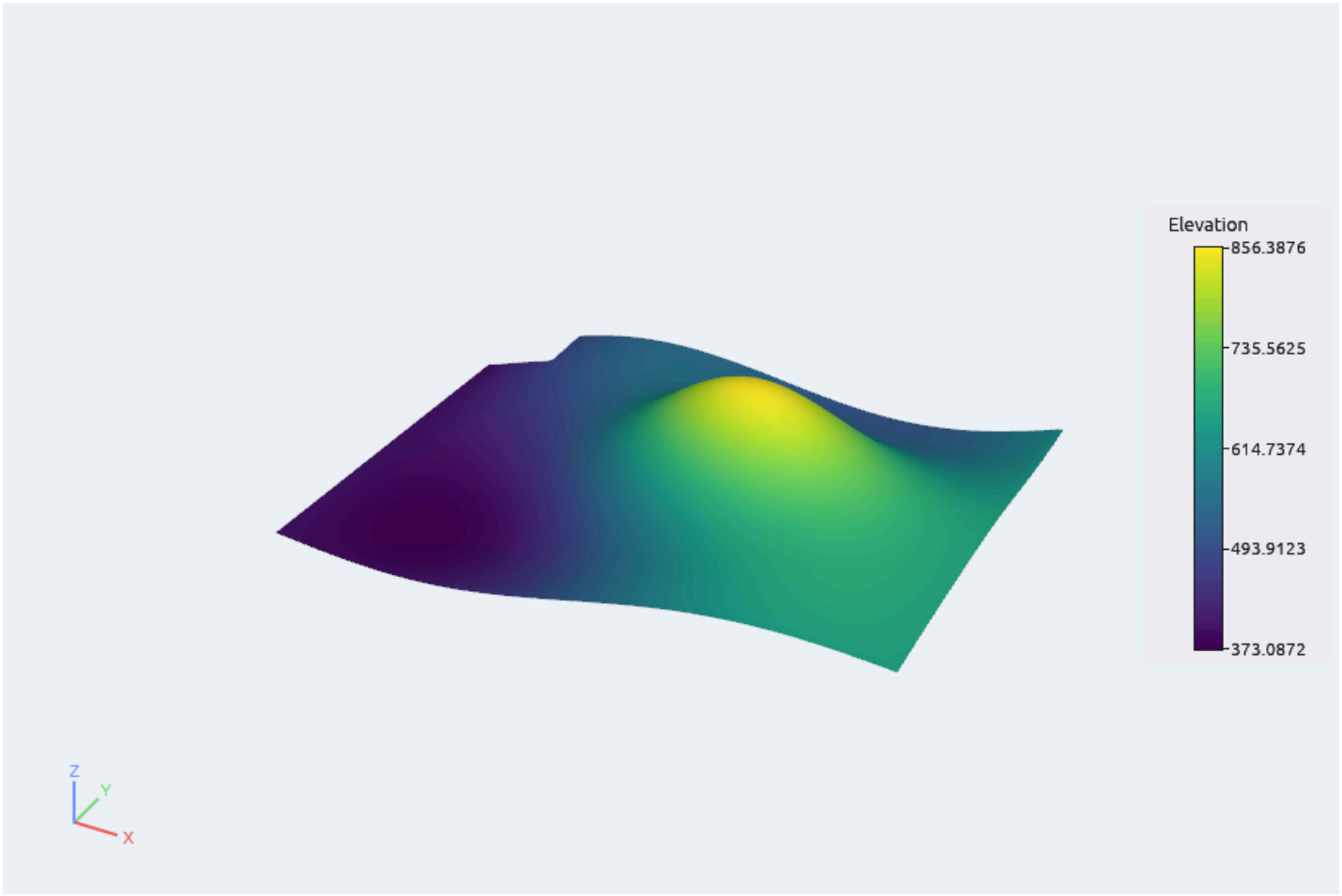
- The raster becomes image data with one cell per pixel, placed at its map coordinates (meters for UTM, degrees for longitude/latitude), with north up.
- Values are cell data. A single band, such as an elevation model, is named `Elevation` . In a file with several bands, the first is `Elevation` and the others are `Band 2` , `Band 3` … An RGB(A) image becomes one `Elevation` array with 3 or 4 components.
- Pixels equal to the file's no-data value become NaN and are drawn grey (after Warp By Scalar they leave a hole, since a NaN height has no place).
- The field data holds `NO_DATA_VALUE` per band and `MAP_PROJECTION` . For a GeoTIFF that is the EPSG code (`EPSG:32633`). For an ASCII grid it is the text of the `.prj` file beside it.
- 8- to 64-bit integer and floating-point bands are read, compressed or not, in strips or tiles, with the bands stored together or one after another. A pixel-is-point GeoTIFF is moved half a pixel to its corner, as GDAL does.
- ERDAS Imagine: every layer of the same size is a band (1-, 2- and 4-bit, 8- to 32-bit integer and 32- and 64-bit floating-point layers, in tiles, plain or run-length compressed), placed by its map information as GDAL does. The no-data value is the layer's "non-initialized" value. `MAP_PROJECTION` is the EPSG code for UTM and longitude/latitude on WGS 84, and the projection's name otherwise. The layers are bands as above: an RGB `.img` gives `Elevation` , `Band 2` and `Band 3` (it does not say which layer is which color). Data kept in a separate `.ige` file (very large images) is not read.

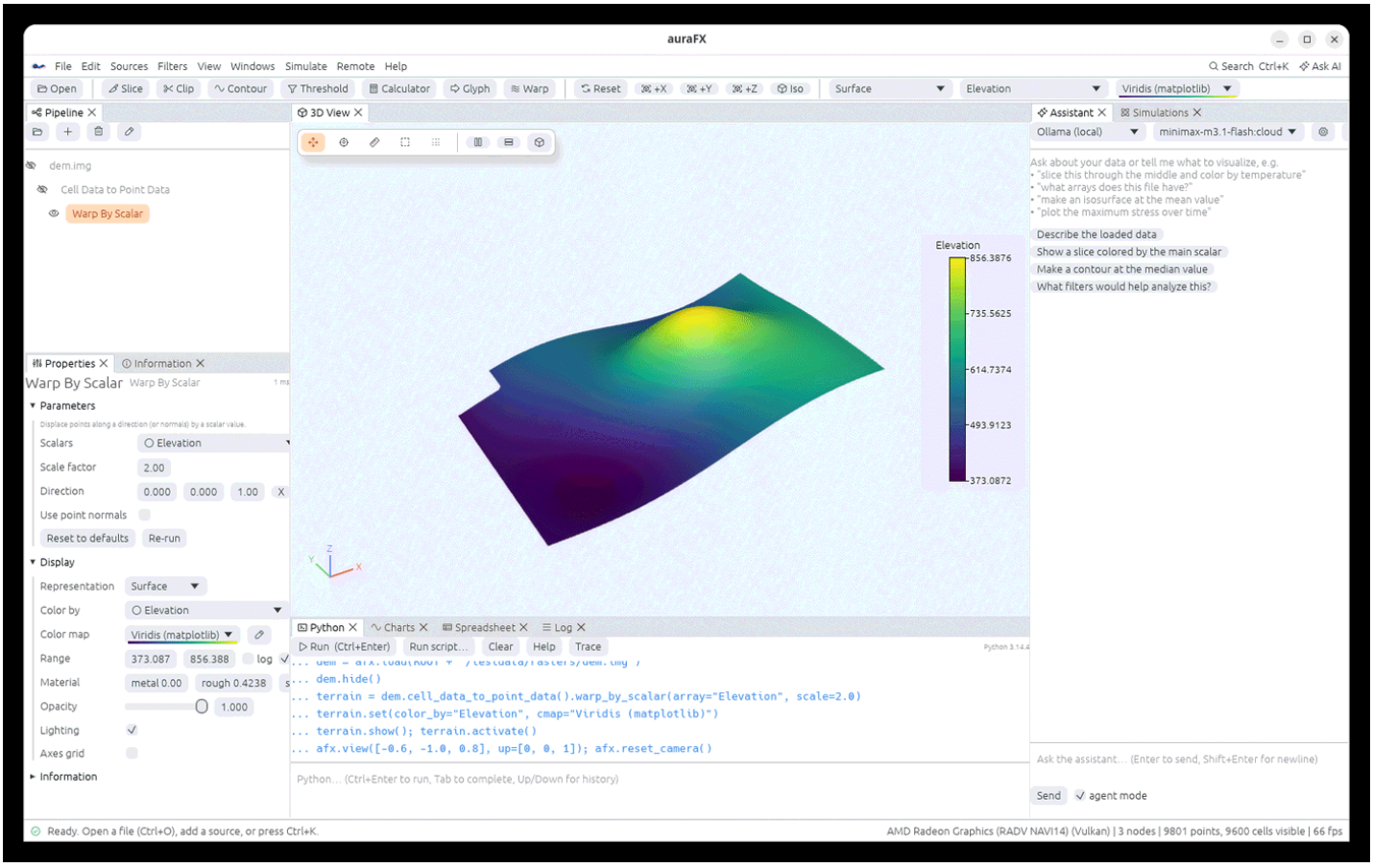
To see an elevation model as terrain, move the values to the points with **Cell Data to Point Data**, then raise them with **Warp By Scalar**:

```
dem = afx.load("dem.tif")                    # or dem.img (ERDAS Imagine)
terrain = dem.cell_data_to_point_data().warp_by_scalar(array="Elevation",
scale=1.0)
terrain.set(color_by="Elevation")
```

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Other GDAL raster formats (NetCDF through GDAL, JPEG 2000…) are not read yet.


Elevation
856.3876
735.5625
614.7374
493.9123
373.0872
Z
Y
X


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Elevation
Viridis (matplotlib)
Pipeline
dem.img
Cell Data to Point Data
Warp By Scalar
3D View
Elevation
856.3876
735.5625
614.7374
493.9123
373.0872
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
Warp By Scalar
Warp By Scalar
Parameters
Displace points along a direction (or normals) by a scalar value.
Scalars
Elevation
Scale factor
2.00
Direction
0.000
0.000
1.00
X
Use point normals
Reset to defaults
Re-run
Display
Representation
Surface
Color by
Elevation
Color map
Viridis (matplotlib)
Range
373.087
856.388
log
Material
metal 0.00
rough 0.4238
Opacity
1.000
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
Python 3.14.4
... dem.hide()
... terrain = dem.cell_data_to_point_data().warp_by_scalar(array="Elevation", scale=2.0)
... terrain.set(color_by="Elevation", cmap="Viridis (matplotlib)")
... terrain.show(); terrain.activate()
... afx.view([-0.6, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 9801 points, 9600 cells visible | 66 fps

GIS vectors

Maps of points, lines and areas, such as countries, roads, rivers, wells or survey sites, open like any other file:

Format	Files
ESRI Shapefile	`name.shp` with `name.dbf` (attributes) and `name.prj` (coordinate system) beside it; open the `.shp`
GeoJSON	`.geojson`, or a `.json` file that holds GeoJSON
KML (Google Earth)	`.kml`; each folder is a layer
KMZ	`.kmz`: a zipped KML (Google Earth's usual download); the KML document inside is read
GeoPackage	`.gpkg`; each feature table is a layer
FlatGeobuf	`.fgb`: a fast binary format for large vector data; one layer
GML	`.gml` (GML 2, 3 and 3.2, as GDAL and most web feature services write it); each feature type is a layer, typed by the `.xsd` schema beside the file when there is one
MapInfo table	`name.tab` with `name.map` (geometry), `name.id` (its index) and `name.dat` (attributes) beside it; open the `.tab`
MapInfo interchange	`name.mif` (geometry, in text) with `name.mid` (attributes) beside it; open the `.mif`
Esri file geodatabase	a `name.gdb` folder; open the folder (or any file in it). Each feature class and table is a layer

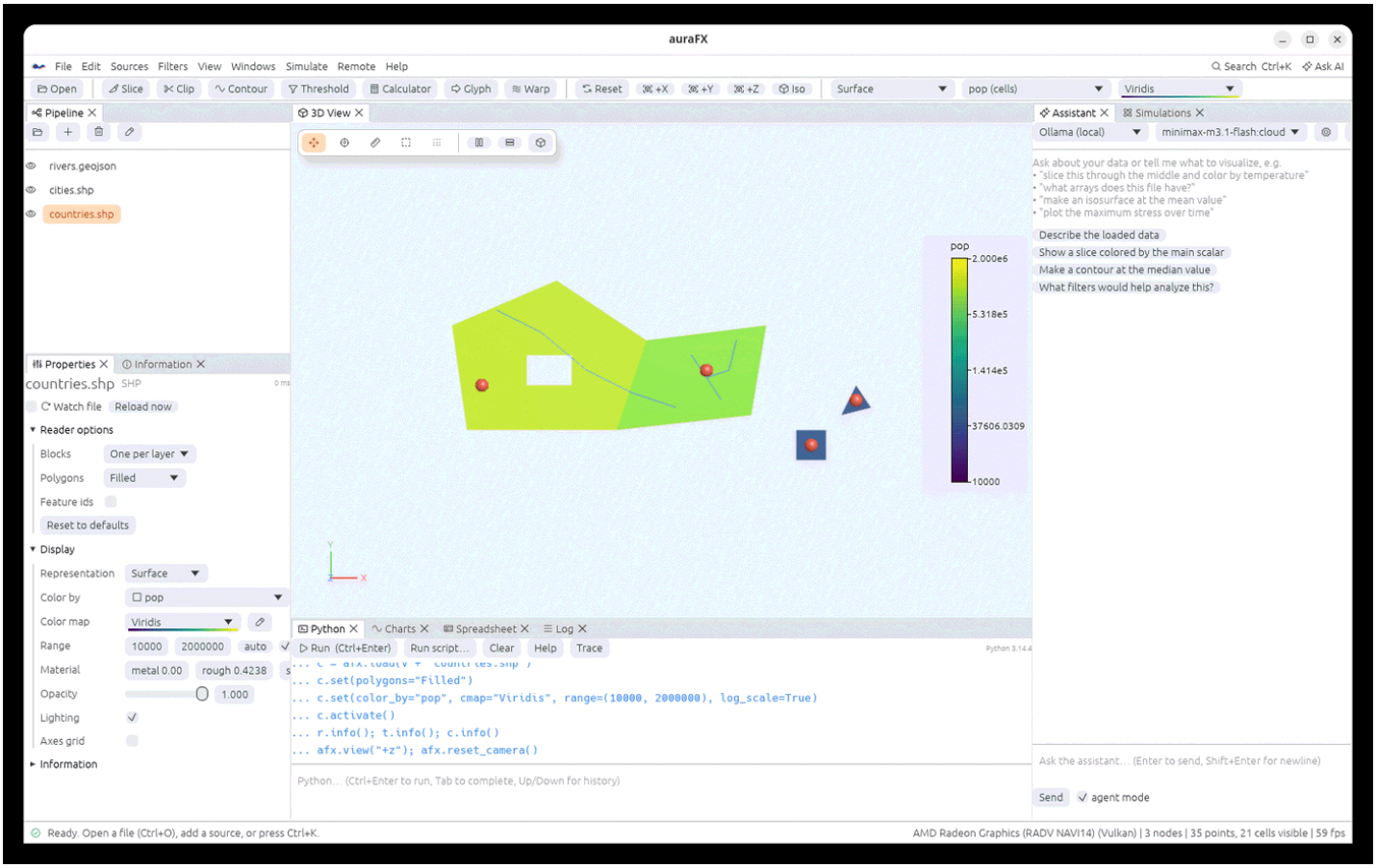
What you get:

- The coordinates stay as stored: longitude and latitude in degrees for most GeoJSON and KML files, or meters for projected shapefiles. Heights (the third coordinate) are kept, so a file with heights in meters on a map in degrees looks very tall. Flatten it with **Transform** (scale z by 0) if that gets in the way.
- Points become vertices, lines become polylines, and polygon rings become closed lines.
- Each feature's attributes become cell data on its cells: whole numbers as integers, decimals as doubles, and anything else (names, dates) as text. 64-bit integers stay numbers.
- `MAP_PROJECTION` in the field data holds the coordinate system: the `.prj` text, the EPSG code of a GeoPackage, FlatGeobuf or GML layer, the coordinate system text of a file geodatabase layer, or `EPSG:4326` (longitude/latitude) for GeoJSON and KML.
- File geodatabases store coordinates on a fine grid and add `Shape_Length` and `Shape_Area` attributes to lines and areas; both come through as stored.

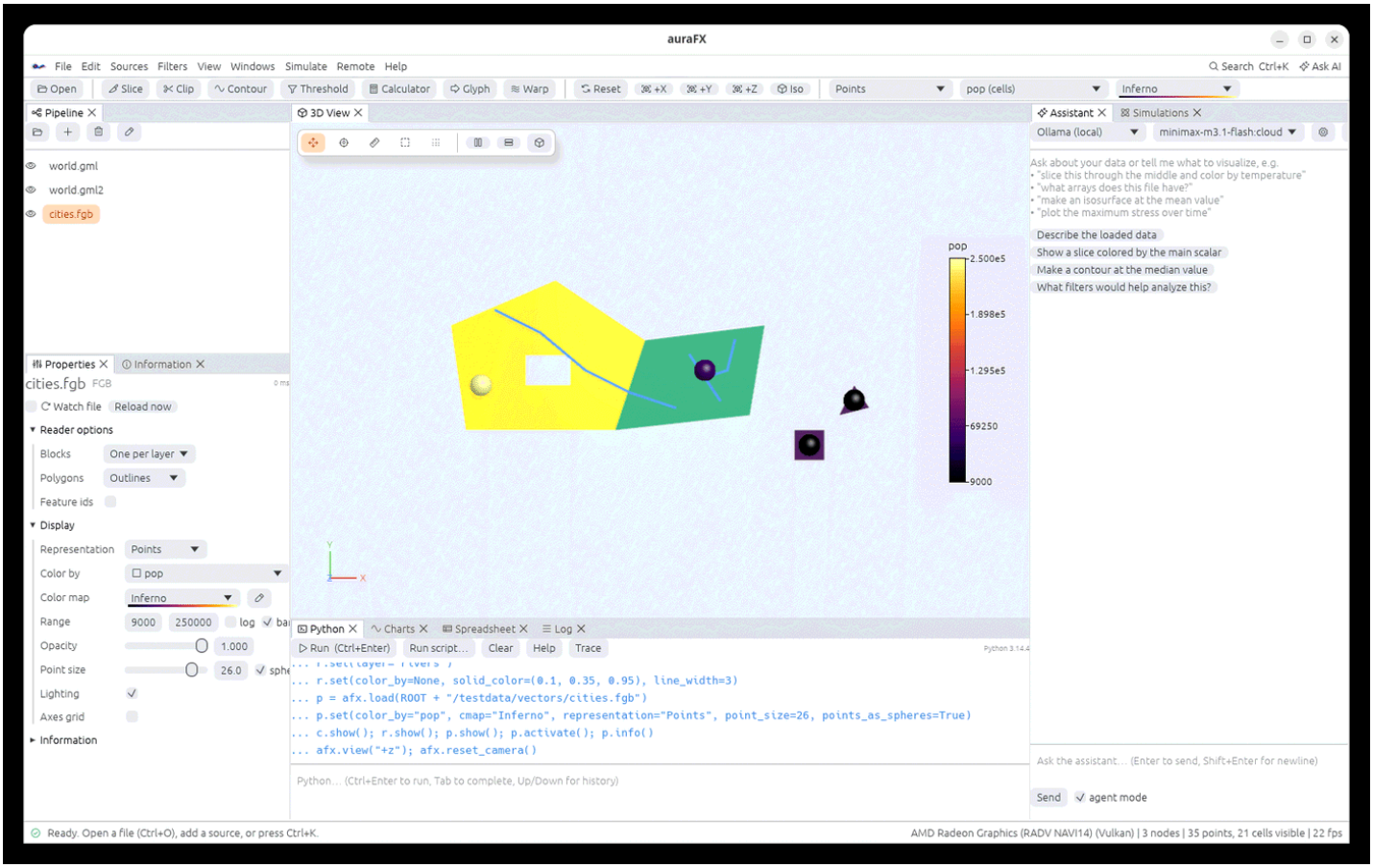
- MapInfo tables store coordinates as whole numbers on a grid set by the file (a millionth of a degree for longitude and latitude). auraFX converts them as GDAL does, rounding to that grid. A MapInfo region's rings are grouped into areas with holes: a `.tab` file says which rings are holes. In a `.mif` file, a ring inside a larger one is a hole, as GDAL decides it.
- GML in a geographic coordinate system named the long way (`urn:ogc:def:crs:EPSG::4326`) stores latitude first. auraFX swaps it back to longitude, latitude, as GDAL does, so GML lines up with the other formats.

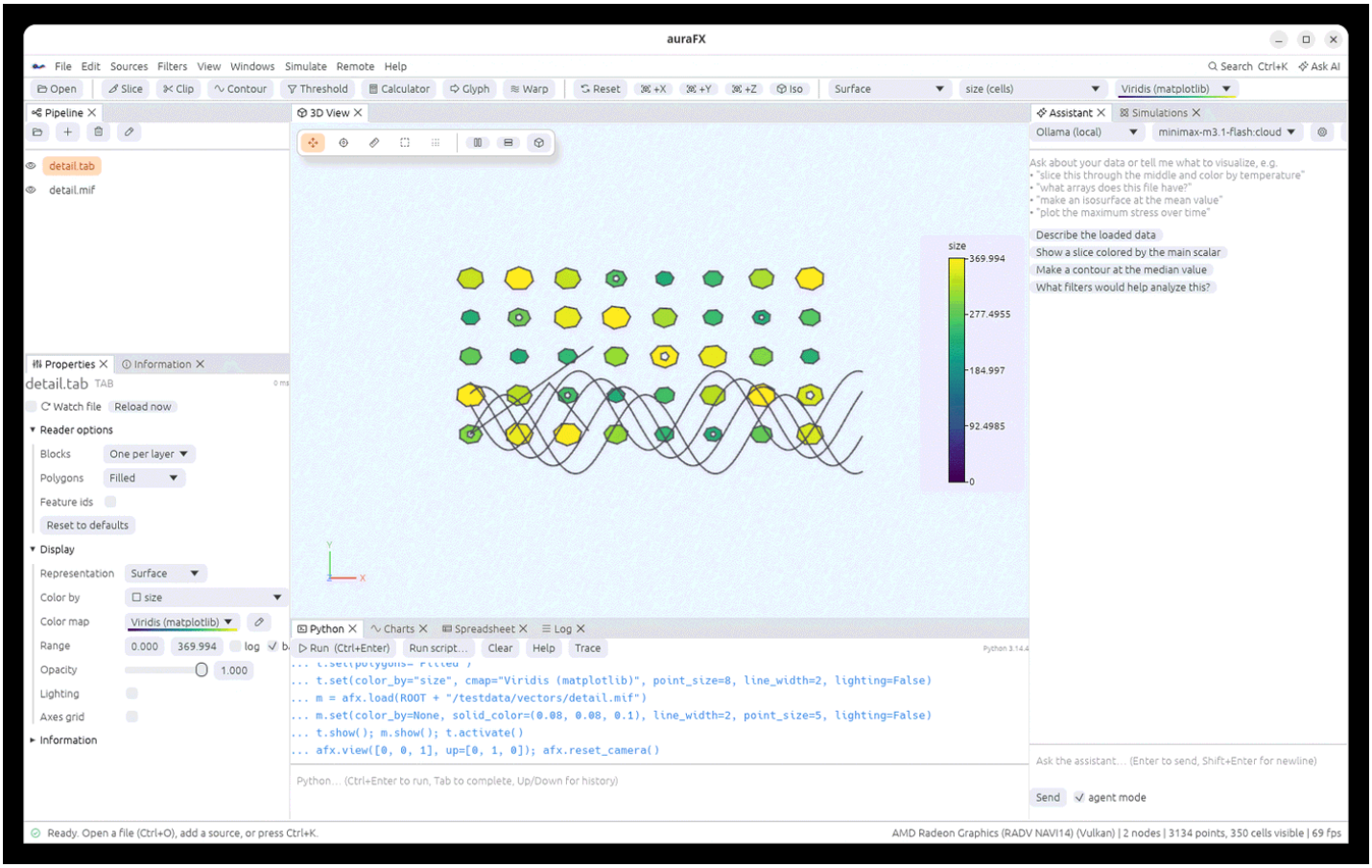
Reader options (Properties › Reader options, or `node.set(...)`):

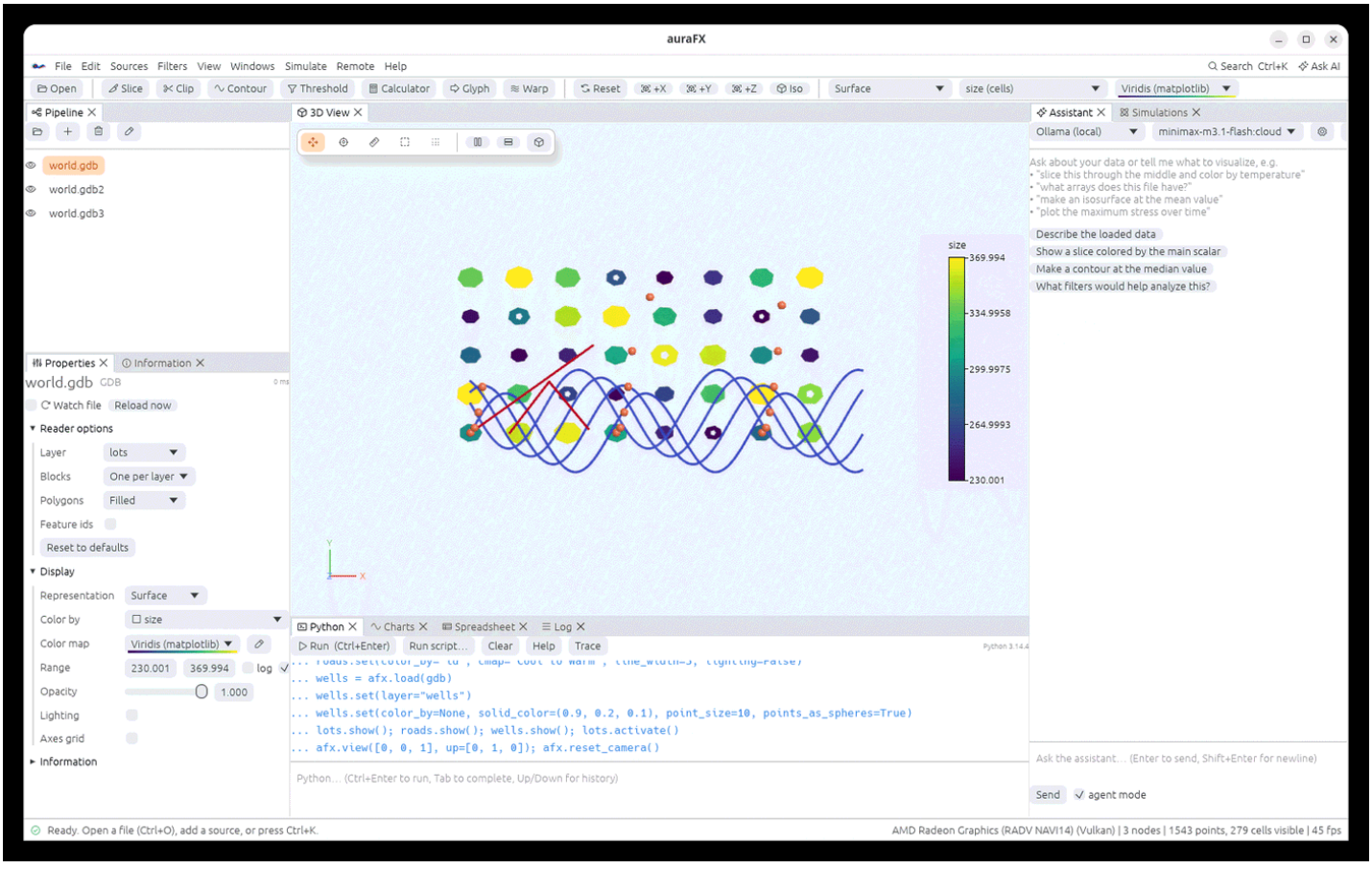
Option	Choices
Layer (files with several layers)	**All** or one layer by name
Blocks	**One per layer** (default: one block per layer, a row of attributes per feature), or **One per feature**: one block per feature, slow for thousands of features
Polygons	**Outlines**, or **Filled**: triangles covering each area, with holes left open, to color areas by an attribute
Feature ids	adds the feature number as `_vtkPedigreeIds`



```
countries = afx.load("countries.shp")
countries.set(polygons="Filled")
countries.set(color_by="pop", log_scale=True)
roads = afx.load("world.gpkg")
roads.set(layer="roads")                    # one layer of a GeoPackage
sites = afx.load("sites.fgb")               # FlatGeobuf
rivers = afx.load("hydro.gml")              # GML, a block per feature type
places = afx.load("trip.kmz")               # Google Earth KMZ
parcels = afx.load("parcels.tab")           # MapInfo (or parcels.mif)
lots = afx.load("city.gdb")                 # an Esri file geodatabase folder
lots.set(layer="lots", polygons="Filled")
```



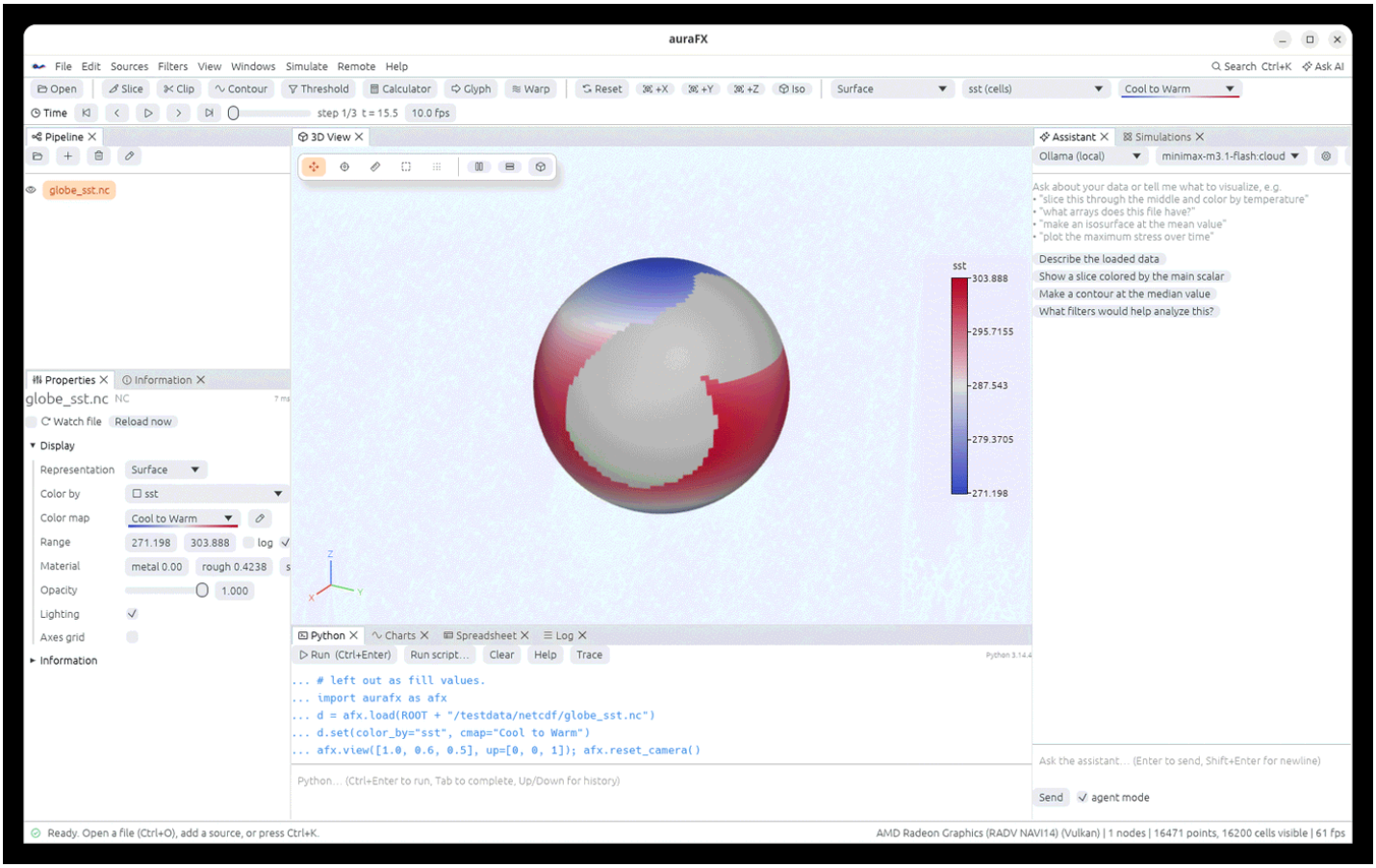

auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
size (cells)
Viridis (matplotlib)
Pipeline
detail.tab
detail.mif
3D View
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
size
369.994
277.4955
184.997
92.4985
0
Properties
Information
detail.tab TAB
Watch file
Reload now
Reader options
Blocks
One per layer
Polygons
Filled
Feature ids
Reset to defaults
Display
Representation
Surface
Color by
size
Color map
Viridis (matplotlib)
Range
0.000
369.994
log
Opacity
1.000
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... m = afx.load(ROOT + "/testdata/vectors/detail.mif")
... m.set(color_by=None, solid_color=(0.08, 0.08, 0.1), line_width=2, point_size=5, lighting=False)
... t.show(); m.show(); t.activate()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 3134 points, 350 cells visible | 69 fps


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
size (cells)
Viridis (matplotlib)
Pipeline
world.gdb
world.gdb2
world.gdb3
3D View
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
size
369.994
334.9958
299.9975
264.9993
230.001
Properties
Information
world.gdb GDB
Watch file
Reload now
Reader options
Layer
lots
Blocks
One per layer
Polygons
Filled
Feature ids
Reset to defaults
Display
Representation
Surface
Color by
size
Color map
Viridis (matplotlib)
Range
230.001
369.994
log
Opacity
1.000
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... wells = afx.load(gdb)
... wells.set(layer="wells")
... wells.set(color_by=None, solid_color=(0.9, 0.2, 0.1), point_size=10, points_as_spheres=True)
... lots.show(); roads.show(); wells.show(); lots.activate()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 1543 points, 279 cells visible | 45 fps

A feature without geometry gives an empty block. Any layer of a file can be picked on its own, and by default all layers are read.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Not read yet: curved geometry (arcs; a file geodatabase's curves are read as straight lines between their vertices) and file geodatabase rasters and multipatches. In MapInfo files, text, arcs, rectangles, ellipses and collections are read without their geometry (their attributes are kept). Shapefile multipatches (3-D surfaces) are skipped.



AMR data (adaptive mesh refinement)

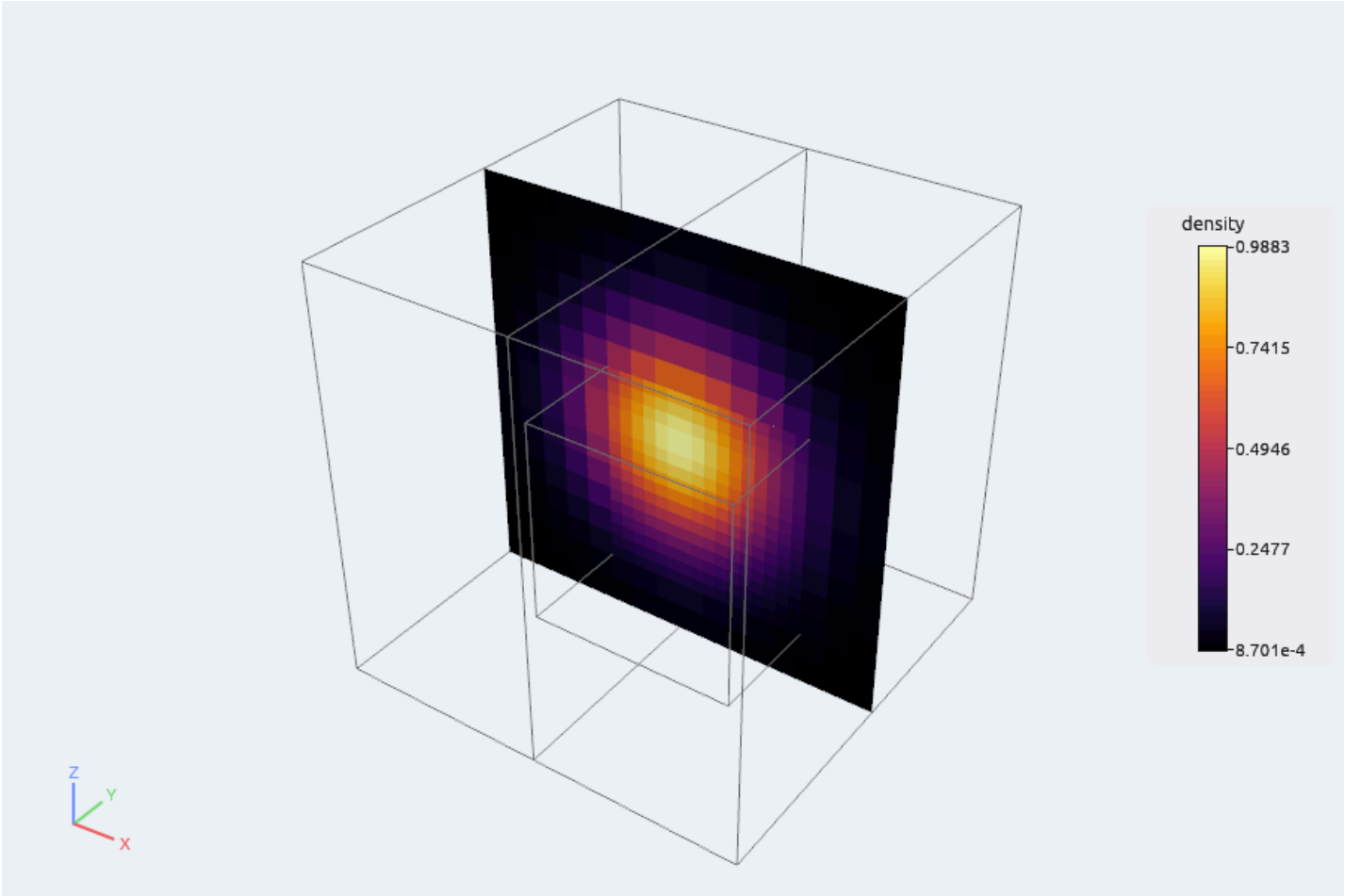
AMR simulations (AMReX, BoxLib, Castro, MAESTRO, Nyx, WarpX, FLASH, Enzo…) store their domain as levels of boxes: a coarse level everywhere and finer boxes where more detail was needed. auraFX reads:

- **AMReX / BoxLib plotfiles**: a folder like `plt00100` with a `Header` file. Open the `Header` file in **File › Open** (choose "All files"), drop the folder on the window, or `afx.load("plt00100")` .
- **VTK AMR files** (`.vthb` , `.vth`).

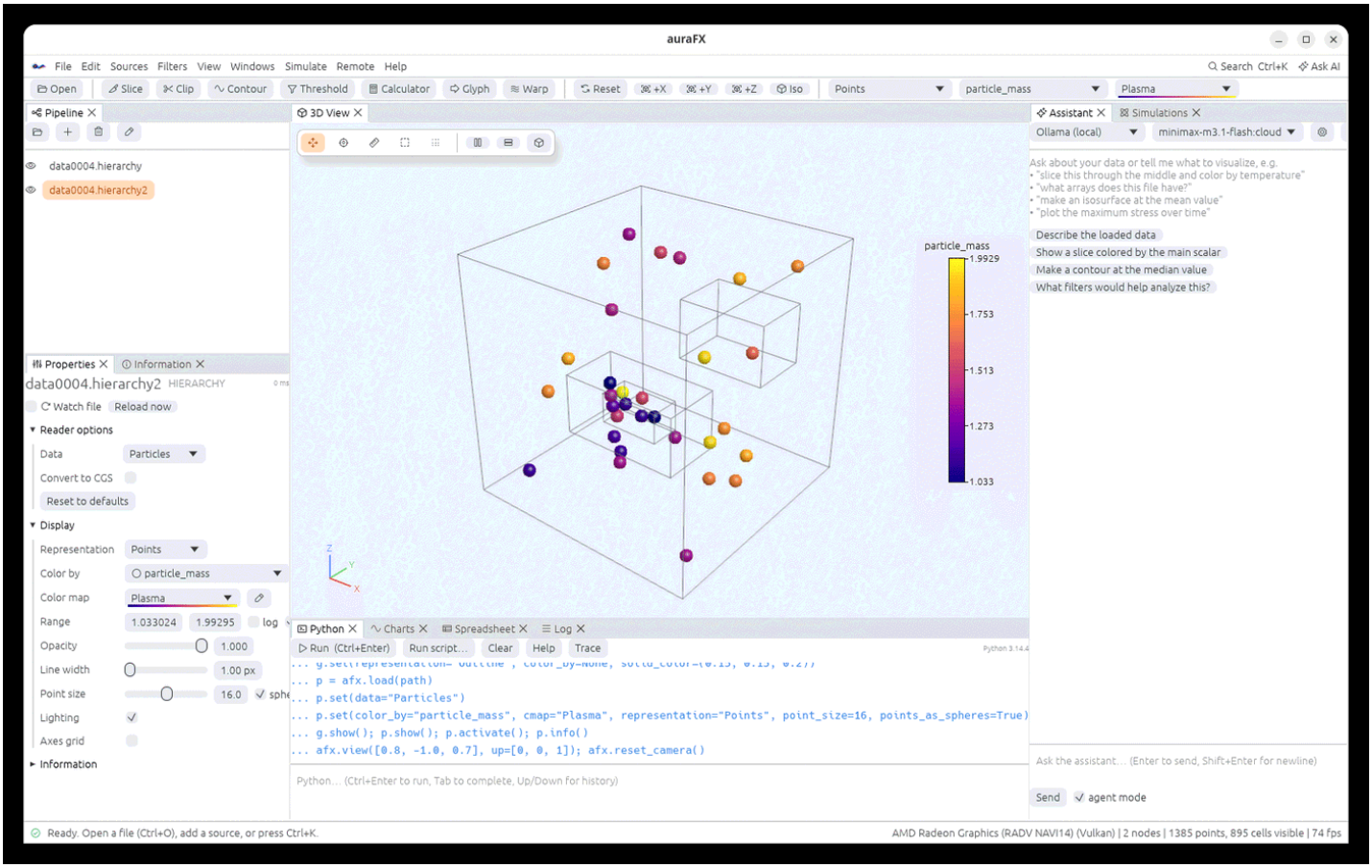
- **FLASH** plot and checkpoint files (`name_hdf5_plt_cnt_0007` , `name_hdf5_chk_0003` ; HDF5, FLASH 2 to 4). They usually have no extension: open them in **File › Open** with “All files”; auraFX recognizes them by their contents. Every block of the PARAMESH tree becomes a box on its refinement level (FLASH’s level 1 is auraFX’s Level 0), with every variable in `unknown names` as cell data in double precision. 1-D, 2-D and 3-D files are read.
- **Enzo** output: open the `.hierarchy` file of a data dump (`DD0010/data0010.hierarchy`). The parameter file (`data0010`) and the grid files (`data0010.cpu0000` …) must be beside it. Each grid is a box on its level; fields keep their stored type (usually single precision). Reader option **Convert to CGS** multiplies each field by its `#DataCGSConversionFactor` from the parameter file. It is off by default.
- **Particles** in FLASH and Enzo files: when a file has them, the reader option **Data** switches between **Grid** (the AMR data) and **Particles**, a point for every particle with its properties as point data (FLASH’s as doubles, Enzo’s in their stored types). Open the file twice to see both, as in the picture below. Chombo files with a `particles` group get the same option (properties as doubles).
- **Chombo** files (`.hdf5` , `.h5` ; 1-D to 3-D), as Chombo-based codes write them (`level_0` , `level_1` … groups with boxes). The component names become cell arrays, or point arrays in node-centered files. Ghost cells stored around each box are dropped, and the boxes are placed from `prob_lo` and the level’s `dx` (or `vec_dx`).
- **4-D Chombo** files (two space axes and two velocity axes, as gyrokinetic codes such as COGENT write them, single level): a 2-D grid over the space axes, whose every cell holds each component as an array with one value per velocity cell (parallel velocity major, magnetic moment minor). `_vpar` and `_mu` hold those cells’ velocity coordinates, and `<component>_sum` the sum over the velocity cells (leaving out velocity cells no box covers, which are NaN in the arrays). Color by a component of the array to see one velocity cell, or by the sum.

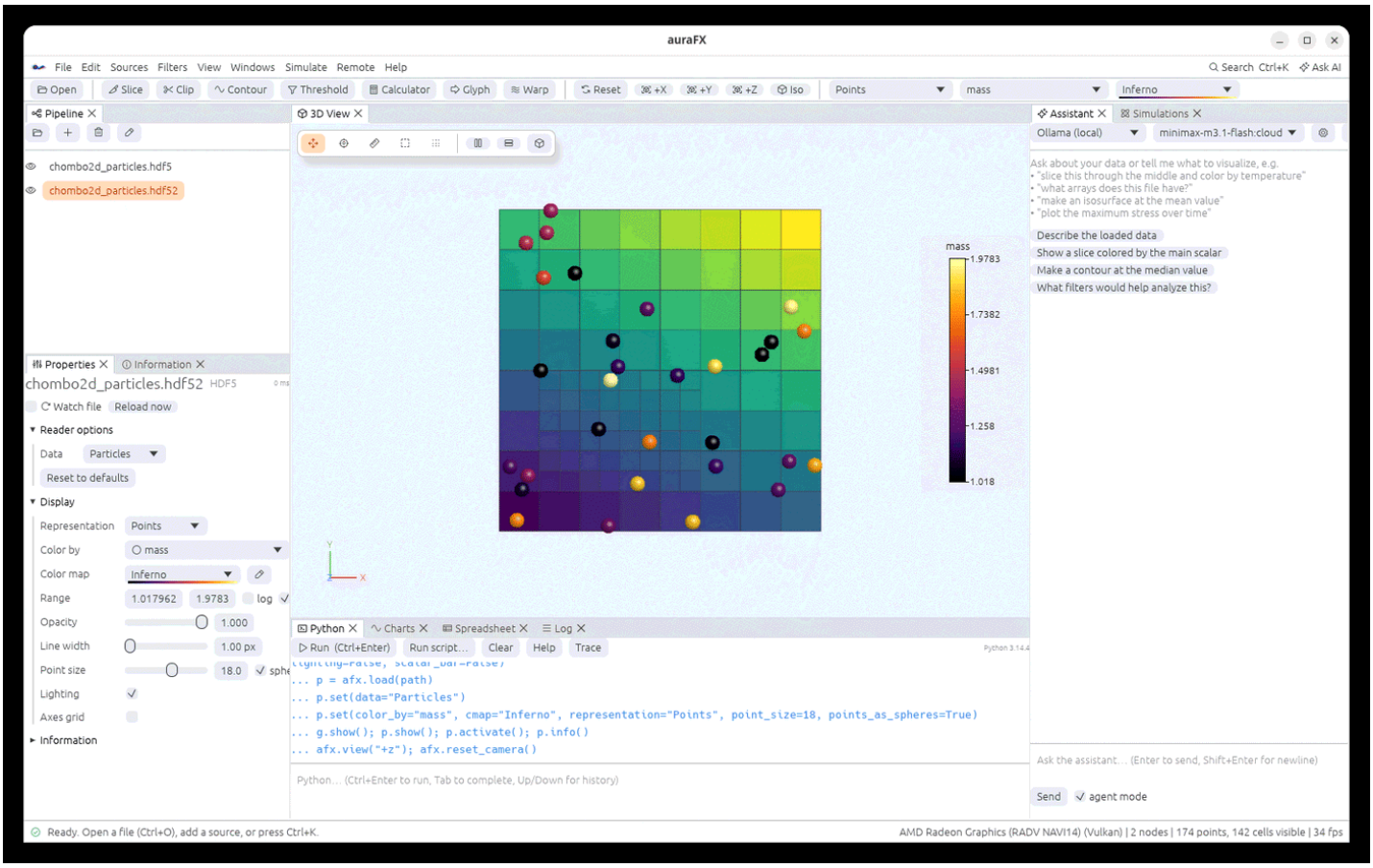
What you get:

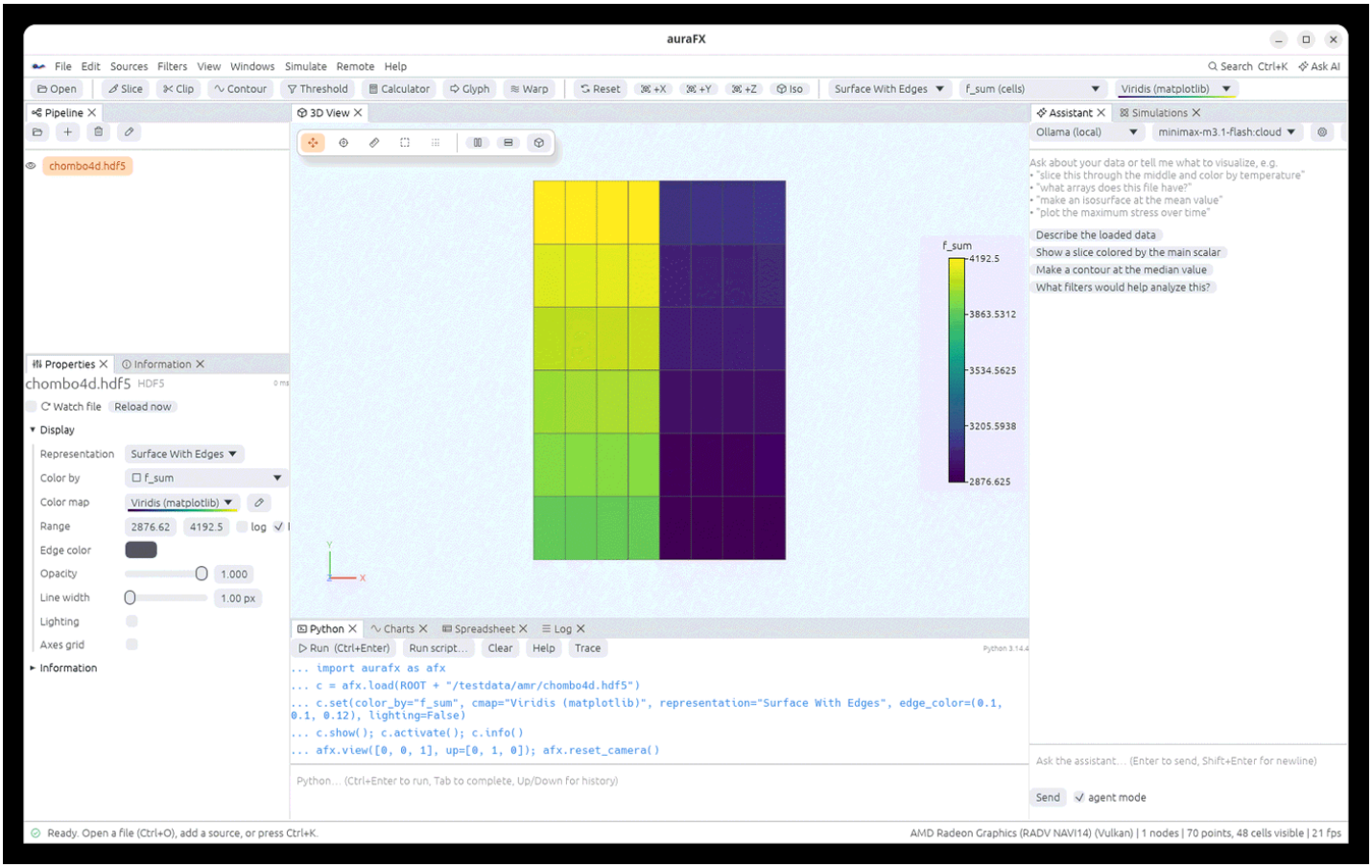
- One block per level (**Level 0**, **Level 1**, …), each holding that level’s boxes as image data with the variables as cell data. The **Information** panel lists them.
- Coarse cells that a finer box covers are marked as hidden in `vtkGhostType` . Surfaces, slices, clips, contours, thresholds, Integrate Variables, Resample To Image and probing skip them, so every place is shown and counted once, at its finest level.
- AMReX vector variables written as `amrexvec_<name>_x` , `_y` , `_z` become one vector `<name>` . In 2-D files the missing z is 0.
- The time of the plotfile or dump is in the field data (`TimeValue`). Single- and double-precision files are read.
- **Outline** draws the outline of every box, which shows where the refinement is.
- Contours of cell data are stitched across levels (see below).

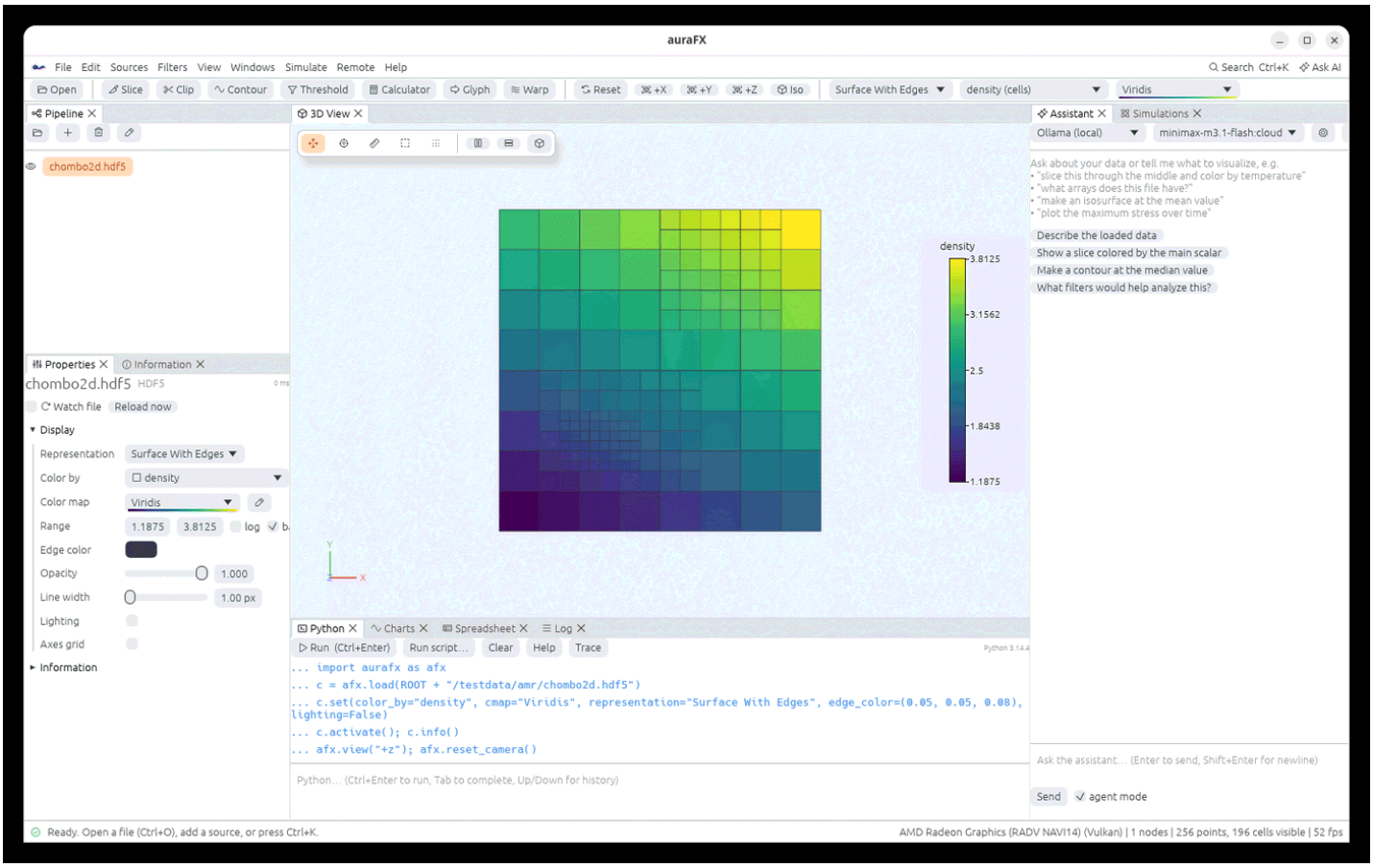


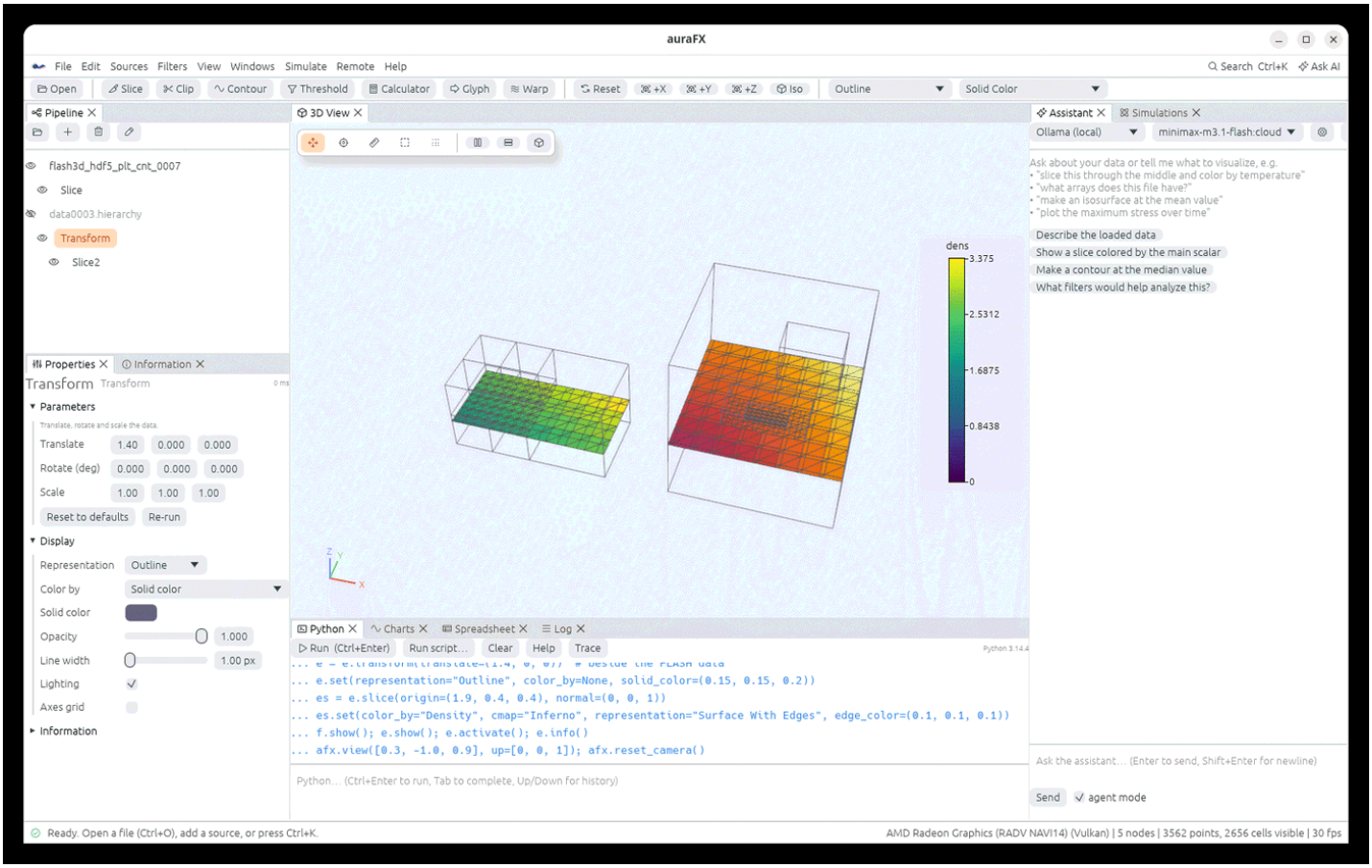

```
amr = afx.load("plt00100")                  # the plotfile folder
amr.set(representation="Outline")           # where the refinement is
s = amr.slice(normal=(0, 0, 1))             # finest data on the cut
s.set(color_by="density")
lev = amr.overlapping_amr_level_ids()       # LevelIdScalars: each cell's level
flash = afx.load("sedov_hdf5_plt_cnt_0010")  # FLASH
enzo = afx.load("DD0010/data0010.hierarchy")  # Enzo
enzo.set(cgs=True)                          # fields in CGS units
chombo = afx.load("plot.0010.2d.hdf5")       # Chombo
stars = afx.load("DD0010/data0010.hierarchy")
stars.set(data="Particles")                 # the particles instead of the grid
```


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso Points particle_mass Plasma
Pipeline
data0004.hierarchy
data0004.hierarchy2
3D View
particle_mass
1.9929
1.753
1.513
1.273
1.033
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
data0004.hierarchy2 HIERARCHY
Watch file
Reload now
Reader options
Data
Particles
Convert to CGS
Reset to defaults
Display
Representation
Points
Color by
particle_mass
Color map
Plasma
Range
1.033024
1.99295
log
Opacity
1.000
Line width
1.00 px
Point size
16.0
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... p = afx.load(path)
... p.set(data="Particles")
... p.set(color_by="particle_mass", cmap="Plasma", representation="Points", point_size=16, points_as_spheres=True)
... g.show(); p.show(); p.activate(); p.info()
... afx.view([0.8, -1.0, 0.7], up=[0, 0, 1]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 1385 points, 895 cells visible | 74 fps


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K Ask AI
Open Slice Clip Contour Threshold Calculator Glyph Warp Reset +X +Y +Z Iso Points mass Inferno
Pipeline
chombo2d_particles.hdf5
chombo2d_particles.hdf52
3D View
mass
1.9783
1.7382
1.4981
1.258
1.018
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
chombo2d_particles.hdf52 HDF5
Watch file
Reload now
Reader options
Data
Particles
Reset to defaults
Display
Representation
Points
Color by
mass
Color map
Inferno
Range
1.017962
1.9783
log
Opacity
1.000
Line width
1.00 px
Point size
18.0
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... p = afx.load(path)
... p.set(data="Particles")
... p.set(color_by="mass", cmap="Inferno", representation="Points", point_size=18, points_as_spheres=True)
... g.show(); p.show(); p.activate(); p.info()
... afx.view("+z"); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 174 points, 142 cells visible | 34 fps

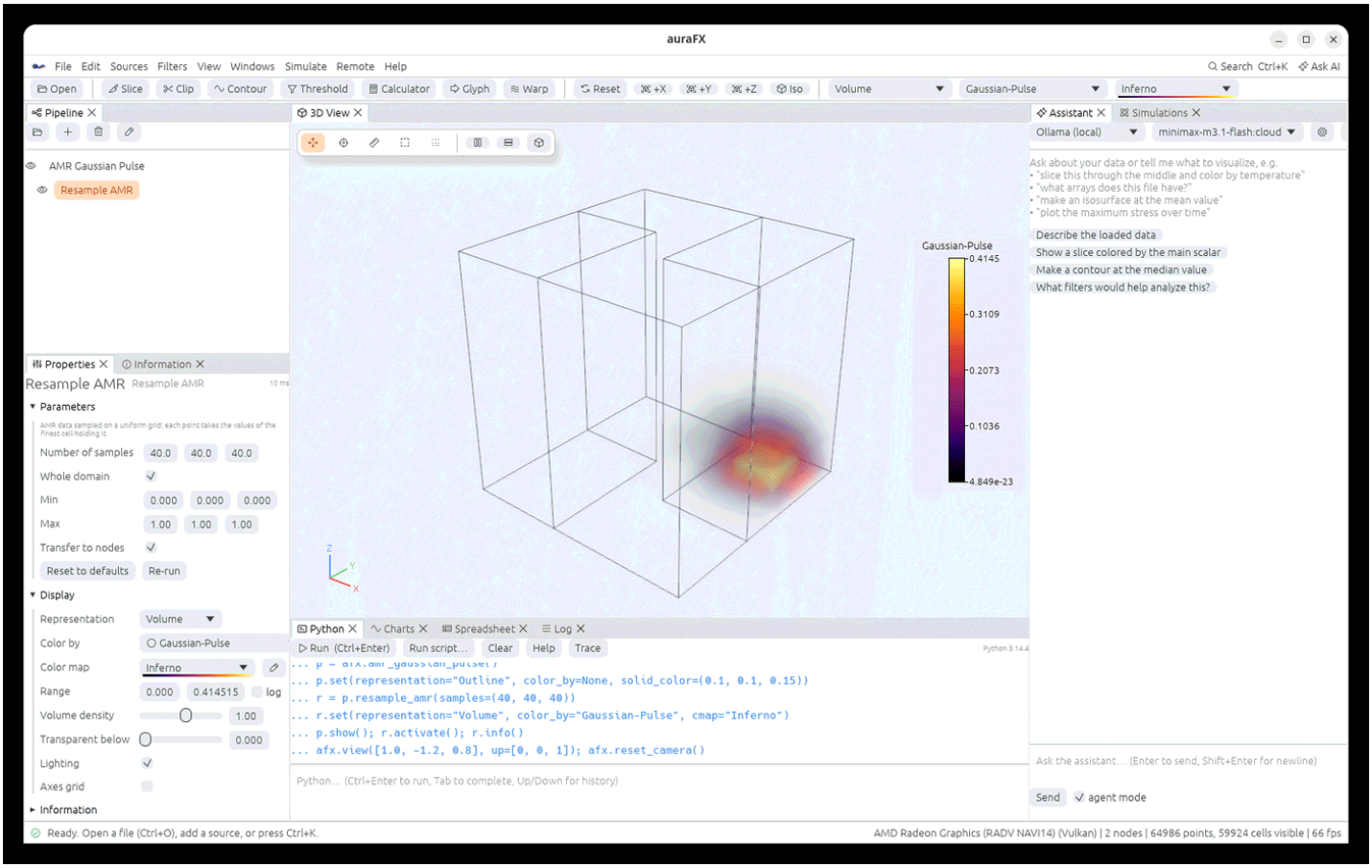

auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
f_sum (cells)
Viridis (matplotlib)
Pipeline
chombo4d.hdf5
3D View
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
f_sum
4192.5
3863.5312
3534.5625
3205.5938
2876.625
Properties
Information
chombo4d.hdf5
HDF5
Watch file
Reload now
Display
Representation
Surface With Edges
Color by
f_sum
Color map
Viridis (matplotlib)
Range
2876.62
4192.5
log
Edge color
Opacity
1.000
Line width
1.00 px
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... import aurafx as afx
... c = afx.load(ROOT + "/testdata/amr/chombo4d.hdf5")
... c.set(color_by="f_sum", cmap="Viridis (matplotlib)", representation="Surface With Edges", edge_color=(0.1, 0.1, 0.12), lighting=False)
... c.show(); c.activate(); c.info()
... afx.view([0, 0, 1], up=[0, 1, 0]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 70 points, 48 cells visible | 21 fps


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
density (cells)
Viridis
Pipeline
chombo2d.hdf5
3D View
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
density
3.8125
3.1562
2.5
1.8438
1.1875
Properties
Information
chombo2d.hdf5
HDF5
Watch file
Reload now
Display
Representation
Surface With Edges
Color by
density
Color map
Viridis
Range
1.1875
3.8125
log
Edge color
Opacity
1.000
Line width
1.00 px
Lighting
Axes grid
Information
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... import aurafx as afx
... c = afx.load(ROOT + "/testdata/amr/chombo2d.hdf5")
... c.set(color_by="density", cmap="Viridis", representation="Surface With Edges", edge_color=(0.05, 0.05, 0.08), lighting=False)
... c.activate(); c.info()
... afx.view("+z"); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 256 points, 196 cells visible | 52 fps



Sources › AMR Gaussian Pulse makes a small AMR test dataset: a coarse box and two finer boxes holding a Gaussian pulse, in 2-D or 3-D. **Filters › Overlapping AMR Level Ids** adds the cell array `LevelIdScalars` with each cell's level.

Filters › Resample AMR samples AMR data on one uniform grid, so tools that want a single image can use it (volume rendering, image filters, export). **Number of samples** sets the grid points per axis; with **Whole domain** off, **Min** and **Max** pick a box (clipped to the data, dropping the samples that fall outside). Each point takes the values of the finest cell holding it. With **Transfer to nodes** off, the values are taken at the cell centers instead and become cell data. Points no box holds are marked hidden.



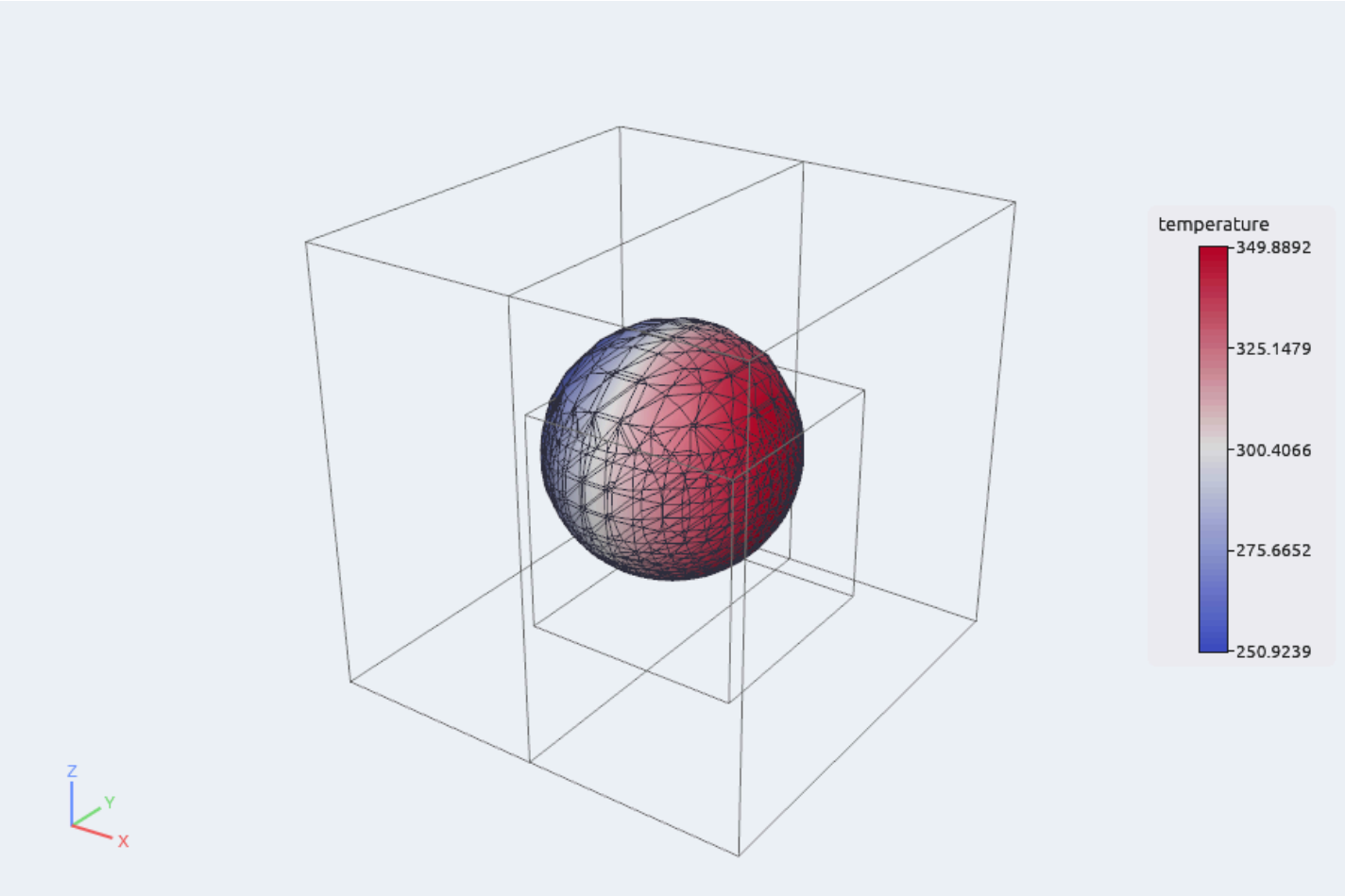

```
grid = amr.resample_amr(samples=(64, 64, 64))
part = amr.resample_amr(samples=(32, 32, 16), whole_domain=False, min=(0, 0, 0),
max=(1, 1, 0.5))
```

Resample AMR always uses every level of the data. 2-D FLASH files are read like 3-D ones. Ghost cells stored in Chombo files are dropped on every level. FLASH particles are given once each, with their own properties.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Contours of AMR data

Values of AMR data sit at cell centers, and cells change size where levels meet. Averaging them to points box by box leaves gaps and overlaps along every level boundary. **Filters › AMR Contour** joins the cell centers of all levels into one mesh, the *dual grid*. Where a finer box meets coarser cells, its border cells reach out to the coarse cell centers, so the isosurface continues across the boundary without a crack. The other cell arrays are interpolated onto the surface. **Contour** on a cell array of AMR data does the same automatically. In 2-D data the result is closed isolines.




```
c = amr.amr_contour(array="density", values=[0.3])
c = amr.contour(array="density", values=[0.3])     # the same, for AMR cell data
dual = amr.amr_dual_grid()                          # cell values on one stitched
mesh
cut = dual.clip(type="Scalar", array="density", value=0.3)   # a crack-free "dual
clip"
```

Filters › AMR Dual Grid gives that mesh itself: one point per cell center, carrying all the cell arrays as point data. Clip, Slice, Stream Tracer and other filters that interpolate point data work on it across levels without seams.

On a test plotfile whose density isosurface is a sphere crossing from the fine box into the coarse ones, the contour is closed (every edge shared by exactly two triangles, the topology of a sphere), its area is within 0.2 % of the sphere's, and its center is within 0.001 of the sphere's center. The box-by-box contour had 346 open edges there. More in the verification notes (`docs/reference/verification.md` in the source tree).

Not read yet: plotfiles without FAB headers (the "NoFabHeader" variants) and AMReX particles. Not available yet: material-interface filters for CTH data (AMR Fragments, Material Interface).

Hyper tree grids

A *hyper tree grid* (HTG) starts from a grid of root cells. Each root cell is the top of a tree: a refined cell splits into 2×2×2 children (3×3×3 with branch factor 3; 2×2 or 3×3 in 2-D), and so on down to the leaves. Every cell of every tree, coarse or leaf, can hold values. A *mask* can hide cells. It is VTK's tree-based format for adaptive meshes. auraFX reads VTK's `.htg` files (XML version 2).

What you get:

- The grid is drawn as its visible leaves. Every filter that works on cells (Slice, Clip, Threshold, Contour after Cell Data to Point Data, Cell Centers, Integrate Variables…) sees those leaves as voxels (pixels in 2-D) with their values.
- **Information** shows the dimension, branch factor, number of visible leaves and depth. The cell count is the number of all cells (coarse ones included).
- A grid with **material interfaces** is drawn cut to its material (see below). Other filters still see whole leaves.
- Saving as `.htg` keeps the grid: its trees, the mask and every cell's values (the version 2 files). Saving as any other format (`.vtu` and the rest) writes the visible leaves as an unstructured grid.

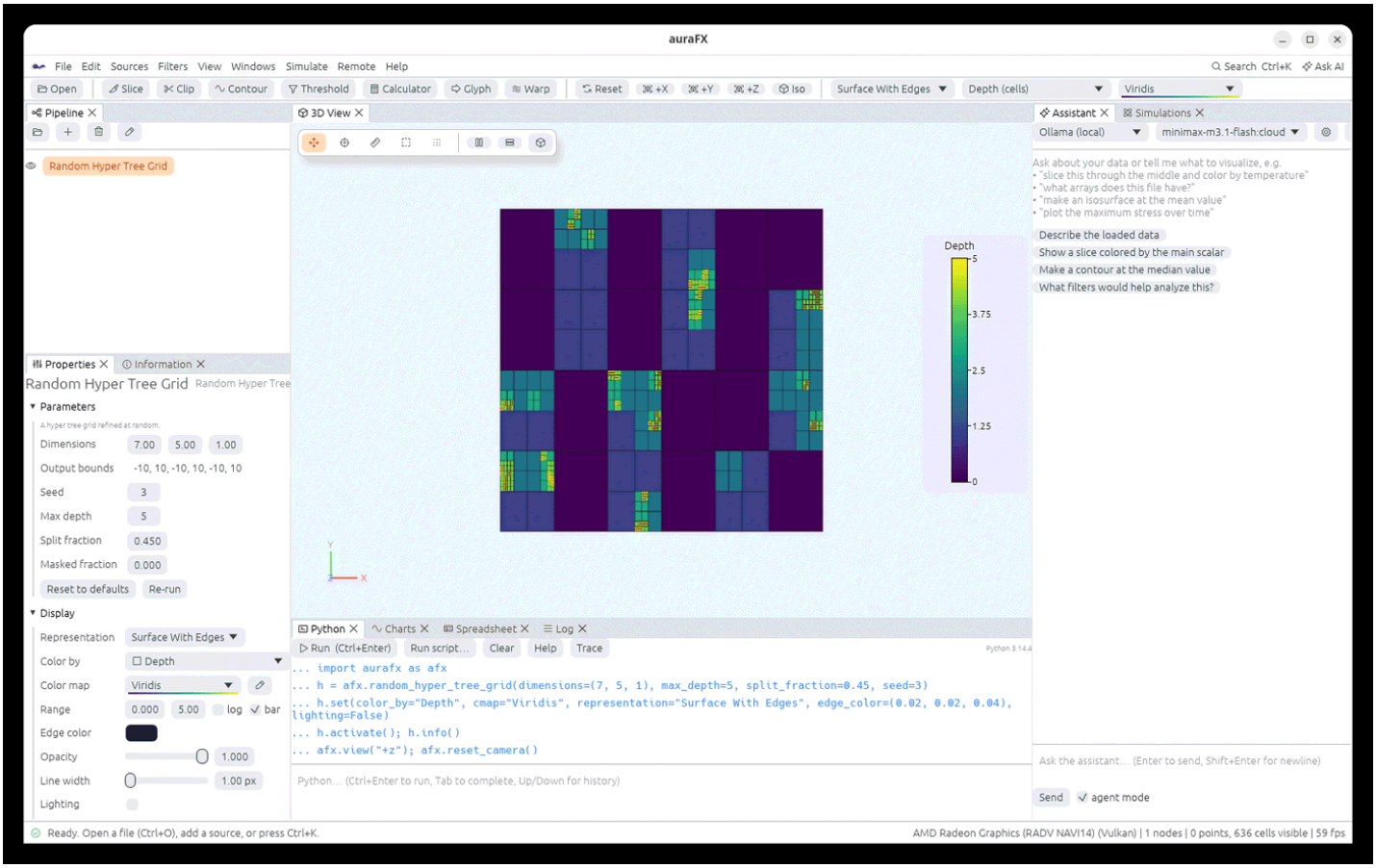
Sources:

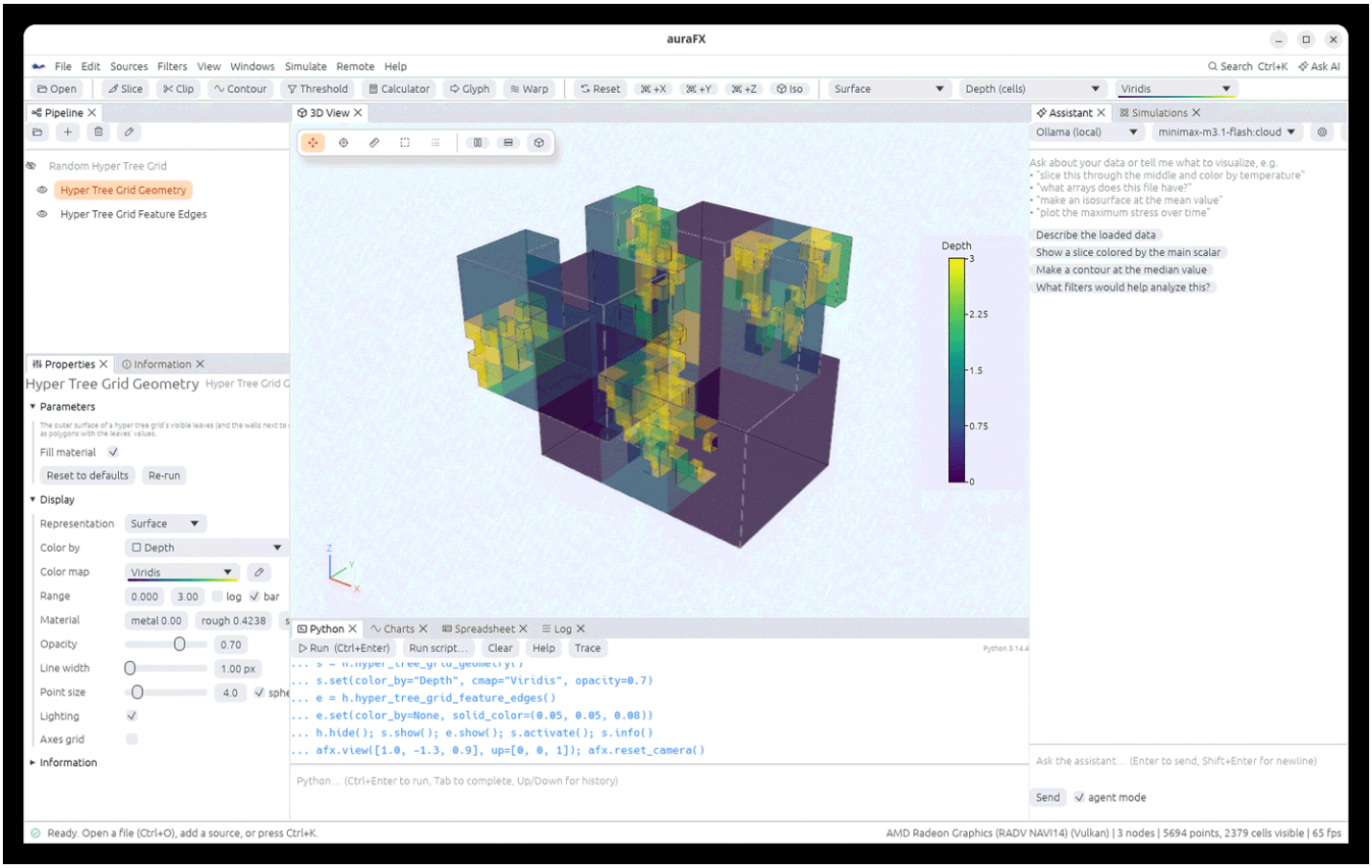
- **Hyper Tree Grid** builds a grid from a text descriptor. Levels are separated by `|`. Each level lists `R` (refined) or `.` (leaf) for every cell of that level, all trees in order; spaces are ignored. **Dimensions** (points per axis; 1 makes an axis flat), **Branch factor**, **Max depth**, **Grid scale**, and **Use mask** with a **Mask** string of the same layout (`0` hides a leaf, `1` keeps it). The mask is ignored on refined cells.
- **Random Hyper Tree Grid** refines cells at random. **Split fraction** is each cell's chance of being refined, down to **Max depth**. The same **Seed** gives the same grid. **Masked fraction** hides about that share of the volume, whole subtrees at a time (the same settings hide the same cells). Its masking can hide a cell while leaving some cells below it unmasked; those stay hidden.

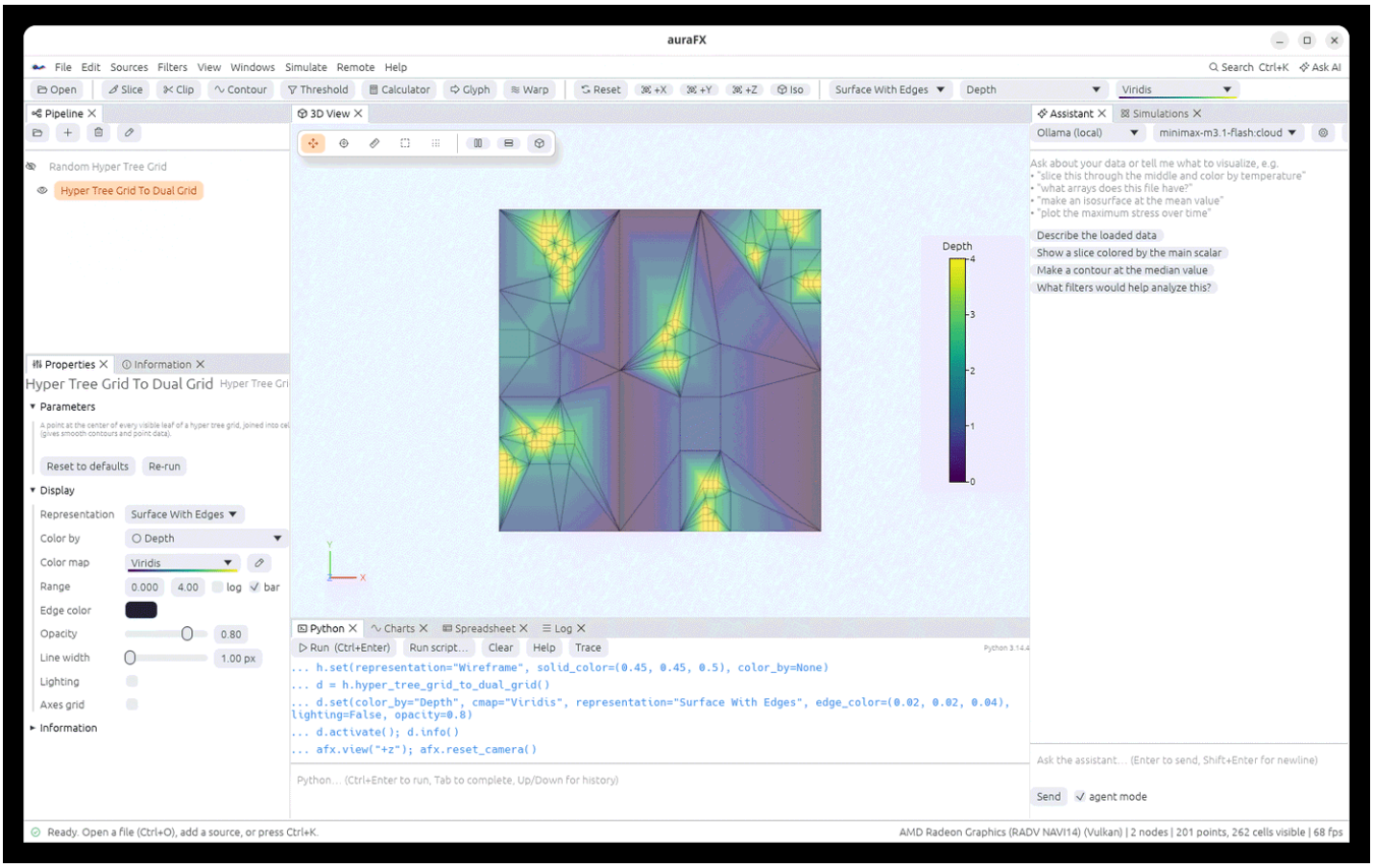
Filters (Filters › Data):

Filter	What it does
Hyper Tree Grid To Unstructured Grid	the visible leaves as an unstructured grid, with `OriginalIds` (each leaf's cell index)
Hyper Tree Grid Depth Limiter	cuts every tree at a depth: cells at that depth become leaves (Depth 0 keeps only the root cells)
Hyper Tree Grid Generate Fields	`CellSize` (volume, area or length of every cell), `ValidCell` (1 on visible leaves), `CellCenter`, and `TotalVisibleVolume` in the field

Filter	What it does
	data
Hyper Tree Grid To Dual Grid	a point at the center of every visible leaf, with the leaf's values as point data, and a cell (voxel, pixel or line) joining the leaves around every corner. Use it for smooth contours and for filters that need point data. Points on the grid's boundary sit on the boundary; next to masked leaves they move onto the face shared with the masked cell
Hyper Tree Grid Geometry	the outside of the visible leaves as polygons with their values: in 3-D the faces on the grid's boundary and the walls next to masked cells (a face of a masked cell next to a coarser visible leaf carries that leaf's values); in 2-D a quad per visible leaf, in 1-D a line
	With material interfaces, mixed cells are cut to their material (see below). **Fill material** (on by default): off draws only the interface lines in 2-D and leaves out pure cells in 1-D and 2-D
Hyper Tree Grid Feature Edges	the edges of the visible region as lines with their leaves' values: the outline in 2-D, the outer corners and folds in 3-D
Hyper Tree Grid Evaluate Coarse	fills the values of refined cells from their children: Min, Max, Sum, Average, Unmasked Average, Elder Child (the first child) or Splatting Average




auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
Depth (cells)
Viridis
Pipeline
Random Hyper Tree Grid
Hyper Tree Grid Geometry
Hyper Tree Grid Feature Edges
3D View
Depth
3
2.25
1.5
0.75
0
Properties
Information
Hyper Tree Grid Geometry
Parameters
Fill material
Reset to defaults
Re-run
Display
Representation
Surface
Color by
Depth
Color map
Viridis
Range
0.000
3.00
log
bar
Material
metal 0.00
rough 0.4238
Opacity
0.70
Line width
1.00 px
Point size
4.0
Lighting
Axes grid
Information
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... e = h.hyper_tree_grid_feature_edges()
... e.set(color_by=None, solid_color=(0.05, 0.05, 0.08))
... h.hide(); s.show(); e.show(); s.activate(); s.info()
... afx.view([1.0, -1.3, 0.9], up=[0, 0, 1]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 3 nodes | 5694 points, 2379 cells visible | 65 fps


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface With Edges
Depth
Viridis
Pipeline
Random Hyper Tree Grid
Hyper Tree Grid To Dual Grid
3D View
Depth
4
3
2
1
0
Properties
Information
Hyper Tree Grid To Dual Grid
Parameters
Reset to defaults
Re-run
Display
Representation
Surface With Edges
Color by
Depth
Color map
Viridis
Range
0.000
4.00
log
bar
Edge color
Opacity
0.80
Line width
1.00 px
Lighting
Axes grid
Information
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... h.set(representation="Wireframe", solid_color=(0.45, 0.45, 0.5), color_by=None)
... d = h.hyper_tree_grid_to_dual_grid()
... d.set(color_by="Depth", cmap="Viridis", representation="Surface With Edges", edge_color=(0.02, 0.02, 0.04), lighting=False, opacity=0.8)
... d.activate(); d.info()
... afx.view("+z"); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 2 nodes | 201 points, 262 cells visible | 68 fps

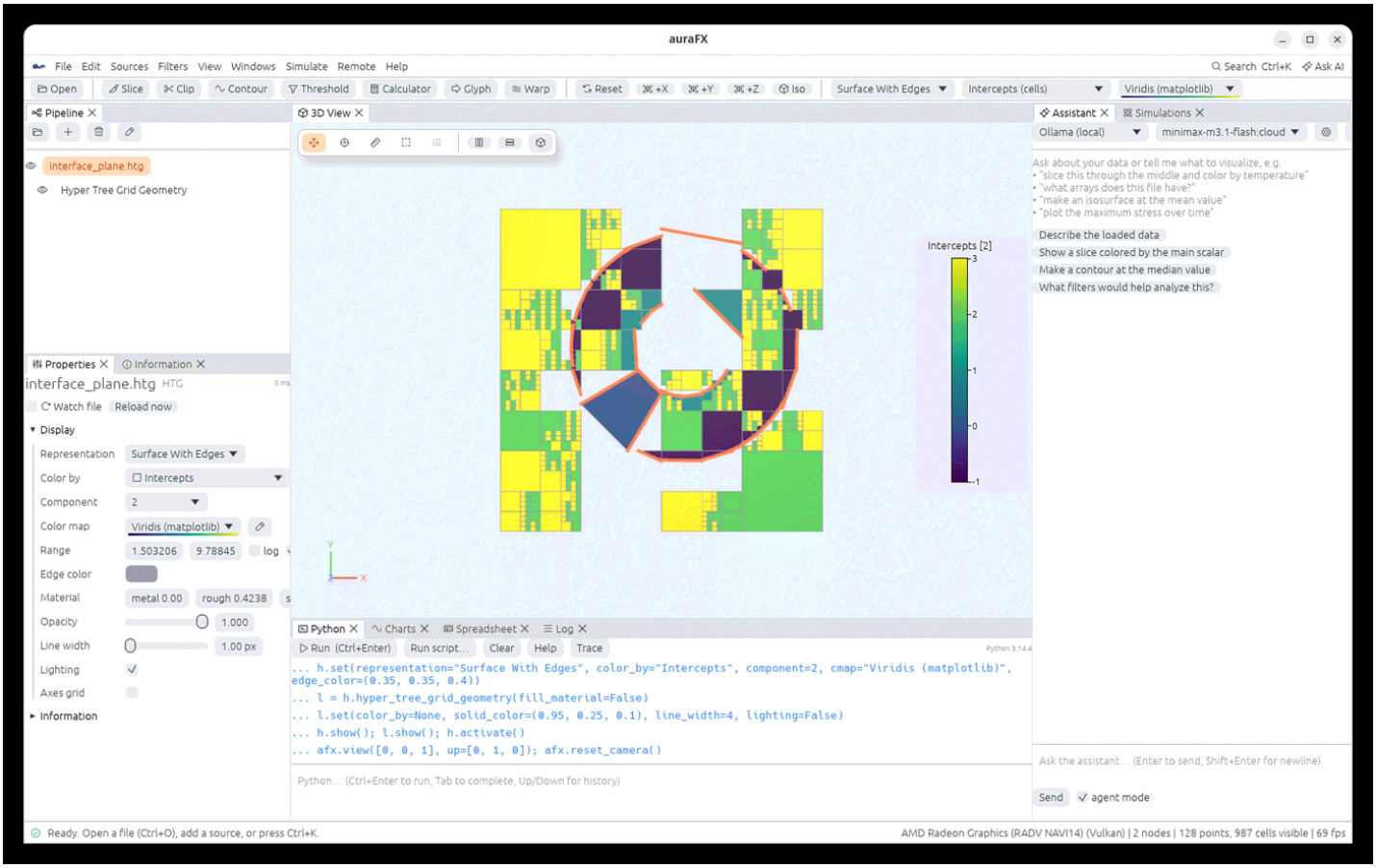
Material interfaces. A grid can describe where a material ends inside some cells, for simulations that track interfaces between materials. The file names two cell arrays (VTK's `InterfaceNormalsName` and `InterfaceInterceptsName`). For each cell they hold a normal *n* (three values) and intercepts (*d1*, *d2*, *type*); each interface is the plane $n \cdot x + d = 0$:

- *type* −1: one plane (*d1*); the material is where $n \cdot x + d1 \geq 0$;
- *type* 1: one plane (*d2*); the material is where $n \cdot x + d2 \leq 0$;
- *type* 0: the material lies between the two planes;
- *type* 2 or more, or a zero normal: the cell is pure (all material).

Hyper Tree Grid Geometry (and the default drawing) cuts each mixed cell by its planes:

- In 1-D and 2-D, each mixed leaf becomes the part of its line or quad that is in the material.
- In 3-D, the faces of mixed cells and of their neighbors are drawn cut to the material, and each mixed cell adds its interface polygons.

With **Fill material** off, 2-D grids give only the interface lines. Saving as `.htg` keeps the interface arrays.



```
h = afx.random_hyper_tree_grid(dimensions=(7, 5, 1), max_depth=5,
split_fraction=0.45, seed=3)
h.set(color_by="Depth", representation="Surface With Edges")
t = afx.hyper_tree_grid(dimensions=(3, 3, 1), max_depth=3, descriptor="R... | .R..
| ....")
coarse = h.hyper_tree_grid_evaluate_coarse(operator="Max")
top = h.hyper_tree_grid_depth_limiter(depth=2)
leaves = afx.load("tree.htg").hyper_tree_grid_to_unstructured_grid()
dual = h.hyper_tree_grid_to_dual_grid()
holes = afx.random_hyper_tree_grid(seed=3, masked_fraction=0.25)
walls = holes.hyper_tree_grid_geometry()
edges = holes.hyper_tree_grid_feature_edges()
h.save("tree.htg")
m = afx.load("interface_plane.htg")       # drawn cut to its material
lines = m.hyper_tree_grid_geometry(fill_material=False)
```

To Unstructured Grid gives every leaf its index in `OriginalIds`, masked files included. To Dual Grid makes points for visible leaves only. **Cell Centers** works on hyper tree grids directly; there is no separate Axis Reflection filter for them. `.htg` files auraFX writes read back unchanged.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Not available yet: version 1 `.htg` files. Ghost Cells only matters for data split across processes. Interfaces are used by Geometry only: slicing or clipping a grid with interfaces works on whole leaves.

Cell grids

A *cell grid* holds cells whose fields are polynomials rather than one value per point or cell. Simulation codes using finite elements of the discontinuous Galerkin (DG) kind write fields this way. On each cell a field is a sum of basis functions, each times a coefficient stored in the file. A field can jump from one cell to the next. It is VTK's newest data type. auraFX reads VTK's `.dg` files (JSON).

Cells can be hexahedra, tetrahedra, wedges, pyramids, quadrilaterals, triangles, edges and vertices. Fields can be:

- **nodal** (values at the corners, linear or quadratic; quadratic cells also have nodes on their edges, faces and middle), either shared between cells or kept per cell;
- **edge** fields (the H(curl) kind, for electric fields: their tangential part is continuous across faces);
- **face** fields (the H(div) kind, for fluxes: their normal part is continuous across faces);
- **constant** fields, one value per cell.

Edge and face fields are turned from the cell's own coordinates into space with the cell's shape (its Jacobian), as in VTK. The cells' shape can itself be quadratic (curved).

What you get:

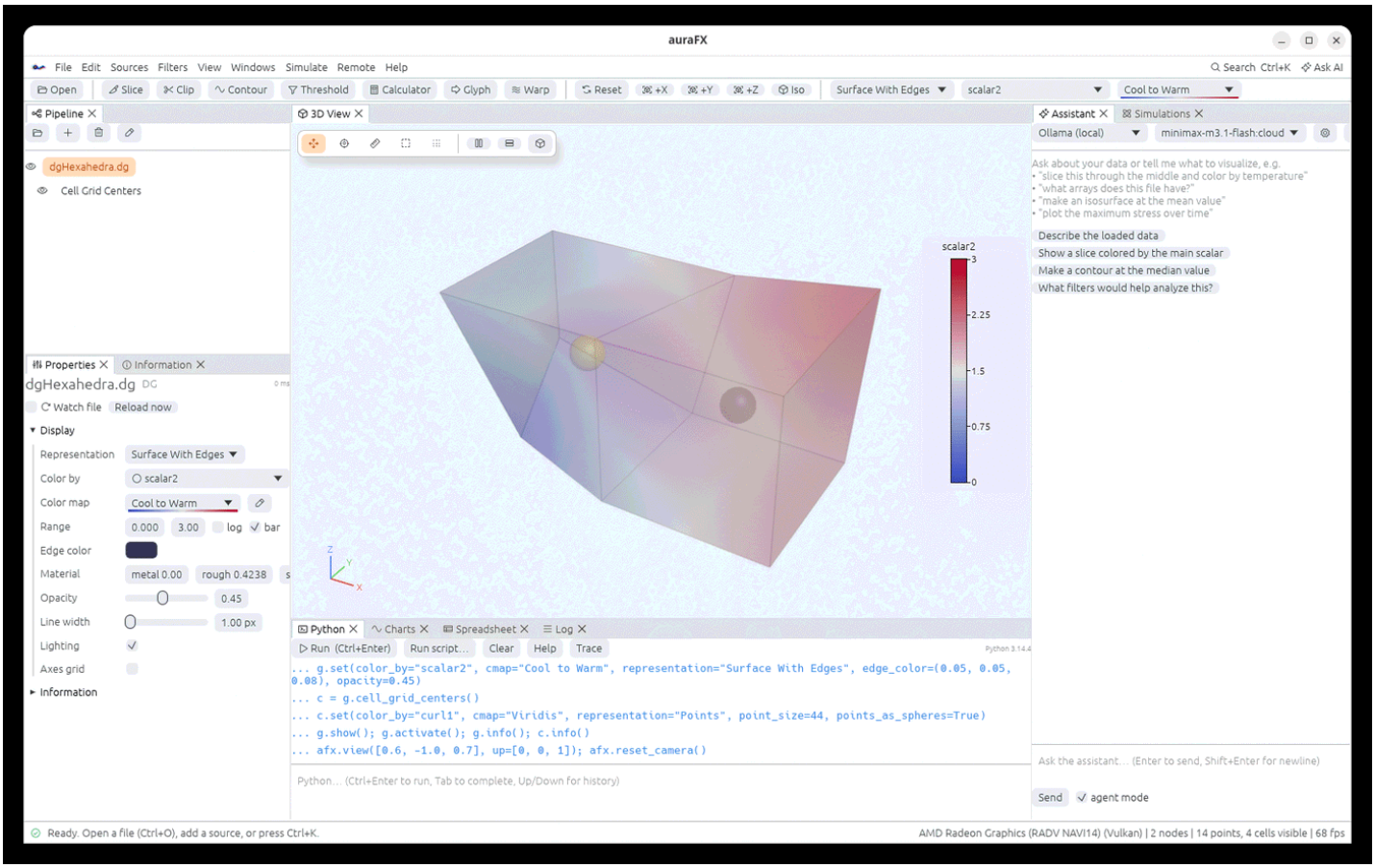
- The grid is drawn as its cells with every field evaluated at the corners. Quadratic cells are drawn as linear cells through their corners. Where cells meet, a field that jumps between them is averaged. Color by, the Spreadsheet, picking and every other filter see that mesh.
- **Information** names the cell types and the number of fields (and, after Compute Sides, the number of sides shown).
- Saving as `.dg` keeps the cell grid (every array, attribute and cell type, in the layout VTK writes). Saving as any other format writes the corner mesh as an unstructured grid.

Source (Sources › Cell Grid): one cell of a chosen **Cell type** with test fields: `hgrad` (1, 0, 0 at the first corner, 0 elsewhere), and on hexahedra, tetrahedra, wedges, quadrilaterals and triangles `hcurl` and `hdiv` (an edge and a face field whose first coefficient is 1). A vertex gets a `constant` field.

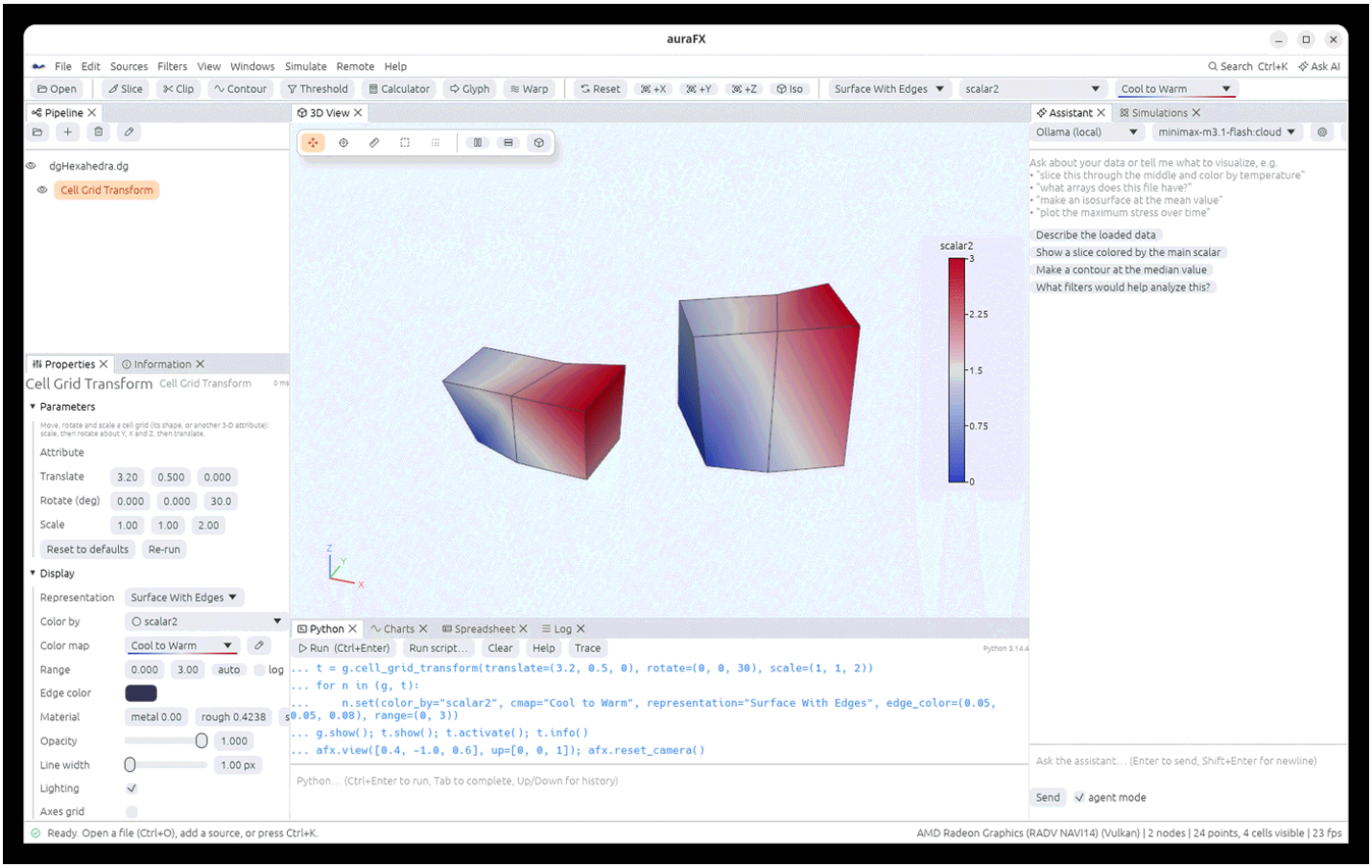
Filters (Filters › Data):

Filter	What it does
Cell Grid To Unstructured Grid	the cells with the fields at their corners (what is drawn); corners at the same place become one point
Cell Grid Centers	a point at the center of every cell with every field evaluated there (exact, not averaged)
Cell Grid Elevation	adds a field: the distance from **Origin** along **Axis** (or, with Spherical distance, from Origin)
Cell Grid Transform	moves, rotates (degrees) and scales the cells, as **Transform** does, or another 3-D attribute shared at points or stored per cell. Edge and face fields follow the new shape
Cell Grid Warp	moves the cells' points by **Scale** times a vector attribute shared at the points (the same basis and order as the shape)
Cell Grid Probe	finds the cells holding the points of a second dataset (the **Probe points**): a point for every cell each one is in, with `ContainingCellID` (the cell's number within its type), `CellType` (the VTK cell type number), `ParametricCoordinates` (where in the cell, in the cell's own coordinates) and, when **Attribute** names a field, the field's exact value there. Points in no cell are left out. A point on a face shared by two cells is listed twice
Cell Grid Compute Sides	finds the sides of the cells and draws them instead of the 3-D cells: by default the faces on the outside (those not shared by two cells).

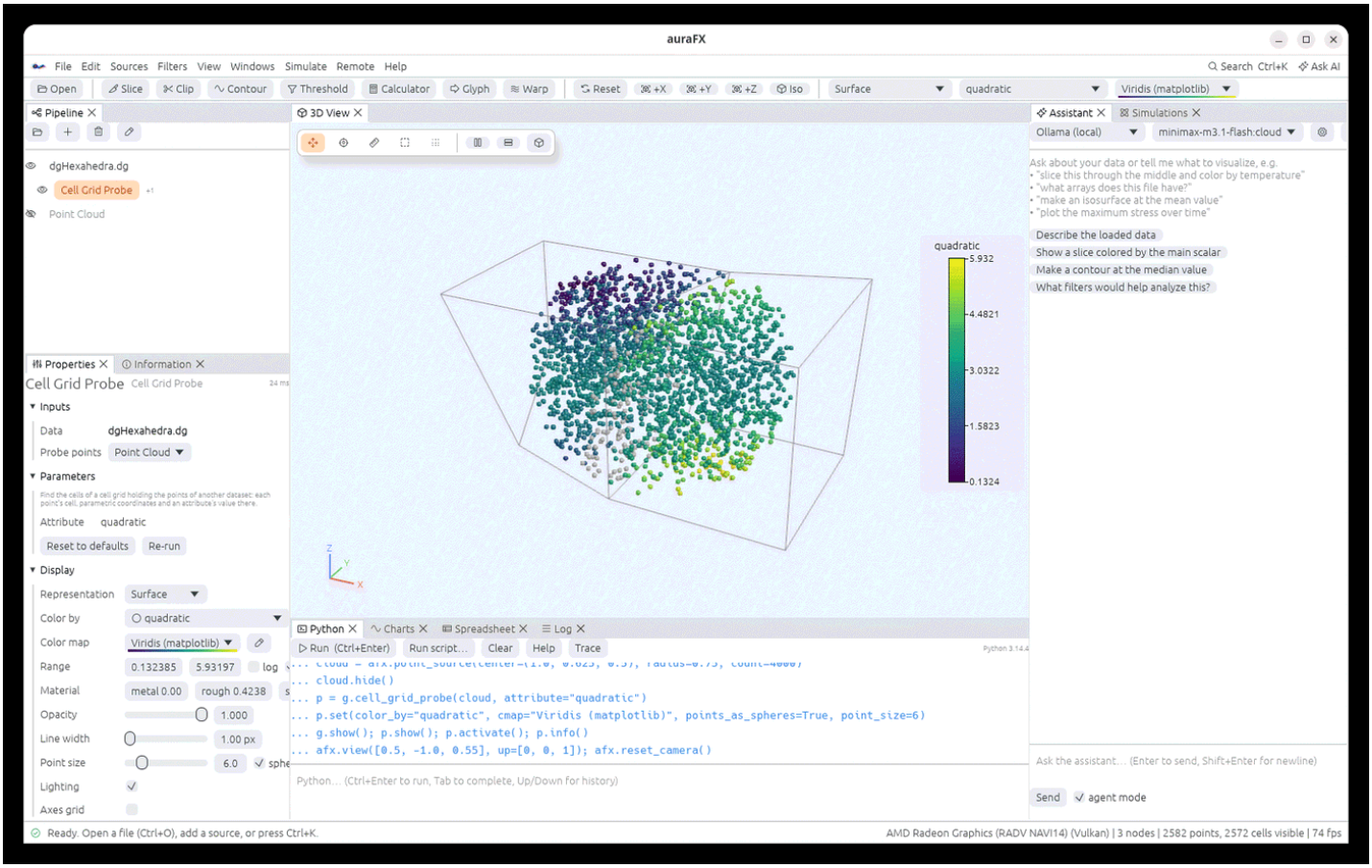
Filter	What it does
	Sides to include picks faces, edges or corners; see below
Convert To Cell Grid	makes a cell grid from a mesh of vertices, lines, triangles, quads, tetrahedra, hexahedra, wedges and pyramids (other cells are skipped): point data becomes fields shared at the points, cell data constant fields (renamed “(cell)” when point data has the same name)




```
g = afx.load("dgHexahedra.dg")
g.set(color_by="scalar2", representation="Surface With Edges")
centers = g.cell_grid_centers()
centers.set(color_by="curl1", points_as_spheres=True)
height = g.cell_grid_elevation(axis=(0, 0, 1))
cell = afx.cell_grid(cell_type="Tetrahedron")
mesh = g.cell_grid_to_unstructured_grid()
turned = g.cell_grid_transform(rotate=(0, 0, 30), scale=(1, 1, 2))
grid = afx.load("mesh.vtu").convert_to_cell_grid()
grid.save("mesh.dg")
cloud = afx.point_source(center=(1, 0.6, 0.5), radius=0.75, count=4000)
found = g.cell_grid_probe(cloud, attribute="quadratic")
outside = g.cell_grid_compute_sides()
edges = g.cell_grid_compute_sides(include="Edges of inputs",
omit_renderable=False)
```



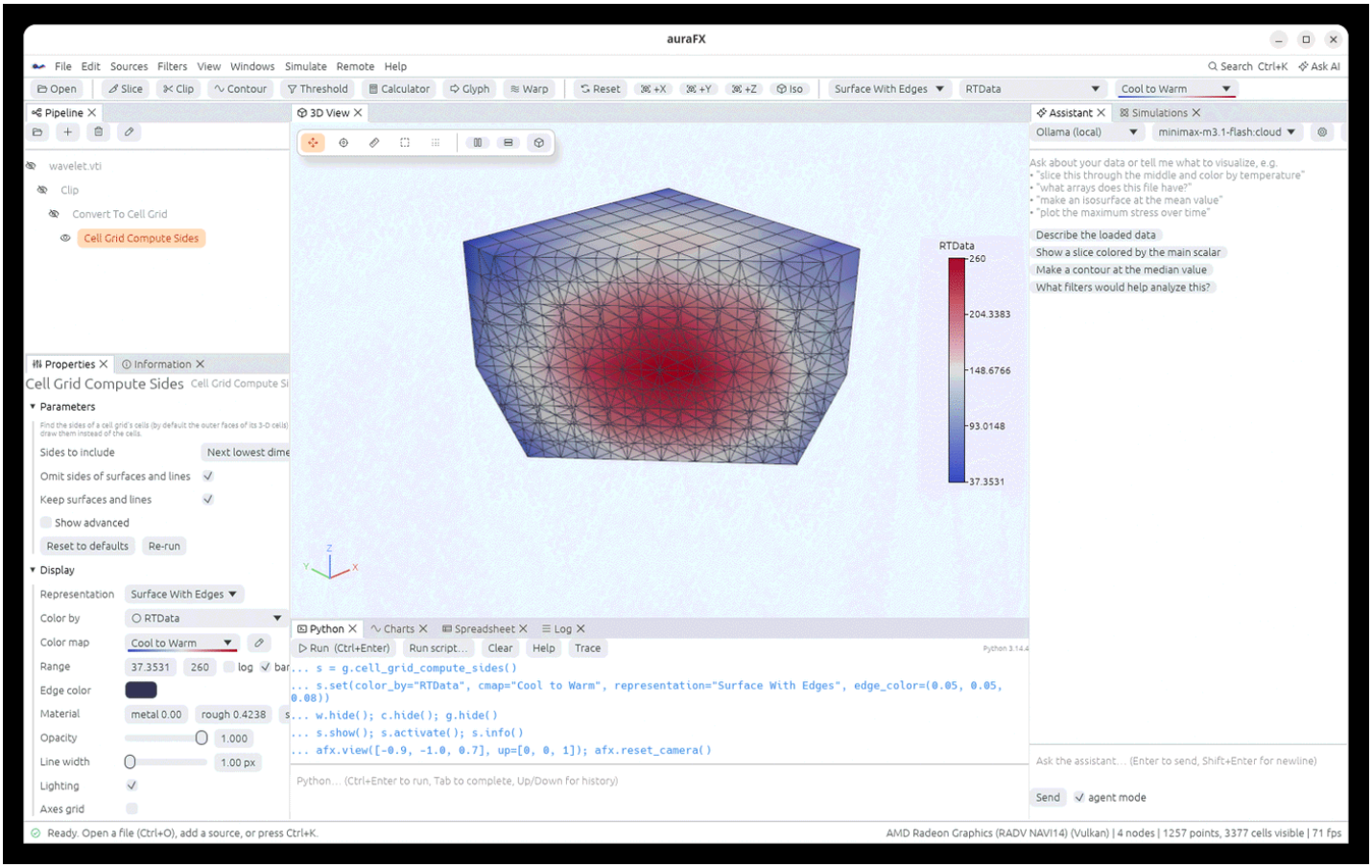
Cell Grid Probe looks up points in the cells. Select the cell grid, Ctrl+click the dataset whose points you want to look up (any dataset: its points are used), then choose Filters › Data › Cell Grid Probe. In Properties, **Probe points** picks the second input and **Attribute** the field to sample (empty: only find the cells). A cell is searched with a quick test first: the point must lie behind each of the cell's faces, taken as flat planes through the corners (on a flat cell, the edges' directions stand in for the normals). Then the point's coordinates in the cell are found step by step from the cell's middle. On a cell with curved faces a point just outside can pass the quick test; it is kept, with empty (NaN) coordinates and values. Points on an edge cell are kept when they lie between the two end planes and within the sphere around the edge, and are then kept only if they lie on the edge. The output points are in single precision, and with several cell types they are listed in the file's order. The grey points in the picture are such points, next to the curved face the two cells share.



Cell Grid Compute Sides finds the sides of the cells. Every side of every cell is listed, by cell and then by side, and a side is matched with the same side of a neighbor cell whichever way round its corners go. A side is kept once, for the first cell that has it. Properties:

- **Sides to include**:
  - *Next lowest dimension* (the default): the faces of 3-D cells, the edges of 2-D cells and the end points of edges;
  - *Surfaces of inputs*: the faces of 3-D cells;
  - *Edges of inputs*: the edges of 3-D and 2-D cells;
  - *Vertices of inputs*: the corners of every cell;
  - *All sides*: all of these.
- **Omit sides of surfaces and lines** (on by default): 2-D and 1-D cells give no sides, since they can be drawn as they are.
- **Keep surfaces and lines** (on by default): keep drawing the 2-D and 1-D cells and the sides already listed. 3-D cells are always hidden behind their sides.
- **Keep** (advanced):
  - *Boundary* (the default): faces of 3-D cells (or edges of 2-D cells, end points of edges) found once, so not shared by two cells, and every lower side (edges and corners of 3-D cells) once;
  - *Odd count*: every side found an odd number of times;
  - *Every side*: each side once.

The sides are stored in the grid as lists of (cell, side number) pairs, one per side shape (“quadrilateral sides of vtkDGHex”), as VTK stores them. The fields on a side are the fields of its cell, so a field that jumps between cells shows the value from the cell the side belongs to. Running Compute Sides on its result adds the sides of those sides, for example the edges of the outer faces. Saving as `.dg` keeps the lists. Any filter other than the cell grid ones works on what is drawn: the sides. VTK’s markings of ghost and hidden points (which skip sides) are not read.



On quadrilaterals and triangles the cell’s shape matrix (its Jacobian) is flat, so it is completed with the cell’s unit normal; edge and face fields then lie in the cell’s plane, and Cell Grid Probe gives coordinates on flat cells and edges.

Verified against a reference implementation; the comparisons are listed in the verification notes (`docs/reference/verification.md` in the source tree).

Not available yet: cell grid range and copy filters. Slicing or clipping works on the corner mesh, so curved cells come out straight.

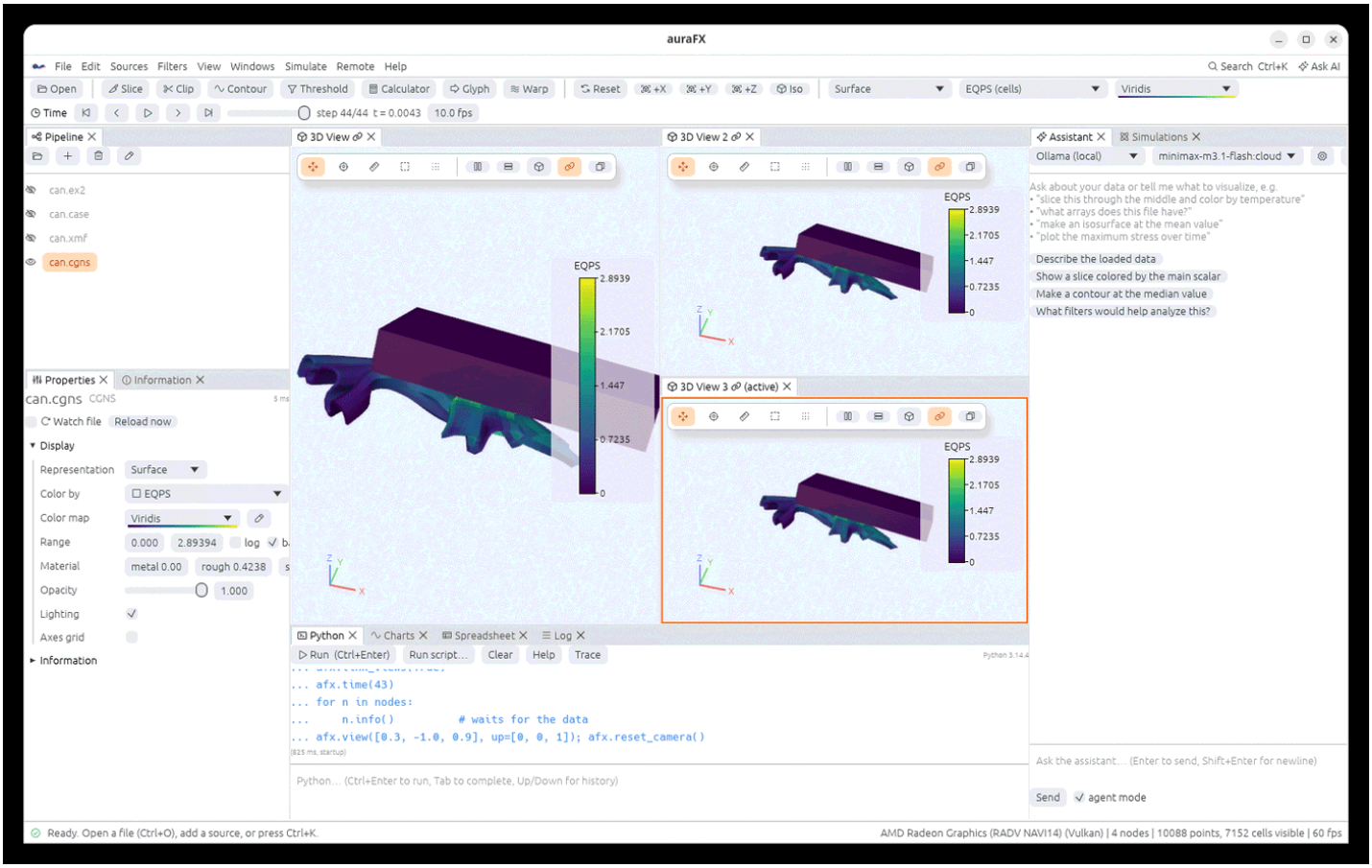
Writers

Format	Extensions	Notes
VTK XML	`.vtu .vtp .vti .vtr .vts .vtm`	appended binary; multiblocks as a folder of pieces
VTK hyper tree grid	`.htg`	XML version 2: the trees, mask and cell data
VTK cell grid	`.dg`	the JSON layout VTK writes: arrays, attributes and cell types
VTK legacy	`.vtk`	
VTKHDF	`.vtkhdf`	all dataset kinds; multiblocks as a collection; time series with `all_times=True`; written with auraFX's own HDF5 encoder (read by h5py and VTK)
Exodus II	`.exo`	blocks, nodal and element variables, every time step with `all_times=True`; HEX20 and HEX27 elements; the element blocks, side sets and node sets of generated meshes (see Geometry and meshing)
Abaqus input	`.inp`	nodes, elements (C3D8, C3D20, C3D27, tetrahedra, wedges, pyramids) in one element set per block, node sets, side sets as element-based surfaces
Gmsh	`.msh`	version 4.1 ASCII: one physical volume per element block, one physical surface of quadrilaterals per side set (node sets are not part of the format)
Fluent	`.cas` (or `name.fluent.msh`)	ASCII: nodes, a cell zone per block, interior faces, a boundary zone per side set and the other outer faces in `wall`, zone names; 2-D or 3-D cells of one dimension
Nastran bulk data	`.bdf .nas`	free field with full precision: GRID, CHEXA/CTETRA/CPENTA/CPYRAM (with mid-side nodes)/CTRIA3/CQUAD4/CBAR, a PSOLID/PSHELL/PBAR per block (named in a comment), node sets as SPC1 cards
MetaImage	`.mha .mhd`	image data's active array (point data, else cell data): all number types, channels, origin, spacing, orientation

Format	Extensions	Notes
Image stacks	`.png` , `.tif`	a volume as one 8- or 16-bit PNG per slice (`name_0000.png` …; other values scaled to 16 bits), or one multi-page TIFF (also 32-bit floats)
Parallel VTK	`.pvtu` `.pvtp` `.pvti` `.pvts` `.pvtr`	the header and a folder of piece files: the blocks of a partitioned dataset, or 4 pieces cut from one dataset (`afx.save` takes `pieces=`)
Time series collection	`.pvd`	with `all_times=True` , one VTK XML file per time step in a folder and the `.pvd` listing them with their times
EnSight Gold	`.case`	C binary; one part per block (image, rectilinear and structured grids as blocks, the rest as elements of every standard type, polygons and polyhedra); every time step with `all_times=True` , the geometry written once when it does not change
CGNS	`.cgns`	HDF5 storage (CGNS 4): a zone per block, image/rectilinear/structured grids as structured zones, the rest as one element section (MIXED when the cell types differ) plus NGON/NFACE for polygons and polyhedra; vectors as `NameX/NameY/NameZ` ; with `all_times=True` one solution per step (and coordinates per step when the points move), but the cells must stay the same
Xdmf 3	`.xmf` + `.h5`	the description in the `.xmf` , the arrays in an HDF5 file beside it; block trees as spatial collections, time steps as a temporal collection that shares unchanged points and cells
Grain orientations	`.tex`	four header lines (the fourth `B N`), then one line per grain id from 1: φ1 Φ φ2 in degrees and a weight of 1 (meshes with grains from Assign Grains or Overlay Mesh); grains without elements get 0 0 0
Microstructure files	`.dream3d`	volumes (an image geometry), point sets and meshes of one element type (vertex, edge, triangle, quad, tetrahedral, hexahedral geometries) with their cell, vertex, feature

Format	Extensions	Notes
		and phase arrays; multiblocks as one data container per block; the pipeline that made the data is recorded in the file (see Grains and crystals)
SPPARKS sites	`.spparks`	the `FeatureIds` of a volume as the `Values` of a simple-cubic lattice filling the box, x fastest
CSV, STL, PLY		a multiblock of surfaces saved as `name.stl` writes one file per block, `name_<block>.stl` (Feature Surfaces gives one per grain)
Scenes	`.glb` `.obj` `.x3d` `.wrl`	see Exporting
Pictures	`.png` `.svg`, GIF/MP4 animations	see Exporting

Files written by the VTK XML, VTKHDF, Exodus, EnSight, Xdmf and CGNS writers were read back with the same points, cells and values (verified against a reference implementation; the comparisons are listed in the verification notes, `docs/reference/verification.md` in the source tree). EnSight groups the cells of each part by type, and CGNS puts polyhedra after the other cells, so cells can come back in a different order (each keeps its values). Cell types a format does not have (Lagrange and Bezier cells, for example) are written by their corner points. A CGNS zone holds cells of one dimension: in a volume mesh, boundary faces and lines are left out.



Curved cells

VTK's Lagrange and Bezier cells of any order, quadratic cells and serendipity cells are read with their node numbering, drawn curved and cut by the filters along their curvature; see Volumes, point clouds and curved cells.

Your own formats

A plugin reader is a Python function with `@afx.reader(extensions=[...])`; it takes precedence over the built-in reader for its extensions and appears in the Open dialog.

Exporting pictures, scenes and animations

Screenshots

File › Save Screenshot… (Ctrl+Shift+S) opens a small dialog:

- **Size**: the view's size, a multiple of it (2×, 4×), 1920×1080, 3840×2160, or any size; **keep the view's shape** ties width and height together. Point sizes and line widths are in pixels, so they look thinner in a larger picture.
- **Background › transparent** saves a PNG with an alpha channel, with clean edges.
- **Annotations** includes the legend, axes-grid labels, time label, text annotations and the orientation axes, laid out as on screen and scaled with the picture.
- **PNG** or **SVG**: an SVG holds the surfaces as shaded polygons sorted back to front, the lines and points, and the annotations as real text, for figures that scale without pixels (where surfaces cut through each other the boundary is jagged; convert to PDF with Inkscape if needed).

With ray tracing on, the picture is traced at the requested size first.

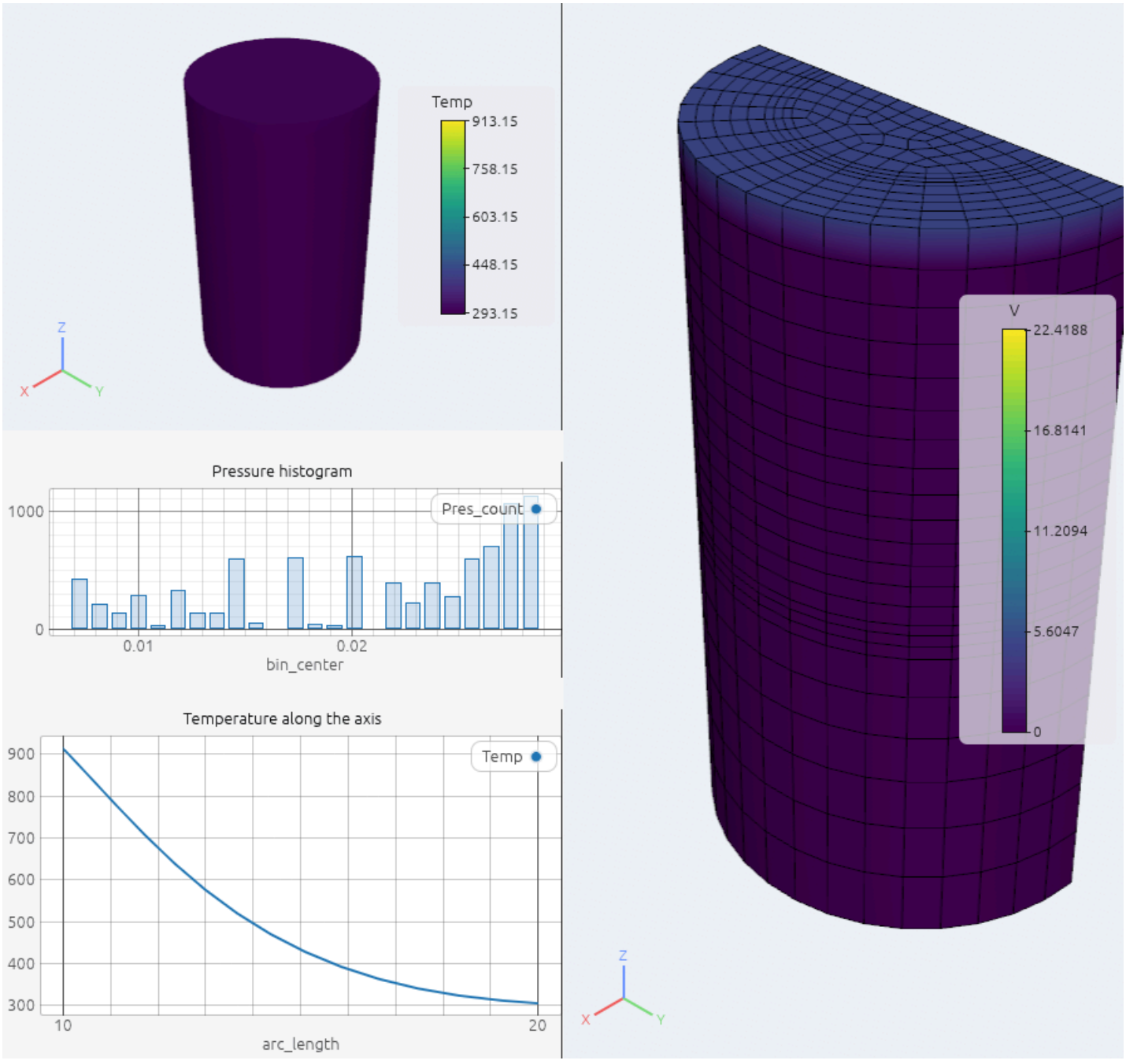
```
afx.screenshot("fig.png")                                  # the view as shown
afx.screenshot("fig.png", width=3840, transparent=True) # keeps the view's shape
afx.screenshot("fig.png", width=1600, height=900, annotations=False)
afx.screenshot("fig.svg")
```

`afx.screenshot` returns only when the file is written, so a script can change the camera or the data between pictures.

All views in one picture

For a slide or a report with several views side by side, choose **Picture of › all views** in the same dialog: every visible 3D view and chart view is saved in one PNG, each where it sits in the window, with tab bars, panels and menus left out and the charts' controls hidden (**keep the chart views' controls** keeps them). **chart views** saves only the charts. The background between the views is the window's, or transparent. The picture has the window's pixels, so a larger (or full-screen) window gives a larger picture. Only views whose tab is in front are in the picture.

```
afx.screenshot("slide.png", views="all")            # every visible view, as laid
out
afx.screenshot("charts.png", views="charts")
afx.screenshot("chart2.png", views="chart 2")       # one chart view ("chart" is
the Charts panel)
afx.screenshot("view2.png", views="view 2", transparent=True)
```



Scenes for other programs

File › Export Scene... (`afx.export_scene("scene.glb")`) writes the visible objects, with their colors, as:

Format	Notes
glTF binary (`.glb`)	for Blender, three.js, PowerPoint, Windows 3D Viewer; vertex colors, normals, translucency, materials (metallic, roughness)
OBJ (`.obj`)	with vertex colors after the coordinates (Blender and MeshLab read them)
X3D (`.x3d`), **VRML** (`.wrl`)	absolute coordinates, vertex colors

Volumes are not exported (they are not geometry).

Animations

File › Animation… records the time steps, a turntable or the keyframes as **GIF**, **MP4** (`ffmpeg` on the PATH) or a folder of PNG frames, of the active 3D view or of all views as laid out (`afx.save_animation("all.mp4", views="all")`); see Time, charts and animation.

Saving data

Right-click an object › **Save data…**, or `node.save("out.vtu")` , writes its output; the format follows the extension:

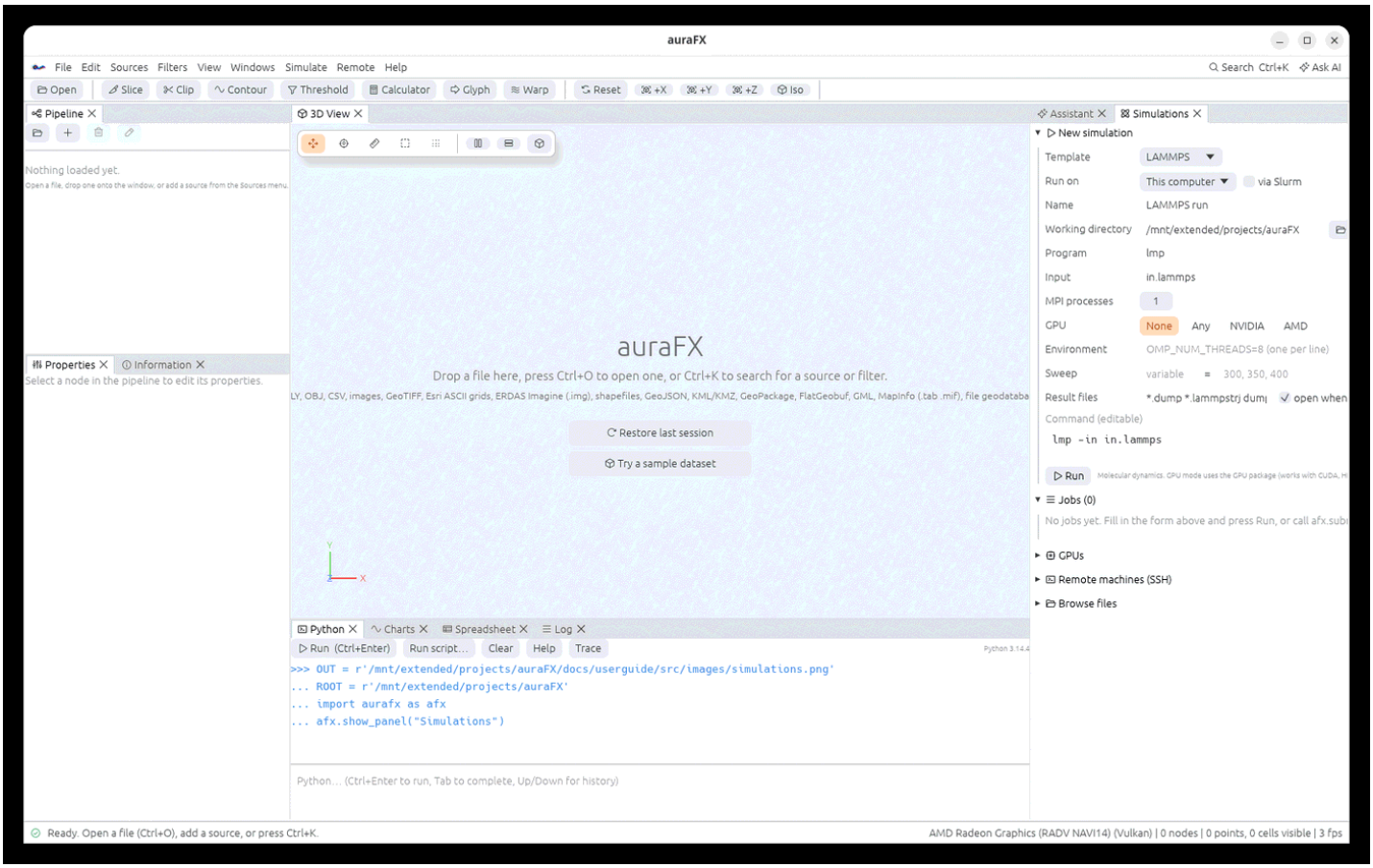
Extension	Format
`.vtu` `.vtp` `.vti` `.vtr` `.vts` `.vtm`	VTK XML (appended binary); `.vtm` writes a folder of pieces
`.vtk`	legacy VTK
`.vtkhdf`	VTKHDF (HDF5); multiblocks as a collection with the block tree; `all_times=True` writes every time step into one file
`.exo`	Exodus II with blocks and variables; `all_times=True` writes every time step
`.case`	EnSight Gold (C binary): the case file plus a geometry file and one file per variable next to it; `all_times=True` writes every time step
`.xmf`	Xdmf 3: the description plus an `.h5` file with the arrays; `all_times=True` writes every time step
`.cgns`	CGNS (HDF5); `all_times=True` writes every time step when the cells stay the same (moving points are fine)

Extension	Format
`.csv`	the point data or the table
`.stl .ply`	surfaces

```
d.save("slice.vtp")
d.save("run_all.exo", all_times=True)
d.save("run_all.vtkhdf", all_times=True)
d.save("run.case", all_times=True)      # run.case, run.geo, run.Temp0000, ...
d.save("run.xmf", all_times=True)       # run.xmf and run.h5
d.save("run.cgns", all_times=True)
```

When the data has several time steps and the chosen format holds them all (Exodus, VTKHDF, EnSight, Xdmf, CGNS), the Save dialog writes every step.

Running simulations



The **Simulations** panel (right side; **Simulate › Run a simulation…**, or **Windows › Simulations**) launches solvers and opens their results when they finish.

A new simulation

Field	Meaning
Template	LAMMPS, MOOSE app, MOOSE polycrystal, SPPARKS grain growth, PINNACLE crystal plasticity, DAMASK, FFT texture file, LAMMPS polycrystal, OpenFOAM, Python pipeline or Custom command; fills in the program, input and result-file patterns (the polycrystal and grain growth templates also write their input files, see below)
Run on	this computer, or a remote machine from the SSH list; **via Slurm** submits with `sbatch` there (partition, time and extra `#SBATCH` lines appear)

Field	Meaning
Name, **Working directory**	where the run lives; a remote directory when running over SSH
Program, **Input**, **MPI processes**	the command line is assembled from these and shown below, editable
GPU	None, Any, NVIDIA or AMD; the panel finds the GPUs on the host and sets the solver's flags (LAMMPS GPU package, `CUDA_VISIBLE_DEVICES` / `HIP_VISIBLE_DEVICES`)
Environment	extra `VAR=value` lines (one per line)
Sweep	a variable name and a list of values: one job per value, `{var}` in the command replaced, each in its own folder
Result files	patterns of the files to open when the run ends (**open when done**)

Run starts the job. Jobs appear in the **Jobs** list with their state, a live log tail, and buttons to cancel, open the results or the folder, and resubmit. Files a running solver writes can be opened while it runs; auraFX reloads them as they change (live reload).

Simulation cases from the pipeline

Six templates write their input files from the data selected in the pipeline, then run the solver:

Template	Selected data	Files written into the working directory
MOOSE polycrystal	a grain mesh (from Assign Grains, or Overlay Mesh of a microstructure)	`mesh.e` (one block per grain, block id = grain id), `grain_euler.txt` (each block's Euler angles, read with `PropertyReadFile`), `grains.tex` (the same orientations for `EulerAngleFileReader`), `slip_systems.txt`, `polycrystal.i`
SPPARKS grain growth	a microstructure volume with `FeatureIds`	`grains.spparks` (the site file), `in.potts` (a Potts-model run that reads it)
PINNACLE crystal plasticity	a grain mesh	`mesh.e`, `orientations_points.csv` (each element's centre with its grain and Euler angles), `case.i`
DAMASK crystal plasticity (grid)	a microstructure volume with `FeatureIds` and orientations	`grid.vti` (cell array `material` = grain id − 1), `material.yaml` (the phase from the material table, then one material per grain with its mean orientation as a quaternion),

Template	Selected data	Files written into the working directory
		`load.yaml` (a strain rate along the axis, the other normal stresses free)
FFT texture file	a microstructure volume	`micro.txt`: one line per voxel, x fastest: φ1 Φ φ2 (Bunge, degrees), x y z (voxel indices from 1), grain id, phase id; `micro_README.txt` with the grid size
LAMMPS polycrystal	a microstructure volume	`atoms.data` (every grain filled with the material's FCC or BCC lattice turned to the grain's orientation; of two atoms closer than half the nearest-neighbour distance across a boundary one is removed; periodic box), `atom_grains.txt` and `atoms.dump` (each atom's grain), `in.lammps` (relax, then heat and hold)

Choose the template, select the data in the pipeline (the panel shows it under **Case from**), and set the case:

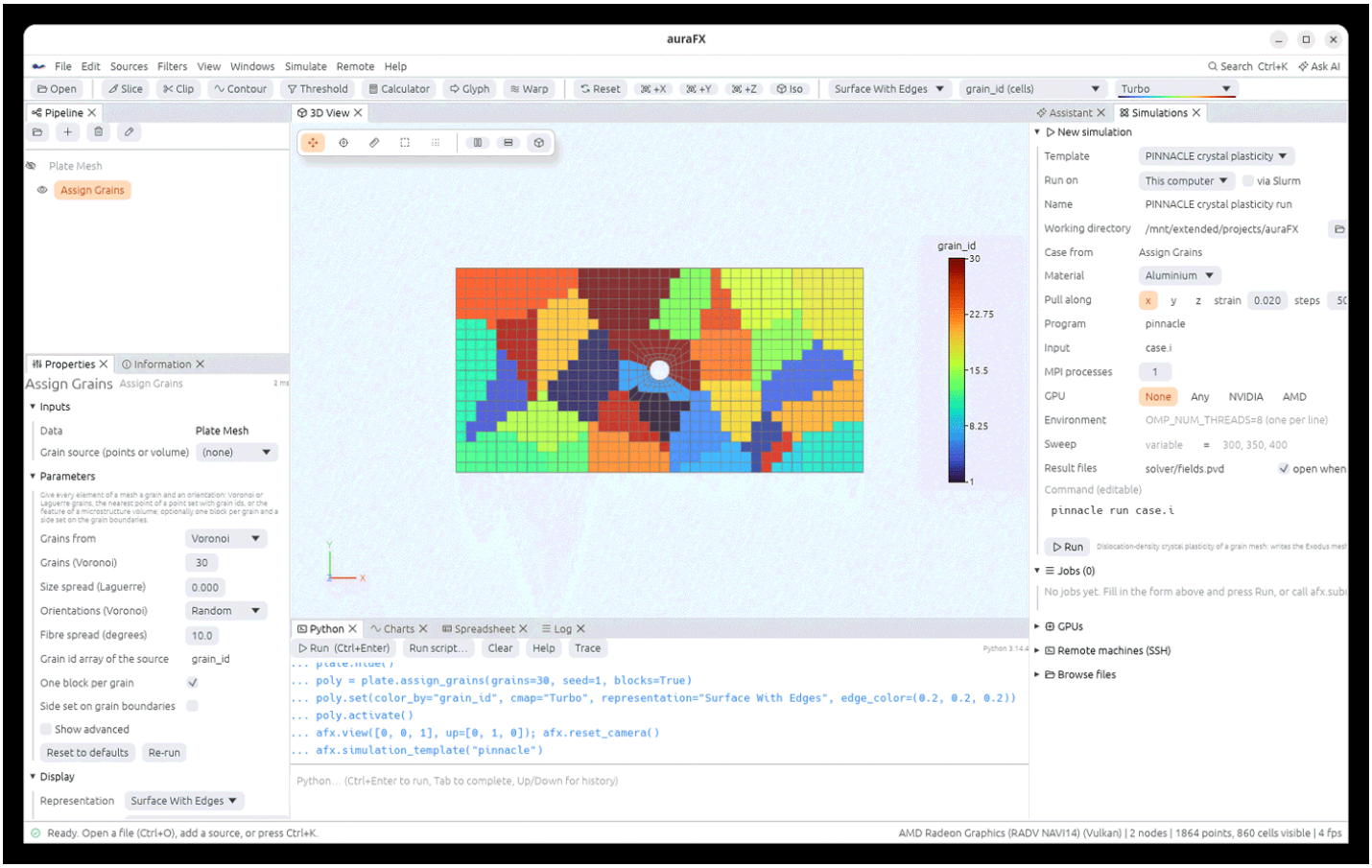
- **Material**: the elastic constants, shear modulus and Burgers vector come from the material table of Crystal Mechanics. Only cubic materials (FCC, BCC) are offered.
- **Pull along**, **strain** and **steps**: the faces named `x-` and `x+` (or `y-` / `y+`, `z-` / `z+`) are held and pulled. The faces of the other two directions are held on their lower side when the mesh names them. Box Mesh, Plate Mesh, Dog-bone Mesh and the Geometry source name their faces this way.
- For SPPARKS: **Monte Carlo steps** and **temperature**.
- For LAMMPS: the lattice constant comes from the material's Burgers vector (a = b√2 for FCC, 2b/√3 for BCC; 4.05 Å for aluminium), and the volume's longest side holds about 40 lattice cells unless `angstrom_per_unit` says how many ångström one unit of the volume is. The potential file named in `in.lammps` (`Al.eam.alloy` and so on) is yours to provide.

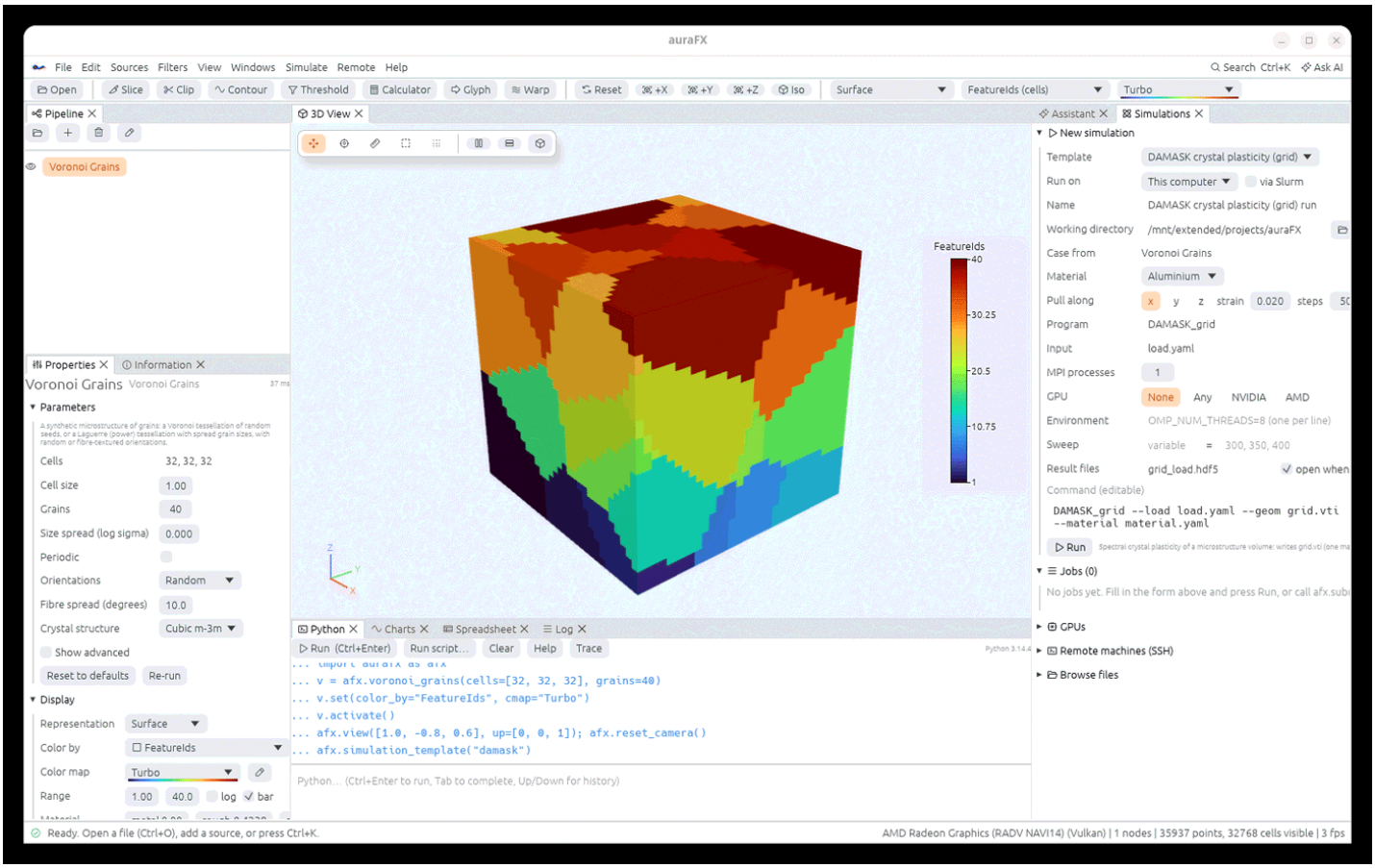
A DAMASK case written this way was checked with DAMASK's own Python package: it loads the grid and the load case, finds the material file complete and valid, and its own conversion of the grains' Euler angles gives the same quaternions.

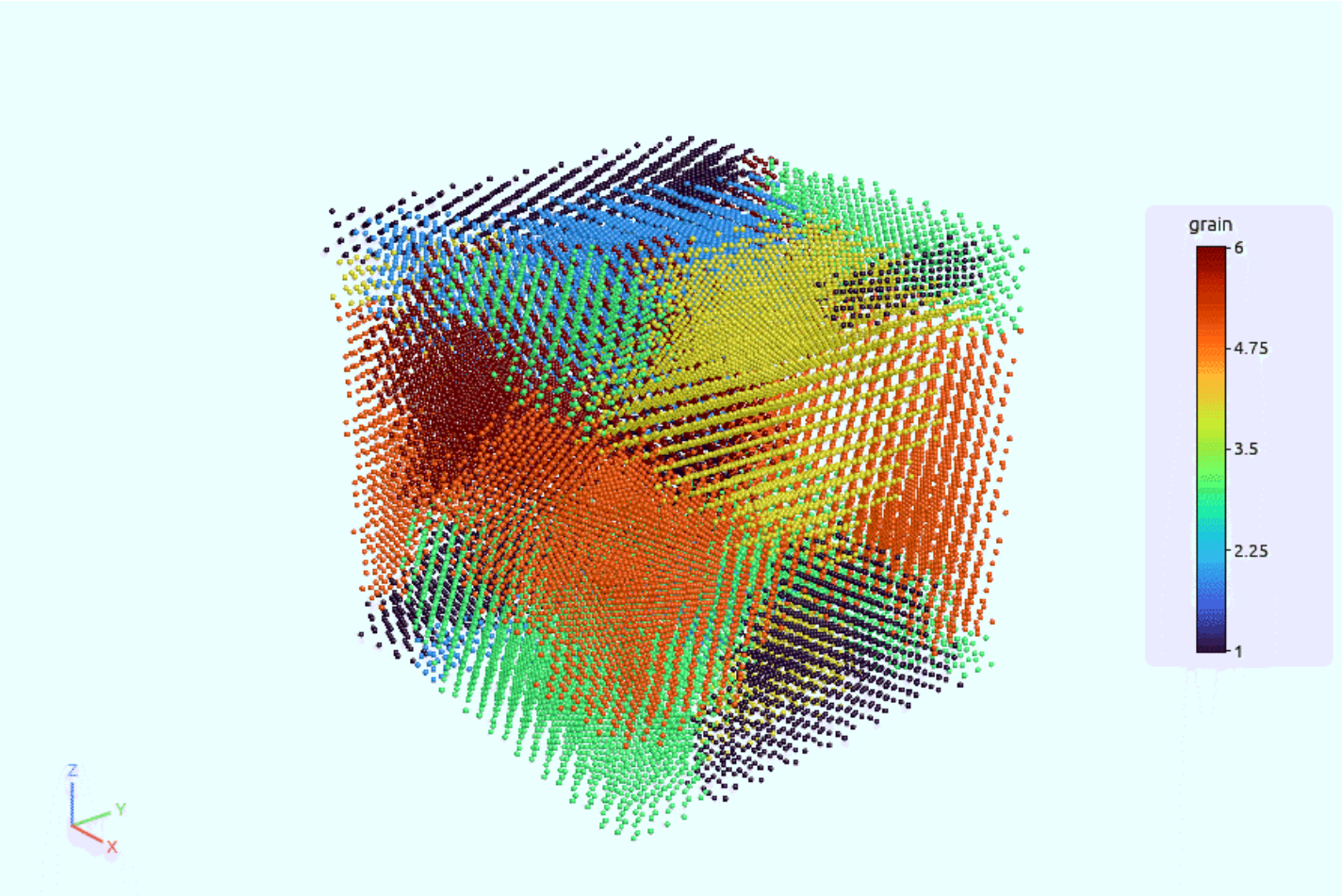
Run writes the files and starts the job. For a remote machine they are written to a local folder and uploaded to the working directory there. With **open when done**, the results open when the run ends, named after the run:

- MOOSE: `polycrystal_out.e`, with one block per grain.
- SPPARKS: `dump.potts`, each snapshot a volume with `FeatureIds`.
- PINNACLE: `solver/fields.pvd`, the field snapshots as a time series.
- DAMASK: `grid_load.hdf5` (DAMASK's result file; post-process it with DAMASK's tools).

- LAMMPS: `dump.lammpstrj`, the atoms over time.







The input files are a starting point. Edit them for the material's hardening and slip parameters, the boundary conditions and the solver settings before production runs.

```
poly = afx.plate_mesh().assign_grains(grains=30, blocks=True)
case = afx.simulation_case(poly, "moose", "~/runs/plate", material="Copper",
axis="x",
                           strain=0.02, steps=50)        # only write the files
print(case["command"], case["files"])
jid = afx.run_case(poly, "pinnacle", "/data/runs/plate", strain=0.02, steps=40,
wait=True)
jid = afx.run_case(volume, "spparks", "~/gg", host="cluster", np=16, mc_steps=500)
v = afx.voronoi_grains(cells=[64, 64, 64], grains=200, periodic=True)
afx.simulation_case(v, "damask", "~/runs/damask", material="Copper", strain=0.05,
steps=100)
afx.simulation_case(v, "fft", "~/runs/fft")
afx.simulation_case(v, "lammps", "~/runs/md", material="Aluminium",
angstrom_per_unit=5)
```

`run_case` takes `host`, `exe` (the solver program), `np`, `slurm`, `auto_load` and `wait` as `afx.submit` does, plus the case options: `material`, `axis`, `strain`, `steps`, `end_time`, `fixed` and `loaded` (other set names), `mc_steps`, `temperature`, `dump_every`, `seed`, and for LAMMPS `lattice_constant` (Å) and `angstrom_per_unit`.

Grains on the results. MOOSE keeps the grain blocks. A PINNACLE result has no grain array, but every element sits on its own point of `orientations_points.csv`, so Assign Grains (nearest point) gives the grains back exactly. Statistics per grain then come from the arrays:

```
res = afx.load("/data/runs/plate/solver/fields.pvd")
g = res.assign_grains(afx.load("/data/runs/plate/orientations_points.csv"),
method="Nearest point")
import numpy as np
gid, sxx = g.array("grain_id", "cells"), g.array("stress", "cells")[:, 0]
for k in np.unique(gid):
    print(k, sxx[gid == k].mean())
```

Limits:

- Every element and site needs a grain: no grain id 0. Fill Bad Data fills unindexed cells of a volume.
- The crystal-plasticity cases cover cubic crystals (FCC and BCC).
- SPPARKS sites lie on a simple-cubic lattice of the volume's cells. The lattice constant is the cell size when the cells are cubes, else 1.

Remote machines

Remote machines (SSH) holds the hosts: a name, the SSH target (`user@machine`, or an alias from `~/.ssh/config`), port, key file, jump host, and **setup lines** run before every command (module loads, `source` of an environment, `AURAFX_SERVER=...`). SSH keys or an agent must be set up so that `ssh host` works without a password prompt. The same hosts serve **File › Open on Server…** (see Data on another machine).

GPUs lists the GPUs of each host; **Browse files** opens a remote file browser to pick inputs or results.

From Python

```
jid = afx.submit("lmp -sf gpu -pk gpu 1 -in in.melt", host="rocm-server",
                 workdir="~/runs/melt", gpu="amd:0", outputs="dump.melt",
auto_load=True)
afx.sweep("lmp -var T {T} -in in.melt", T=[300, 600, 900], host="cluster",
workdir="~/sweep")
afx.jobs(); afx.job(jid); afx.cancel(jid)
afx.gpus("rocm-server"); afx.hosts()
afx.load_remote("rocm-server", "~/runs/melt/dump.melt")   # copy a file here and
open it
```

`gpu` is `None`, `"any"`, `"nvidia"`, `"amd"` or e.g. `"nvidia:0,1"`. `afx.job(id)` returns the state, exit code, log tail and output files.

Data on another machine

When the data lives on a cluster or a GPU box, auraFX can read and filter it **there** and bring back only what you look at, with nothing to set up beyond SSH.

How it works

`aurafx-server` is a small program without a window. The app starts it on the host over SSH (`ssh host aurafx-server --stdio`) and talks to it through that connection, so keys, jump hosts and `~/.ssh/config` work as usual and no port is opened. Files you open on the server are read there; filters you add below them (Slice, Clip, Contour, Threshold, Stream Tracer, temporal filters and so on) run there too. What comes back:

Situation	What is transferred
Visible, output smaller than 64 MB	the whole output
Visible, larger	its outer surface (all arrays), which is what the 3D view draws
Hidden (for example a reader with a slice below it)	only its summary: arrays, ranges, bounds, time steps
Bring all data ticked in Properties, or a filter below it that runs here	the whole output

The Information panel always describes the full data on the server. Python filters (Programmable Filter, Python Calculator) and plugin filters run on this computer, on the data brought over. The server keeps results, so changing one filter only re-runs that filter there.

Setting up a host

Everything is in one window: **Remote › Open data on another machine…** (also **File › Open on Server…**). It walks through four steps:

1. **The machine.** Pick it, or open **Add a machine** and type its SSH target (`user@machine` or an alias from `~/.ssh/config` ; the aliases found there are offered as buttons), with a port, key file or jump host if needed. `ssh machine` must work without a password prompt. Machines added here are also the Simulations panel's, and the other way round. "This computer" runs the server locally, which is handy for trying it out.

2. **aurafx-server on it. Check server** shows its version and GPU; **Install server** copies the one shipped with the app to `~/.local/bin` (Linux hosts of the same architecture; the packages run on glibc 2.35 or newer). You can also build it there with `cargo build --release -p aurafx-server`, or point `AURAFX_SERVER` at it in the host's setup lines.
3. **Where the picture is drawn.** *Auto* brings the geometry here unless even its surface is too big to send; *On the server* renders it with the server's GPU and sends only pictures (the 3D view on this computer shows them, merged with local objects by depth; picking does not work on them); *On this computer* always brings the geometry. Each object keeps the choice, and **Properties › Display › Draw on** changes it later.
4. **Open a file.** Browse to it and double-click, or select it and press **Open on …**.

Opening files

A cloud icon marks objects that run on a server; hover it, or look at the top of Properties, to see what was transferred and how long the server took. From Python:

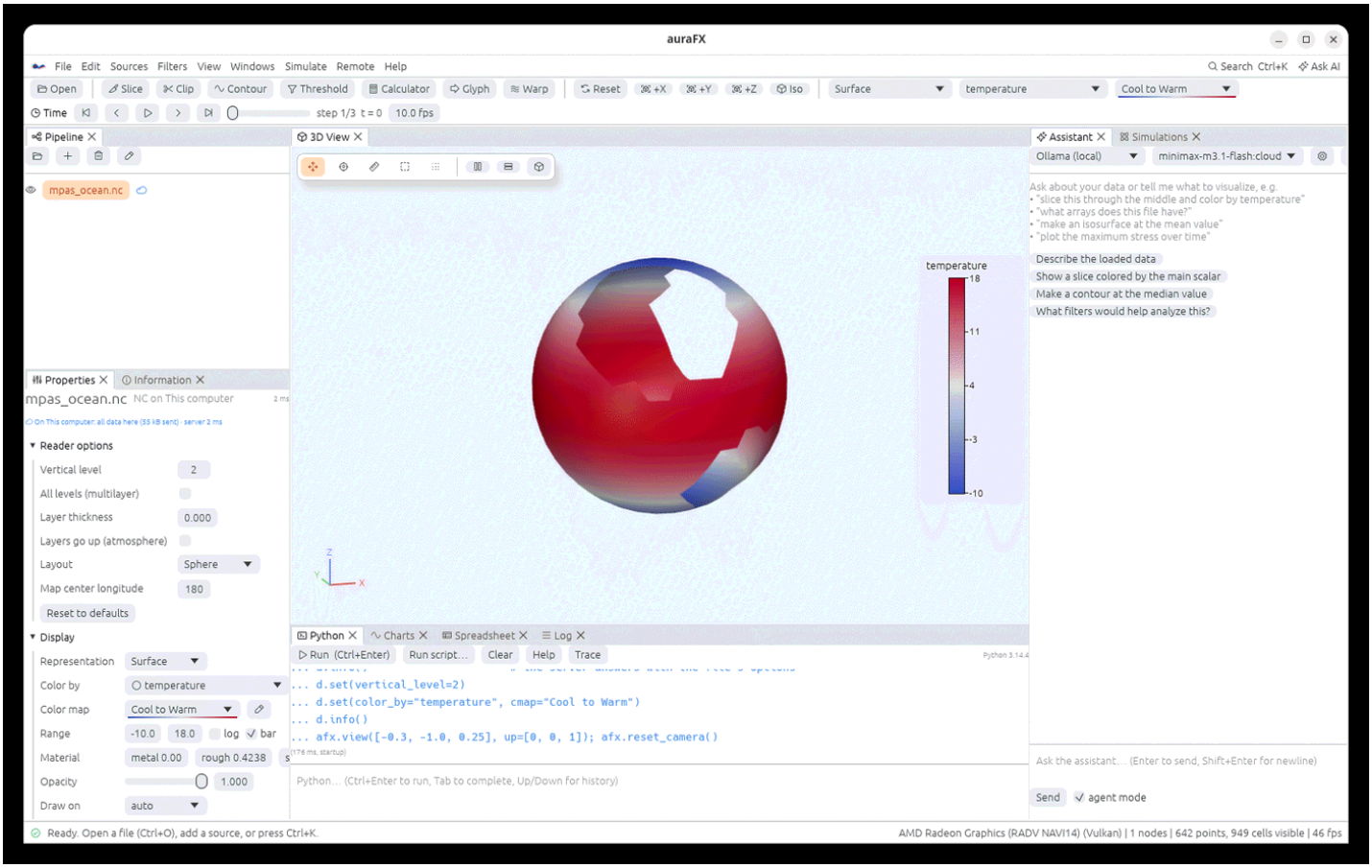
```
d = afx.load("/scratch/run42/out.e", host="cluster")   # a Simulations host, ssh
alias or user@machine
d = afx.load("ssh://cluster/scratch/run42/out.e")       # the same
s = d.slice(normal=(0, 0, 1))                            # runs on the server
s.set(all_data=True)                                     # bring the whole output
over
afx.servers()                                            # connections and the
machines' details
```

Reader options on the server

Files on a server have the same **Reader options** as local ones (an MPAS file's vertical level and layout, a NetCDF file's grid, a CAM-SE file's connectivity file…). The server looks at the file and reports its options with its first answer, so they appear under Properties a moment after the file opens. Changing one has the server read the file again with it; on a parallel server, every process reads its piece with it. Paths in options, such as a CAM-SE connectivity file, are paths on the server.

```
d = afx.load("/scratch/ocean/output.nc", host="cluster")
d.set(vertical_level=10)          # read on the server
```

From Python, an option the file does not have is refused once the server has said which ones it has; before that, the server ignores names it does not know. Saved states keep the options.



Drawing on the server

Some outputs are too big even as a surface. Then the server draws the object with its own GPU and sends pictures (a color and a depth image per view, compressed), which the 3D view blends by depth with the local objects, so everything hides and is hidden correctly. While you move the camera the last picture stays up until the next one arrives.

Display › Draw on in Properties picks per object: **auto** (drawn here unless the surface is over 64 MB), **this computer**, or **the server** (`node.set(draw_on="server")`). Picking, rulers and selections need the geometry here, so switch to “this computer” for those. Translucent objects, volumes and Surface LIC are always drawn here. The view’s lights and shadows travel with each request; screen effects (ambient occlusion, Eye-Dome Lighting, FXAA, tone mapping) are applied here to the whole picture.

Parallel servers

Data too big for one process can be spread over several, like `mpirun pvserver` : **Open on Server › Processes**, or `afx.load(path, host="cluster", processes=8)` , starts eight worker

processes on the host (`aurafx-server --workers node1:16,node2:16` spreads them over machines sharing a file system). Partitioned VTK files are read piece by piece; other files are read by every process and split. Cell-by-cell filters run on every piece with automatic values taken from the whole data; the pieces come back appended, as a merged surface, or as pictures composited by depth. Results match a single process exactly.

Not yet: MPI launchers (`srun`) for the workers, ghost cells, sharing one server between people, temporal filters on parallel servers.

Grains and crystals

auraFX reads electron backscatter diffraction (EBSD) maps, serial-section stacks and voxel microstructures, and works with crystal orientations. It colors them, converts them between representations and re-expresses them in a turned reference frame. It also crops, resamples and aligns voxel volumes. It finds the grains (features) and measures them: sizes, shapes, average orientations, misorientations, neighbours and boundaries. Texture analysis and building synthetic microstructures come in later releases.

EBSD maps

Open a map with **File › Open** or `afx.load("scan.ang")`.

Format	Extension	Notes
TSL/EDAX	`.ang`	square grids, and hexagonal grids (resampled, see below)
Oxford	`.ctf`	angles in degrees in the file, converted to radians
EBSD stacks	`.h5ebsd`	serial sections from TSL or Oxford (HKL) maps, read as one volume
EDAX OIM HDF5	`.h5`	square-grid scans; choose the scan with the **Scan** reader option
Bruker Esprit HDF5	`.h5`	angles in degrees in the file; phases from their space group
Microstructures	`.dream3d`	voxel volumes and meshes with their cell, feature and phase arrays; see below

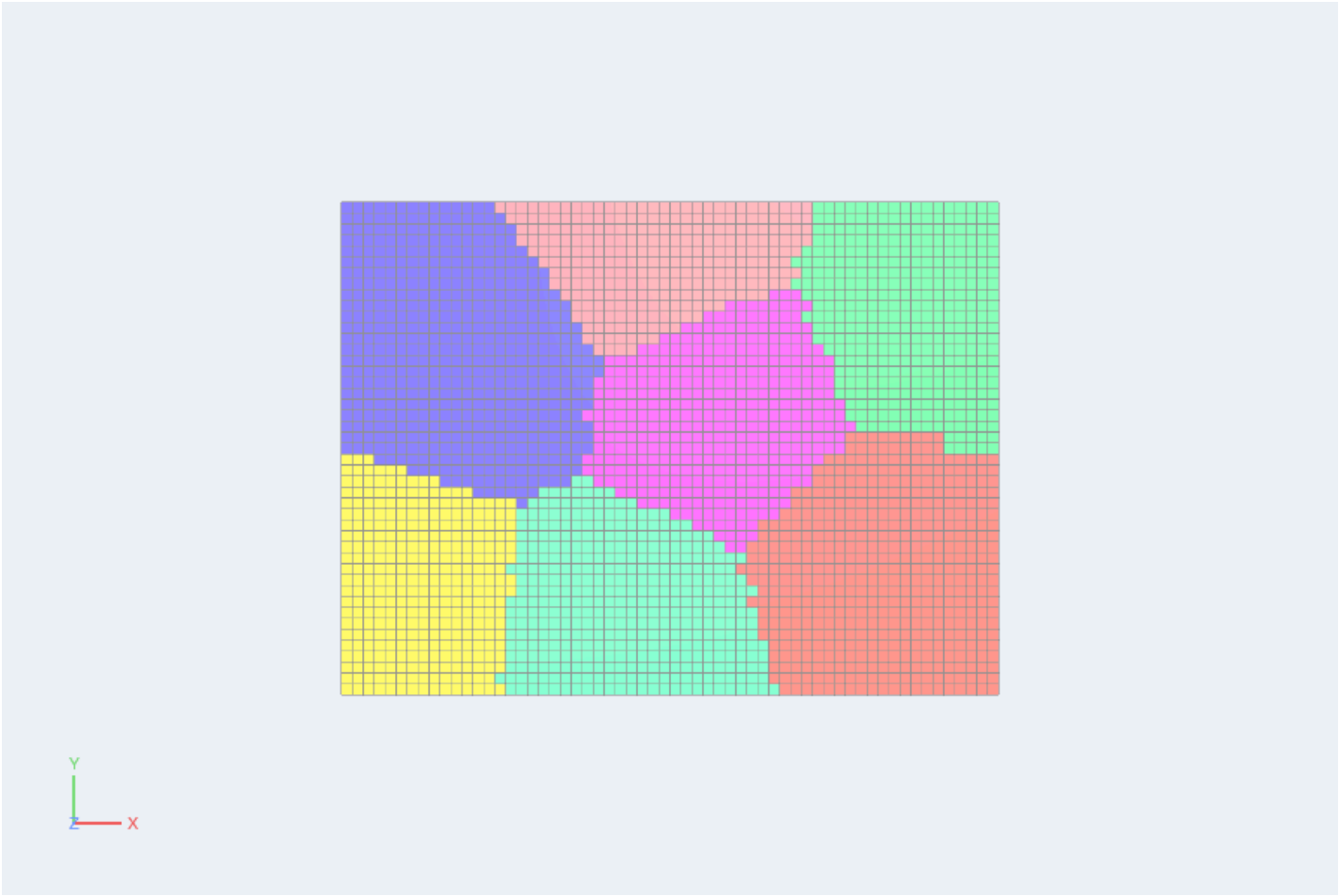
`.h5` files are recognised by their contents, so other HDF5 files with that extension still open with their own readers.

A map becomes image data with one cell per measured point, the cells centered on the points (so the map's corner lies half a step before the first point). Its arrays are:

- **Cell data:** `EulerAngles` (three Bunge angles φ1, Φ, φ2 in radians, as stored in the file) and `Phases` (the phase of each point, 0 where the pattern was not indexed). A `.ang` file adds `Image Quality`, `Confidence Index`, `SEM Signal` and `Fit`. A `.ctf` file adds `Bands`, `Error`, `MAD`, `BC` and `BS`.
- **Field data:** one entry per phase, entry 0 standing for "not indexed". `CrystalStructures` holds the crystal class of each phase (see the numbers below), `MaterialName` its name

and `LatticeConstants` a, b, c, α, β, γ.

Hexagonal grids. A `.ang` scan on a hexagonal grid (rows of `NCOLS_ODD` and `NCOLS_EVEN` points, every other row shifted by half a step) is read on a square grid whose cells are the scan's x step: each square cell takes every value of the scan point nearest to it. The field array `ResampledFromHexGrid` says so and gives both grids, so the map's filters (segmentation, IPF colors, pole figures) work on it as on any square map.



A single-phase `.ang` scan writes phase 0 for every point. auraFX counts those points as phase 1, unless their confidence index is negative.

An `.h5ebsd` stack becomes one volume with a cell layer per slice, the slices `Z Resolution` apart. The stack's sample and Euler transformations are not applied. They are kept as the field arrays `SampleTransformationAngle`, `SampleTransformationAxis`, `EulerTransformationAngle` and `EulerTransformationAxis`. Apply them with Rotate Orientations (and Transform for the sample) if you need them.

Hexagonal axes. Oxford files describe hexagonal crystals with the [10-10] direction along x. auraFX, `.ang` files and the color key put [2-1-10] along x. By default the `.ctf` reader adds 30° to φ2 of hexagonal and trigonal phases to match. To keep the angles as stored, set the reader option **Hexagonal axes** to *As stored* in the Properties panel, or in Python:

```
m = afx.load("titanium.ctf", hexagonal_axes="As stored")
```

Crystal classes

Number	Class	Number	Class
0	Hexagonal 6/mmm	6	Orthorhombic mmm
1	Cubic m-3m	7	Tetragonal 4/m
2	Hexagonal 6/m	8	Tetragonal 4/mmm
3	Cubic m-3	9	Trigonal -3
4	Triclinic -1	10	Trigonal -3m
5	Monoclinic 2/m	999	Unknown

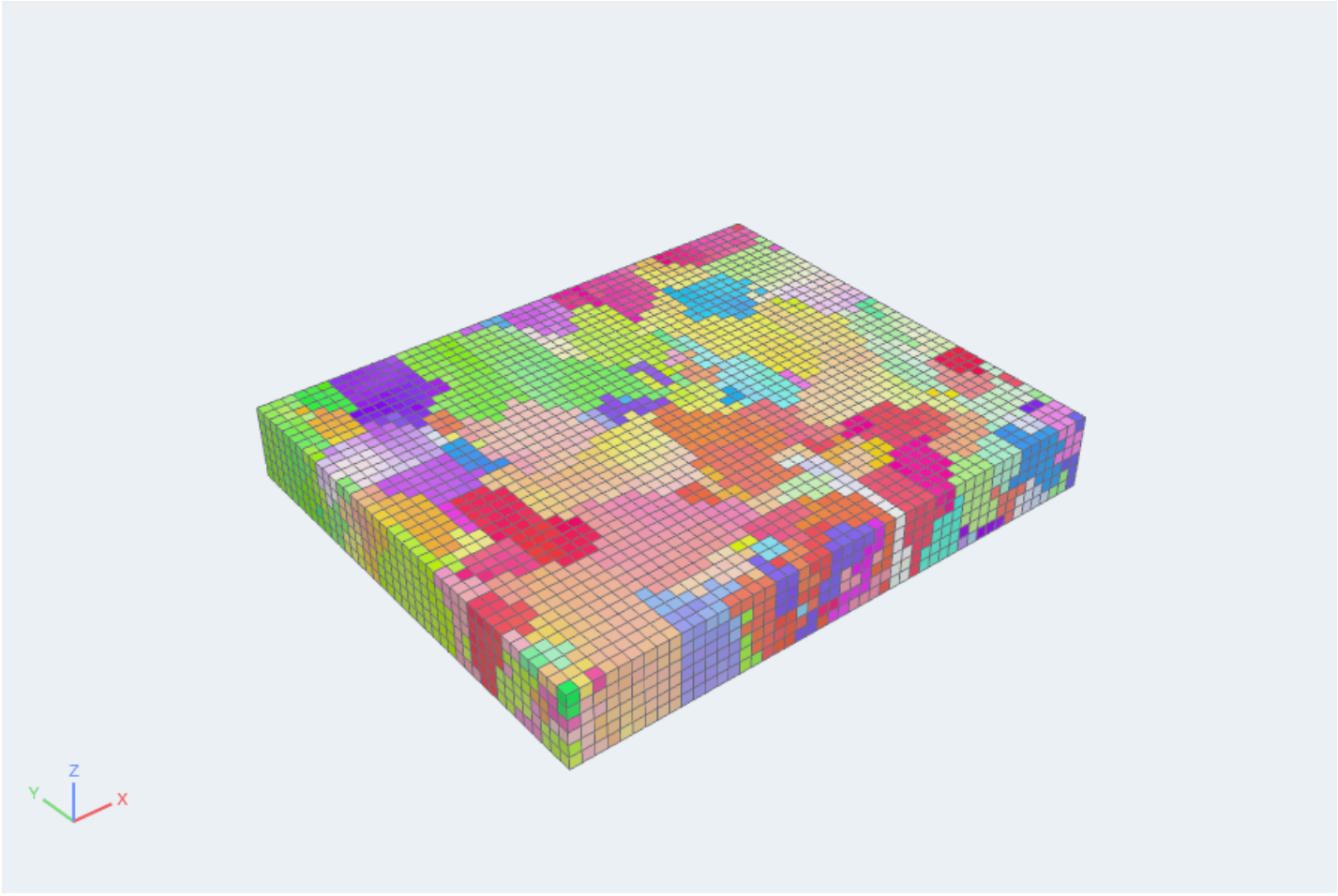
Face-centered and body-centered cubic metals (nickel, aluminium, iron) are Cubic m-3m. Hexagonal close-packed metals (titanium, magnesium, zirconium) are Hexagonal 6/mmm.

Microstructure files (.dream3d)

A `.dream3d` file holds one or more data containers, each a geometry with arrays. auraFX reads image (voxel), rectilinear, vertex, edge, triangle, quadrilateral, tetrahedral and hexahedral geometries.

- Cell arrays (voxels, triangles, ...) become cell data, and vertex arrays become point data.
- Feature arrays (one row per feature, such as `AvgQuats`, `Volumes` or `Centroids`) and phase arrays (`CrystalStructures`, `LatticeConstants`, `MaterialName`) become field data.
- Neighbor lists are not read.
- A file with several data containers opens as blocks. The **Data container** reader option picks one.
- Large volumes load faster with only the arrays you need. Give their names in the **Arrays** reader option, separated by commas.

```
v = afx.load("volume.dream3d", arrays="EulerAngles, Phases, FeatureIds")
```



Saving. File › Save Data writes `.dream3d` files of volumes, meshes and point sets (see File formats). The file also records the pipeline that made the data.

Pipelines in .dream3d files

File › Import Pipeline… builds a pipeline from a pipeline file:

- a `.json` pipeline with numbered steps, the form `.dream3d` files record;
- the pipeline inside a `.dream3d` file.

Files auraFX wrote rebuild exactly: the same filters, parameters and inputs, starting from the same files.

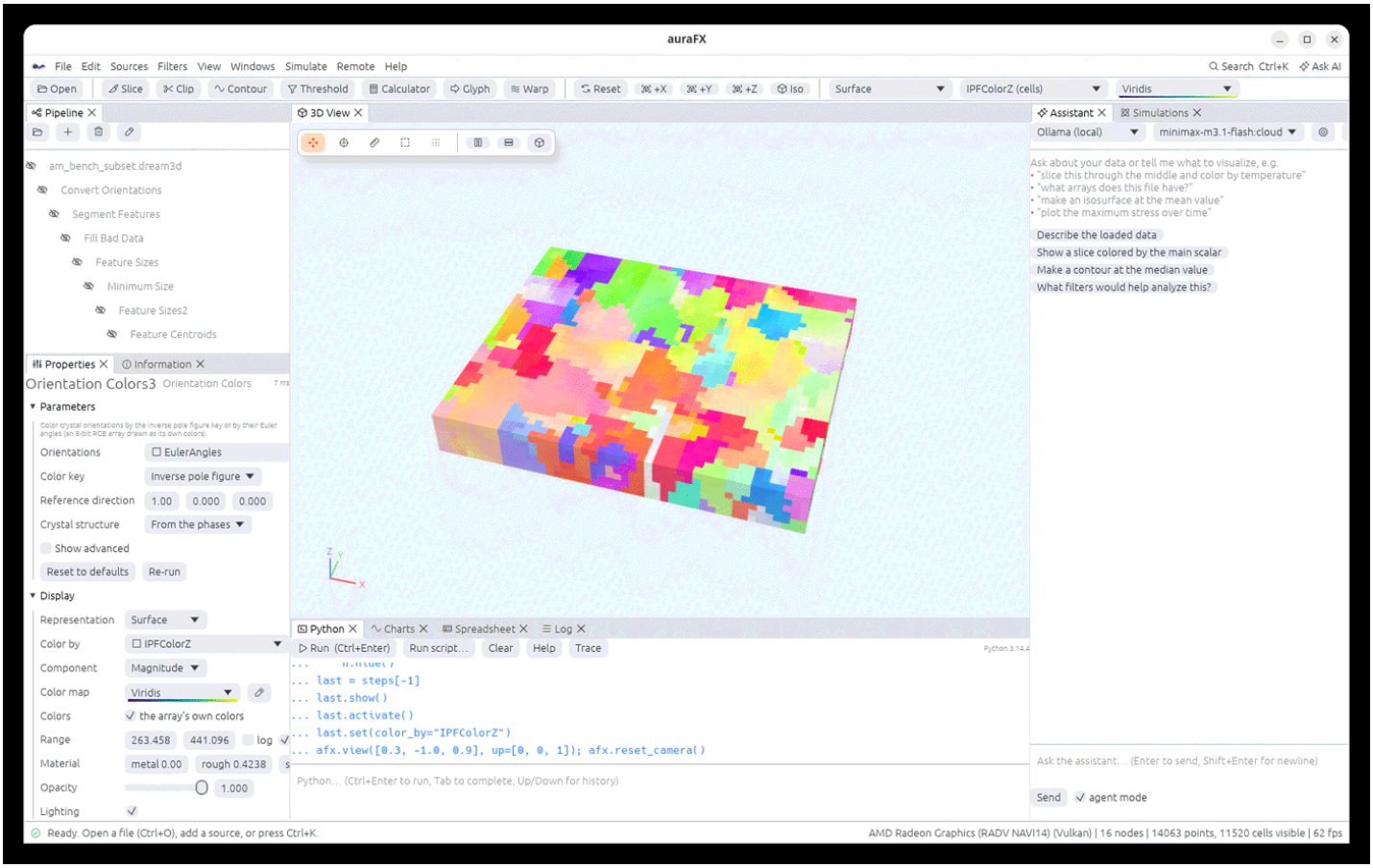
Pipelines from other `.dream3d` files are built as a chain of the auraFX filters that do the same work:

- reading steps open their file when it exists; otherwise the chain starts from the data selected in the pipeline;
- Crop, Change Resolution, Rotate Orientations, Convert Orientations;
- thresholds (as a Calculator mask), Segment Features, Fill Bad Data, Minimum Size;

- the per-feature steps (sizes, centroids, shapes, neighbors, average and reference orientations, kernel average misorientation, distances, merging twins);
- IPF colors, Align Sections, Quick Surface Mesh and Smooth Boundaries.

Steps without a counterpart are listed in the Log and the notice: writing files, deleting arrays, image filters, point cloud alignment and the like.

```
v = afx.load("window.dream3d")
steps = afx.import_pipeline("segmentation.json", v)   # the new nodes; skipped
steps are printed
seg = v.segment_features(tolerance=3, mask="SampleVoxels")
seg.save("segmented.dream3d")                          # records Open File and
Segment Features
print(seg.pipeline_json())
again = afx.import_pipeline("segmented.dream3d")       # the same pipeline,
rebuilt
```



Limits:

- Pipelines with numbered steps (version 6) are read. The newer layout with a `pipeline` list is not.
- A pipeline from another program becomes one chain, so steps that worked on another data container continue from the previous step.

Orientation conventions

An orientation turns the sample axes into the crystal axes. A direction with sample coordinates *s* has crystal coordinates *g s*, where *g* is the orientation matrix. Euler angles are Bunge angles in radians: *g* = Rz(φ2) Rx(Φ) Rz(φ1). Quaternion arrays store (x, y, z, w), with w ≥ 0. Orientation matrices are stored row by row (9 components).

Orientation Colors

Filters › Grains › Orientation Colors adds an 8-bit RGB array named `IPFColor`. Each point is colored by one of two keys:

- **Inverse pole figure** (the default). The reference direction, z by default (the map normal), is turned into crystal coordinates. The crystal's symmetry brings it into the standard triangle of its class, and its place there gives the color. For cubic crystals, [001] is red, [101] green and [111] blue. For hexagonal crystals, [0001] is red, [2-1-10] green and [10-10] blue. Choose x or y as the reference direction to see the orientation along the map's edges.
- **Euler angles**: φ1, Φ and φ2 scaled to red, green and blue.

The class of each point comes from its phase and `CrystalStructures`, as EBSD files give them. For data without phases, choose the class under **Crystal structure**. Points not indexed, of an unknown class, or where the optional **Mask** array is 0, are black. The orientation array can hold Euler angles (3 components), quaternions (4) or orientation matrices (9). If none is chosen, the filter uses `EulerAngles`, `Quats` or `OrientationMatrix`, whichever it finds first.

```
m = afx.load("scan.ang")
ipf = m.orientation_colors(direction=(0, 0, 1))      # key="Euler angles" for the
other key
ipf.set(color_by="IPFColor")
```

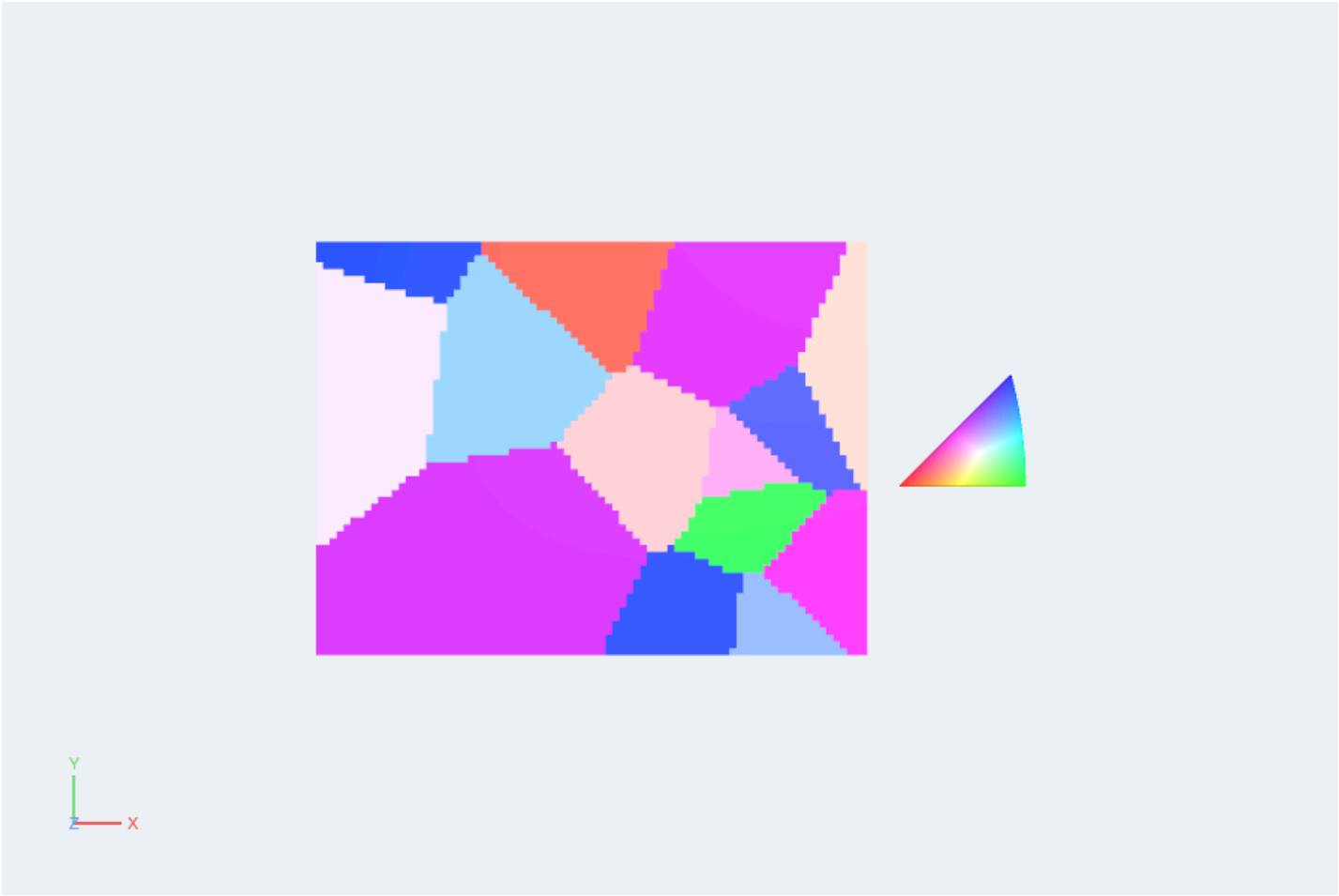
Arrays drawn as their own colors

An array of 8-bit values with 3 or 4 components, like `IPFColor`, is drawn with its own colors when you color by it, not through a color map. No legend is shown for it. To map such an array through a color map instead, clear **Colors: the array's own colors** under Color by in the Properties panel, or use `node.set(direct_colors=False)`. Choosing one component of the array also uses the color map.

IPF Color Key

Sources › IPF Color Key draws the color key of a crystal class: its standard triangle in stereographic projection, colored as Orientation Colors colors it. It lies around the origin in the xy plane, about half a unit across, so move and enlarge it with **Transform** to place it next to a map.

```
key = afx.ipf_color_key(structure="Cubic m-3m", resolution=60)
key.transform(scale=(20, 20, 1), translate=(42, 12, 0)).set(color_by="IPFColor")
```



Convert Orientations

Filters › Grains › Convert Orientations writes the orientations in another representation. It reads and writes Euler angles, quaternions, orientation matrices, axis-angle pairs (the unit axis and the angle in radians), Rodrigues vectors (the unit axis and tan(ω/2)) and homochoric vectors. With **Reduce to the fundamental zone**, each orientation is replaced by its symmetry equivalent with the smallest rotation angle (for cubic crystals, at most 62.8°). The output is named after the representation (`Quats` , `OrientationMatrix` , ...) unless you give a name.

```
q = m.convert_orientations(input_type="Euler angles", output_type="Quaternions",
reduce=True)
```

Rotate Orientations

Filters › Grains › Rotate Orientations re-expresses the orientations after the reference frame is turned by an angle (degrees) about an axis. The points do not move; only the description of their orientations changes.

- **Sample** frame: use this when the map's axes differ from those you want, such as a scan taken with the sample turned 90° about its normal.
- **Crystal** frame: turns the crystal axes the orientations refer to.

```
r = m.rotate_orientations(frame="Sample", axis=(0, 0, 1), angle=90)
```

Crop

Filters › Grains › Crop keeps the cells of a voxel volume or EBSD map whose centres lie inside a box, given in sample coordinates (leave a side unset to keep everything on that side). The cells keep their place in space. With **Renumber features** (on by default), the remaining feature ids are numbered from 1 and the per-feature field arrays keep only the rows of those features. To crop by grid index instead, use **Extract Subset**.

```
c = v.crop_volume(min=(0, 0, 0), max=(100, 80, 20))
```

Change Resolution

Filters › Grains › Change Resolution resamples a voxel volume or EBSD map onto cells of another size. Each new cell takes the values of the old cell that holds its centre, so orientations, phases and feature ids stay valid. The volume keeps its corner and, rounded to whole cells, its size. Features that disappear are dropped and the rest renumbered, as in Crop. Point data is not carried over.

```
coarse = v.change_resolution(spacing=(2, 2, 2))
```

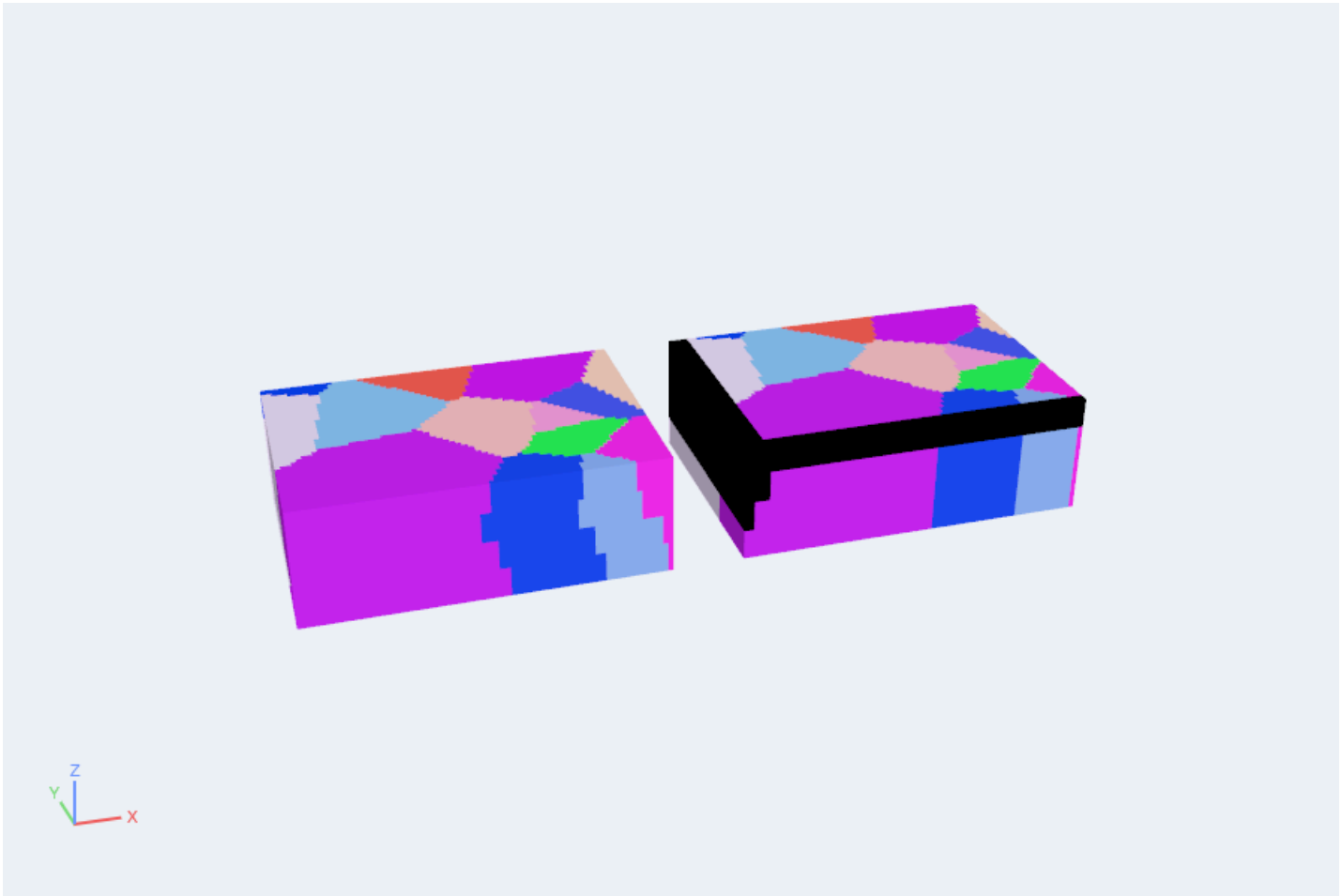
Align Sections

Filters › Grains › Align Sections moves each z slice of a serial-section volume in x and y, by whole cells, so it matches the slice below. Choose how slices are compared:

- **Misorientation** (the default): two cells match when their orientations differ by less than the **Tolerance** (5° by default), with the crystal class of their phase.
- **Feature ids**: two cells match when they belong to the same feature.
- **Given shifts**: no search. Each slice moves by the x and y cells you give, against the slice below.

The search starts from no move and steps to the neighbouring shift with fewer differing cells until none is better, at most **Largest shift** cells each way. A **Mask** array (a sample or good-data mask) limits the comparison to cells where it is not 0. Cells uncovered by a move are set to 0 (phase 0, not indexed). The total shift of each slice is kept as the field array `SectionShifts` (x, y cells). Point data is not carried over.

```
a = stack.align_sections(method="Misorientation", tolerance=5,
mask="SampleVoxels")
a.align_sections(method="Given shifts", shifts=[2, -1, 3, 1, -1, 2])   # 3 slices
after the first
```

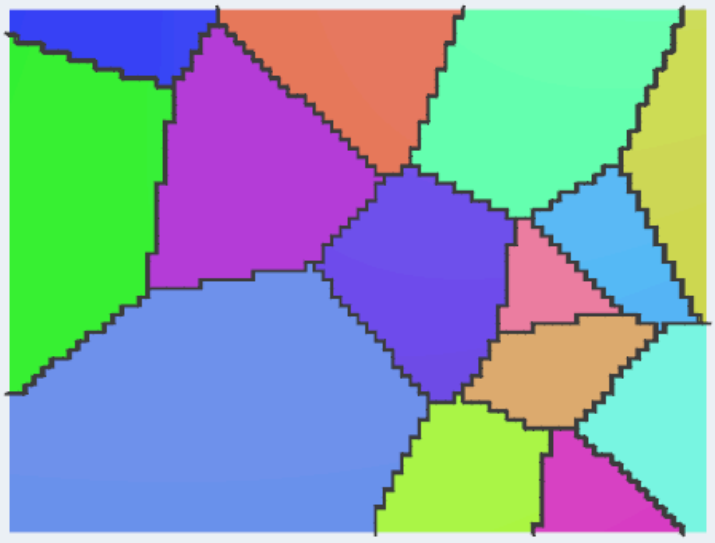


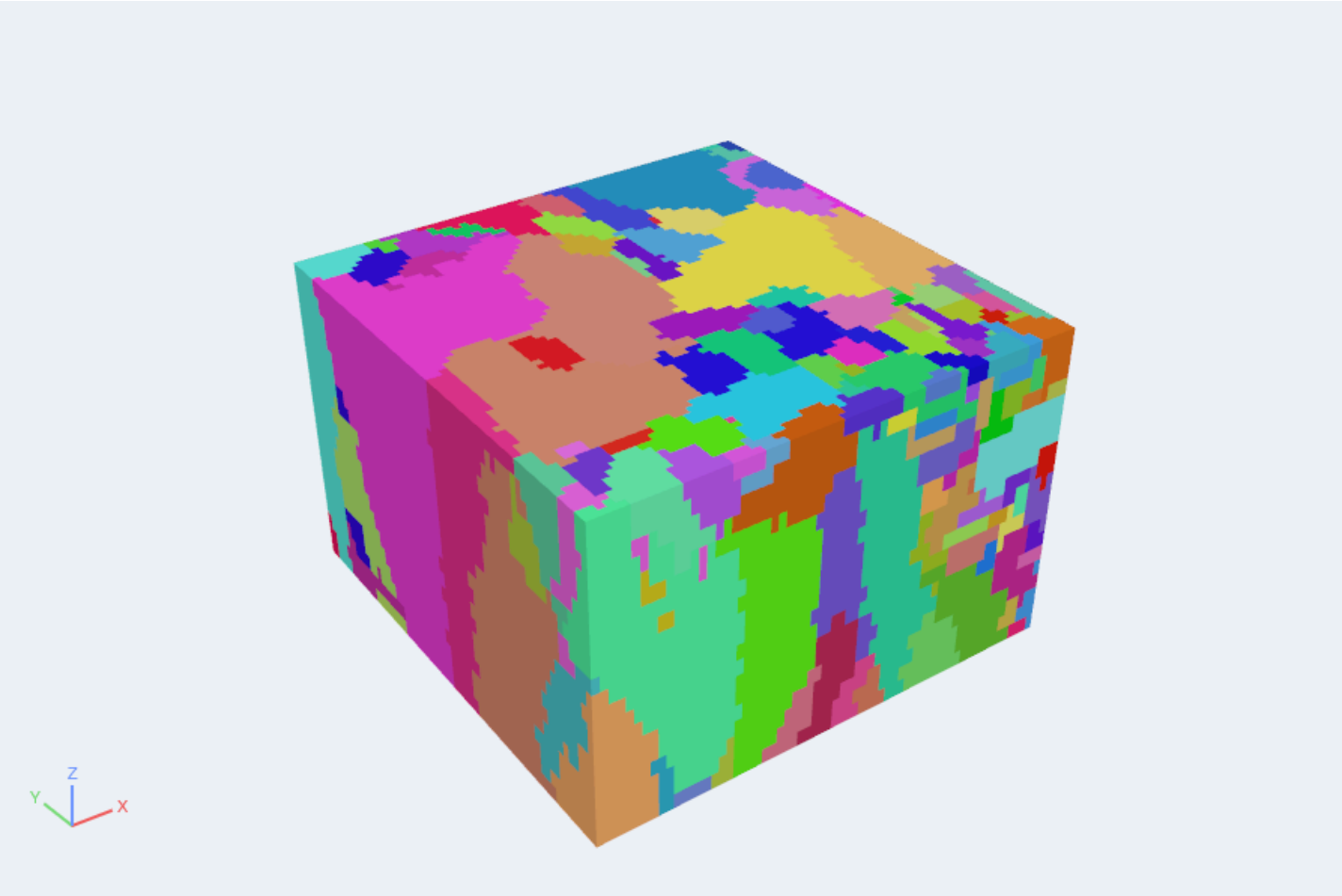
Features (grains)

A feature is a connected set of cells, cells sharing a face, that belong together: usually a grain. **Segment Features** finds them and writes the cell array `FeatureIds` (0 for cells in no feature). The other feature filters then add per-feature results as field arrays, one row per feature id (row 0 stands for id 0). All of them are under **Filters › Grains** and work on voxel volumes and on flat EBSD maps.

A typical sequence for an EBSD volume:

```
v = afx.load("scan.dream3d")
g = (v.segment_features(tolerance=2, mask="SampleVoxels")
      .fill_bad_data(min_defect=7)
      .minimum_size(min_cells=7)
      .feature_sizes().feature_centroids().feature_shapes()
      .average_orientations().reference_misorientations()
      .kernel_average_misorientation(kernel=(2, 2, 1))
      .feature_neighbors())
table = g.feature_table()           # one row per grain, for the Spreadsheet,
charts and CSV
g.feature_colors().set(color_by="FeatureColors")
```





Segment Features

Neighbouring cells join the same feature when they pass the test of **Group by**:

- **Misorientation**: same phase, and misorientation below the **Tolerance** in degrees.
- **C-axis alignment**: same phase, and crystal c axes within the tolerance either way round. This is for hexagonal crystals.
- **Scalar**: values of the chosen cell array differ by at most the tolerance.

Cells outside the **Mask** (cells where it is 0), and cells of no known crystal class, stay at 0. Features are numbered in the order their first cell appears (x fastest, then y, then z). **Shuffle ids** numbers them in a shuffled order instead, which gives better colors when coloring by id. The field arrays `Active` and `FeaturePhases` (the phase of each feature) are added, and per-feature arrays from an earlier segmentation are removed.

Cleaning up

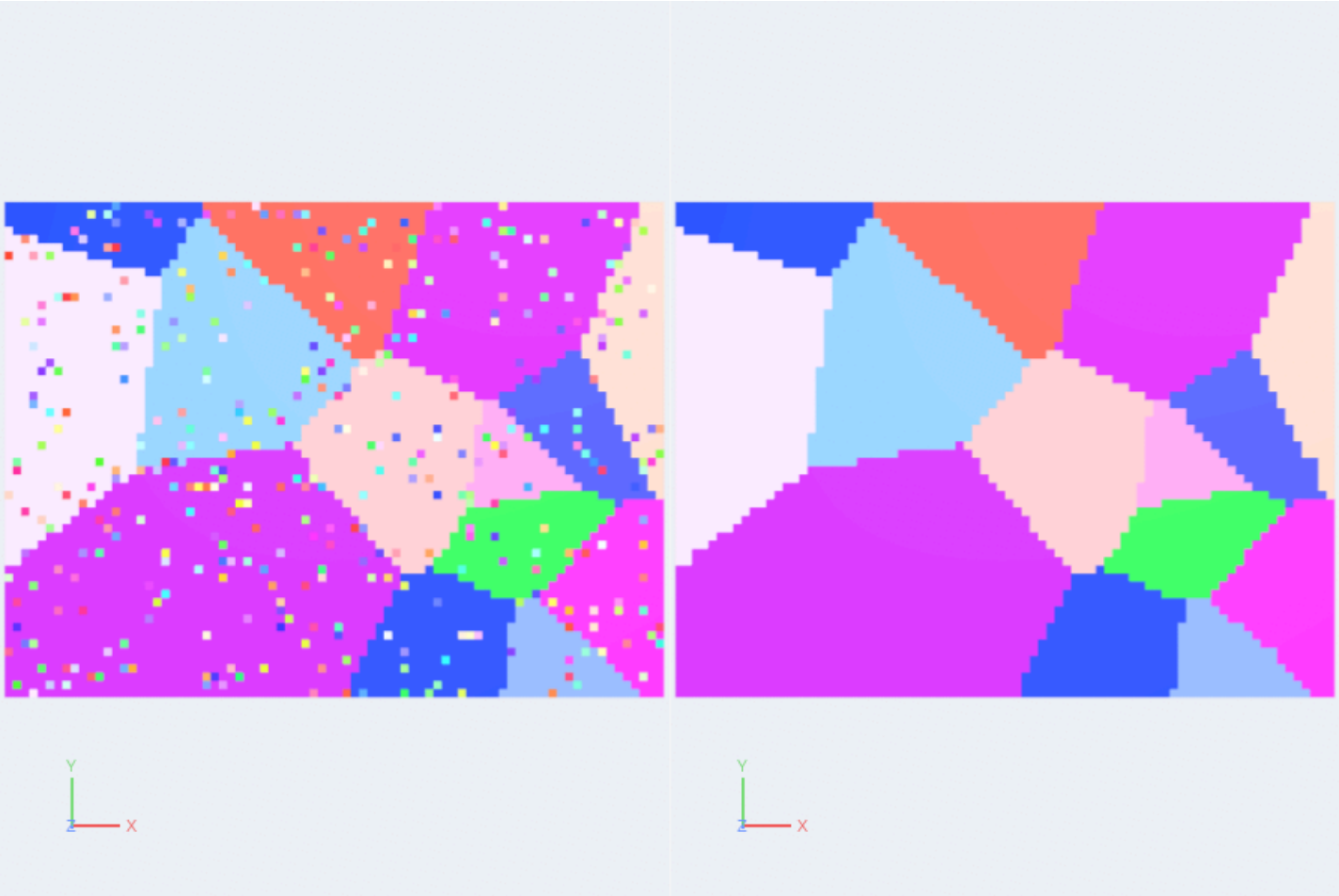
- **Fill Bad Data**: clusters of cells without a feature that are smaller than **Smallest kept defect** are filled from the surrounding features. Each cell takes every cell array from the

neighbour whose feature most of its neighbours belong to, sweep after sweep. Larger clusters (pores, the outside of the sample) stay 0.
- **Minimum Size**: features with fewer cells than **Fewest cells** are removed and filled from their neighbours the same way. Feature 0 can win, so a small feature against the outside may become outside. The remaining features are renumbered from 1 in their old order, and per-feature field arrays keep their rows.

Before or instead of segmenting, noisy maps can be cleaned cell by cell (every cell array moves with the cell it comes from):

- **Replace Element Attributes with Neighbor** (neighbour confidence correlation): each cell whose confidence (an array such as `Confidence Index`) is below a minimum takes the values of its most confident neighbour, again and again if asked.
- **Neighbor Orientation Correlation**: each low-confidence cell takes the values of the neighbour that most other neighbours agree with in orientation (within a misorientation tolerance), when enough of them agree, from 6 agreeing neighbours down to 6 minus the *cleanup level*.
- **Bad Data Neighbor Orientation Check**: a cell outside the mask (bad) with enough good neighbours of nearly its own orientation becomes good.
- **Erode Dilate Bad Data**: the features grow into the bad cells (erode) or the bad cells into the features (dilate), a number of times, along chosen axes.
- **Erode Dilate Coordination Number**: a cell with at least this many face neighbours in one other feature joins that feature: smooths jagged boundaries.
- **Identify Sample**: the largest connected region of good cells is the sample; other good cells become bad, and with *fill holes* bad pockets inside the sample become good. **Isolate Largest Feature** keeps only the largest feature.

A 20 × 20 map of four grains with one cell in 17 spoilt (wrong orientation, low confidence) comes back to its four grains exactly with Replace Element Attributes with Neighbor, Neighbor Orientation Correlation, or one pass of Erode Dilate Bad Data.



To put a sample the right way round: **Rotate Sample Reference Frame** turns the grid and its data (not the crystal orientations; use Rotate Orientations for those) by an angle about an axis, exactly for multiples of 90°; **Apply Transformation to Geometry** scales, turns and moves the grid (only the origin and spacing change when there is no turn), also by a 4 × 4 matrix; **Set Origin and Spacing** gives the volume a new origin and cell size; **Resample Rectilinear To Image** puts an unevenly spaced grid on an even one.

Image filters on volumes

In **Filters › Data Analysis**, for any image data (an EBSD map's quality arrays, an optical or X-ray volume): **Median Image** (the median of a box of cells, against speckle), **Gaussian Smooth Image** (a standard deviation in cells), **Otsu Threshold** (one to four thresholds that best split the histogram; a label array and the thresholds as field data) and **Binary Morphology** (erode, dilate, open or close the non-zero cells with a ball of cells). The results agree with scipy.ndimage (median, Gaussian, binary erosion and dilation) on a test volume.

```
m = volume.median_image(array="ImageData", radius=(1, 1, 1))
o = m.otsu_threshold(array="Median", thresholds=1)
clean = o.binary_morphology(array="Otsu", operation="Open", radius=1)
```

Per-feature results

Filter	Field arrays (per feature)	Cell arrays
Feature Sizes	`NumElements`, `Volumes`, `EquivalentDiameters` (sphere of equal volume; circle of equal area on flat maps)	
Feature Centroids	`Centroids` (mean of the cell centres)	
Feature Shapes	`AxisLengths` (semi-axes a ≥ b ≥ c of the ellipsoid with the same moments of inertia), `AspectRatios` (b/a, c/a), `AxisEulerAngles` (Bunge angles of the axes: the rows of the rotation are the a, b and c directions), `Omega3s` (1 for an ellipsoid, smaller for other shapes)	
Average Orientations	`AvgQuats` (x, y, z, w), `AvgEulerAngles`	
Reference Misorientations	`FeatureAvgMisorientations`	`FeatureReferenceMisorientations` (degrees from the feature's average orientation)
Kernel Average Misorientation		`KernelAverageMisorientations` (degrees)
Feature Neighbors	`NumNeighbors`, `SurfaceFeatures` (1 when touching the edge of the volume or cells without a feature), `AvgMisorientations` (mean misorientation to the neighbouring features of the same crystal class, when average orientations exist)	`BoundaryCells` (face neighbours in another feature)
Distance to Boundaries		`GBEuclideanDistances`, `TJEuclideanDistances`, `QPEuclideanDistances` (distance between cell centres to the nearest

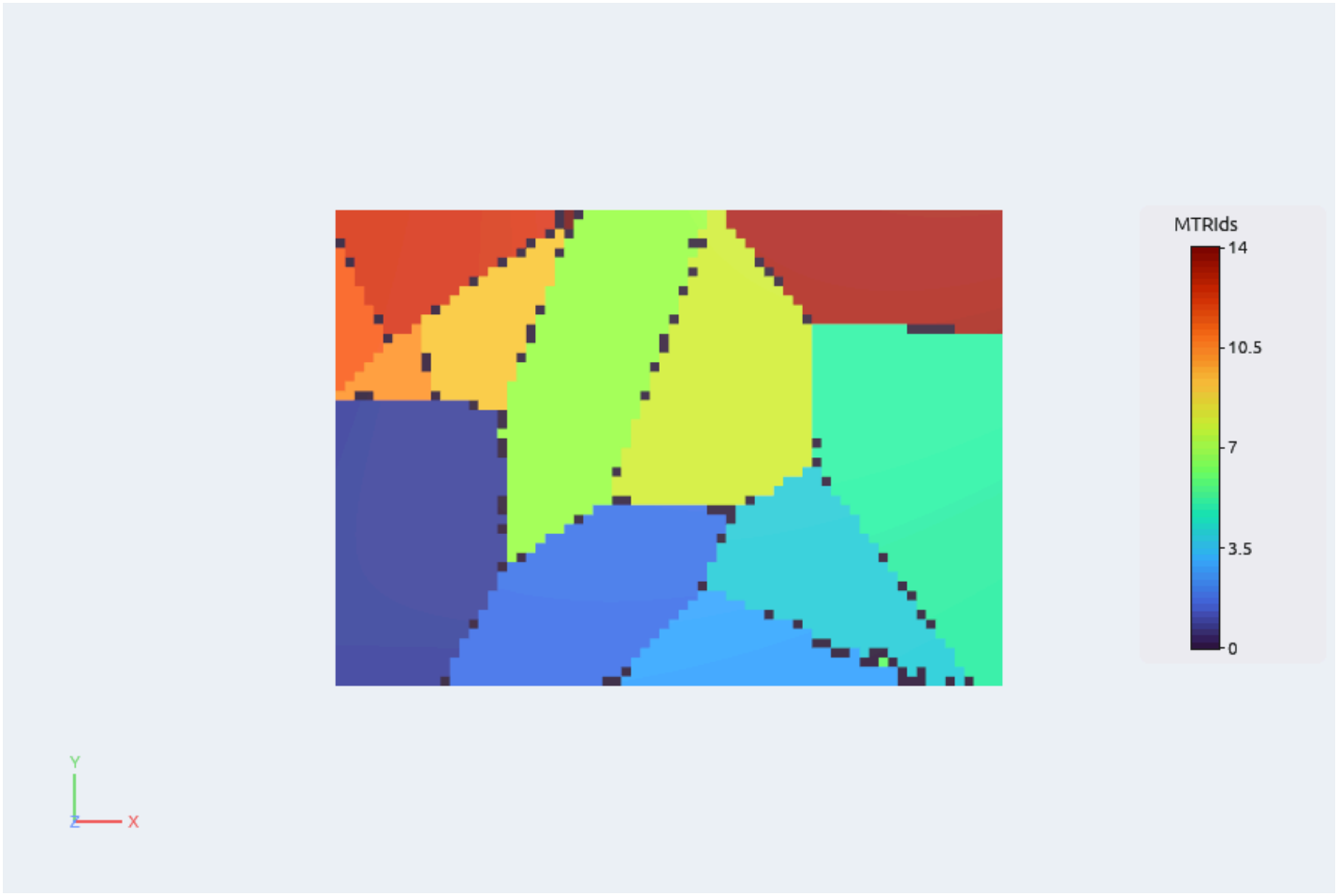
Filter	Field arrays (per feature)	Cell arrays
		boundary, triple-line and quadruple-point cell; -1 where there is none)
Merge Twins	`ParentIds`	`ParentIds`

- **Feature Shapes** uses the exact second moments of the cells: each cell counted at its centre with its own inertia. On flat maps it gives the ellipse of the same moments: a and b, b/a, and the angle of the long axis in the plane as the first Euler angle.
- **Average Orientations** adds, cell by cell, the symmetry equivalent nearest the running average (the identity for the first cell), then normalises.
- **Kernel Average Misorientation** averages a cell's misorientation to the cells of the same feature within **Kernel** cells along x, y and z (1 1 1 means the 26 neighbours). The cell itself counts, with 0. Cells without a feature get 0.
- **Merge Twins** gives neighbouring cubic m-3m features a common parent when their average orientations differ by 60° about a ⟨111⟩ axis, within the **Axis** and **Angle** tolerances. Chains of twins share one parent. Parents are numbered in the order of their first feature.

More measures of features and their boundaries:

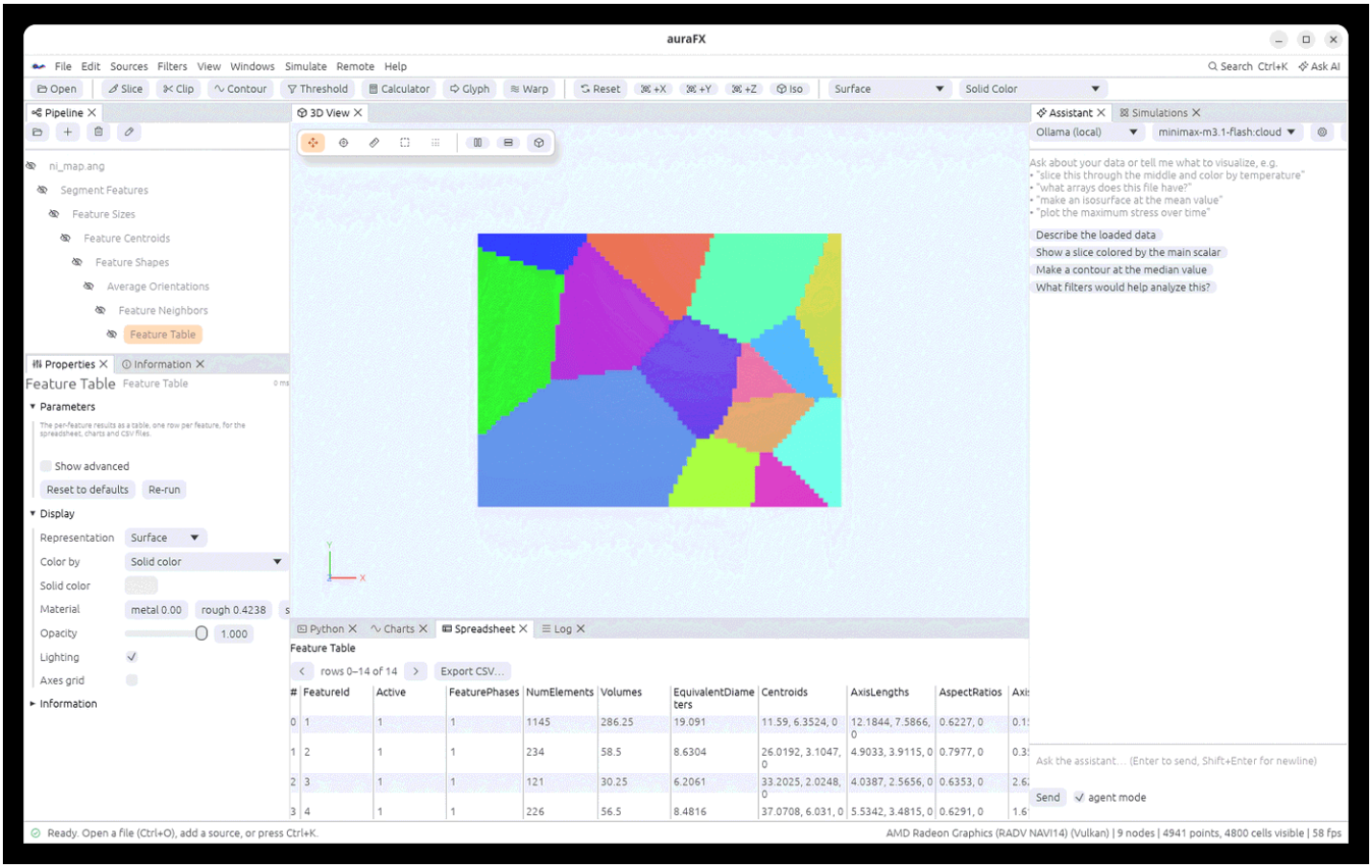
Filter	Gives
Find Neighborhoods	`Neighborhoods` : how many feature centres lie within some multiple of the mean equivalent diameter
Find Largest Cross Sections	`LargestCrossSections` : the largest area of a feature in one slice normal to x, y or z
Find Feature Clustering	a table of the radial distribution function of feature centres (of one phase), for the Charts panel
Find Surface Area to Volume	`SurfaceAreas` , `SurfaceAreaVolumeRatio` and `Sphericity` per feature
Find Boundary Strengths	a table per pair of neighbouring grains: m' (
Find Twin Boundary Schmid Factors	for every Σ3 twin boundary of cubic grains, the Schmid factors of the three slip directions in the twin plane, in each grain
Find C-Axis Locations	`CAxisLocation` of hexagonal cells (unit vectors)
Find Feature Reference C-Axis Misorientations	the angle of each cell's c axis from its grain's mean c axis
Identify Micro-Texture Regions	`MTRIds` : neighbouring hexagonal grains whose c axes line up (within a tolerance) joined into regions

Filter	Gives
Generate Precipitate Statistics	a table for one phase: count, volume fraction, number density, lognormal size fit, mean nearest-neighbour distance
Copy Feature Array to Cells	a per-feature array (equivalent diameter…) on every cell of the feature, to color by



Boundaries, colors and the table

- **Feature Boundaries** gives the cell faces between different features as a surface. Each face has `FeatureIdA` and `FeatureIdB`, the `Misorientation` (degrees) between the two cells, and `Twin` (1 on twin boundaries of cubic m-3m crystals). **Include the outer surface** adds the faces against cells without a feature and the edge of the volume. Draw it as **Wireframe** over a flat map to see the boundaries as lines.
- **Feature Colors** gives each feature a distinct, stable color (`FeatureColors`, drawn as its own colors).
- **Feature Table** turns the per-feature field arrays into a table, one row per feature with a `FeatureId` column, for the Spreadsheet, charts and **Export CSV**.



Texture

The texture filters (**Filters › Grains**) describe how the orientations of a map or volume are distributed. Every density is in multiples of a random distribution (MRD): 1 is what randomly oriented crystals give, 3 means three times as often as random. Each density is a smoothed count: every orientation spreads over a Gaussian of the **Smoothing** width.

The texture filters work on one phase:

- **Phase** chooses it. With 0, every indexed point is used, and they must all have one crystal class.
- **Mask** can leave out more points.
- **Orientations used at most** caps how many are counted (60 000 for the figures, 10 000 for the ODF). When there are more, a random subset with a fixed seed is taken, so the result is the same each time.

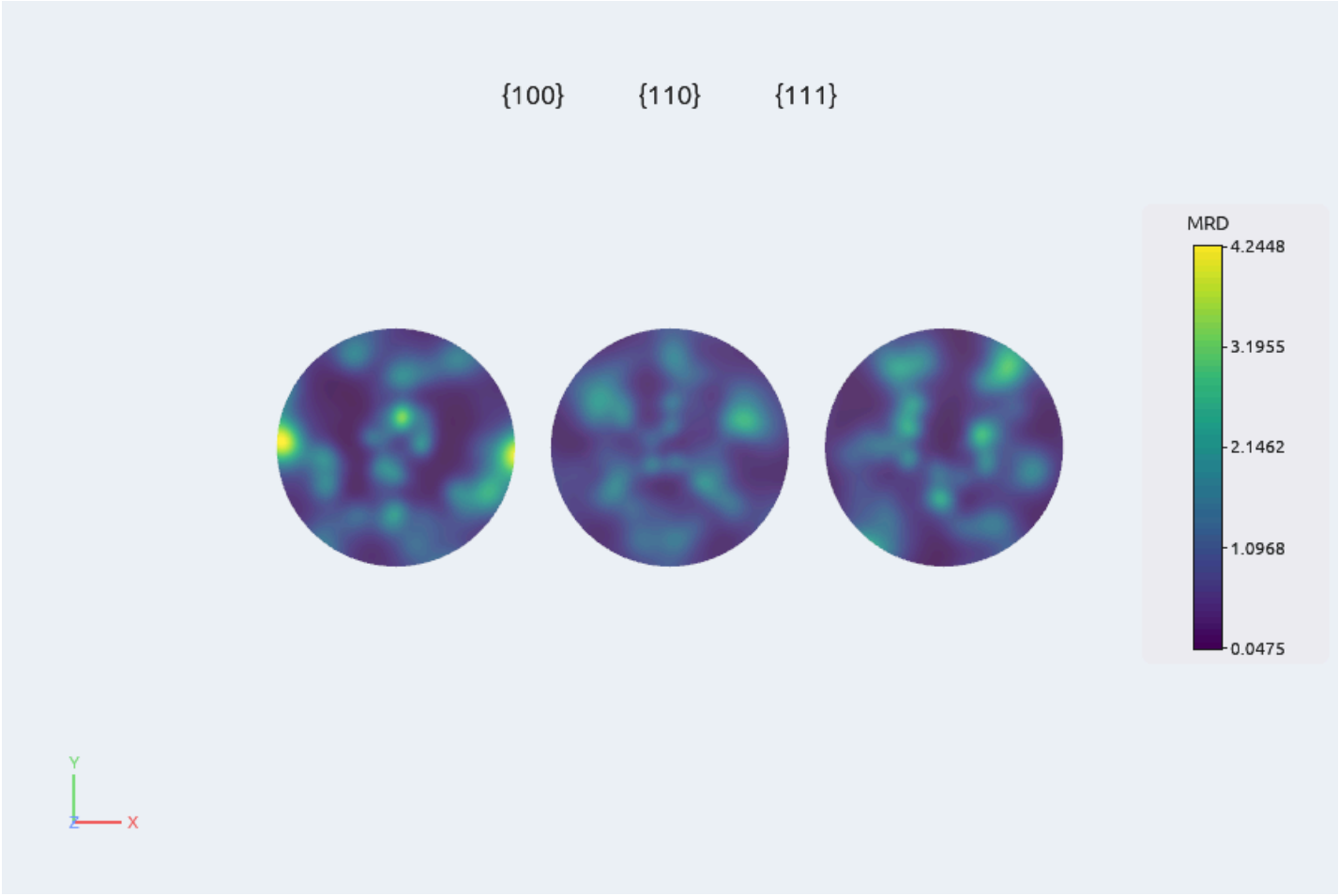
The figures come out as flat panels side by side in the view, colored by `MRD` or `ODF`. To keep the numbers, save the output as CSV: every panel goes into one file, with a `Block` column naming the panel and the direction or angles of each point.

Pole Figure

Pole Figure shows where the crystal directions of a family point in the sample.

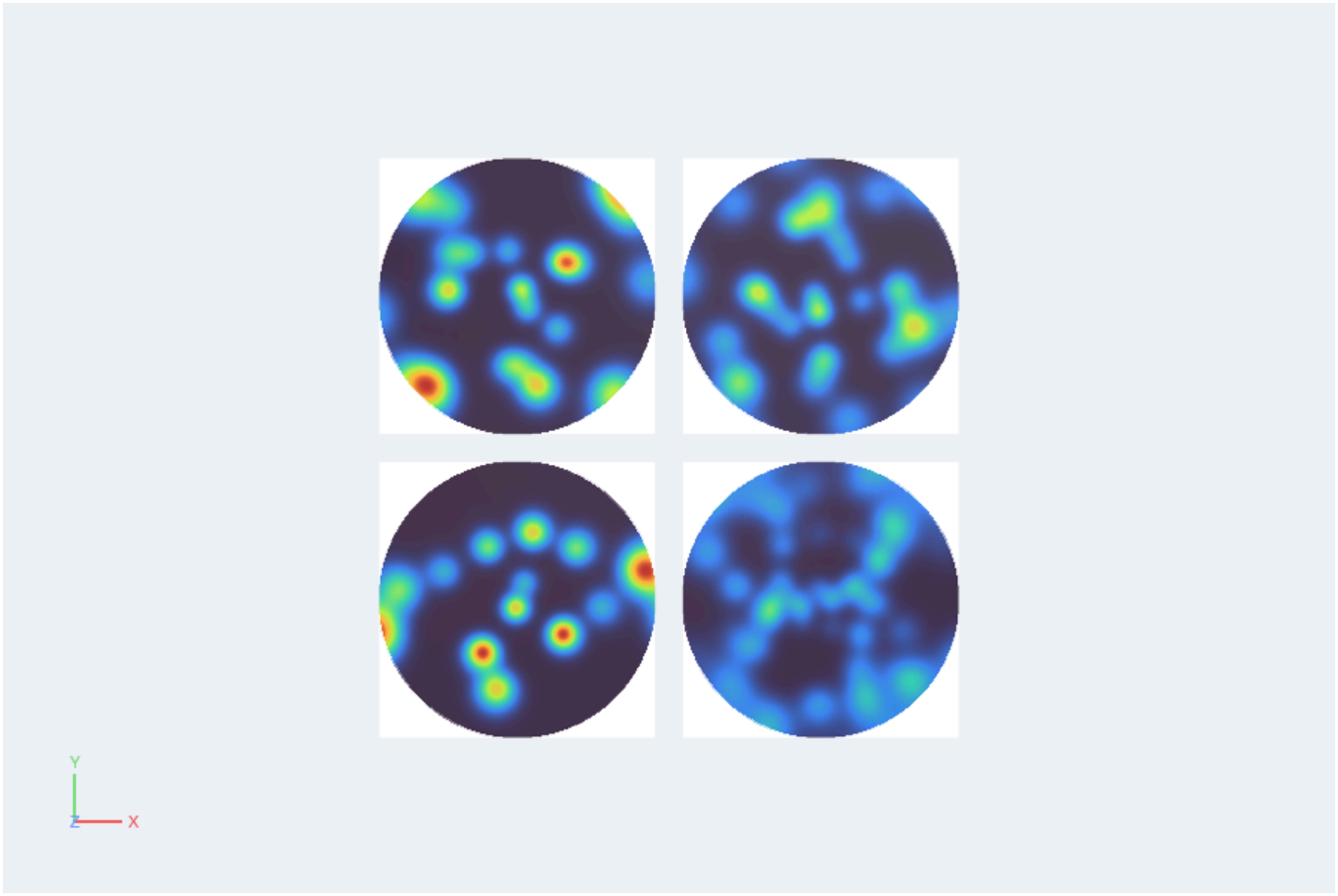
- **Directions** lists the families, written like `100, 110, 111`, `-112` or `1 2 3`. Every symmetry-equivalent direction and its opposite are counted.
- Each family is a disk: sample X to the right, Y up, sample Z at the centre and the horizontal plane at the rim.
- **Projection** is *Stereographic*, which keeps angles, or *Equal area*, which keeps areas.
- The directions are first gathered in **Bin size** cells (2° by default) before smoothing. This keeps large maps fast.

```
pf = ebsd.pole_figure(families="100, 110, 111", projection="Stereographic",
sigma=7.5)
```



To keep the pole figures as pictures, one per phase and family, call `save_pole_figures` (Python only): each phase's figures share one scale, from 0 (dark blue) to the largest MRD of that phase (red), with sample X to the right and Y up, white outside the disk.

```
files = ebsd.save_pole_figures("pole_figures", families="100, 110, 111", size=256)
# [('pole_figures/phase1_100.png', 3.1), ...]: the file and its largest MRD
```



Inverse Pole Figure

Inverse Pole Figure shows which crystal directions lie along a sample direction (X, Y or Z, or all three side by side). The density is drawn on the standard triangle of the crystal class, the same region the orientation color key uses. A fibre texture shows here as one bright corner or spot.

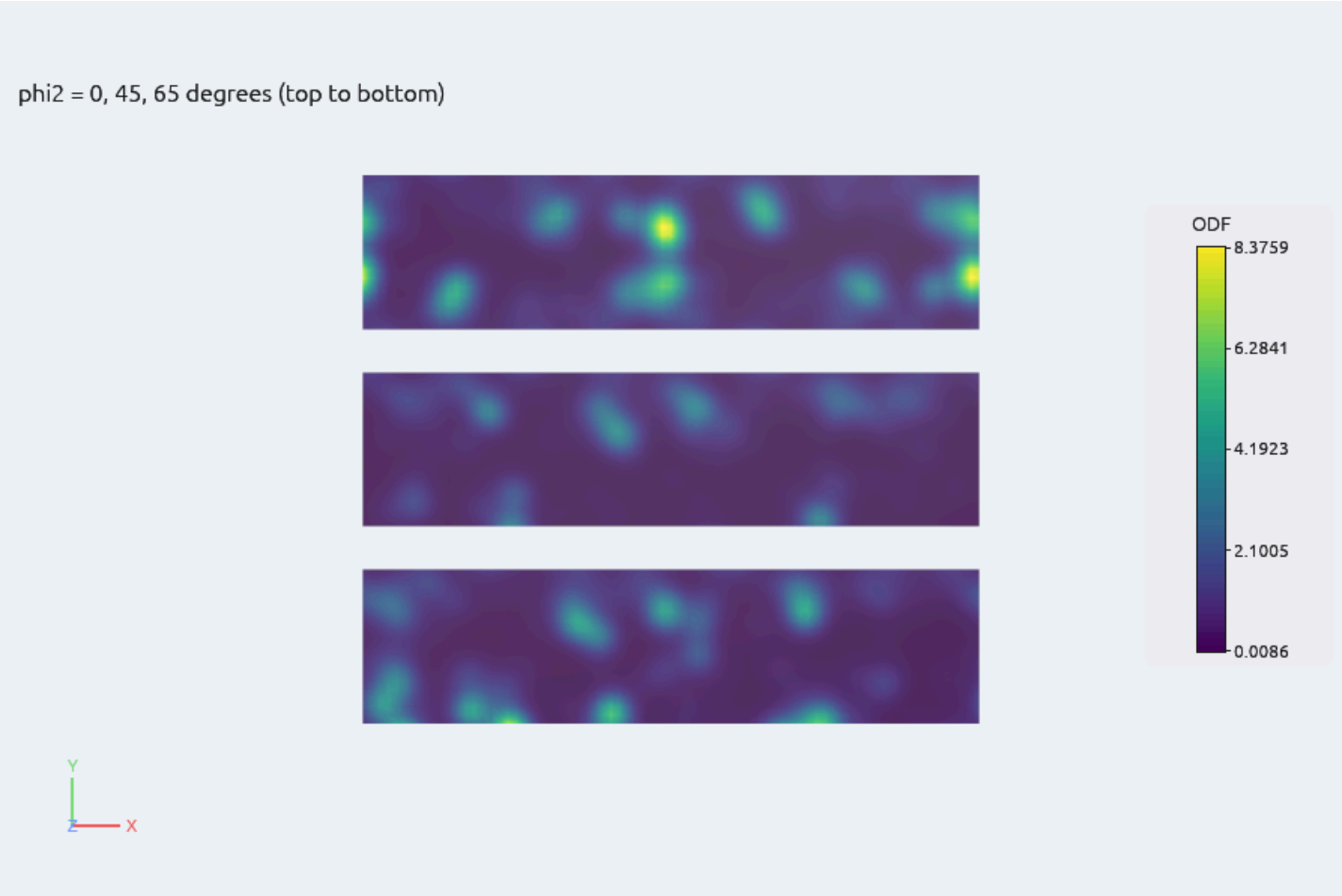
```
ipf = ebsd.inverse_pole_figure(directions="X, Y and Z")
```

ODF Sections

ODF Sections computes the orientation distribution function (ODF): how often each orientation occurs, smoothed over the disorientation angle.

- Each section has a constant φ2 (**Sections**, 0, 45 and 65° by default). φ1 runs across from 0 to 360°, and Φ runs down from 0 to 90° for classes with a two-fold axis across the c axis (otherwise to 180°).
- Each point's Euler angles are kept with its value.
- The output also carries the **texture index** `TextureIndex`: the mean of the squared ODF over all orientations. It is 1 for a random texture and grows as the texture sharpens.
- `MaxODF` is the ODF's largest value. Both come from a grid of **Texture index step** degrees over the whole orientation space.
- With a finite number of orientations, even a random set gives an index a little above 1: about 1.001 for 20 000 random orientations with 7.5° smoothing.

```
odf = ebsd.odf_sections(sections="0, 45, 65", step=5)
print(odf.array("TextureIndex", "field"))
```



Misorientation Distribution

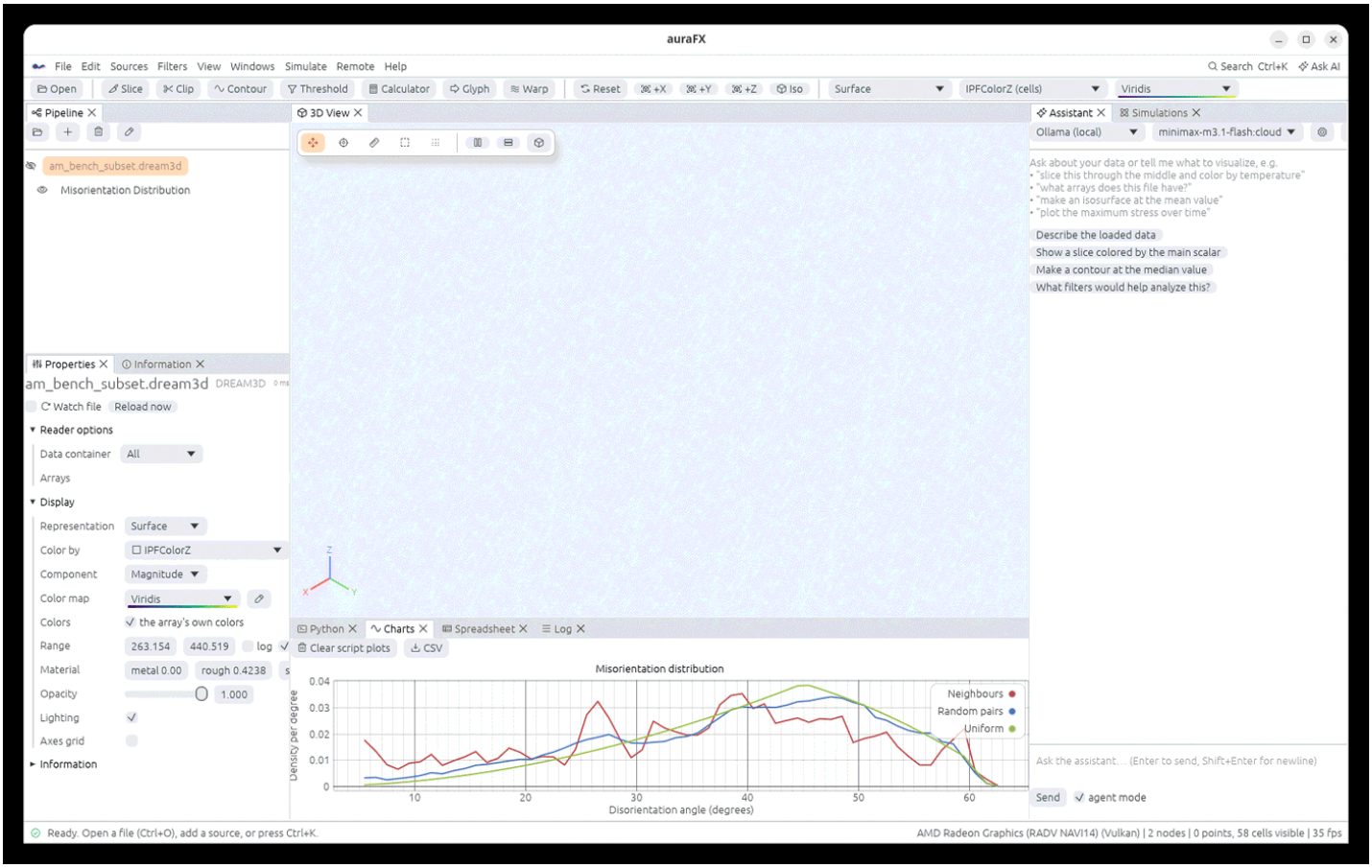
Misorientation Distribution makes a table of disorientation angles (the smallest rotation between two orientations, allowing for crystal symmetry) with three columns:

- **Neighbours:** between neighbouring points of a map or volume.
- **Random pairs:** between randomly chosen pairs of points (**Random pairs** of them). This shows what the texture alone gives, without any spatial correlation.
- **Uniform:** between randomly oriented crystals, computed exactly for the crystal class.

Each column is a density per degree, so all three can be plotted on one axis. Angles below **Smallest angle** (5° by default, meant to leave out the small angles inside grains) are left out of all three. Peaks in Neighbours that are missing from the other two point to special boundaries, such as twins near 60° in cubic metals.

The table plots in the **Charts** panel with `bin_center` on the x axis. Its field data counts the neighbour pairs and gives their mean angle.

```
m = ebsd.misorientation_distribution(min_angle=5, bin=1)
x = m.array("bin_center")
afx.plot(x, m.array("Neighbours"), label="Neighbours", clear=True)
afx.plot(x, m.array("Uniform"), label="Uniform")
```



Texture Components

Texture Components makes a table of the fraction of orientations that have a crystal direction of a family within **Cone** degrees of a sample axis. It has one row per direction and axis, such as `<100> Z` for orientations with a cube axis along Z. The defaults are the families `100, 110, 111` against the axes `X, Y, Z`, with a 15° cone.

```
tc = ebsd.texture_components(families="100, 111", axes="Z", cone=15)
```

Chord Lengths

Chord Lengths measures grain size along straight lines through a map or volume.

- **Lines.** They run in x, y and, for a volume, z. **Line spacing** is the distance between them, in length units; 0 uses every row of points.

- **Boundaries.** A boundary lies between two neighbouring points whose disorientation is at least **Boundary angle** (10° by default). A chord is the distance between two boundaries that follow each other along a line.
- **Unused points.** Points of other phases, unindexed points and masked points are skipped. A run of more than **Largest gap** of them ends the line segment there.
- **Partial chords.** A chord that touches a segment's end, or the edge of the data, is left out, because its true length is unknown.
- **Specks. Ignore single-point specks** stops a single stray point, whose two neighbours agree with each other, from counting as two boundaries.

Output gives either every chord (its direction and length) or a histogram with one column per direction. Both carry the mean chord and the number of chords per direction as field data (`MeanChordX`, `ChordsX`, …).

```
ch = ebsd.chord_lengths(threshold=10, output="Histogram", bin=2)
print(ch.array("MeanChordX", "field"))
```

Synthetic microstructures

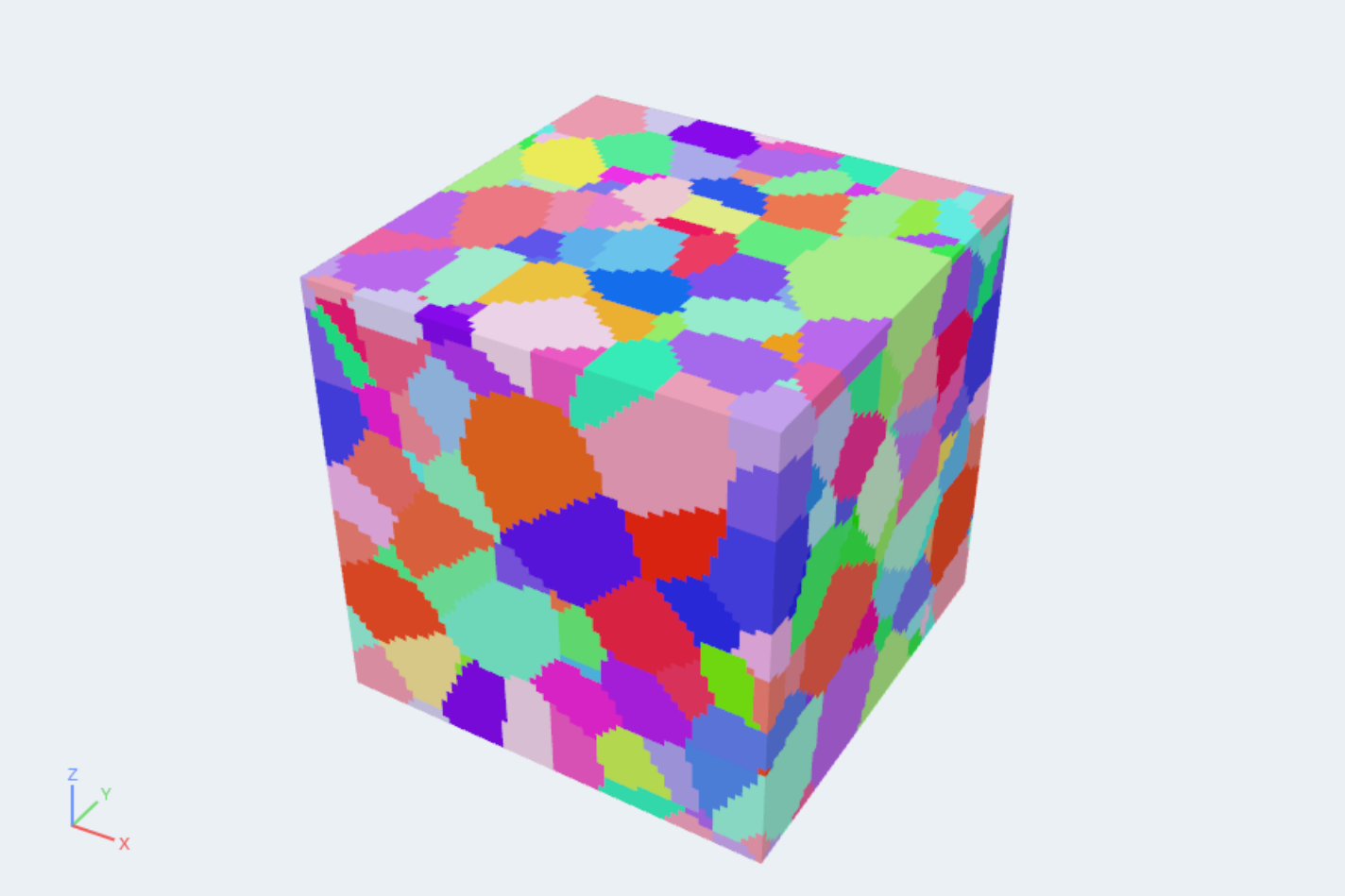
These sources build microstructures without measured data. They have the same arrays an EBSD reader gives: `FeatureIds`, `Phases` and `EulerAngles` per cell, and `CrystalStructures`. Every filter in this chapter therefore works on them, and Overlay Mesh turns them into a mesh. A fixed **Random seed** gives the same microstructure every time.

Voronoi Grains

Sources › Voronoi Grains places **Grains** seeds at random in a grid of **Cells** and gives every cell to its nearest seed.

- **Size spread** above 0 makes Laguerre grains: each seed gets a radius drawn from a lognormal distribution with that log-sigma, and a cell goes to the seed with the smallest squared distance minus the squared radius. Grain sizes then vary much more than in Voronoi grains.
- **Periodic** wraps the grains across opposite faces, for representative volume elements.
- **Orientations** are random, or a fibre with `<100>` or `<111>` along Z, scattered by **Fibre spread** and random about Z.
- The seed positions are kept as the field array `Seeds`.

```
v = afx.voronoi_grains(cells=[64, 64, 64], grains=300, size_sigma=0.4,
periodic=True)
```



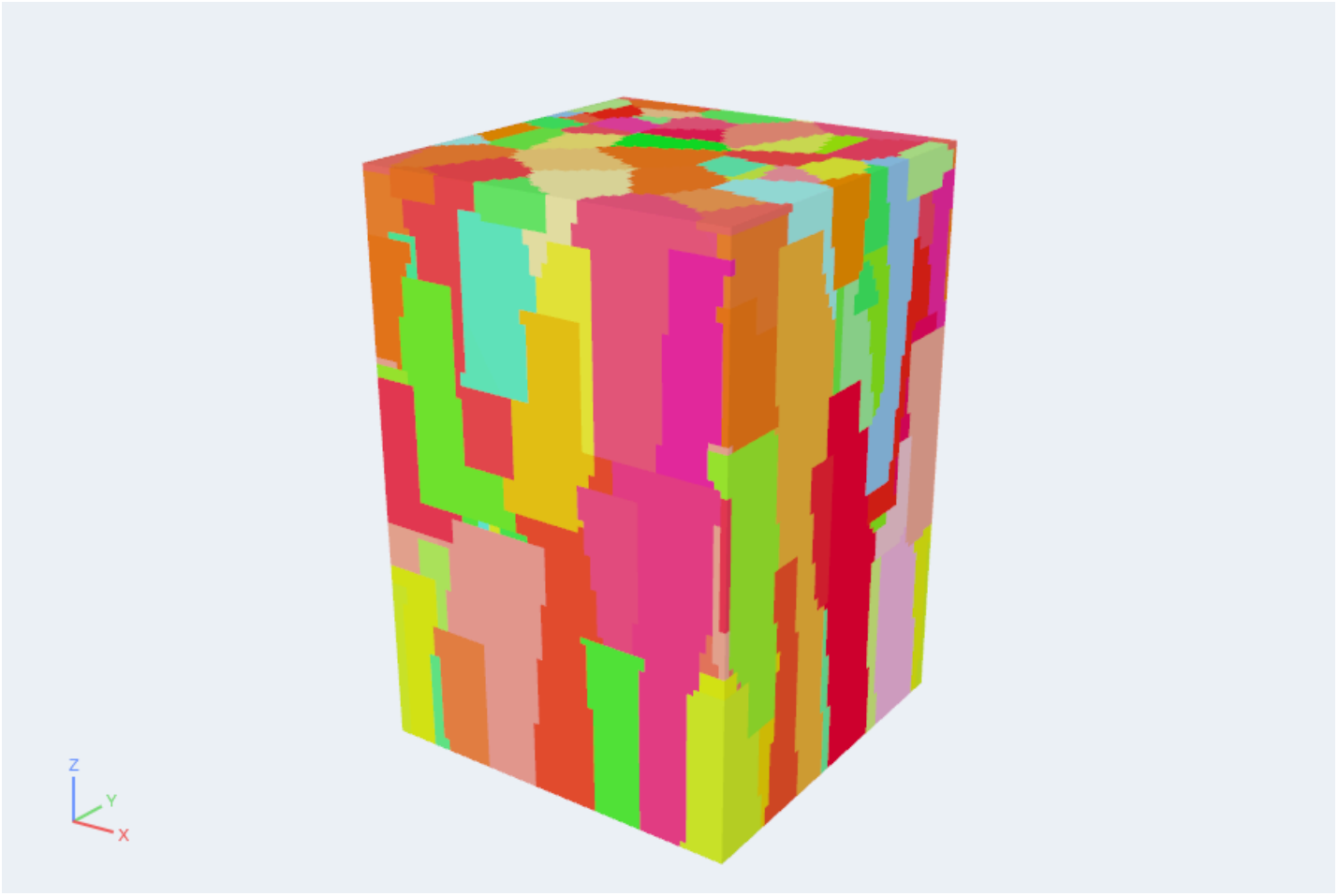
Columnar Grains

Sources › Columnar Grains grows the columnar grains of an additive build along Z:

- **Columns.** As many columns as fit one **Grain width** apart are alive at every height. Each is tilted at random (**Column tilt**) and grows to a lognormal height (**Median grain height**, **Height spread**).
- **Ending.** A column also ends when its neighbours grow over it. Each slice of the box is shared among the columns alive there, so a column can lose its whole cross-section.
- **Renewal.** A column that ends is replaced at that height by a new grain at a random position.
- **Widths. Width spread** and **Width grows with height** make the columns differ in width, with tall grains wider.
- **Orientations.** By default every grain has a `<100>` direction near Z, as in many additively built cubic metals.

- **Per-grain field arrays:** `GrainBase`, `GrainTop`, `GrainTilt` and `Originating` (1 for grains that start inside the build rather than on the base plate).

```
g = afx.columnar_grains(size=(300, 300, 450), voxel=5, width=50, height=150)
```



Add Orientation Noise and Jumble Orientations

Add Orientation Noise turns every point's orientation by a small random rotation. The rotation is about a random axis, with an angle drawn from a normal distribution with standard deviation **Angle**, so the mean turn is about 0.8 times **Angle**. With **One rotation per feature**, every point of a grain turns alike.

Jumble Orientations gives the grains each other's orientations at random. The texture stays the same, but any link between a grain's orientation and its neighbours' or its shape is lost. Each grain takes the orientation of the first point of the grain it swapped with.

```
noisy = v.orientation_noise(sigma=2)
mixed = v.jumble_orientations()
```

Generate Ensemble Statistics

Generate Ensemble Statistics describes a segmented microstructure (one with `FeatureIds`) as a table of fits:

- **Size:** a lognormal fit of the equivalent diameters (`ESD mu` and `ESD sigma` of their logarithms), with the mean, smallest and largest diameter.
- **Shape:** beta-distribution fits of the aspect ratios b/a and c/a (a, b and c are a grain's axes, longest first).
- **Neighbours:** a lognormal fit of the neighbour counts.

Features touching the outside are left out unless **Include features on the surface** is on, because their size and neighbours are cut short. The values are also field data of the table.

```
st = v.ensemble_statistics(include_surface=True)
print(st.array("ESD sigma", "field"))
```

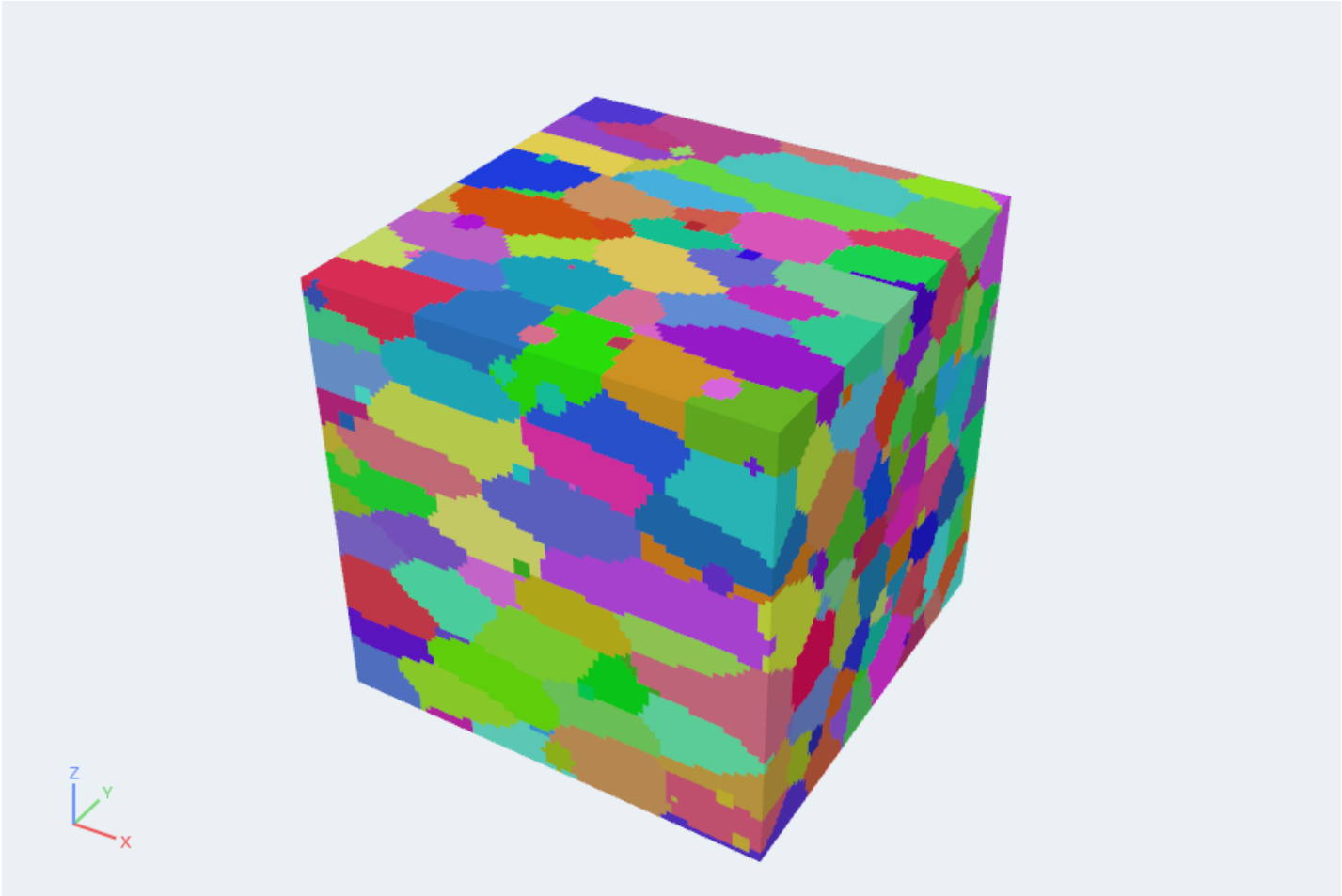
Pack Grains

Sources › Pack Grains builds a microstructure from goal statistics.

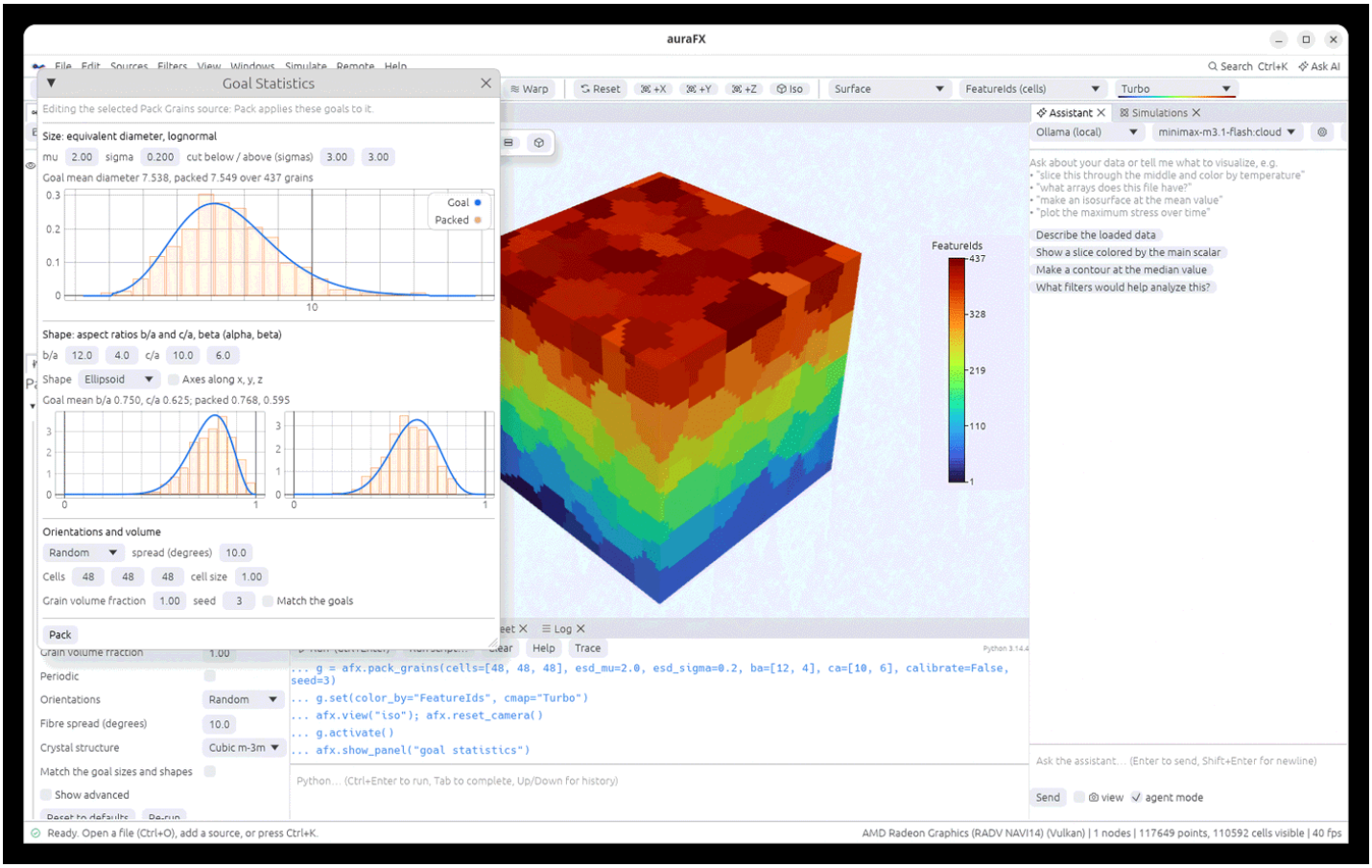
- **Size.** Equivalent diameters are lognormal: **Size mu** and **Size sigma** describe their logarithms. The advanced cut-offs limit the draws to a few sigmas either side.
- **Shape.** The aspect ratios follow beta distributions: **b/a** and **c/a**, each given as (alpha, beta), so their mean is alpha / (alpha + beta).
- **Grain shape** is an ellipsoid, a superellipsoid (boxier as its exponent grows), a cube-octahedron or a cylinder. Its axes are turned at random, or lie along x, y, z with **Axes along x, y, z**.
- **Filling.** Grains are drawn until their volume fills the box. Each is then moved about at random, and a move is kept when it lowers the overlap and the gaps. Every cell goes to the grain it lies deepest inside, and the gaps are filled by growing the grains a layer at a time.
- **Matrix.** With a **Grain volume fraction** below 1, grains are added until they cover that fraction. The rest of the box becomes a matrix phase (phase 2, one feature).
- **Calibration.** Grains that fill space come out a little smaller and flatter than the shapes drawn. **Match the goal sizes and shapes** (on by default) measures the result, shifts the goals by what is missing and packs again, up to four times. The grains away from the box faces then match the goal mean size and aspect ratios to within about 0.02.

`OverlapFraction` in the output's field data is the share of cells that more than one grain reached during packing.

```
g = afx.pack_grains(cells=[80, 80, 80], esd_mu=2.48, esd_sigma=0.15, ba=[12, 4],
ca=[10, 6])
```



View › Goal Statistics edits the goals with live plots: the lognormal size distribution (mu, sigma and where it is cut), the beta distributions of b/a and c/a, the shape, whether the grain axes follow x, y and z, the orientations (random or a fibre with its spread), the volume, the matrix fraction and the seed. With a Pack Grains source selected the panel edits it and draws the packed volume's measured sizes and aspect ratios as bars under the goal curves, with the goal and packed means; with none selected, **Pack** adds one. `afx.show_panel("goal statistics")` opens it from Python.



Pack Like

Pack Like makes a statistical copy of a segmented microstructure in a box of any size:

- It measures the input like Generate Ensemble Statistics.
- It packs grains with those size and shape statistics.
- The new grains take orientations of the measured grains, chosen in proportion to their volume, so the copy has the same texture.

```
copy = measured.pack_like(cells=[100, 100, 100])
```

Insert Precipitates

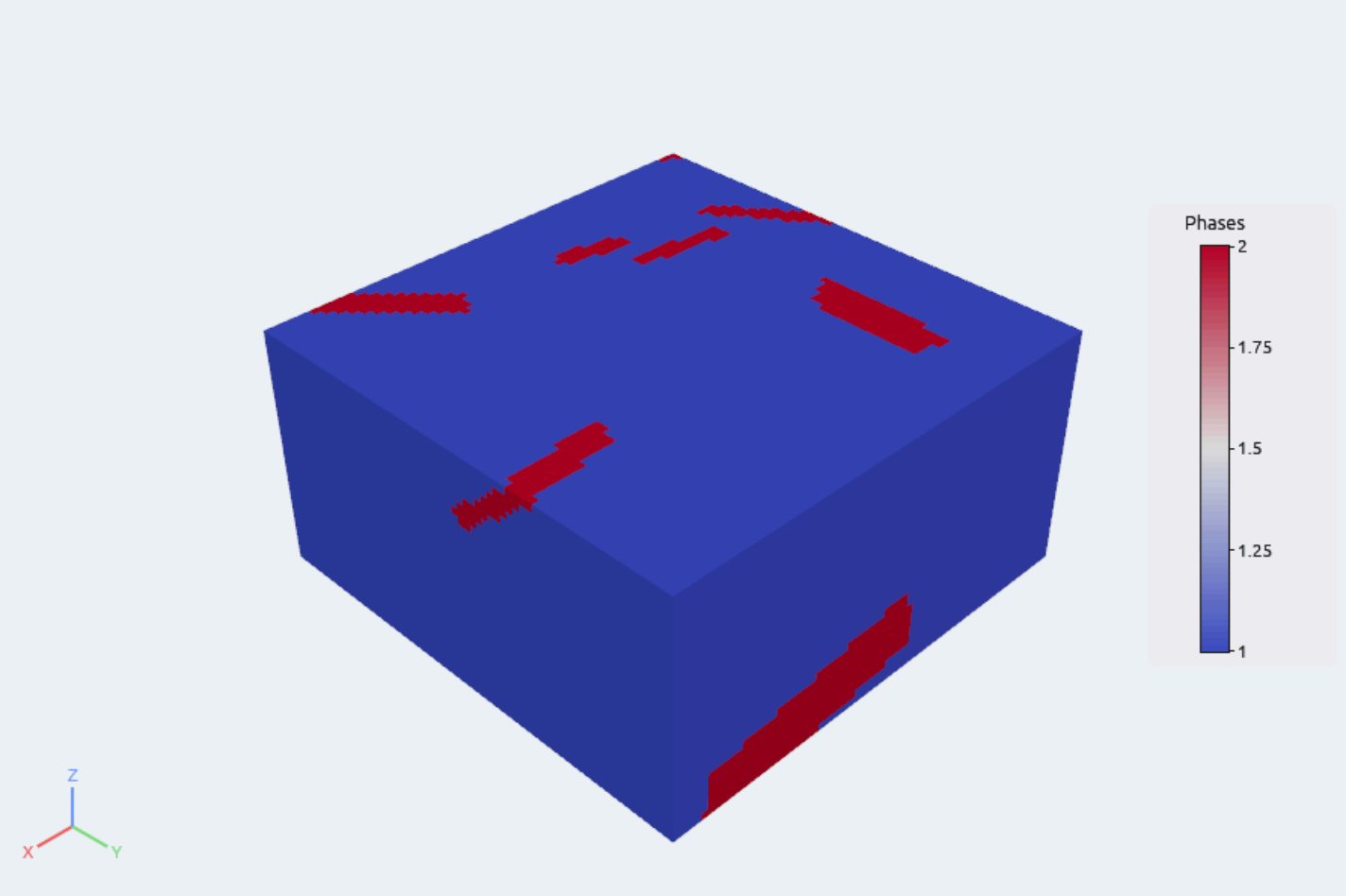
Insert Precipitates adds particles of a new phase to a microstructure until they fill the **Volume fraction**:

- **Shape and size.** Particles are spheres whose diameters are lognormal (**Size mu**, **Size sigma**).
- **Placement.** They are placed at random, or on grain boundaries with **On grain boundaries**. No two particles touch.

- **Output.** Each particle is a feature with a random orientation, in the next phase number, whose crystal structure is added to `CrystalStructures`.

Insert Transformation Phases and Adjust Volume

Insert Transformation Phases puts plates into a fraction of the grains of a parent phase: a plate of a thickness in cells through the grain, on one of its {111} planes, becomes a new feature of a new phase, turned from its parent by an angle about the plane normal (60°: an annealing twin). `TransformationParentIds` gives each plate's parent. **Adjust Volume** moves cells on feature boundaries between neighbours so that each feature's size comes closer to a target (a per-feature array of cell counts or equivalent diameters).



Match Crystallography

Match Crystallography takes a second input: a reference microstructure, normally the measured one. It works in two steps:

1. **Texture.** Each grain of phase 1 gets an orientation drawn from the reference's grains by volume, which reproduces the texture.

2. **Boundaries.** It swaps orientations between grains, or draws new ones, whenever that brings the distribution of disorientation angles across grain boundaries closer to the reference's. The distributions are weighted by boundary area, in 5° bins.

The field data `MDFErrorBefore` and `MDFErrorAfter` give the summed difference between the two distributions before and after the swaps.

```
matched = synthetic.match_crystallography(measured)
```

Grain boundary surfaces

These filters work on the boundaries between the features of a segmented volume.

Quick Surface Mesh

Quick Surface Mesh turns every voxel face between two different features into two triangles, including the faces on the outside of the volume.

- `FaceLabels` (two per triangle) are the features on either side. The triangle's normal points out of the first and into the second, and -1 is the outside.
- `NodeType` counts the features meeting at each node: 2 on a boundary, 3 on a triple line, 4 at a quadruple point. Nodes on the outer faces of the volume get 10 more (12, 13, 14).
- **Orientations.** When the volume has orientations, the surface keeps each feature's mean orientation (`AvgQuats`) and phase (`FeaturePhases`) for the filters below.

Smooth Boundaries

Smooth Boundaries removes the voxel staircase.

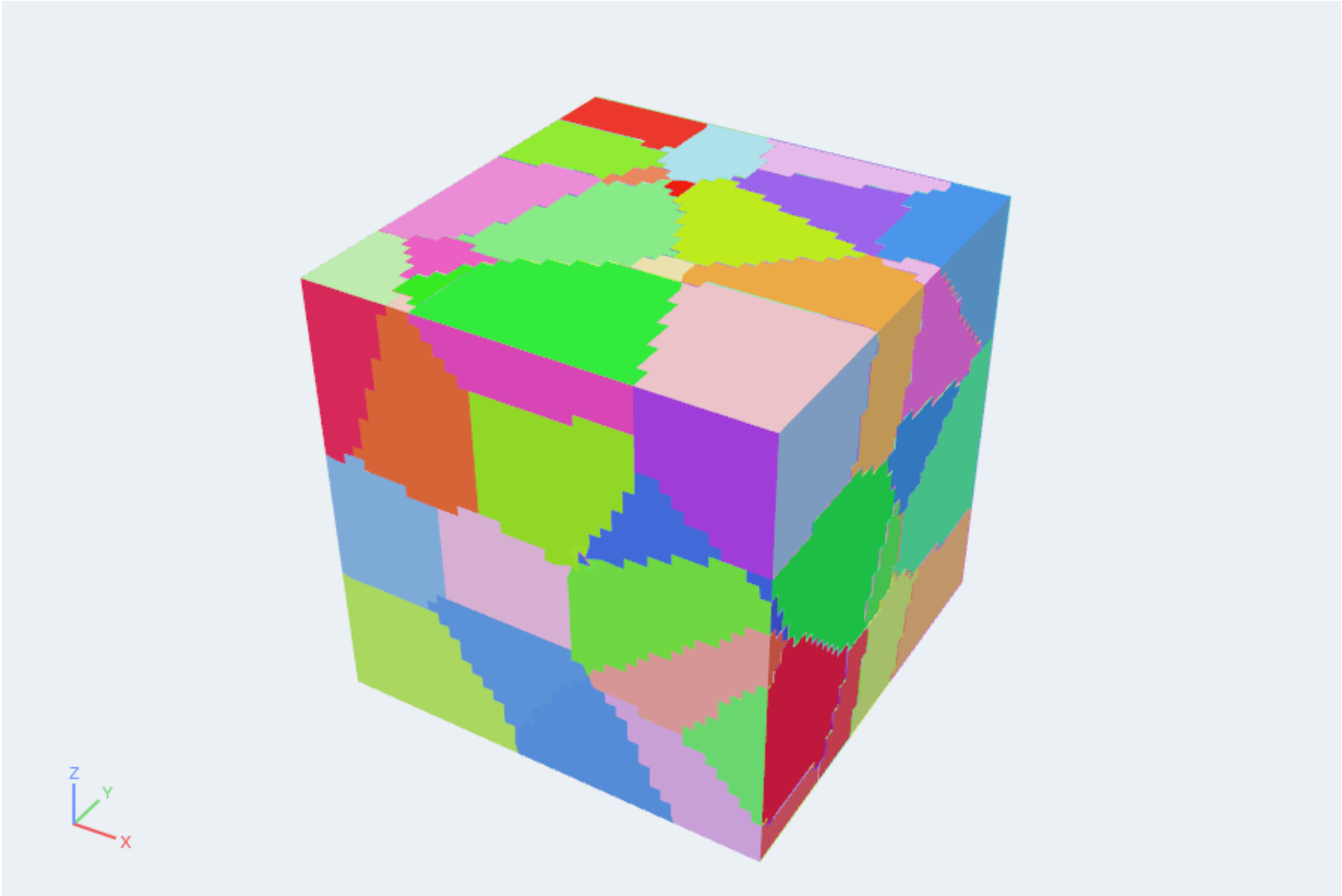
- **Steps.** Each node moves towards the mean of its neighbours by **Step (lambda)**, and on every other pass back by **Inflating step (mu)**. This is Taubin smoothing, which keeps the grains' volumes. With mu 0 it is plain Laplacian smoothing, which shrinks them.
- **Triple lines** follow only their neighbours on triple lines, with their own step. **Quadruple points** stay put by default.
- **Outer faces.** Nodes on the outer faces of the volume move only within those faces.

Triangle Geometry and Feature Surfaces

Triangle Geometry measures each triangle: `FaceAreas`, `FaceNormals`, `FaceCentroids`, and `FaceMinimumAngles` (its smallest corner angle, a measure of its quality). For every feature it also adds, as field data, the volume its boundary encloses (`SurfaceVolumes`) and its boundary area (`SurfaceAreas`).

Feature Surfaces gives the closed surface of each feature, its triangles facing outwards, one block per feature. Saved as STL (`grains.stl`), it writes one file per feature: `grains_feature_1.stl`, `grains_feature_2.stl`, … **Features** chooses some of them.

```
s = volume.quick_surface_mesh().smooth_boundaries(iterations=20)
s.feature_surfaces().save("grains.stl")
```



Triangle angles and boundary curvature

Find Triangle Angles gives each triangle of a surface mesh its smallest and largest interior angle and the largest dihedral angle with its neighbours on the same boundary. **Find Feature Face Curvature** measures the mean and Gaussian curvature of every boundary (one pair of

`FaceLabels`) on its own. A grain surface mesh is also written as text files with `mesh.save("grains.triangles")`: `grains.nodes` (node id, type, x, y, z) and `grains.triangles` (triangle id, its nodes, the two features on either side).

Face Orientation Colors

Face Orientation Colors colours each triangle three ways:

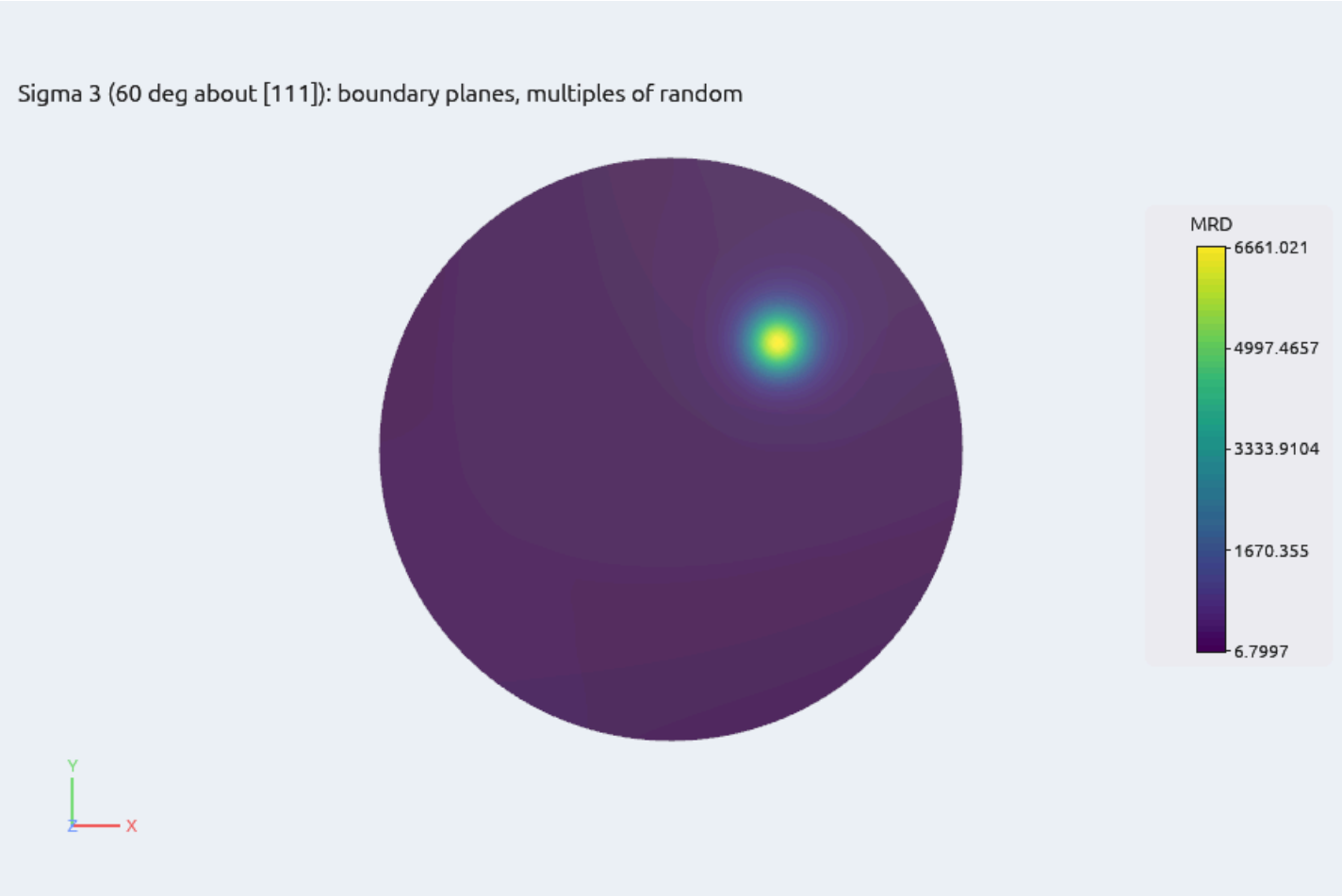
- `FaceIPFColors`: the inverse pole figure key of its plane, in the crystal of the feature it faces out of (on the outer surface, the feature inside).
- `FaceMisorientations`: the disorientation angle across it, in degrees.
- `FaceMisorientationColors`: a colour for that disorientation. The hue comes from where its axis falls in the standard triangle, and it is darker for smaller angles.

GBCD Section

The grain boundary character distribution (GBCD) tells how often each boundary plane occurs for each misorientation, in multiples of random. **GBCD Section** shows one misorientation (**Misorientation angle** about **Misorientation axis**, within **Tolerance**). It is a disk of crystal directions (stereographic or equal area, [001] at the centre), colored by how often boundaries of that misorientation lie on each plane.

- **How it is counted.** Every boundary triangle counts from both sides, with every symmetry-equivalent description of its misorientation, weighted by its area.
- **Random.** A microstructure with random orientations and planes gives 1 everywhere.
- `MisorientationMRD` is how much more often than random the boundaries have that misorientation at all.
- **Smooth first.** The voxel staircase points the faces along the grid axes, so a GBCD of an unsmoothed surface smears each plane into a ring. Use Smooth Boundaries first.

```
g = s.gbcd_section(angle=60, axis=(1, 1, 1), tolerance=5)
```



Mechanics and additive manufacturing

Crystal Mechanics

Crystal Mechanics works out, for every orientation and a **Loading direction** in the sample, three quantities:

- `SchmidFactor`: the largest Schmid factor of the material's slip systems, with the system in `SlipSystem` (numbered as in the field array `SlipSystemNames`).
- `TaylorFactor`: the full-constraint Taylor factor for straining along the direction. It is exact: every set of five independent slip systems is tried, weighted by the relative resistance of their families.
- `YoungsModulus`: in GPa, from the elastic constants.

The field data gives the means, the Voigt (`YoungsModulusVoigt`), Reuss and Hill averages of the modulus, and, for materials with a known base strength, a yield estimate:

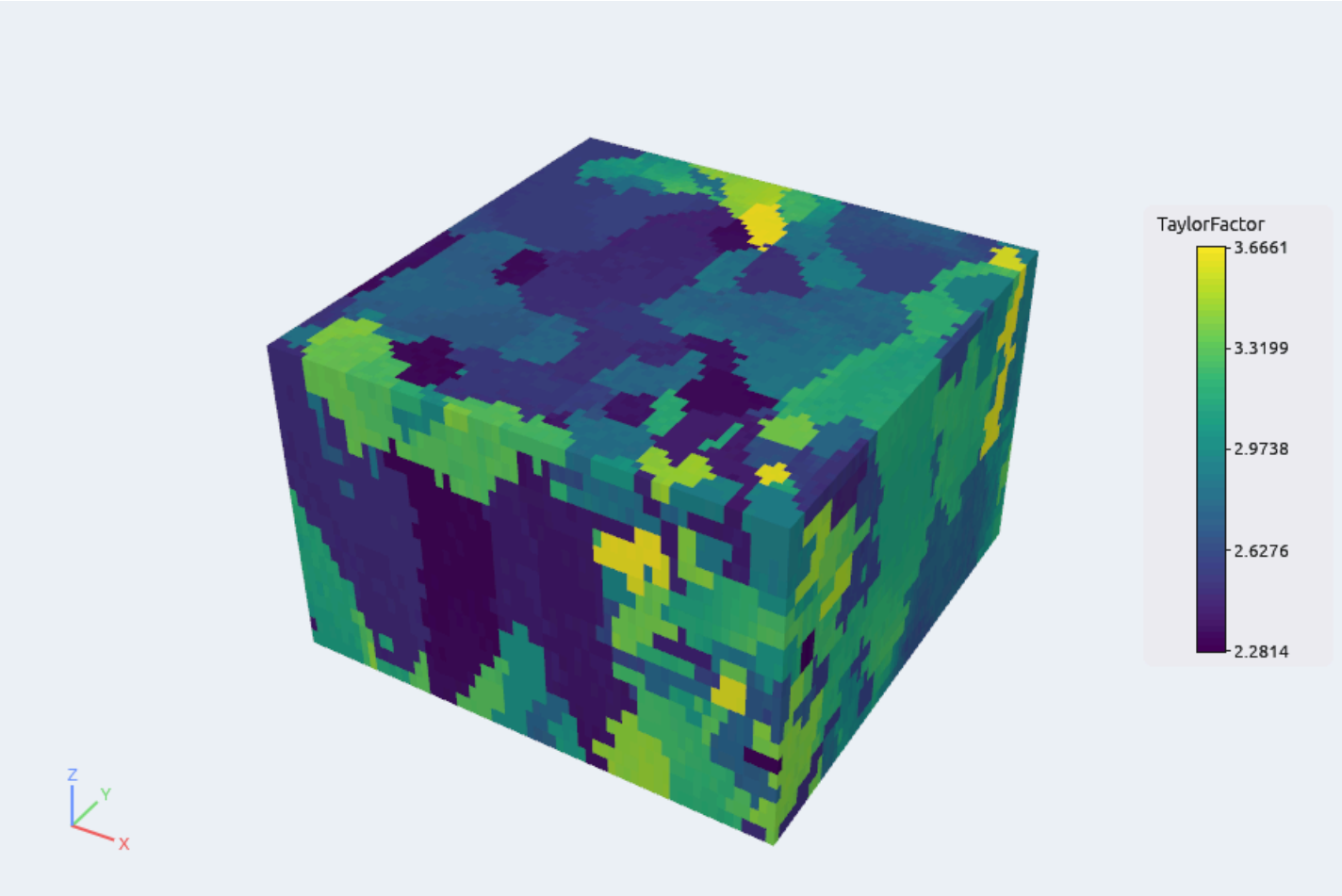
$$\sigma_y = \sigma_{base} \cdot M / 3.06 + \alpha \, \mu \, b \, M \sqrt{\rho}$$

Here M is the mean Taylor factor, 3.06 is a random texture's, and ρ is the mean of an optional dislocation-density array (from GND Density). The result is in `YieldEstimateMPa`, with its two parts in `YieldTextureMPa` and `YieldDislocationMPa`.

Material	Lattice	Slip families (relative resistance)	Base strength
IN718, CoNi GammaPrint-700	FCC	{111}<110>	647.6 MPa, 400 MPa
Nickel, Copper, Aluminium	FCC	{111}<110>	
Iron (BCC)	BCC	{110}<111>, {112}<111>	
Titanium (HCP)	HCP, c/a 1.587	basal 1, prismatic 0.7, pyramidal 1, pyramidal <c+a> 3	
Magnesium (HCP)	HCP, c/a 1.624	basal 1, prismatic 10, pyramidal 10, pyramidal <c+a> 15	

Custom takes the lattice, the elastic constants (C11 C12 C44, or C11 C12 C13 C33 C44 for hexagonal crystals), the shear modulus, the Burgers vector, c/a and the slip families (written like `Basal <a>=1, Prismatic <a>=0.7`). The families available are {111}<110> for FCC; {110}, {112} and {123}<111> for BCC; basal, prismatic and pyramidal , and pyramidal <c+a>, for HCP.

```
m = volume.crystal_mechanics(material="IN718", direction=(0, 0, 1))
print(m.array("YieldEstimateMPa", "field"))
```



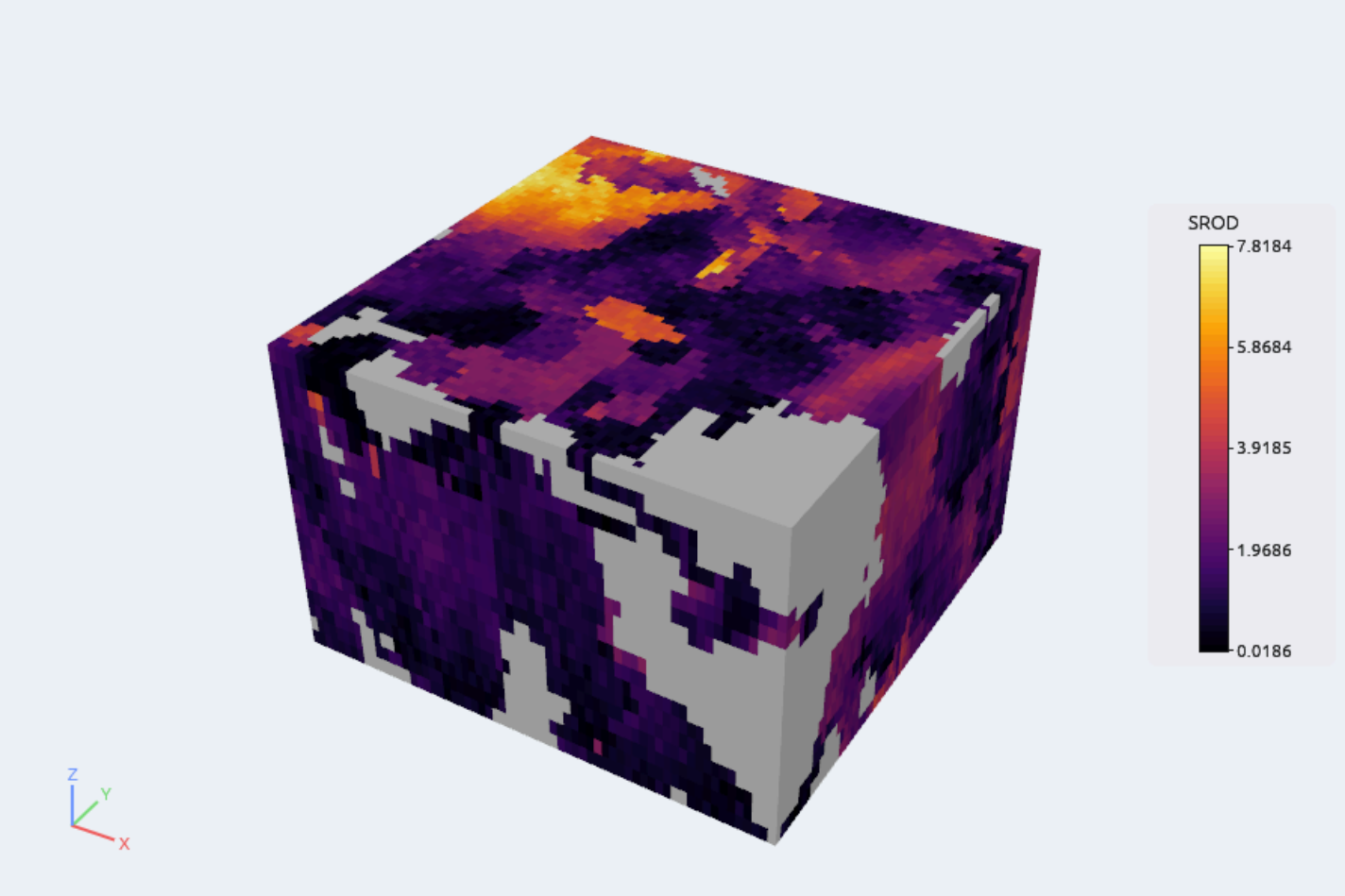
SROD

In an additive build, a grain's orientation turns slowly as it grows layer by layer. **SROD** measures, for every grain, how far its orientation has turned from the orientation it started with, with z the build direction.

- **Reference.** A grain's starting orientation is the mean of its lowest sections that hold the **Reference volume** (at most a few sections).
- **Cells.** Every cell gets its disorientation from that reference (`SROD` , degrees). Cells of grains smaller than **Smallest grain** are left out and drawn grey.
- **Bins.** Bins of one **Layer thickness**, counted from the grain's lowest section, give the disorientation of their mean orientation.
- **Rate.** A grain's rate (`SRODRate` , degrees per layer) is the straight-line slope of its bins after the first.
- **Classes**, by where a grain starts relative to the **Plate top**:
  - *plate*: wholly below it;
  - *epitaxial*: growing out of the plate;
  - *interface*: starting within **Interface height** of it;
  - *originating*: starting higher in the build.

Output *Volume* keeps the volume with `SROD` and the per-grain field arrays (class, base, top, bins, first-bin and largest SROD, rate). *Bins* gives one row per grain and bin, with the spread of its cells (mean and 10th to 90th percentiles). *By height* gives the mean SROD of each class layer by layer, for the Charts panel.

```
s = volume.srod(layer=40, plate_top=0)
by_height = volume.srod(layer=40, plate_top=0, output="By height")
```



GND Density

GND Density estimates the density of geometrically necessary dislocations in every cell (`GND` , 1/m²):

- **Nye tensor** fits the lattice curvature inside the grain over a **Kernel** of cells, by least squares on the small rotations to the neighbouring cells. It turns the curvature into Nye's dislocation tensor and sums its entries over the **Burgers vector**. Cells with too few neighbours in their grain are left empty.
- **From KAM** converts a kernel average misorientation array (Kernel Average Misorientation) as KAM / (step · b).

The field data gives the mean, median and 90th percentile.

On a 2-D map the curvature across the map cannot be measured, so the Nye tensor method sums the five components of the tensor a map gives (a lower bound of the density). A map whose lattice turns steadily about its normal gives back that rate over b within 2 %.

```
g = volume.gnd_density(method="Nye tensor", burgers=2.54e-10, kernel=3)
```

Limits

- Hexagonal-grid EDAX `.h5` files, Oxford `.h5oina` files, and `.dream3d` neighbor lists are not read yet (hexagonal `.ang` files are, resampled on a square grid).
- Cubochoric vectors are not among the representations.
- Neighbor lists (each feature's neighbours and shared areas) are not kept as arrays; Feature Boundaries holds the same information face by face.
- Merge Twins handles cubic m-3m crystals only.
- Texture directions are Cartesian crystal directions. For cubic crystals they are the Miller indices. For hexagonal and trigonal crystals, `001` is the c axis and `100` the a1 axis. Four-index Miller-Bravais notation is not read.
- The texture filters assume no sample symmetry and count every point equally.
- Voronoi and Laguerre grains are tessellations of random seeds, and columnar grains are power diagrams slice by slice; use Pack Grains to match given size and shape statistics.
- Pack Grains matches the mean size and mean aspect ratios. The spread of sizes and the neighbour counts follow from the packing rather than from a goal. Grains cut by the box faces of a periodic volume are measured as pieces.
- Match Crystallography matches the boundary misorientation angles, not their axes. Precipitates are spheres.
- Quick Surface Mesh works on volumes; for 2-D maps use Feature Boundaries. Smoothing keeps the volumes to within about 0.5 % but does not stop a triangle from folding when it runs for many passes on very thin features.
- The GBCD is shown one misorientation at a time; the full five-parameter distribution is not stored as an array.
- The Taylor factor takes at most 30 slip systems (all of FCC, of {110} and {112} BCC, and of the four HCP families together). Twinning is not counted. The yield estimate is the texture and dislocation scaling above, not a crystal-plasticity calculation.
- The feature filters copy the volume they work on: a volume of 80 million cells needs several gigabytes of memory. Segmentation of such a volume takes about 10 seconds.
- The orientation mathematics is checked against independent references: rotations from SciPy, disorientation angles by brute force over every crystal class, and the color key by a separate implementation. On a window of a real volume, the inverse pole figure colors

agree with those stored in the file within one level of 255. The verification notes (`docs/reference/verification.md` in the source tree) list the comparisons.

Geometry and meshing

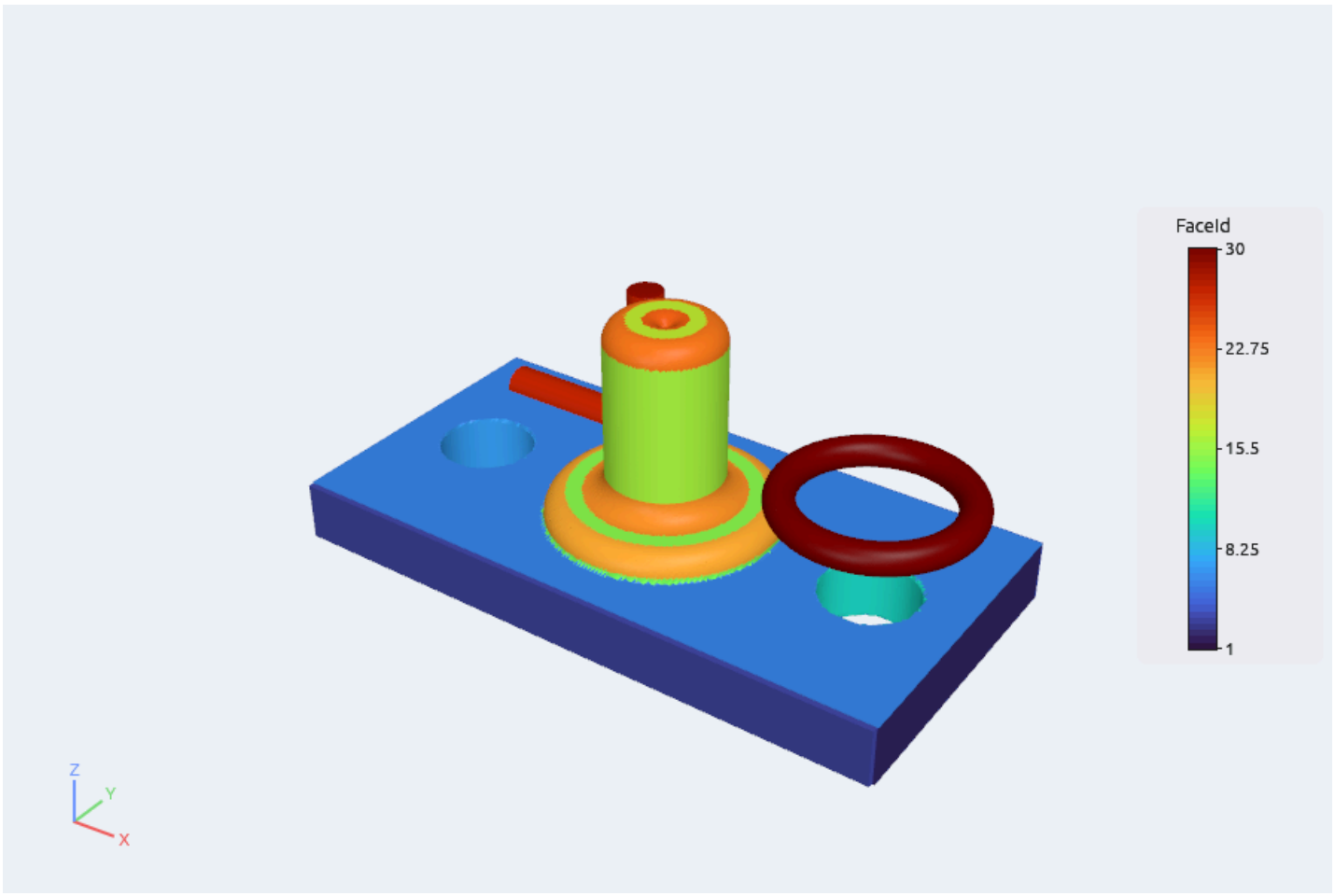
auraFX builds solid geometry from short commands: primitives, booleans, outlines extruded, revolved or swept, and imported surfaces. Each body keeps named faces, so later steps can refer to the top of a plate or the side of a hole. It also makes structured hexahedral meshes: boxes, and boxes of matrix holding inclusions whose surfaces the elements follow exactly. The meshes carry element blocks, side sets and node sets, check their quality, and write Exodus, Abaqus and Gmsh files for solvers. Plates with holes, dog-bone specimens and swept surfaces have their own generators, and bricks and cylinders built with commands can be meshed with `mesh body` commands.

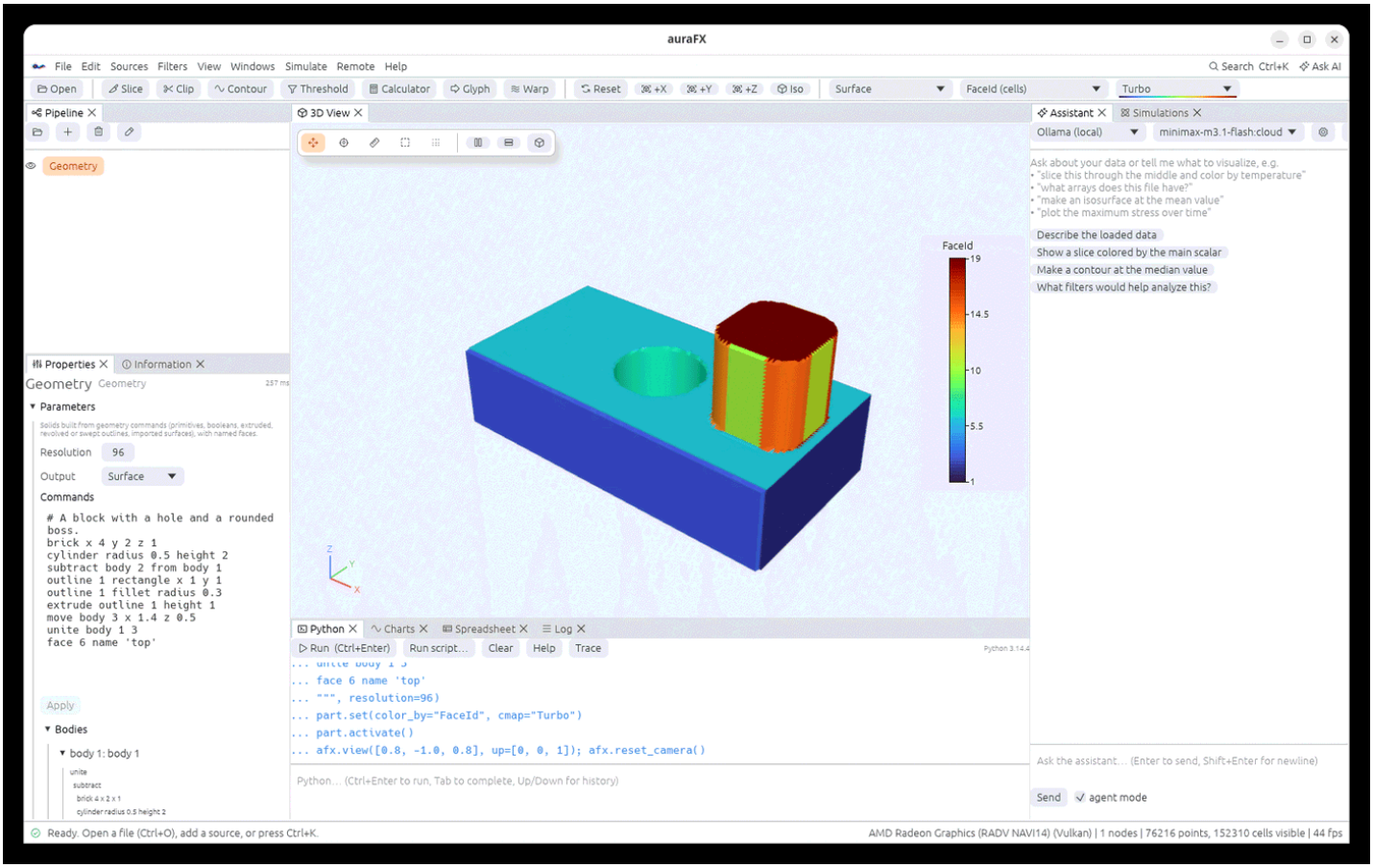
The Geometry source

Add it with **Sources › Geometry**. Its Properties show:

- **Commands**: the command list, edited in the box and applied with **Apply**.
- **Bodies**: a tree of the bodies the commands make. It shows how each body was built and lists its faces with their numbers and names. A mistake in a command shows here, with the line it is on.
- **Resolution**: cells along the longest side of each body when its surface is drawn.
- **Output**: *Surface* gives the bodies' surfaces. *Distance field* gives the signed distance to the nearest body on a grid (negative inside), with the body and face nearest each point.

The surface output carries the cell arrays `BodyId` and `FaceId`. Its field arrays are `FaceNames` (indexed by face number), `BodyIds`, `BodyNames`, and each body's `Volume` and `SurfaceArea`. Color by `FaceId` to see the faces.


FaceId
30
22.75
15.5
8.25
1
Z
Y
X


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
FaceId (cells)
Turbo
Pipeline
Geometry
3D View
FaceId
19
14.5
10
5.5
1
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
Geometry
Parameters
Resolution
96
Output
Surface
Commands
A block with a hole and a rounded boss.
brick x 4 y 2 z 1
cylinder radius 0.5 height 2
subtract body 2 from body 1
outline 1 rectangle x 1 y 1
outline 1 fillet radius 0.3
extrude outline 1 height 1
move body 3 x 1.4 z 0.5
unite body 1 3
face 6 name 'top'
Apply
Bodies
body 1: body 1
unite
subtract
brick 4 x 2 x 1
cylinder radius 0.5 height 2
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
... face 6 name 'top'
... """, resolution=96)
... part.set(color_by="FaceId", cmap="Turbo")
... part.activate()
... afx.view([0.8, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 76216 points, 152310 cells visible | 44 fps

Commands

Write one command per line, or separate commands with `;`. Text after `#` is a comment. Bodies, outlines and faces are numbered from 1 in the order they are made. Sizes are in model units and angles in degrees.

Primitives

All primitives are centred on the origin, and round ones stand along z.

Command	Makes
`brick x 4 y 2 z 1` (or `brick 2` for a cube)	a box; faces `x-`, `x+`, `y-`, `y+`, `z-`, `z+`
`sphere radius 1`	a sphere
`cylinder radius 0.5 height 2`	a cylinder; faces `side`, `bottom`, `top`
`cone radius 1 top 0.4 height 2`	a cone or frustum (top 0 for a point)
`torus major 2 minor 0.5`	a torus in the xy plane
`prism sides 6 radius 1 height 2`	a regular prism; faces `side 1` … , `bottom`, `top`
`superquadric x 1 y 0.8 z 0.6 e1 0.5 e2 0.7`	a superellipsoid (exponents 1 give an ellipsoid; smaller ones square it off)
`harmonic radius 1 coefficient 2 0 0.3 coefficient 3 2 0.1`	a star-shaped body with radius r (1 + Σ c Y_{lm}), real spherical harmonics, for particle and grain shapes
`import 'part.stl' scale 0.001`	a closed surface from a file (STL, PLY, OBJ, VTK polygonal data)

Outlines and profile solids

An outline is a closed 2-D curve. Build it, then extrude, revolve or sweep it.

Command	Does
`outline 1 polyline 0 0 2 0 2 1 0 1`	straight edges through the points, closed back to the start
`outline 1 rectangle x 2 y 1 center 0 0`	a rectangle

Command	Does
`outline 1 circle radius 1 center 0 0`	a circle
`outline 1 ellipse x 2 y 1`	an ellipse
`outline 1 regular sides 6 radius 1`	a regular polygon
`outline 1 spline 0 0 2 0 2 1 0 1`	a smooth closed curve through the points
`outline 1 add line 3 1 3 2`	straight edges appended to the outline
`outline 1 add arc center 0 0 radius 1 start 90 end -90`	an arc appended (counterclockwise when end > start)
`outline 1 fillet radius 0.2`	every corner between straight edges rounded
`extrude outline 1 height 2`	the outline from z = 0 to the height
`revolve outline 1 angle 360`	the outline turned about z; x is the distance from the axis and y the height
`sweep outline 1 path 0 0 0 4 0 0 4 3 0`	the outline carried along a path of points, kept normal to it

Faces of profile solids are named after the outline's curves: `edge 1`, `edge 2`, … for straight edges, then `arc 1`, `fillet 1`, `circle`, `spline 1`. Extrusions add `bottom` and `top`, partial revolutions `start` and `end`, and sweeps `start` and `end`.

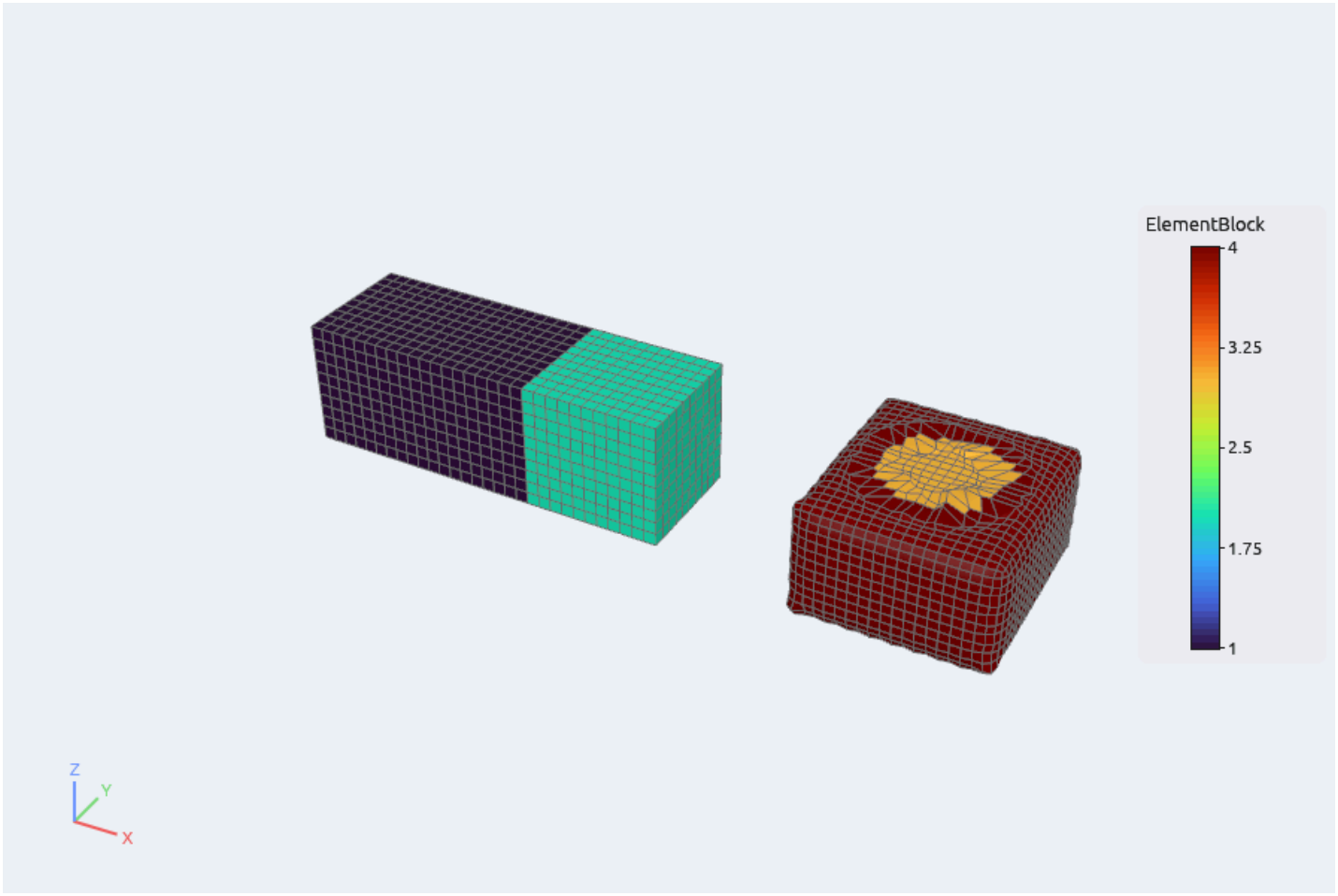
Moving and combining bodies

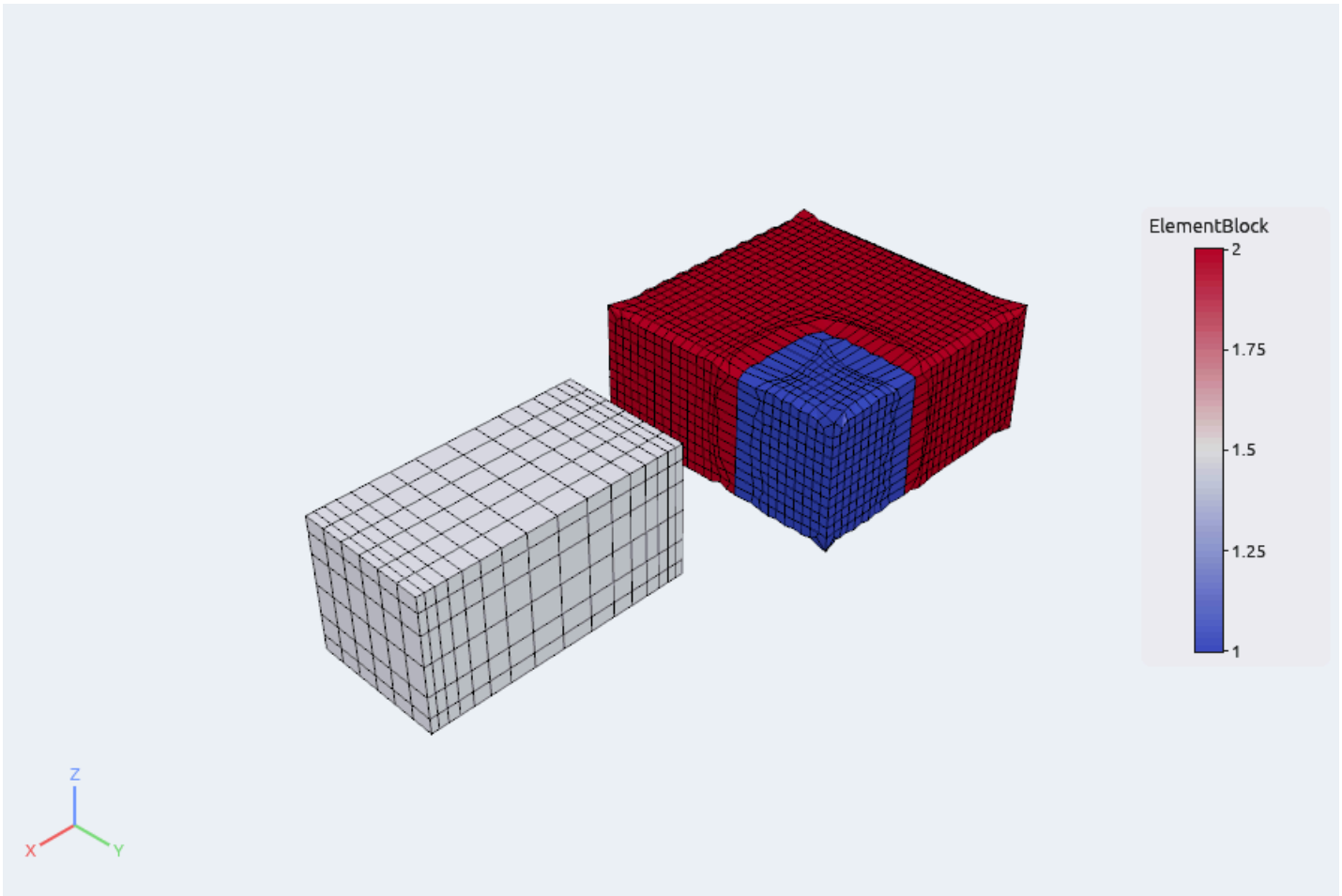
Command	Does
`move body 2 x 1 y 0 z 0.5`	translates
`rotate body 2 angle 30 about z center 0 0 0`	rotates (`about x`, `y`, `z` or a direction `1 1 0`)
`scale body 2 1.5` or `scale body 2 x 2 y 1 z 1`	scales about the origin
`unite body 1 2 3`	one body (numbered as the first) from several
`subtract body 2 3 from body 1`	removes bodies 2 and 3 from body 1 (add `keep` to keep them)
`intersect body 1 2`	the common part

Command	Does
`webcut body 1 with plane xplane offset 0.3`	cuts body 1 in two along a plane (`xplane` , `yplane` , `zplane` , or `normal 1 1 0` , at `offset` along the normal): the part on the side the normal points away from keeps the number, the rest is a new body, both sharing the face `webcut 1`
`webcut body 1 with cylinder radius 0.4 axis z center 0 0 0`	cuts a core out of the body (the core keeps the number); `with body 3` cuts with another body's shape
`webcut body 1 with sweep outline 4 direction 0 0 1`	cuts along a curve swept through the body: the points of outline 4 (in the xy plane, used as an open curve and continued past its ends) swept along the direction; the part on the left of the curve as it runs from its first point to its last (seen from the tip of the direction) keeps the number
`webcut body 1 with sheet 'cut.stl'`	cuts with an open surface read from a file (it must reach across the body): the part its triangles face away from keeps the number
`imprint body 1 2` (or `imprint all`)	touching bodies are meshed with one element size (the smallest asked for), so their meshes meet node to node on the faces they share
`merge body 1 2` (or `merge all`)	the bodies' meshes share the nodes of their common faces (with one size, as imprint); meshing stops with a message if they cannot meet node to node there (bodies offset by part of a cell)
`copy body 1`	a copy as a new body
`delete body 2`	removes a body
`body 1 name 'bracket'`	names a body
`face 6 name 'inlet'`	names a face

A face keeps its name through booleans, so the faces of a hole are still `cylinder 2 side` after the cylinder is subtracted, unless you rename them.

Webcuts make blocks you can sweep. A brick cut along one of its own axes (even moved or turned) gives two bricks, so both stay structured, and their meshes meet node to node on the cut, which becomes an internal surface with a side set of its own. Cut a long part into a coarse and a fine region this way and imprint them so the sizes agree. Other cuts (a slanted plane, a cylinder, another body) make general pieces, meshed together on one overlay grid. When a plane runs exactly through a row of grid-cell centres, the overlay gives those cells to the first body.


ElementBlock
4
3.25
2.5
1.75
1
Z
Y
X


ElementBlock
2
1.75
1.5
1.25
1
Z
X
Y

Meshing bodies

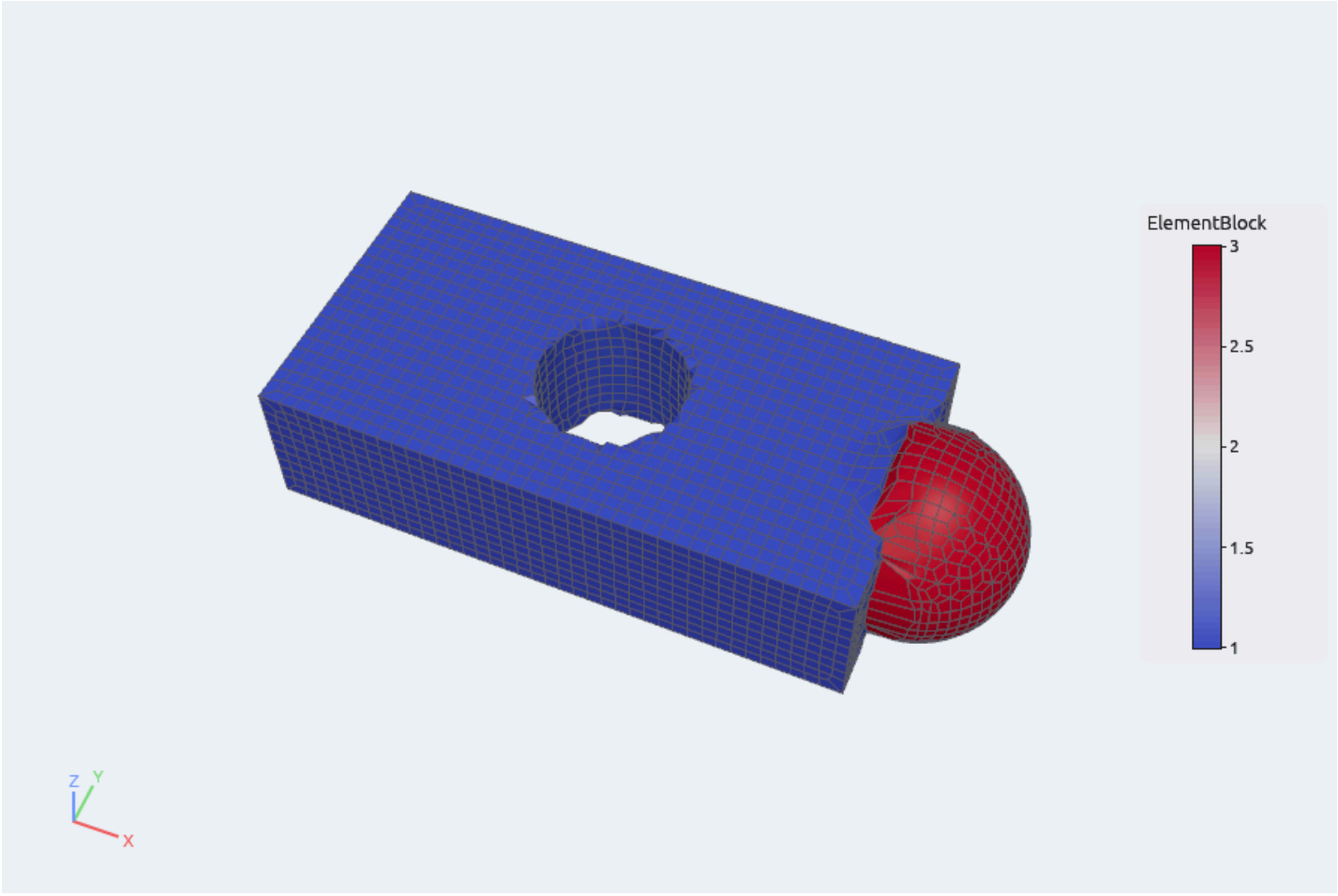
These commands make a hexahedral mesh, shown when the Geometry source's **Output** is *Hex mesh*:

Command	Does
`mesh body 1 2 size 0.1 element HEX20`	meshes bodies with elements of about that size (HEX8 by default)
`mesh body 3 size 0.1 scheme overlay`	chooses how: `structured`, `overlay` or `stairstep` (by default bricks and cylinders are structured, everything else overlay)
`mesh body 1 size 0.1 intervals 20 8 4 bias 3 1 1 cluster start`	a structured body's pieces along its own x, y and z (a cylinder: a quarter of the way round, across the radius, along the height) instead of from the size, and per axis the largest piece over the smallest, the small ones at the `start`, `end` or `both`
`block 7 add body 2 name 'rod'`	puts bodies in an element block (by default each body is its own block, with its id and name)
`sideset 1 add face 6 9 name 'top'`	a side set of the mesh faces on those geometry faces
`nodeset 2 add face 1 name 'left'`	a node set of the nodes on those faces

Bricks become structured boxes, and cylinders become an O-grid disk swept along their axis, each moved, turned and scaled like its body. Every geometry face also becomes a side set named after it, such as `brick 1 x+` or `cylinder 2 side`. With HEX20 or HEX27 elements, the new nodes on a cylinder's side lie on the exact surface. Bodies meshed together are welded where their nodes coincide, and the faces between two bodies' elements form a side set and a node set named `interface <body> - <body>` (for structured and overlay bodies alike).

Every other body (spheres, booleans, outlines, sweeps, imported surfaces) is meshed on an overlay grid; see Overlay meshing below.

- **One grid.** All overlay bodies share one grid of the smallest size asked for, so bodies that touch meet node to node. Each body is its own block, and the faces between two bodies form a side set named `interface <body> - <body>`.
- **Overlap.** Where bodies overlap, the one listed first in the `mesh` commands keeps the overlap.
- **Curved elements.** With HEX20 or HEX27 elements, the new nodes on a body's surface are moved onto it.
- **Stair-step.** `scheme stairstep` keeps whole grid cells, for a quick look.



Python

`afx.mesh.geometry()` builds geometry command by command. Each command is checked at once, so a mistake raises an error naming the line.

```
g = afx.mesh.geometry()
g.cmd("brick x 10 y 5 z 2")
g.cmd("cylinder radius 1 height 4; subtract body 2 from body 1")
for b in g.bodies():                    # id, name, history, faces, bounds
    print(b["id"], b["name"], b["faces"])
part = g.result(resolution=96)          # adds the Geometry source to the pipeline
print(g.journal())                      # every command so far
```

The Geometry source is also an ordinary source: `afx.geometry(commands="sphere radius 1")`. Edits made in the command box are recorded in the trace (**Trace** in the Python panel), so the journal can be replayed.

Signed Distance Field

Filters › Mesh › Signed Distance Field turns a closed surface, such as an STL file or the Geometry output, into an image grid with a `Distance` point array. The array is negative inside. **Resolution** sets the cells along the surface's longest side, and **Margin** adds space around it as a fraction of its size. Contour it at 0 to get the surface back, or threshold it to keep the inside.

```
d = afx.load("part.stl").signed_distance(resolution=128, margin=0.1)
```

Meshes, blocks and sets

A generated mesh is an unstructured grid of hexahedra with the cell array `ElementBlock` (the block each element belongs to) and field arrays for the rest: `ElementBlockIds` and `ElementBlockNames`, one `SideSet <name>` per side set (element index and Exodus side number 1 to 6 per face) and one `NodeSet <name>` per node set (node indices). Exodus files keep all of them, as do Abaqus files (element sets, node sets and surfaces). Gmsh files keep the blocks and side sets as physical groups. Exodus files written elsewhere read back the same way, with their side sets and node sets.

Filters › Mesh › Extract Set shows one set: a side set's faces as a surface (with the owning element's block and element variables) or a node set's nodes as points.

```
walls = mesh.extract_set(set="x-")
mesh.save("part.e")          # or part.inp, part.msh
```

Box Mesh

Sources › Box Mesh makes a box of hexahedra: its **Size**, **Origin** and **Cells** along x, y and z, optionally **Graded**. The grading is the ratio of the largest to the smallest interval along each axis, with the small intervals at the **Start**, the **End** or **Both** ends. It has one block, `block_1`, and side sets and node sets `x-`, `x+`, `y-`, `y+`, `z-` and `z+` on its faces. **Element** gives HEX8, HEX20 or HEX27 elements.

```
b = afx.box_mesh(size=(2, 1, 1), cells=(40, 20, 20), grading=(3, 1, 1),
cluster="Start")
```

Inclusion Mesh

Sources › Inclusion Mesh makes a box of matrix with randomly placed inclusions, meshed with conforming hexahedra. This is the method of hexaform, a structured mesher written by the author of auraFX:

- **Background grid.** A grid of cubic cells covers the box. **Cells along the shortest side** sets the cell size.
- **Packing.** Each inclusion gets a box of whole grid cells. The boxes are placed at random, largest first, at least **Cells between boxes** apart and **Cells from the walls** away from the box faces. So anything that is placed can be meshed.
- **Blocks.** Each inclusion box is filled with a block:
  - a core filled from its six sides;
  - layers out to the exact inclusion surface, where every node lies on the surface;
  - layers out to the flat box faces, whose nodes are the grid's own.
- **Welding.** The pieces are welded exactly, so there are no hanging nodes and the elements fill the box exactly.

Packing *Random* places the inclusions once each, largest first. *Collective* then keeps moving the placed ones, each by a random step or a push towards free space, to open room for the rest. This reaches denser packings, but takes longer. **Shape** chooses spheres, ellipsoids or superquadrics (with **Semi-axes / radius** and the superquadric **Exponents**), or harmonic grains with a random surface of the given **Roughness**. **Radius** and **Radius spread** set the size distribution, and **Random orientation** turns the non-spherical shapes. **Seed** fixes the placement.

The mesh has:

- block 1 `matrix` (the grid and the outer layers) and block 100 `inclusions`, or one block per inclusion with **One block per inclusion**;
- side sets `x-` … `z+` on the box faces and `interface` (the matrix side of every inclusion surface), and node sets `x-` … `z+`;
- the element variable `particle_id`: the inclusion's number inside it, minus that number in its outer layers, 0 elsewhere;
- field arrays `InclusionsPlaced`, `InclusionsRequested`, `VolumeFraction` and `CellSize`.

With **Element** HEX20 or HEX27, the new nodes on the inclusion surfaces are placed on the exact surface too. **Periodic** pairs every node on a box face with the node opposite it (matched by their other two coordinates). It adds node sets `x-_node_set` / `x+_node_set` (and y, z) listing the pairs in the same order, and the field arrays `PeriodicPairs x` (and y, z), for periodic boundary conditions.

The advanced settings shape the blocks:

- the number of layers inside and outside the surface, and their grading;
- the core size and roundness;
- the box size relative to the inclusion;
- the spacing of the nodes on the surface.

The default blocks have a smallest scaled Jacobian of 1/√3 = 0.577, at the corners of the inclusion boxes.

```
m = afx.inclusion_mesh(size=(1, 1, 1), cells=24, count=8, radius=0.075,
shape="Sphere", seed=7)
m.save("rve.e")
```

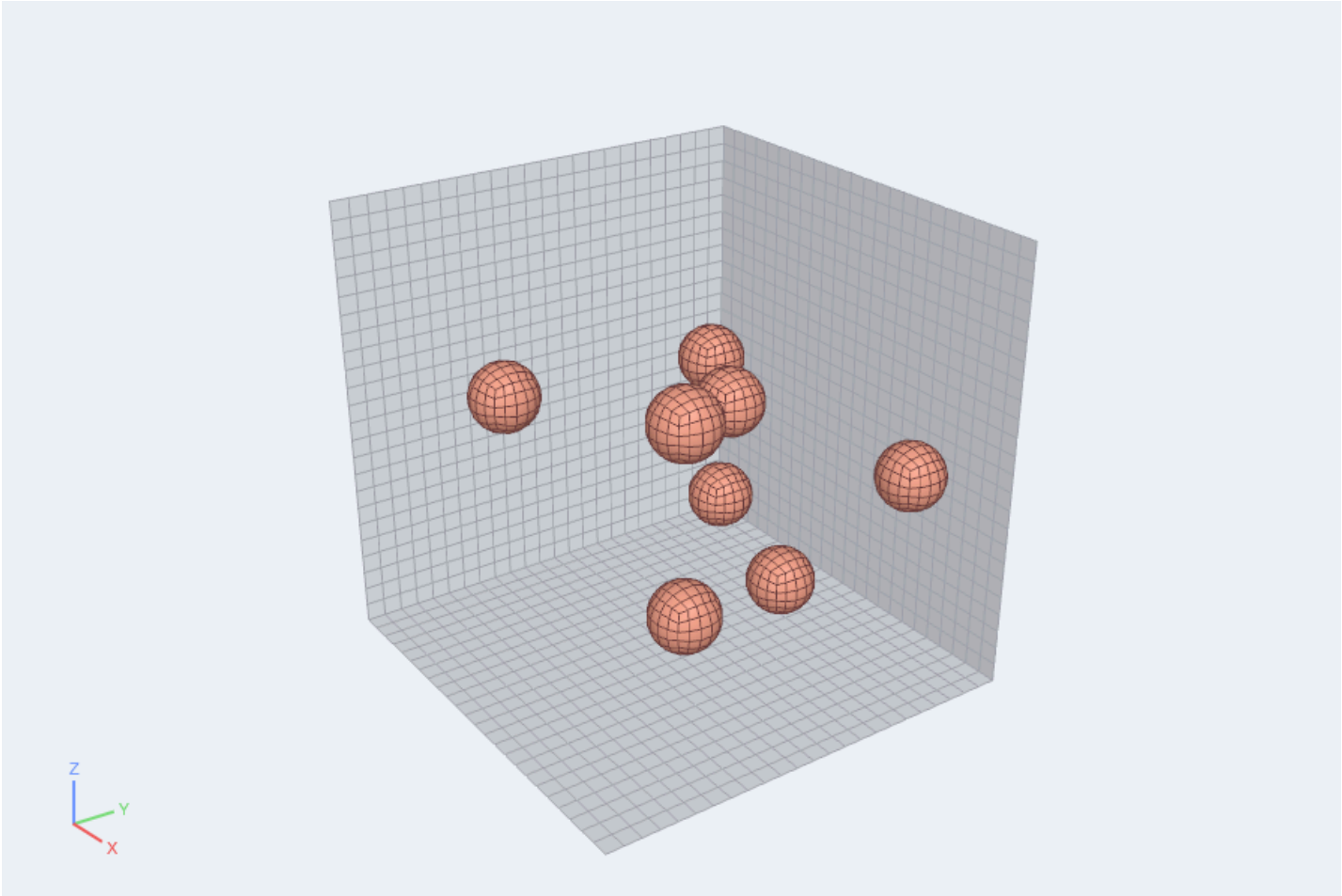


Plate Mesh

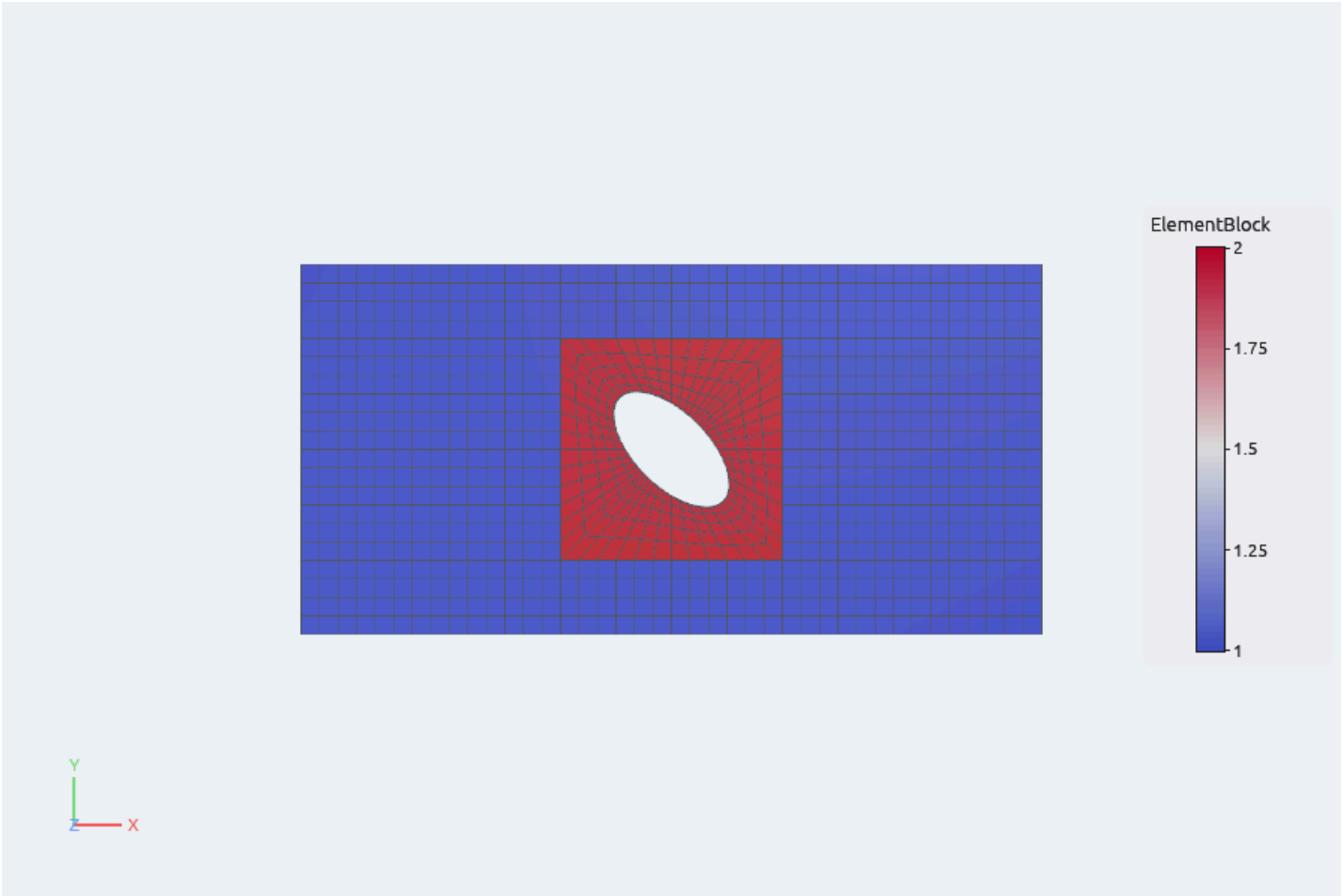
Sources › Plate Mesh makes a plate of **Cell size** hexahedra, one element through the thickness. The width and height must be whole multiples of the cell size. It can have a circular or elliptical through-hole, given by its semi-axes **a** and **b**, the ellipse's **angle** and its centre (empty for the plate centre).

The hole is meshed by a ring of elements whose inner nodes lie on the exact curve and whose outer edge is a rectangle of whole background cells. So the ring hands over to the plate without transition elements. The ring's layers grow from about the hole's arc spacing to the cell size.

The mesh has:

- blocks 1 `plate` and 2 `ring`;
- side sets and node sets `x-` … `z+`;
- node sets `outer` (the four edges) and `grips` (both x faces);
- a node set and a side set `hole`.

```
p = afx.plate_mesh(size=(100, 50, 1), cell=2.5, hole="Circle", a=2.5)
```



Dog-bone Mesh

Sources › Dog-bone Mesh makes a flat tensile specimen as one mapped block:

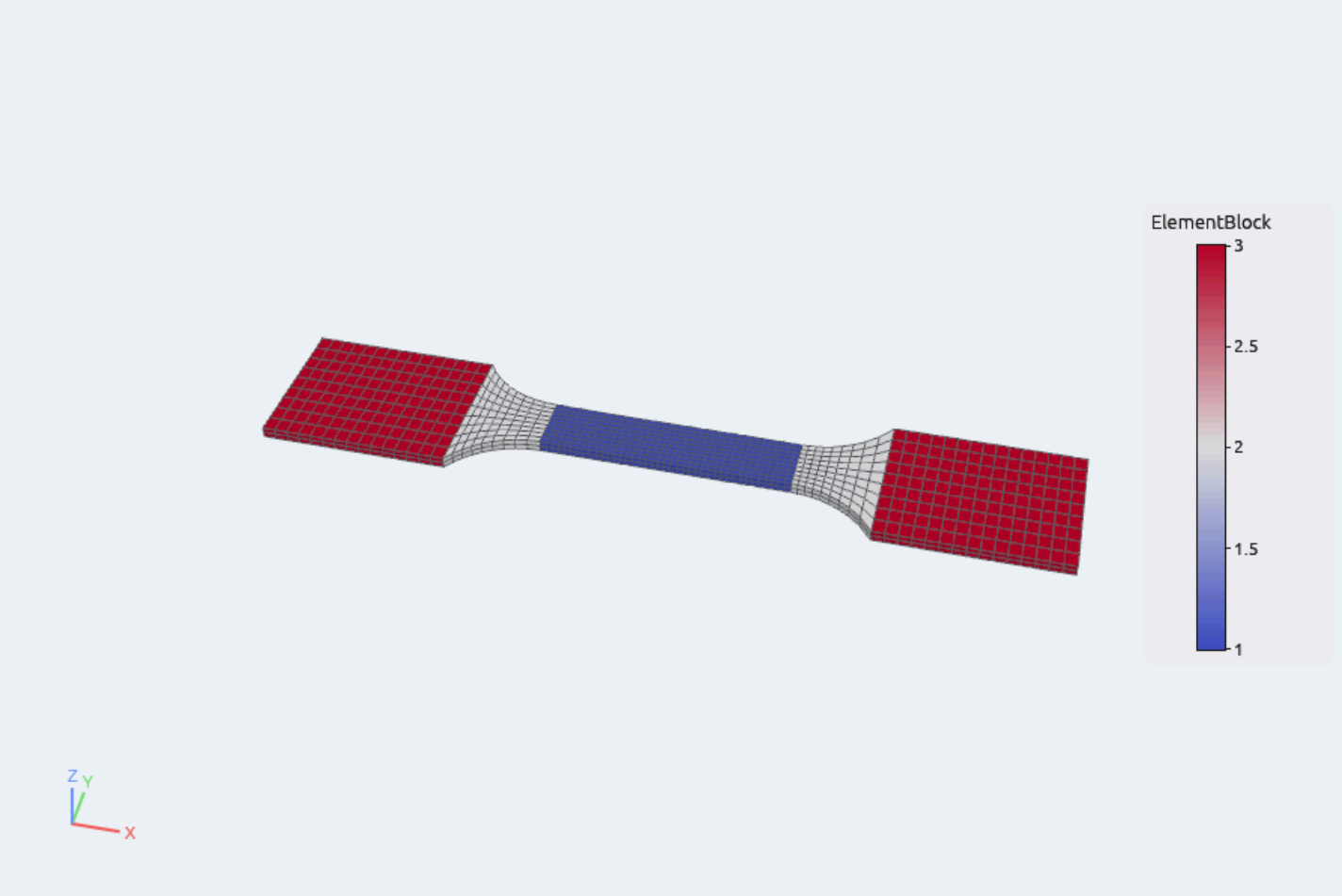
- **Planform.** Grips of **Grip width** at both ends and a gage of **Gage width** and **Gage length** in the middle, joined by circular fillets of **Fillet radius**.

- **Thickness.** Constant in the grips. Along the gage it follows a spline through the **Thickness stations** (pairs of position from the gage start and thickness), and it blends linearly across the fillets.
- **Cells. Cell size** in the gage, growing to **Grip cell size** in the grips, with **Layers through the thickness** layers.

The mesh has:

- blocks 1 `gage`, 2 `shoulder` and 3 `grip`;
- side sets and node sets `x-` and `x+`;
- node sets `grips`, `z-`, `z+`, `edge-`, `edge+`, `gage-` and `gage+`, and `station_1`, … for each thickness station.

```
d = afx.dogbone_mesh(stations=[0, 0.37, 4, 0.35, 8, 0.346], layers=2)
```

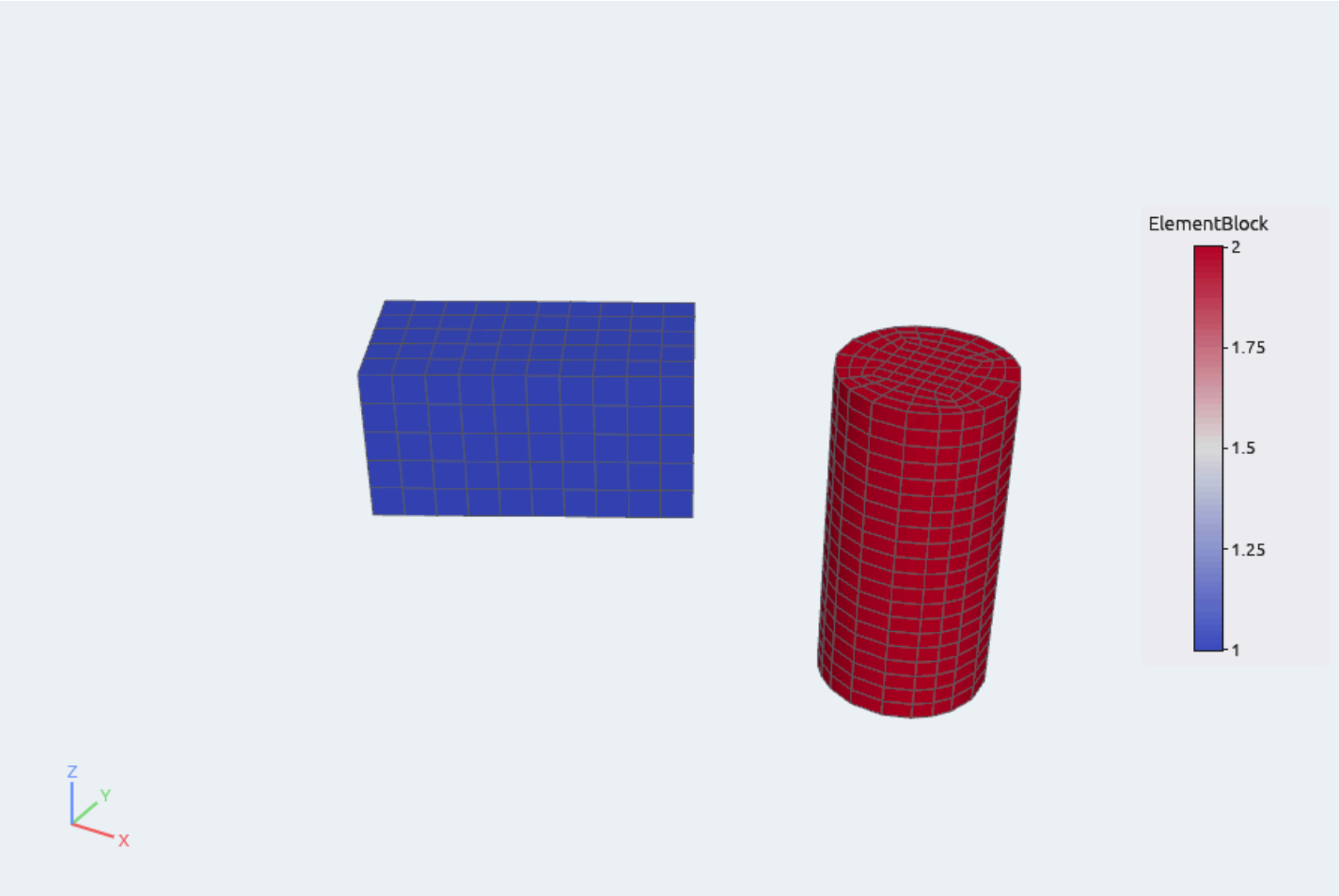


Sweep Quads

Filters › Mesh › Sweep Quads sweeps a surface of quadrilaterals into layers of hexahedra along a **Direction** over a **Distance**. The layers can be graded like Box Mesh's cells. Blocks come

from the surface's `ElementBlock` array. The side sets are `bottom` (the swept surface), `top` and `side` (swept from the surface's boundary edges), with node sets of the same names. Any quadrilateral surface can be swept, such as one face of a mesh from Extract Set or a surface read from a file.

```
thick = plate.extract_set(set="z-").sweep_quads(direction=(0, 0, 1), distance=4,
layers=3)
```



Face Mesh: triangles and quadrilaterals on a planar face

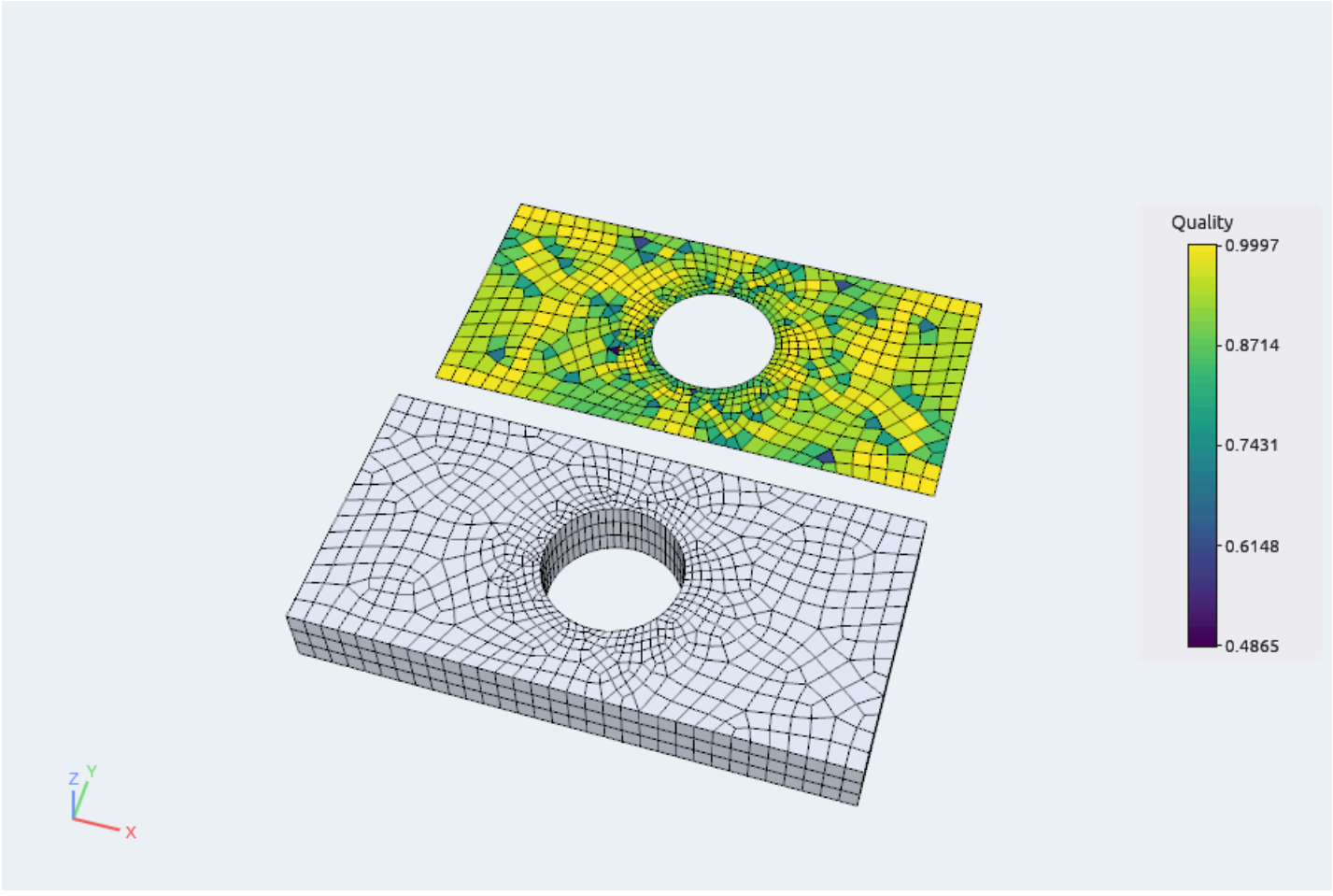
Sources › Face Mesh meshes a flat face drawn with the outline commands (see Outlines and profile solids): an **Outer outline** with **Holes** cut out (outline numbers), at a **Size**, with *Quadrilaterals* (the default) or *Triangles*:

- The outlines are divided at the size, corners kept. **Pieces per outline** fixes the number of pieces of each outline instead (outer first).
- **Size at the holes** and **Growth** refine near the holes: the size there grows by Growth per unit of distance until it reaches Size. **Smallest size** and **Largest size** limit it.

- Triangles are made by Delaunay triangulation and improved (long edges split, short ones removed, edges swapped, nodes smoothed). Quadrilaterals come from triangles of twice the size: pairs of triangles that make good quads are joined, then every quad is cut into four and every triangle left into three, so the face is all quads.
- The cell array `Quality` is each quad's scaled Jacobian (a triangle's: its smallest angle over 60°), and `MinQuality` in the field data the worst.

A plate 4 × 2 with a hole of radius 0.5 at size 0.15 gives roughly its area over the size squared in quads (about 320), the worst scaled Jacobian above 0.3; swept 3 layers through 0.5 with **Sweep Quads** it is 3 hexahedra per quad, all with scaled Jacobians above 0.3.

```
face = afx.face_mesh(commands="outline 1 rectangle x 4 y 2 center 2 1\noutline 2
circle radius 0.5 center 2 1",
                     outer=1, holes="2", size=0.15, near=0.05, growth=0.3)
plate = face.sweep_quads(direction=(0, 0, 1), distance=0.5, layers=3)
```



Remesh Surface

Filters › Mesh › Remesh Surface meshes any triangle surface again (from a file, Extract Surface or a Geometry source), with *Triangles* or *Quadrilaterals*:

- **Size** (0: the input's mean edge length), or **Size from a point array** (the size wanted at each point, such as a Calculator result), or **Curvature: chord tolerance**, which makes the size small where the surface curves so that the mesh stays within the tolerance of it (a chord of length 2 √(2 R t) on a radius R).
- Edges where the surface turns by more than **Sharp edge angle**, and open boundaries, are kept; the other nodes slide along the surface and are put back onto it.

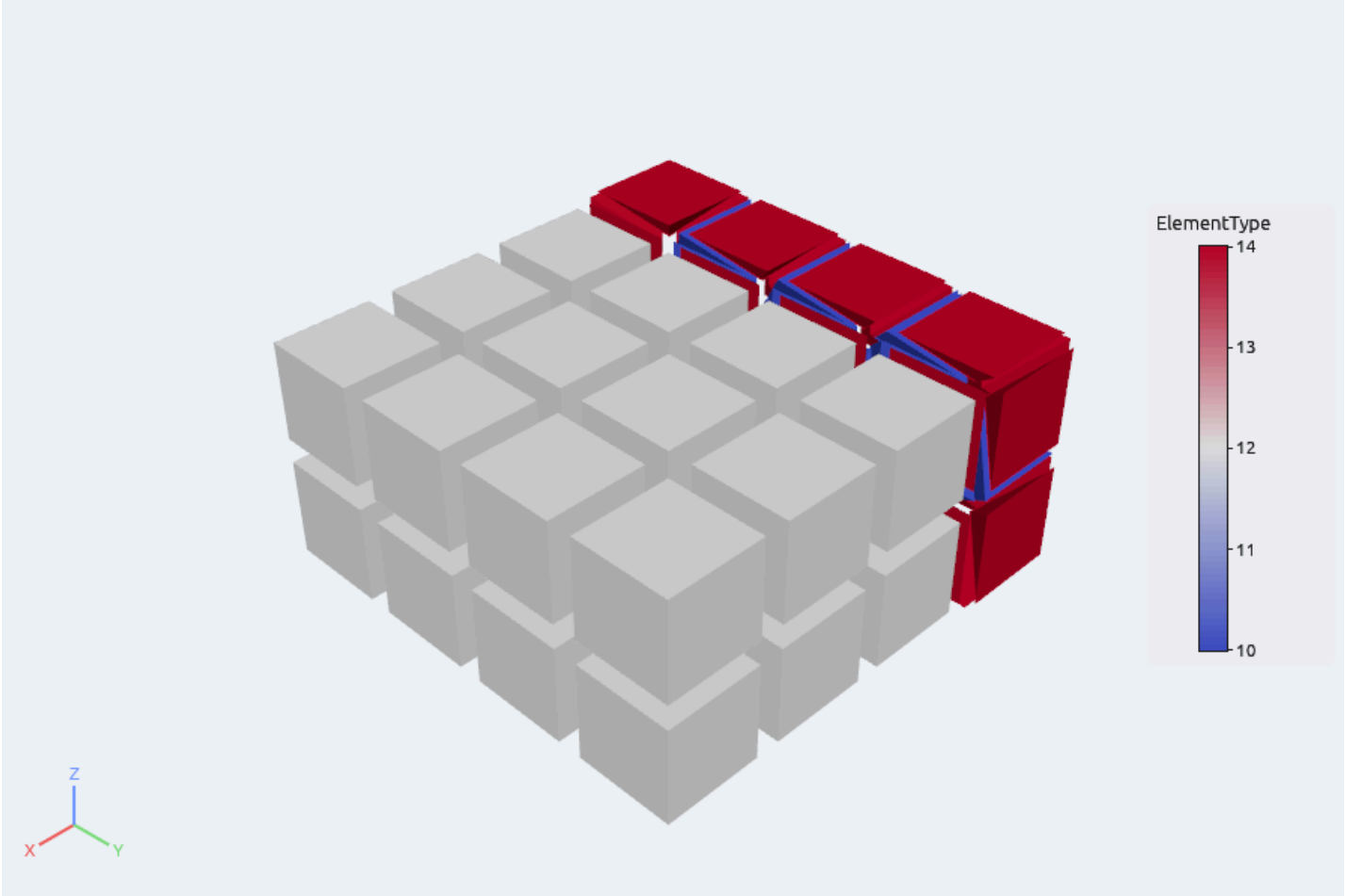
A sphere of radius 0.5 remeshed at 0.05 has edges within 20 % of 0.05; with a chord tolerance of 0.002 its edges are close to 0.089.

```
r = part.remesh_surface(size=0.05)
c = part.remesh_surface(size=1, tolerance=0.002, element="Quadrilaterals")
```

Mesh sets, merging and mixed meshes

- **Filters › Mesh › Define Mesh Set** adds a *Side set* (the boundary faces whose nodes all satisfy an expression) or a *Node set* (the nodes that do), named, from an expression of `coordsX`, `coordsY`, `coordsZ` and the point arrays, like the Calculator: `abs(coordsZ - 1) < 1e-6` or `coordsX*coordsX + coordsY*coordsY < 0.25`.
- **Merge Nodes** joins nodes closer than a tolerance (meshes made apart that touch), renumbering the cells and node sets; `MergedNodes` in the field data says how many went.
- **Tetrahedra in a Region** turns the hexahedra all of whose nodes satisfy an expression into tetrahedra. Each gets a node at its centre: across a face between two converted hexahedra there are four tetrahedra, and on a face against a hexahedron that stays (or on the boundary) a pyramid, so the mixed mesh is conforming. The volume is unchanged; cell arrays follow, and a side on a converted hexahedron moves to its pyramid's base.
- To move, turn or scale a mesh, use **Transform**: blocks and sets stay with it.

```
m = afx.box_mesh(size=(1, 1, 1), cells=(8, 8, 8))
m = m.define_mesh_set(kind="Side set", name="hot", expression="coordsZ > 1 - 1e-6")
mixed = m.hex_to_tets(expression="coordsX < 0.5")
```



Overlay meshing

Overlay meshing fills any shape with hexahedra. It works in five steps:

1. **Grid.** A grid of cubes is laid over the shape, and each cell takes the material found at most of its sample points.

2. **Clean-up.** Pieces smaller than **Smallest piece**, and cells that stick out or form layers one cell thick, join their surroundings. Around every grid node, each material and the rest must each be connected through faces, and no material may form a saddle (faces pointing both ways along an axis). Single cells are changed until this holds, so the next step always has room.

3. **Pillowing.** Each material is wrapped in a layer of elements. No element then has more than one face on a surface.

4. **Surfaces.** The nodes on each surface move onto the true surface. Where two faces of the geometry meet at a convex edge, the nodes nearest the edge are put on it, and on its corners. For imported surfaces, the faces are the patches between creases of more than 30°. Where the shape is only known as voxels, the nodes are smoothed along the surface instead.

5. **Smoothing.** The rest of the mesh is smoothed in stages:

   - Laplacian smoothing;
   - small moves of the nodes of elements whose scaled Jacobian is below 0.6;
   - a wider search below 0.3;
   - untangling of anything still flat.

   Every move is kept only if the elements around it stay valid.

The result has:

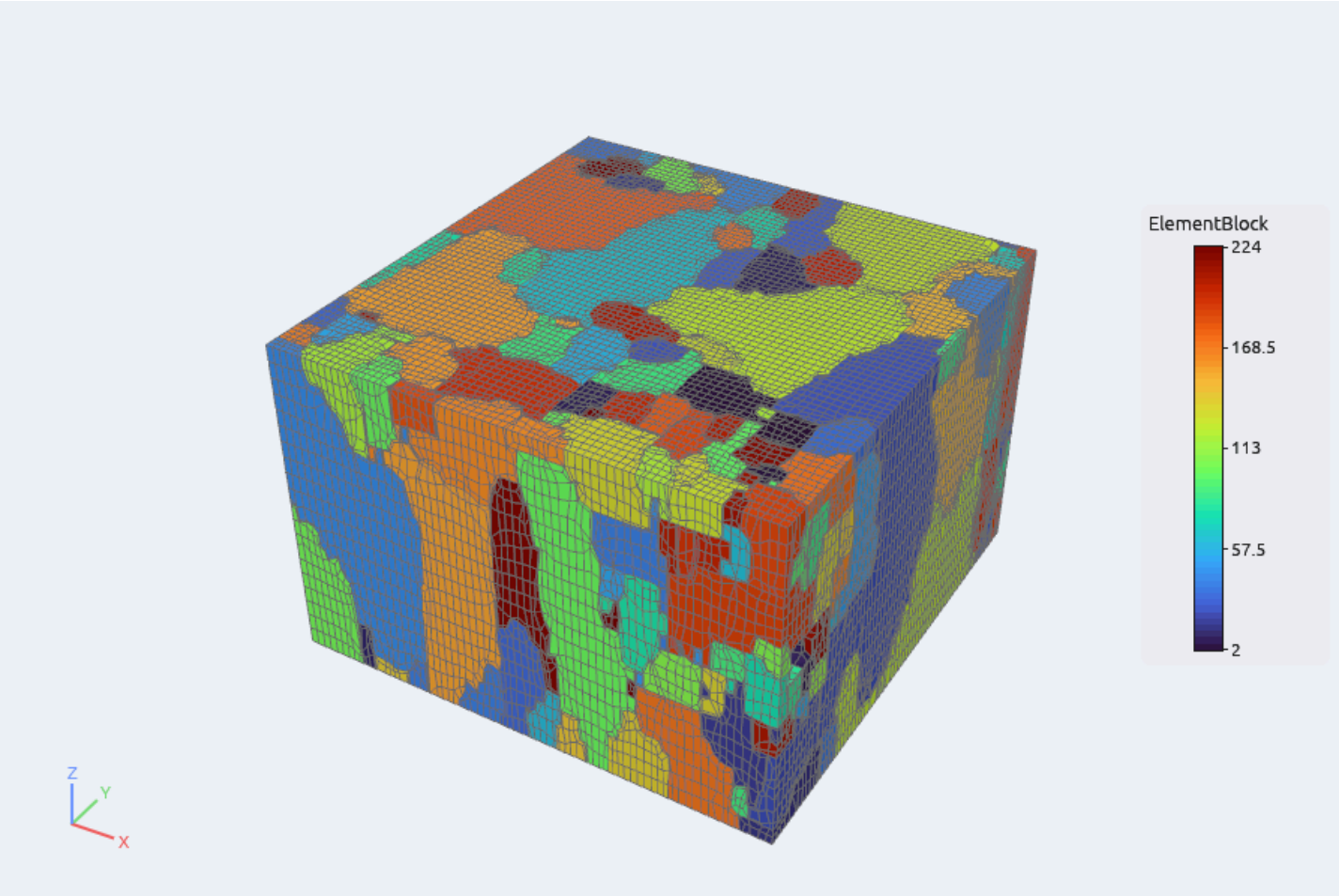
- one block per material;
- side sets and node sets for the outer faces of the grid (`x-` … `z+`), for each surface of the model (named after the geometry face, or `surface <block>`), and for each interface between two blocks (`interface <a> - <b>`);
- field data `MinScaledJacobian` and `InvertedElements` , to check the quality at a glance.

Filters › Mesh › Overlay Mesh runs it on data:

- **Image data with a label array** (`FeatureIds` by default): each label becomes a block, named `grain N` for grain numbers. Label 0 is empty space. **Cell size** 0 meshes on the image's own voxels.

- **A closed surface** (STL, OBJ, PLY, …): the solid inside it. **Cell size** 0 uses a thirtieth of its largest size. Each patch between creases gets a side set (`surface 1`, `surface 2`, …), and **Keep sharp edges** puts nodes on the creases and corners.
- **Interfaces** *Smooth* runs all five steps; *Stair-step* keeps whole cells, every element a perfect cube.
- **Mesh the empty space** meshes label 0, or the space around a surface, as a block of its own (`void`).
- **Periodic** keeps the outer faces of the grid flat and unmoved, and pairs their nodes across opposite faces (`PeriodicPairs x/y/z`), for representative volume elements.

```
mesh = ebsd_volume.overlay_mesh(array="FeatureIds", periodic=True)
part = afx.load("part.stl").overlay_mesh(cell=0.5)
```



Tetrahedral meshes

The Geometry source's **Output** *Tet mesh* fills every body named in a `mesh body` command with tetrahedra of the size asked for. Each body is a block, and each geometry face a side set (with

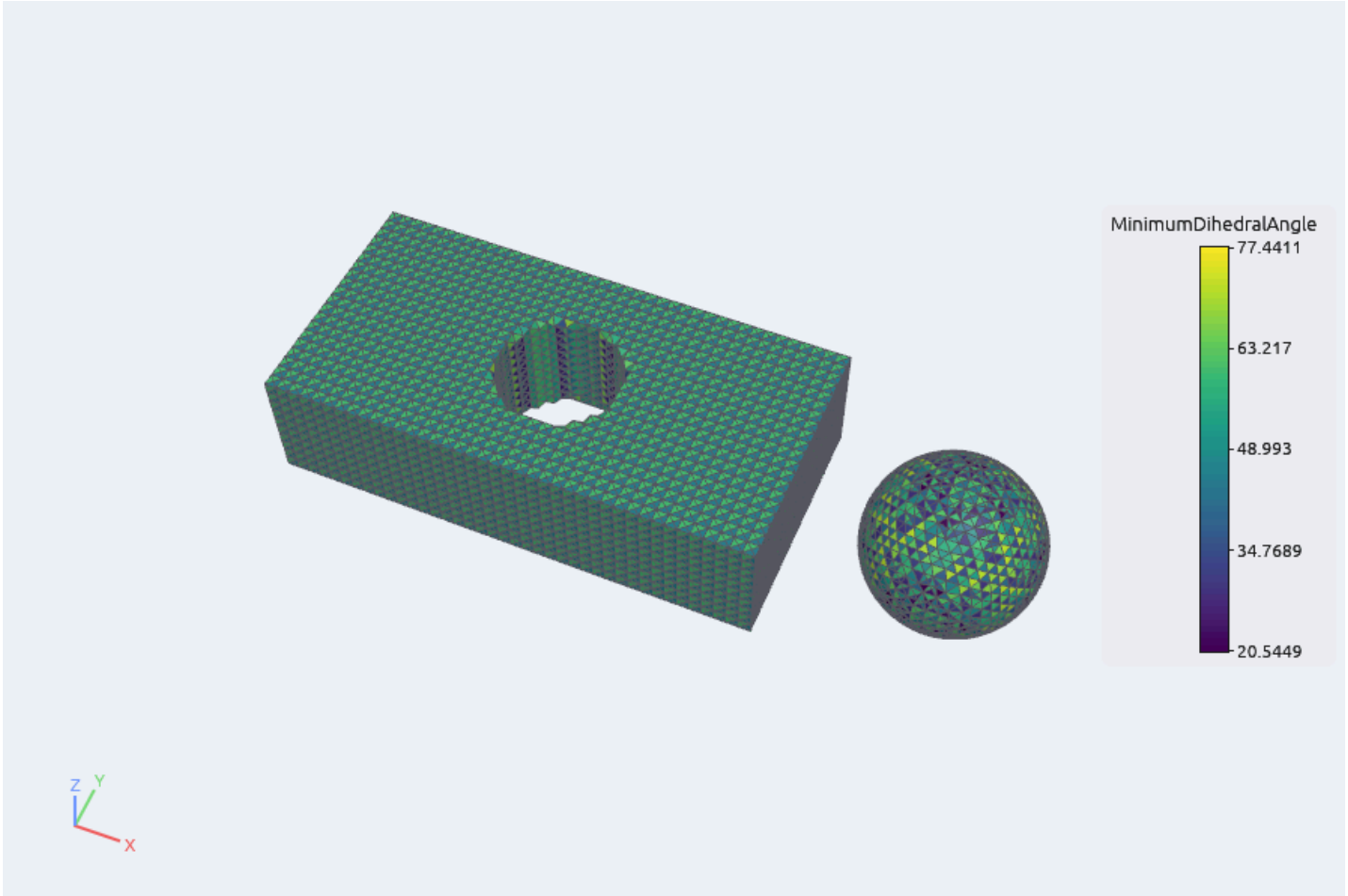
those of `sideset` and `nodeset` commands). **Filters › Mesh › Tet Mesh** does the same for a closed surface such as an STL file, with a side set per patch of the surface between creases.

The tetrahedra are made in three steps:

1. **Lattice.** A regular lattice of tetrahedra, from cubes and their centres, is laid over the shape.
2. **Snapping.** Lattice nodes close to the surface are moved onto it, so no element is cut thin.
3. **Cutting.** The tetrahedra the surface crosses are cut, and their inside parts split into tetrahedra.

Every element is positive and the mesh is conforming. The smallest dihedral angle stays around 20 degrees or more (above 21 on the test shapes). The volume converges to the shape's as the element size shrinks: a sphere meshed with elements a tenth of its radius is within 0.3 %.

```
part = afx.geometry(output="Tet mesh", commands="sphere radius 1; mesh body 1 size
0.1")
tets = afx.load("part.stl").tet_mesh(cell=0.5)
```



Refine Region

Filters › Mesh › Refine Region refines part of a tetrahedral mesh:

- **Which elements.** The tetrahedra whose centre lies in the box from **Box min** to **Box max**, or whose value of a chosen cell array reaches **is at least**.
- **How.** Each of them splits into eight.
- **Neighbours.** A neighbour that shares split edges gets a centre point joined to its faces, so the mesh stays conforming. This gives finer elements where they are needed.
- **Sets.** Side sets and node sets follow.

Refine Mesh

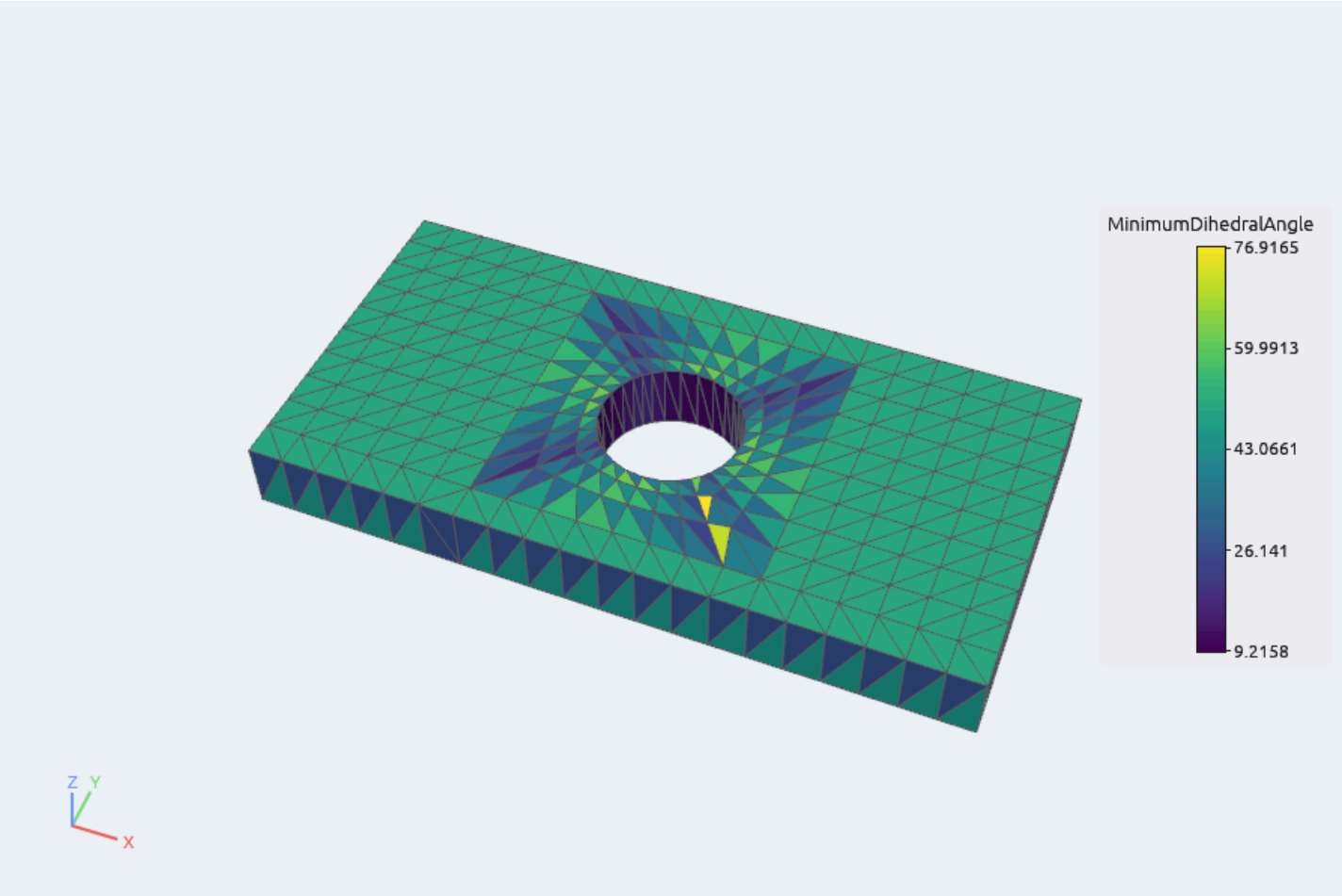
Filters › Mesh › Refine Mesh splits every element into eight, **Levels** times.

- **Hexahedra** (HEX8) split at their edge midpoints, face centres and centre.
- **Tetrahedra** keep their four corners and split the inner octahedron along its shortest diagonal.
- New nodes are shared between neighbours, so the mesh stays conforming.
- Blocks and element variables carry over. Each side-set face becomes its four sub-faces, and a node set gains the new nodes that lie on it.

Convert Elements

Filters › Mesh › Convert Elements changes the element kind, keeping blocks and sets:

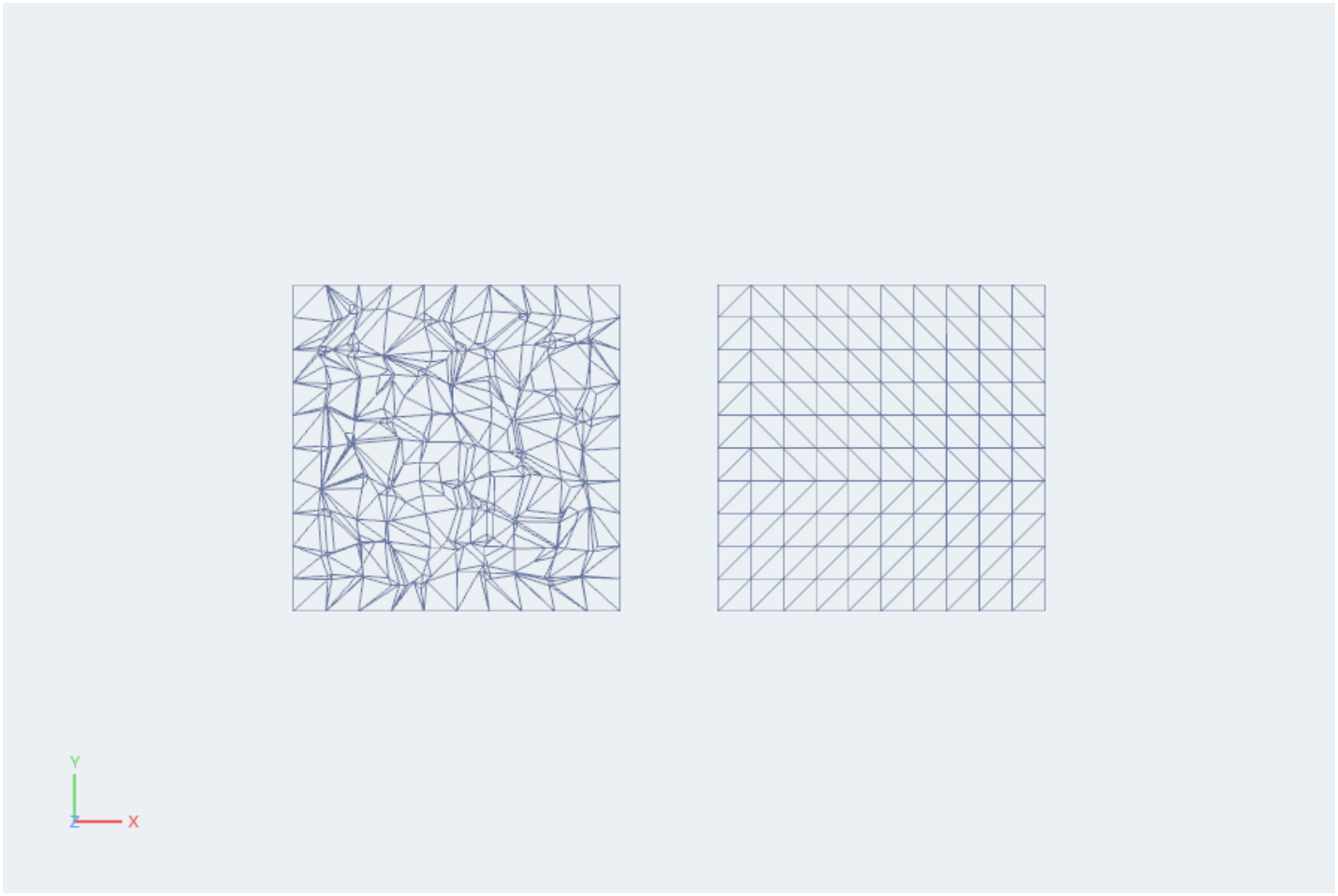
- **Hexahedra to tetrahedra** cuts each face of a hexahedron into two triangles, along the diagonal through its lowest-numbered node so that neighbouring hexahedra agree. Each triangle is joined to the centre, giving 12 tetrahedra per hexahedron.
- **Tetrahedra to hexahedra** gives 4 hexahedra per tetrahedron, one at each corner. They meet at the edge midpoints, face centres and centre.



Smooth Mesh

Filters › Mesh › Smooth Mesh moves the inner nodes of a hexahedral or tetrahedral mesh. Nodes on its boundary, and those of an optional node set, stay put.

- **Smart Laplacian** (the default) moves each node to the mean of its neighbours, keeping the move only when the worst element around it does not get worse.
- **Laplacian** always moves; on a badly shaped domain it can fold elements.
- **Condition number** searches around each node for the position that lowers the condition numbers of its elements. It is slower but untangles better.
- **Winslow** moves each node whose eight hexahedra are arranged like a structured grid to the solution of the Winslow (elliptic) equations over its 3 × 3 × 3 block of nodes, which spreads the nodes evenly without folding; other nodes use Smart Laplacian. A grid whose inner nodes were moved by up to a third of a cell comes back to within 5 % of where they were.



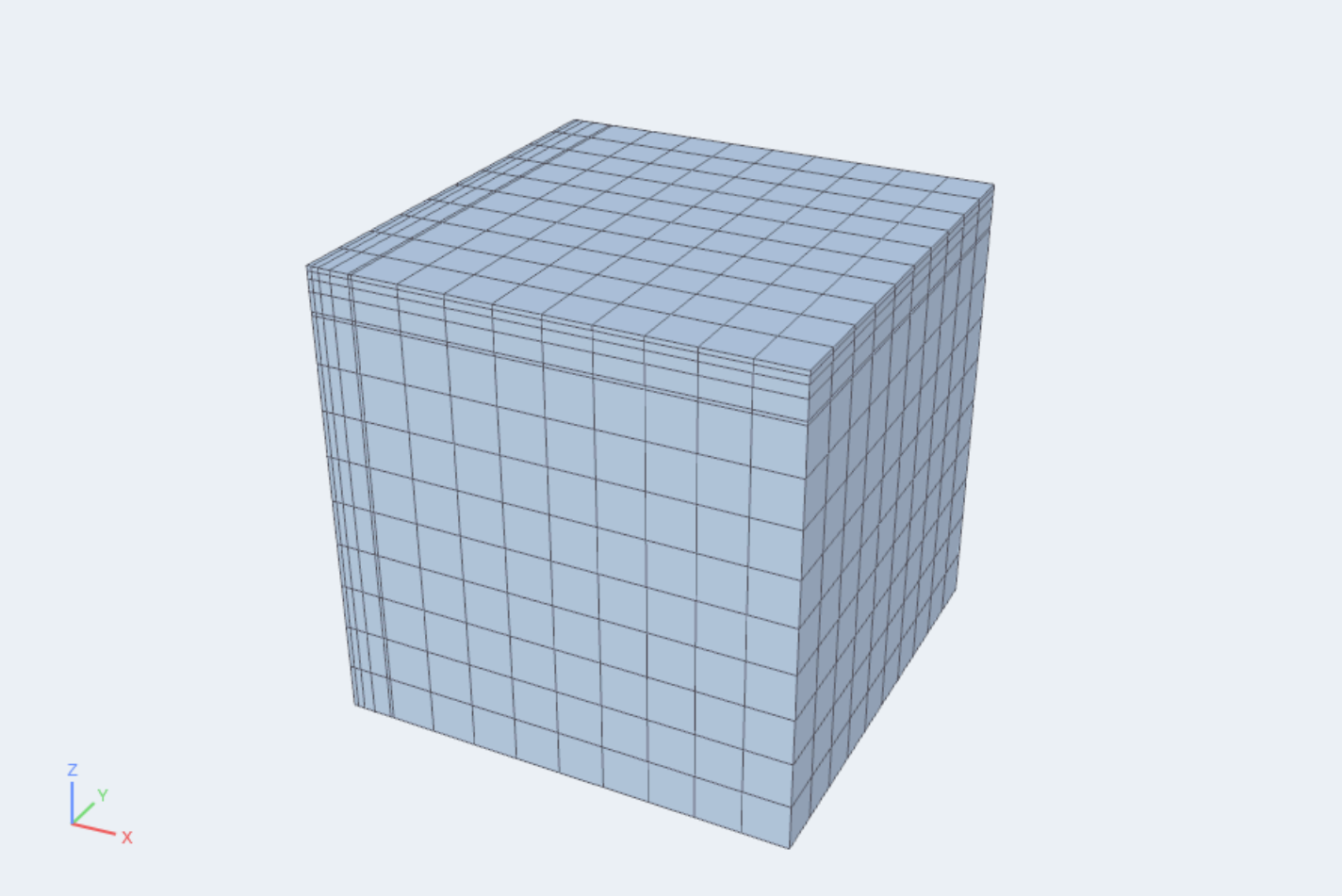
The output's field data `MinScaledJacobian` gives the worst element after smoothing.

Boundary Layers

Filters › Mesh › Boundary Layers inserts **Layers** thin layers of hexahedra under the faces of a side set, for wall-resolved flow or contact.

- **Thickness.** The first layer is **First layer thickness** thick, and each next one **Growth ratio** times thicker.
- **Placement.** The faces stay where they are, and the mesh behind them is pushed inwards by the total thickness. At the edge of the side set, the layers slide along the neighbouring boundary faces.
- **Sets.** The side set moves to the outer layer, and the layers' side faces join the side sets beside them. If the layers are thicker than the elements behind them would allow, the filter refuses rather than fold elements.

```
fine = mesh.refine_mesh(levels=1)
tets = mesh.convert_elements(to="Hexahedra to tetrahedra")
walls = mesh.boundary_layers(side_set="z+", layers=4, first=0.01, growth=1.3)
smooth = tets.smooth_mesh(scheme="Smart Laplacian", iterations=10)
```



Grains on meshes

Assign Grains

Filters › Mesh › Assign Grains gives every element of a mesh a grain, as the cell array `grain_id`, and that grain's orientation, as `EulerAngles` (radians). **Grains from** sets where the grains come from:

- **Voronoi**: **Grains** seeds placed at random in the mesh's box. Each element takes the grain of the seed nearest its centre. **Size spread** above 0 makes Laguerre grains of different sizes. Orientations are random or a fibre. The seeds are kept as `GrainSeeds`.
- **Nearest point**: a second input of points, or a table with `x`, `y` and `z` columns, carrying grain ids (**Grain id array**, `grain_id` by default). Each element takes the grain of the point

nearest its centre. If the points have orientations, as `EulerAngles` in radians or `phi1`, `Phi` and `phi2` columns in degrees, the grain takes them.

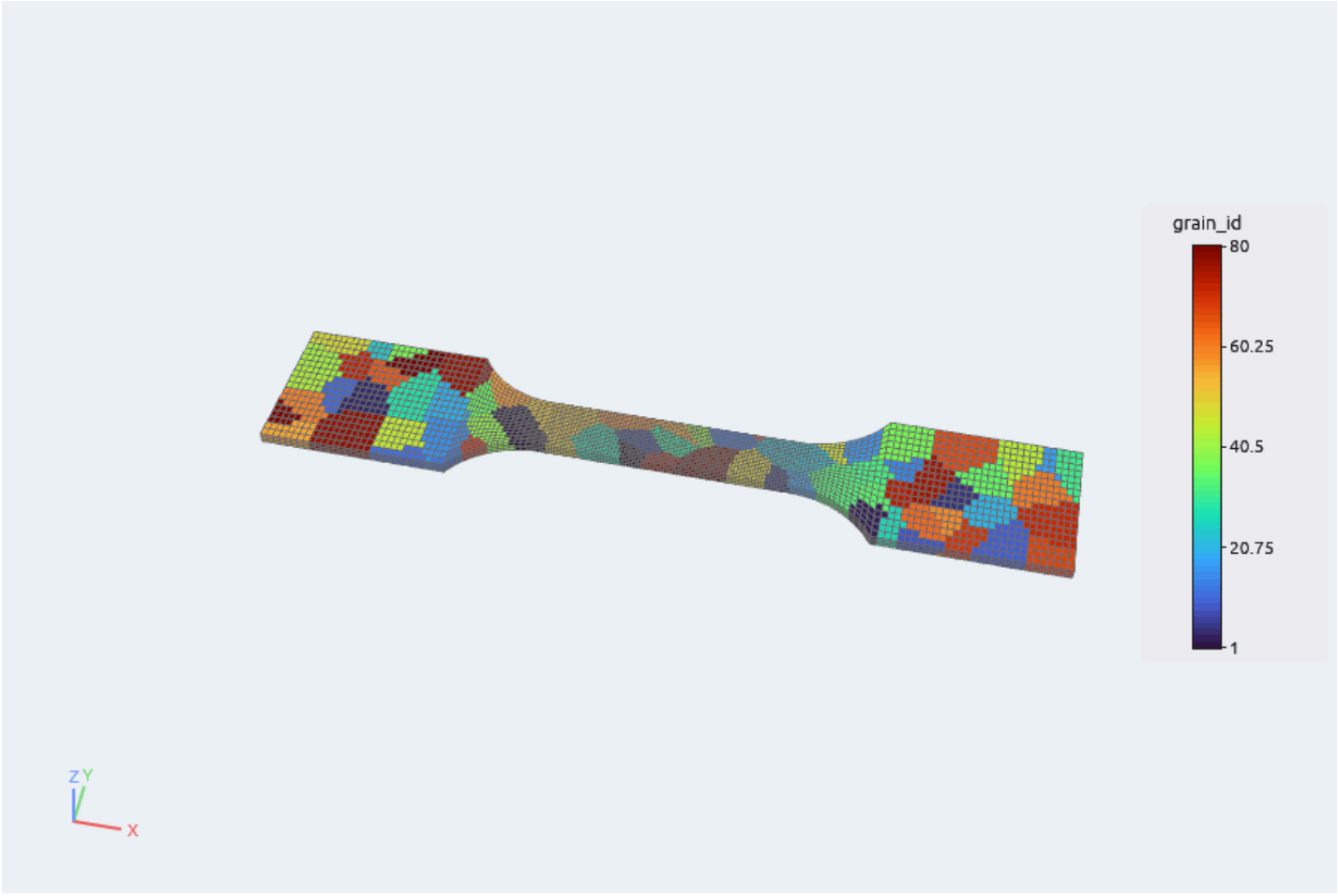
- **From a volume**: a second input that is a microstructure, for example an EBSD volume or a synthetic one, with `FeatureIds`. Each element takes the feature of the cell its centre lies in, with that feature's mean orientation.

One block per grain makes each grain an element block named `grain N`. **Side set on grain boundaries** adds a side set, and a node set, named `grain boundaries` on the faces between elements of different grains. Grains without an orientation in the source get a random one. The grains' orientations are also kept as the field array `GrainEulerAngles`, one row per grain id.

Mesh grains from a microstructure. Overlay Mesh on a volume with `FeatureIds` and orientations already gives one block per grain. Each element also gets its grain's `grain_id` and mean orientation.

Orientation files. Saving a mesh with grain orientations as `.tex` writes one line per grain, in order of grain id: φ1 Φ φ2 in degrees. Crystal-plasticity input decks read orientations in this form.

```
grains = mesh.assign_grains(grains=80, blocks=True, boundaries=True)
from_points = mesh.assign_grains(afx.load("points.csv"), method="Nearest point")
grains.save("specimen.e")
grains.save("specimen.tex")
```



Worked example: geometry to a solver-ready polycrystal

```
import aurafx as afx
part = afx.geometry(output="Hex mesh", commands='''
brick x 4 y 1 z 0.5
cylinder radius 0.2 height 1
subtract body 2 from body 1
mesh body 1 size 0.05 scheme overlay
''')
poly = part.assign_grains(grains=200, blocks=True, boundaries=True)
q = poly.mesh_quality(output="Summary table")       # check the elements
poly.save("plate.e")                                  # blocks per grain, side
sets per face
poly.save("plate.tex")                                # one orientation per grain
```

The Exodus file holds:

- one block per grain;
- the plate's faces as side sets (`brick 1 x-`, `cylinder 2 side`, …);
- the grain boundaries as a side set.

The `.tex` file lists the grains' orientations, ready for a crystal-plasticity input deck.

The case templates of Running simulations write and run the solver's input from the grain mesh directly, here or on a remote machine, and open the results when the run ends:

```
jid = afx.run_case(poly, "moose", "~/runs/plate", host="cluster", np=32,
                   material="Nickel", axis="x", strain=0.02, steps=100)
```

On the results, per-grain statistics come from the grain blocks (MOOSE) or from Assign Grains with the case's point file (PINNACLE).

Elevate

Filters › Mesh › Elevate turns 8-node hexahedra into 20-node (edge midpoints) or 27-node (also face and cell centres) ones. Neighbouring elements share their new nodes, and the blocks, sets and element variables are kept. The new nodes lie at the midpoints. Inclusion Mesh's own **Element** setting also puts those on the inclusion surfaces onto the surface.

Mesh Quality

Filters › Data › Mesh Quality gives every cell its edge ratio (longest over shortest edge) and smallest corner angle. For hexahedra it also adds the quality metrics as cell arrays: `Volume`, `Jacobian`, `ScaledJacobian`, `MaxEdgeRatio`, `MaxAspectFrobenius`, `MeanAspectFrobenius`, `Skew`, `Taper`, `Stretch`, `Diagonal`, `Dimension`, `Oddy`, `Shear`, `Shape`, `RelativeSizeSquared`, `ShapeAndSize`, `ShearAndSize` and `Distortion`. They follow the definitions of the Verdict library (Sandia National Laboratories), which most mesh tools share. **Output › Summary table** gives instead one row per metric: its minimum, maximum and mean over the hexahedra, its acceptable range and how many cells fall outside it.

Metric	Acceptable	Perfect cube
ScaledJacobian	0.5 to 1	1
MaxAspectFrobenius, MeanAspectFrobenius	1 to 3	1
MaxEdgeRatio	1 to 1.3	1
Skew, Taper, Oddy	0 to 0.5	0
Stretch	0.25 to 1	1
Diagonal	0.65 to 1	1
Shear, Shape	0.3 to 1	1
RelativeSizeSquared, Distortion	0.5 to 1	1

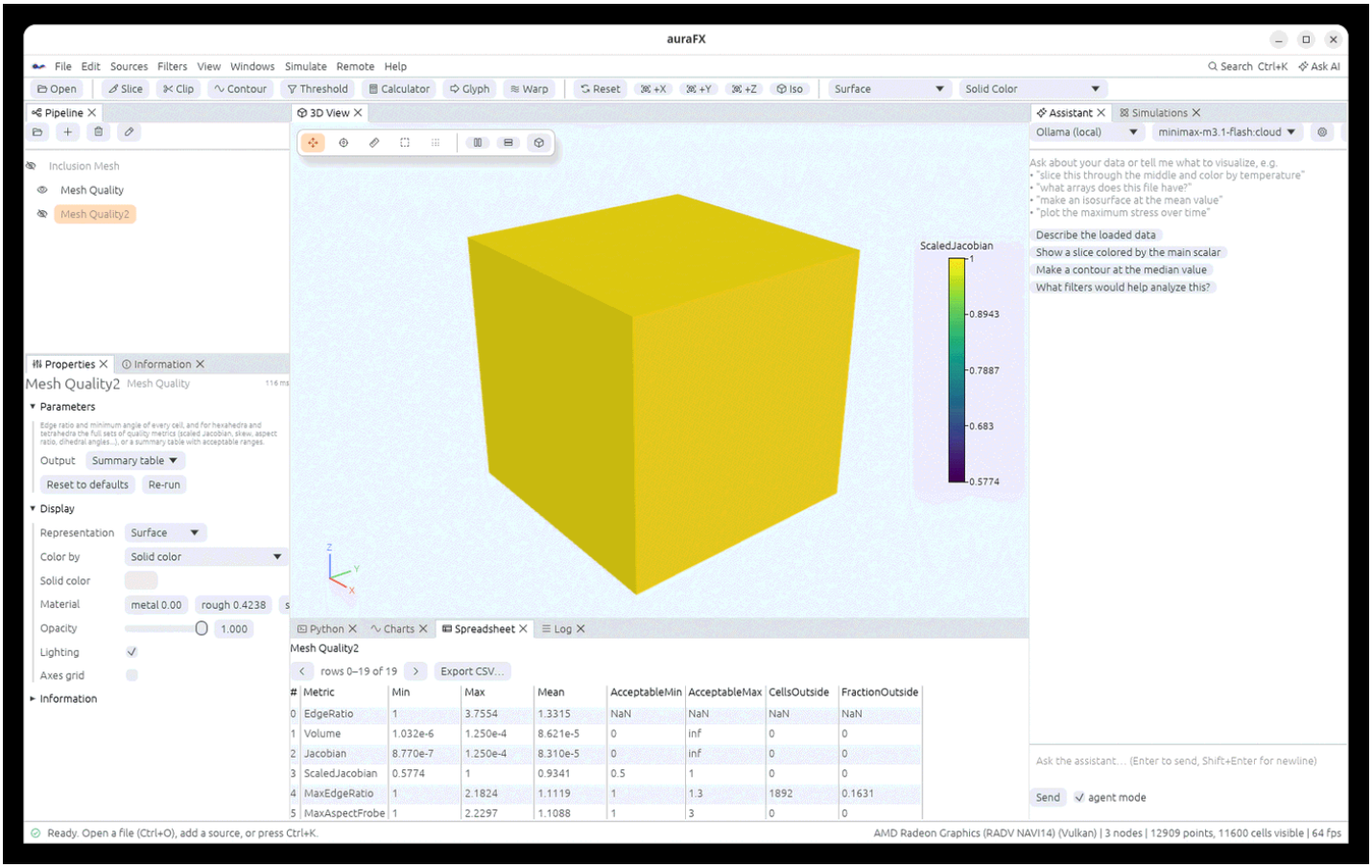
Metric	Acceptable	Perfect cube
ShapeAndSize, ShearAndSize	0.2 to 1	1
Volume, Jacobian, Dimension	above 0	

A negative Jacobian or scaled Jacobian marks an inverted element. HEX20 and HEX27 elements are measured by their corners.

For tetrahedra the arrays are `Volume`, `Jacobian`, `ScaledJacobian`, `AspectRatio`, `AspectFrobenius`, `AspectGamma`, `RadiusRatio`, `MinimumDihedralAngle` (degrees), `CollapseRatio`, `Condition`, `Shape`, `RelativeSizeSquared`, `ShapeAndSize`, `Distortion`, `EquiangleSkew`, `SquishIndex`, `MeanRatio`, `Inradius` and `NormalizedInradius`. Each array holds the hexahedron value for hexahedra and the tetrahedron value for tetrahedra. In the summary, a mesh with both kinds lists the tetrahedra's rows with "(tetrahedra)" after the name.

Tetrahedron metric	Acceptable	Regular tetrahedron
ScaledJacobian	0.5 to 1	1
AspectRatio, AspectGamma, RadiusRatio, Condition, EdgeRatio	1 to 3	1
AspectFrobenius	1 to 1.3	1
MinimumDihedralAngle	40 to 70.53	70.53
CollapseRatio	above 0.1	0.82
Shape, RelativeSizeSquared	0.3 to 1	1
ShapeAndSize	0.2 to 1	1

```
q = m.mesh_quality()                                  # cell arrays
t = m.mesh_quality(output="Summary table")            # one row per metric
```



Limits

- Surfaces are drawn by contouring each body's distance field, so sharp edges are rounded over about one cell. Raise **Resolution** for crisper edges. The `Volume` and `SurfaceArea` of smooth bodies are within about 0.5 % at a resolution of 96.
- The distances of superquadrics, harmonic shapes, partial revolutions, sweeps and non-uniform scaling are approximate away from the surface. The surface itself is exact.
- Sweeps along sharply bent paths join the pieces with a mitre, not a rounded bend.
- Imported surfaces must be closed. Their inside is found from the direction the triangles face.
- Inclusions must be star-shaped (one surface point along every direction from their centre) and cannot touch: each lies in its own box of grid cells, which caps how dense a packing can get even with collective packing.
- Assign Grains places a grain by each element's centre: grain boundaries follow the element faces (a finer mesh resolves them better), and the tetrahedral and hexahedral side sets need the mesh to be all one kind.
- Tetrahedral meshes have one element size, refined locally with Refine Region; they round sharp edges over about an element, and touching bodies are meshed separately (their

nodes do not match). Local refinement works on tetrahedra only (hexahedra are refined everywhere, or turned into tetrahedra in a region).

- Overlay meshes keep convex edges and corners. Concave edges, such as where two crossed cylinders join, are rounded over about a cell, and so are features thinner than a cell.
- The clean-up may give a few cells at grain boundaries to the neighbouring grain, so very small grains lose some volume: a voxel sphere 5 voxels in radius comes out about 8 % smaller than the exact sphere.
- Overlay meshing of 100 000 voxels takes about 3 seconds.
- Gmsh files carry no node sets.
- Face Mesh meshes flat faces; curved faces are meshed with Remesh Surface from a surface of the part. Remesh Surface keeps the nodes of sharp edges where they are (it adds nodes along them but does not remove any). On a faceted surface, such as a contoured Geometry surface, a few quadrilaterals come out poorly shaped (on a torus about 0.5 % below a scaled Jacobian of 0.2): check the `Quality` array.

CAD modeling

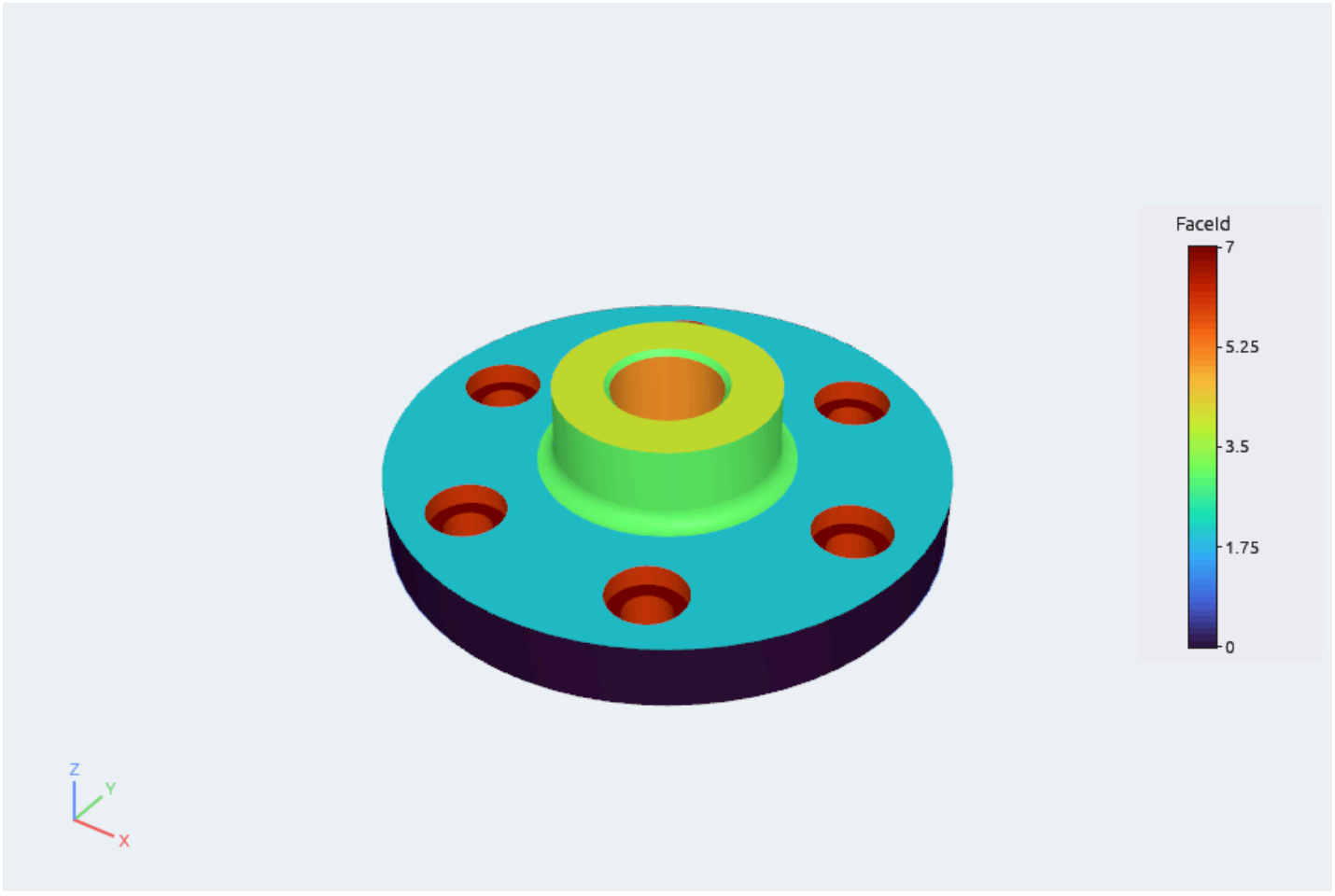
auraFX has a parametric part modeler. A part is a short list of **features** (a sketch extruded, a hole cut, a boss added) with **named dimensions**. Change a dimension and the whole part rebuilds, and everything built on it (meshes, grains, solver cases) can be re-run. The part keeps **named faces**, so a later step can say “the wall of the bore” instead of a number that changes when the part does.

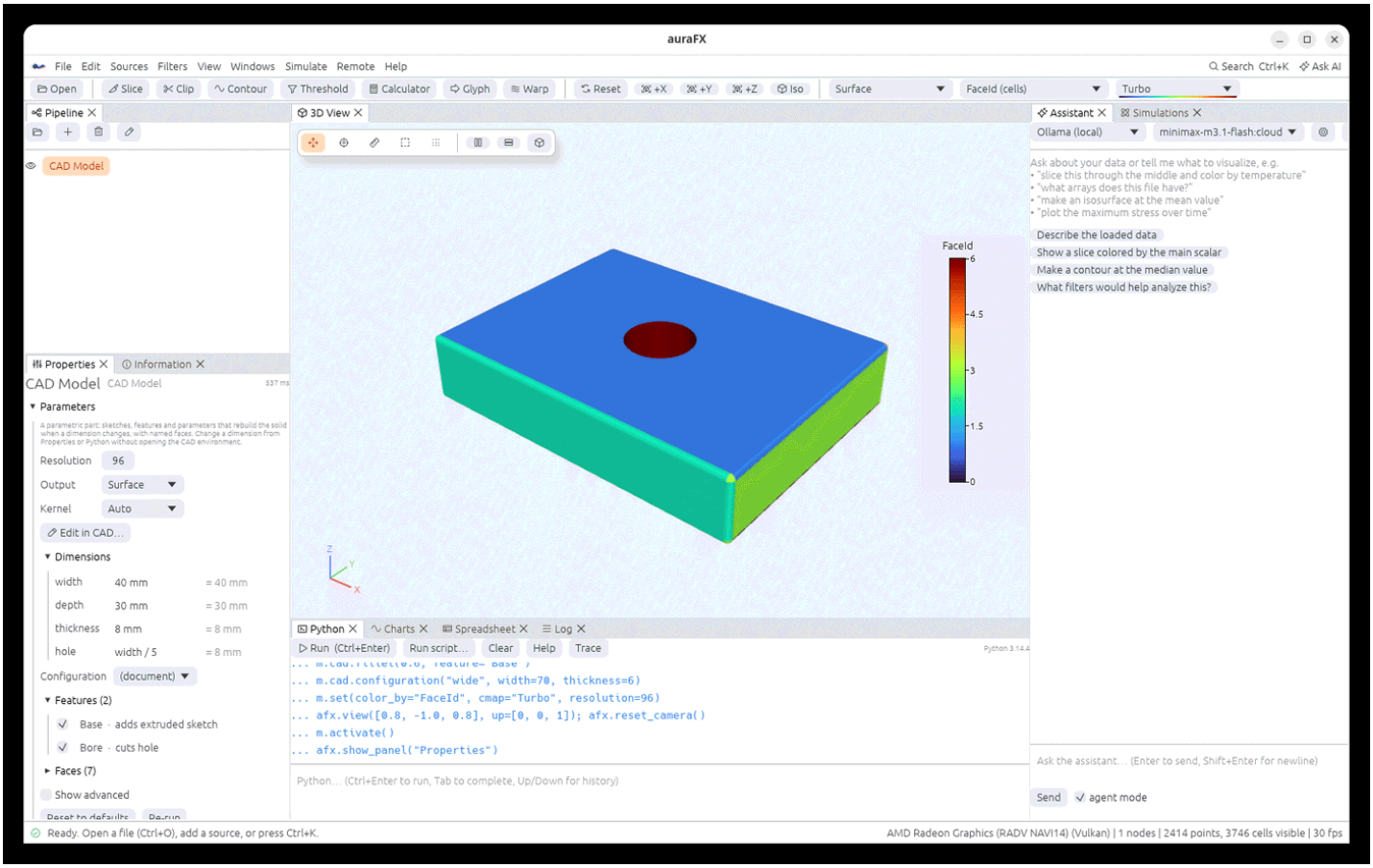
It sits beside the command-based Geometry source of the previous chapter. Use Geometry for quick specimens written as text; use a CAD model when the part has dimensions you want to change, sketches, patterns and holes.

The CAD Model source

Add one with **Sources › CAD Model** (or the command palette, `Ctrl+K`). It starts as a plate. Its Properties show:

- **Dimensions**: the part’s named dimensions. Type a new value or an expression such as `width / 2` and press **Enter**; the part and everything downstream update. The value the expression gives is shown next to it. **reset** returns to the value stored in the part. A dimension changed here belongs to this node and leaves the part untouched, so two nodes can show two sizes of one part.
- **Configuration**: shown when the part has configurations (named sets of dimensions such as *small* and *large*); pick one to build that variant.
- **Features**: the part’s history in order. The checkbox suppresses a feature (the part is rebuilt without it).
- **Faces**: every face of the result with its number and name, its volume and area.
- **Resolution** and **Output**: cells along the longest side when the surface is drawn, and *Surface* or *Distance field* (the signed distance to the part, negative inside).


FaceId
7
5.25
3.5
1.75
0
Z
Y
X


auraFX
File Edit Sources Filters View Windows Simulate Remote Help
Search Ctrl+K
Ask AI
Open
Slice
Clip
Contour
Threshold
Calculator
Glyph
Warp
Reset
+X
+Y
+Z
Iso
Surface
FaceId (cells)
Turbo
Pipeline
CAD Model
3D View
FaceId
6
4.5
3
1.5
0
Assistant
Simulations
Ollama (local)
minimax-m3.1-flash:cloud
Ask about your data or tell me what to visualize, e.g.
• "slice this through the middle and color by temperature"
• "what arrays does this file have?"
• "make an isosurface at the mean value"
• "plot the maximum stress over time"
Describe the loaded data
Show a slice colored by the main scalar
Make a contour at the median value
What filters would help analyze this?
Properties
Information
CAD Model
CAD Model
537 ms
Parameters
A parametric part: sketches, features and parameters that rebuild the solid when a dimension changes, with named faces. Change a dimension from Properties or Python without opening the CAD environment.
Resolution 96
Output Surface
Kernel Auto
Edit in CAD...
Dimensions
width 40 mm = 40 mm
depth 30 mm = 30 mm
thickness 8 mm = 8 mm
hole width / 5 = 8 mm
Configuration (document)
Features (2)
Base · adds extruded sketch
Bore · cuts hole
Faces (7)
Show advanced
Reset to defaults
Re-run
Python
Charts
Spreadsheet
Log
Run (Ctrl+Enter)
Run script...
Clear
Help
Trace
Python 3.14.4
... m.cad.configuration("wide", width=70, thickness=6)
... m.set(color_by="FaceId", cmap="Turbo", resolution=96)
... afx.view([0.8, -1.0, 0.8], up=[0, 0, 1]); afx.reset_camera()
... m.activate()
... afx.show_panel("Properties")
Python... (Ctrl+Enter to run, Tab to complete, Up/Down for history)
Ask the assistant... (Enter to send, Shift+Enter for newline)
Send
agent mode
Ready. Open a file (Ctrl+O), add a source, or press Ctrl+K.
AMD Radeon Graphics (RADV NAVI14) (Vulkan) | 1 nodes | 2414 points, 3746 cells visible | 30 fps

Faces and their names

The surface has the cell arrays `BodyId` and `FaceId`, and the field arrays `FaceNames` (indexed by face number), `BodyNames`, `Volume` and `SurfaceArea`: the same layout as the Geometry source, so everything that works on a Geometry result works on a CAD model. Color by `FaceId` to see the faces.

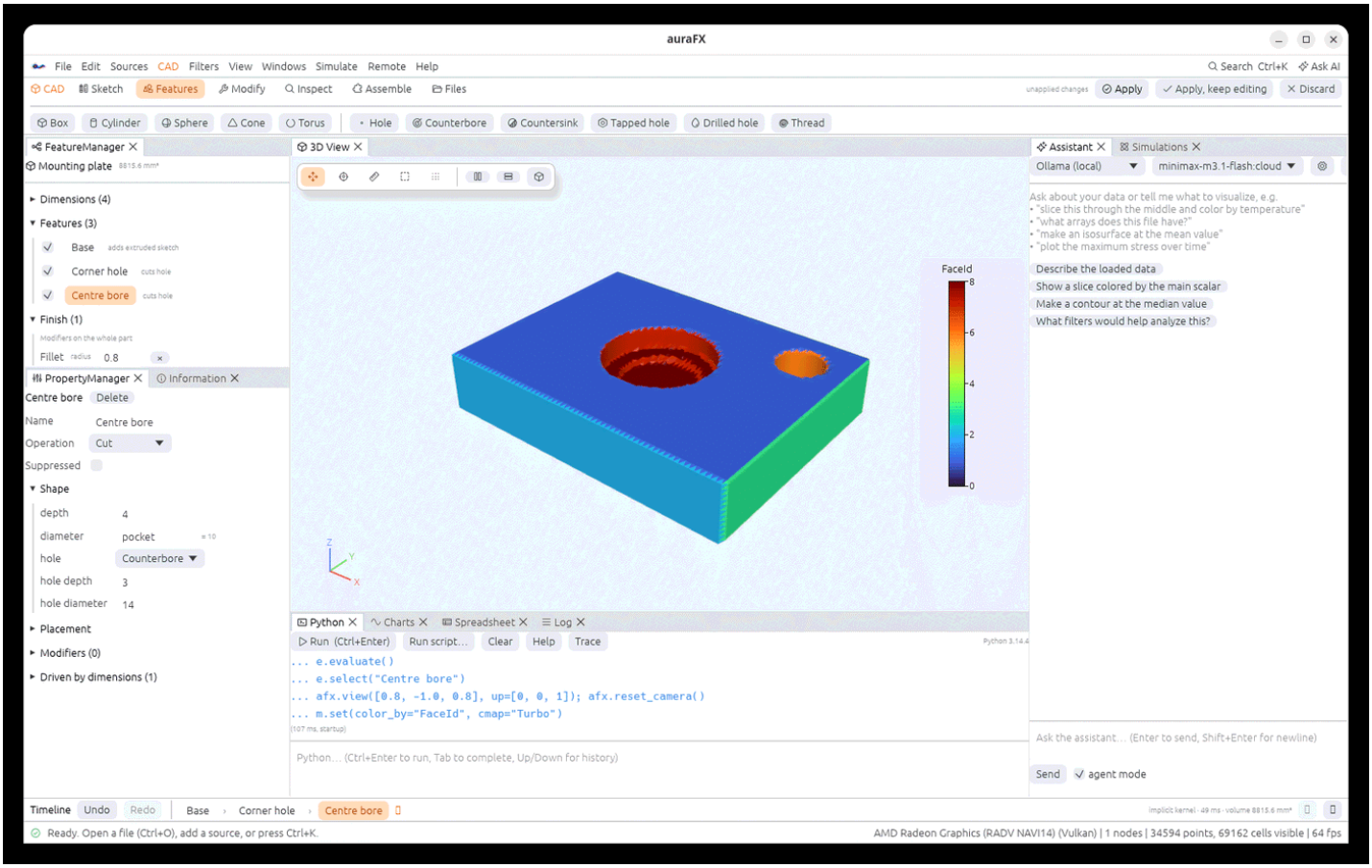
A face is named after the feature that made it and its place on that feature: a brick has `x-`, `x+`, `y-`, `y+`, `z-` and `z+`; a cylinder or a hole has `side`, `bottom` and `top`; an extruded sketch has `bottom`, `top` and one `side N` for each line, arc or circle of the sketch (the sketch's own order). `Bore side` is the wall of a hole named *Bore*; `Base z+` is the top of a brick named *Base*. Rename a feature and its faces are renamed with it, which the later *downstream check* uses to follow them.

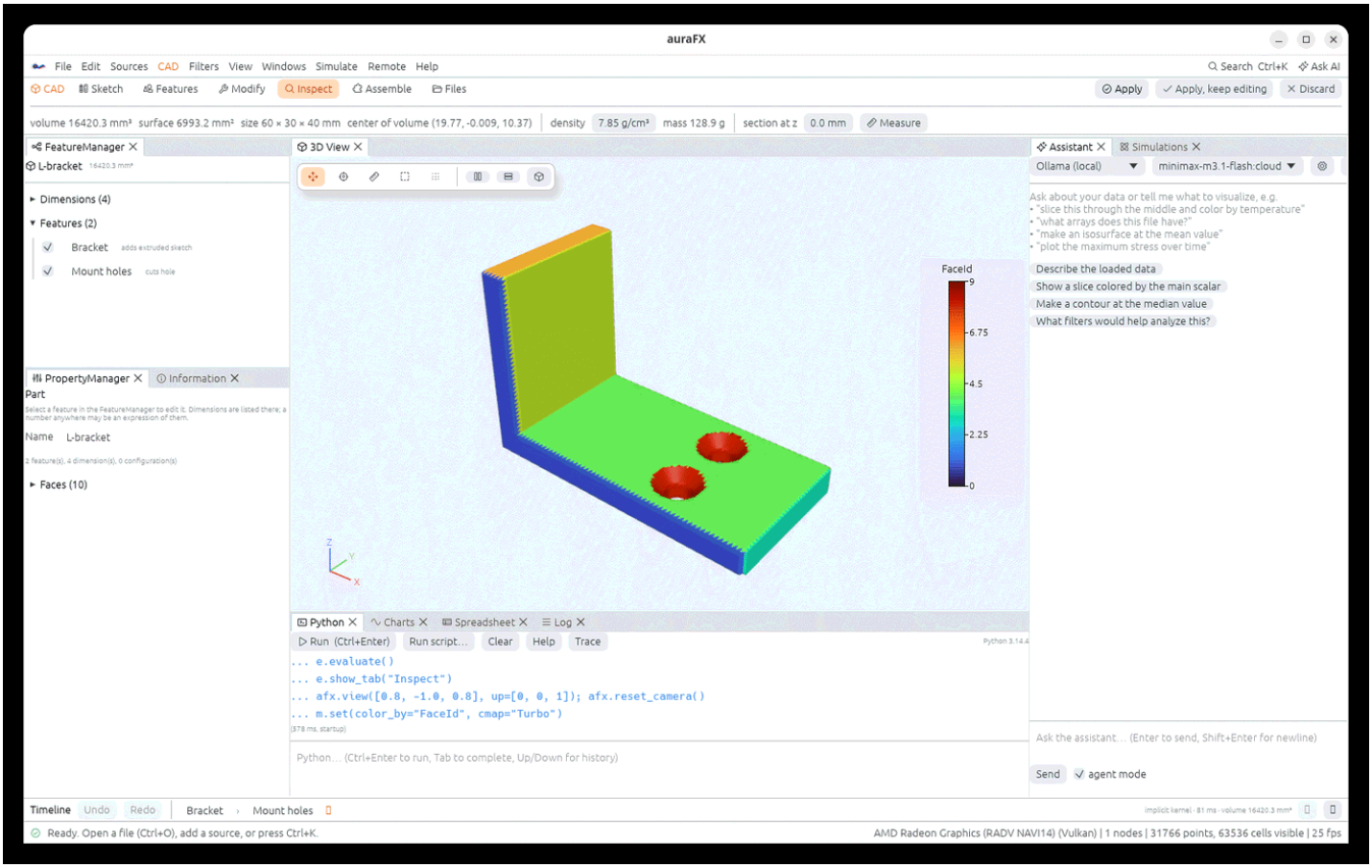
The CAD environment

Editing happens in one mode of the same window. Open it from **Sources › CAD Model…** (which adds a new part and opens it), from **Edit in CAD…** (right-click a CAD Model in the Pipeline, or the button at the top of its Properties), or from Python with `part.edit_cad()`. While it is open:

- the toolbar becomes the **ribbon** (every button with an icon and its name) with tabs *Sketch*, *Features*, *Modify*, *Inspect*, *Assemble* and *Files* (Alt+1 to Alt+6). Each button adds a feature with sensible sizes at the top of the part; you then set its numbers in the PropertyManager. *Inspect* reads the part (volume, surface, size, center, mass for a density, the area of a cross-section); *Assemble* places other parts (see below); *Files* starts from a template or opens and saves a document.
- the Pipeline panel becomes the **FeatureManager**: the part's dimensions, its features in order (the checkbox suppresses one; double-click renames; right-click for *Suppress*, *Roll back to here*, *Move up*, *Move down*, *Delete*), the modifiers on the whole part and the configurations.
- the Properties panel becomes the **PropertyManager**: every field of the selected feature. Each number accepts an expression of the dimensions: type `width / 2` instead of a number and the field follows the dimension (the value it gives is shown beside it). Choices (the end condition of an extrusion, the type of a hole, a mirror plane) are menus. With no feature selected it shows the part itself and its faces.
- the **timeline bar** at the bottom lists the features in order. The orange marker is the *rollback marker*: only the features before it are built, so you can look at the part as it was after the third feature, or insert a hole earlier in the history. ◀ ▶ move it.
- the menu bar gains a **CAD** menu, and the part is built again after every edit, in the background (the timeline shows the time it took and the part's volume).

- **the pipeline is paused.** Other 3D views, charts and spreadsheets stay as they were and say so in a grey banner; nothing downstream re-runs on every edit.





Starter parts. *Files › New from template* (or `afx.cad_model(template="flange")`) replaces the part by one of six starters, all driven by named dimensions (the enclosure follows `width`, `depth`, `height` and `wall`; the flange its diameter, bolt circle and number of holes):

The right end of the ribbon leaves the environment:

- **Apply** hands the part to the CAD Model node and closes the environment; the nodes below it re-run. A part that does not build is refused, with the reason.
- **Apply, keep editing** does the same and stays.
- **Discard** (or `Esc`, which asks) closes it and leaves everything as it was.

All the edits of a session are **one undo step** in the main window (*Edit › Undo* after Apply takes back the whole session). Inside the environment, *Undo* and *Redo* (`Ctrl+Z`, `Ctrl+Y`, the timeline bar, the CAD menu) step through single edits.

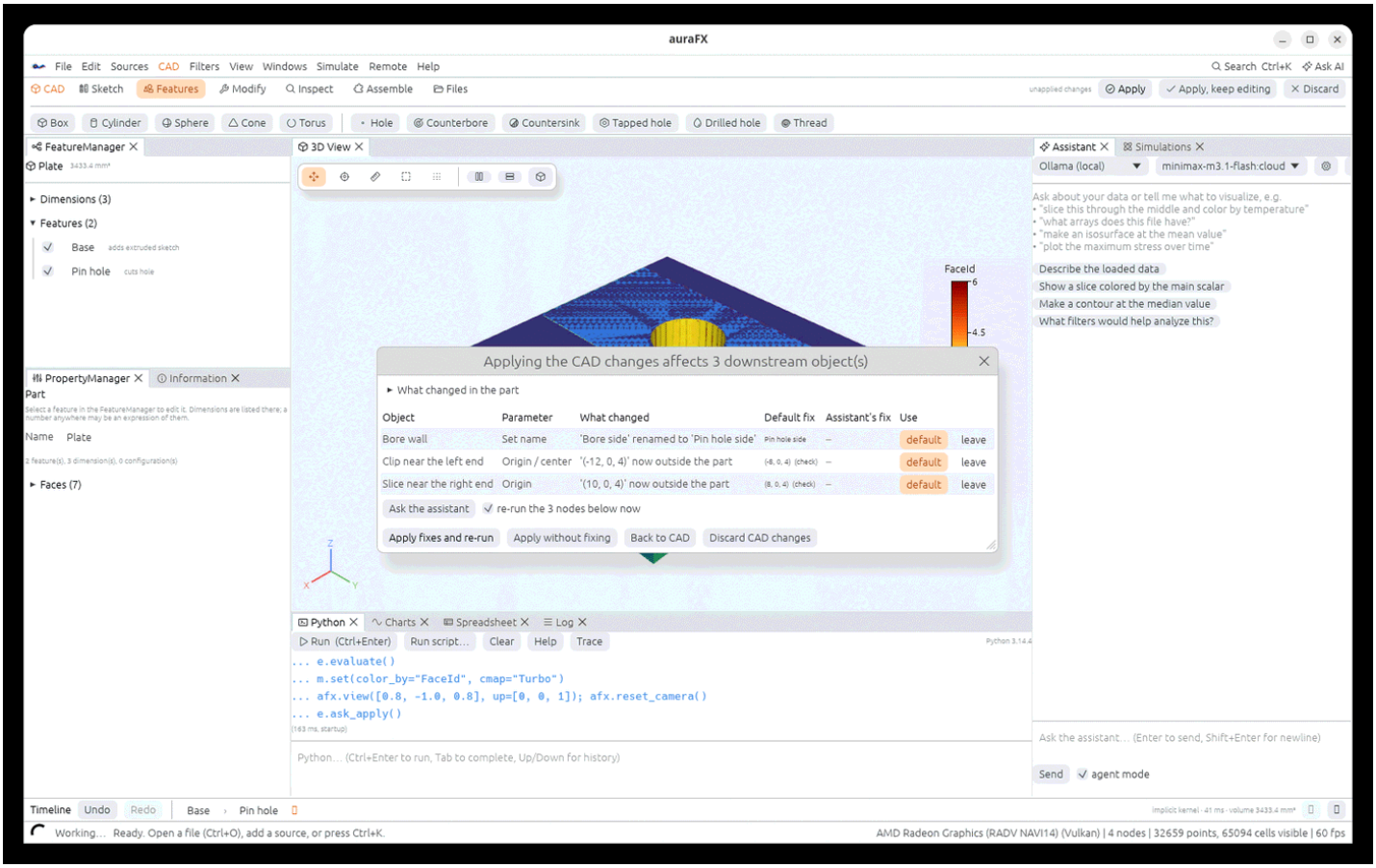
The *Inspect* tab reads the part: volume, surface, size, center of volume, mass for a density, and the area of a cross-section at a height. The *Files* tab starts from a template, opens an `.afxcad` file, or saves the part as one.

From Python:

```
e = part.edit_cad()            # open the environment; e is part.cad
e.box(size=(10, 10, 6), name="Boss", at=(0, 0, 11))
e.evaluate()                   # build it now (the window does this by itself)
e.rollback(1)                  # build only the first feature
e.undo(); e.redo()
e.apply()                      # or e.apply(keep_editing=True) / e.discard()
```

What the nodes below depend on

Other nodes refer to a part by the names inside it: an Extract Set that names `Bore side`, a mesh command that names a face, a clip plane placed inside the plate. Editing the part can break these. So **Apply checks them first**. It compares the part as the nodes below were built against with the part you made, finds what they refer to that changed, and opens a dialog:



- **What changed in the part** lists faces that were renamed, replaced, removed or moved, dimensions added or removed, and the new bounds and volume.
- Each row is something below the part that refers to a changed thing: the object, its parameter, what happened to the thing it refers to (*renamed to…*, *removed*, *now outside the part*, *one of several*), the **default fix**, the **assistant's fix** when you asked for one, and a choice per row (*default*, *assistant*, *leave*).

- The default fixes are the safe ones. A renamed face is renamed where it is used, including inside command and expression text (whole names only: `Bore side` is not found inside `Bore sides`). A face that is gone but has one successor in the same place is replaced by it, and marked *(check)* because that is a guess. A position that fell outside the part is moved onto its nearest point. A face that is simply gone, or has several possible successors, has no default: you choose, or ask the assistant.
- **Ask the assistant** sends the changes, the conflicts and each affected node's filter and parameters to the assistant, which answers with a list of changes; they appear in the *Assistant's fix* column and the same chat shows its reasoning.
- **Apply fixes and re-run** applies the part and your choices, together, as one undo step; **Apply without fixing** applies the part and leaves the references (those nodes will show their own errors); **Back to CAD** returns; **Discard CAD changes** closes the environment as it was. The checkbox *re-run the nodes below now* postpones their re-run: they keep their old results, marked by a banner with **Run now**.

When nothing below is affected the dialog is only the question *Re-run N downstream filters?* with **Run now**, **Later** and **Back to CAD**. A Python filter's code is searched for the names as text: such rows say *(in code)* and have no default fix, because the name may not be a reference.

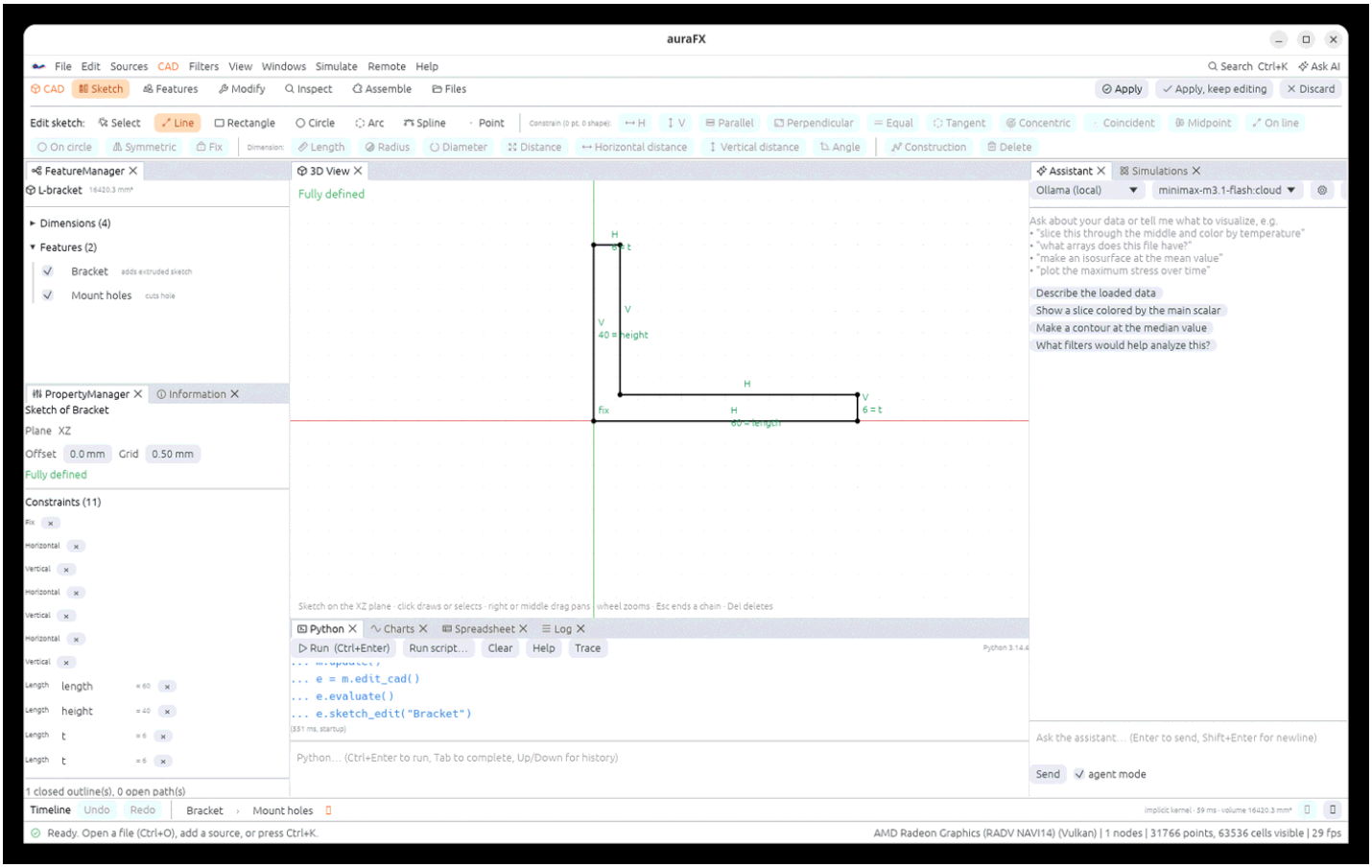
From Python:

```
e = part.edit_cad()
e.rename("Bore", "Pin hole")
report = e.check()              # {'diff': ..., 'conflicts': [...], 'downstream':
[...], 'prompt': ...}
e.apply()                       # fix="default": applies the default fixes
e.apply(fix="none")             # leave what the check found as it is
e.apply(fix=answer_text)        # the text of an assistant's answer (a JSON list of
fixes)
e.apply(run="later")            # the nodes below wait; part.cad.run_held() runs
them
```

Sketching

A sketch is a drawing on a plane that a feature extrudes or revolves. Start one from the ribbon's **Sketch** tab: **Top (XY)**, **Front (XZ)** or **Right (YZ)**, or **Top of the part** (the plane of the highest face, to add a boss or cut a pocket). To sketch on a flat face, select no feature, open *Faces* in the PropertyManager and press **Sketch here** next to a face whose normal points along x, y or z. To change an existing feature's sketch, right-click it in the FeatureManager and choose **Edit sketch…**, or press **Edit sketch…** in its properties.

The canvas takes the place of the 3D view while a sketch is open:



- **Draw** with the tools in the ribbon: *Line* (connected lines; right-click ends the chain), *Rectangle* (two corners), *Circle* (center, then a point on it), *Arc* (center, start, end), *Spline*, *Point*. New points snap to existing ones (which joins shapes) and to a grid (set in the PropertyManager). A line drawn nearly horizontal or vertical becomes so.
- **Select** shapes and points by clicking (Shift adds); drag a point to move it. Right or middle mouse drag pans, the wheel zooms, `Esc` ends a chain, `Del` deletes.
- **Constrain** the selection with the ribbon: horizontal, vertical, parallel, perpendicular, equal, tangent, concentric, coincident, midpoint, on line, on circle, symmetric, fix. **Dimension** it: length, radius, diameter, distance (also horizontal and vertical) and angle.
- A dimension starts at the current size. Edit it in the **Constraints** list in the PropertyManager: type a number, or an expression of the part's dimensions (`width / 2`, `3 in`), and the sketch follows the dimension from then on.
- The **solver runs after every change**. Shapes are **blue** while the sketch can still move (the status says how many degrees of freedom are left), **white or black** when it is fully defined, and **red** when constraints conflict (remove one). *Construction* turns the selected shapes into guides that are not part of the outline.

Finish from the PropertyManager: **Extrude** (or **Cut**, for a pocket) with a depth that may be an expression, **reverse** to go against the plane's normal, **Revolve** about the plane's vertical axis (draw on the Front plane with x as the radius), or **Apply sketch** when you edited the sketch of an existing feature. The sketch then becomes a feature with named faces: `side 1`, `side 2`… follow the order the shapes were drawn.

From Python, `e.sketch_new(plane="XY", items=[...])` and `e.sketch_edit("Bracket")` open the canvas in the window, and `e.sketch(...)` with `e.extrude(...)` builds the same features without it.

Kernels, STEP and other files

A part is built by one of two kernels, chosen in the CAD Model's Properties (**Kernel**):

- **B-rep** builds *exact* solids with true faces and edges (a cylinder is a cylinder, a hole's wall is a real cylindrical face). It maps primitives, extruded and revolved sketches, lofts, holes, patterns (as copies), rounding and hollowing of a feature or of the whole part, the rounded junction a feature's *blend* makes with the part, drafts, mirrors about a face, combined parts, and cap chamfers. This is what **STEP** and **IGES** are written from, and it reads imported solids.
- **Implicit** describes the part as a distance field. It builds every feature, including threads, sweeps and a loft's shell, and never fails; edges are not exact. It leaves a picked fillet or a draft undone and says so.
- **Auto** (the default) uses B-rep when it can build the part and Implicit otherwise. The result carries a field `Kernel` (which one built it) and `KernelNotes` (why B-rep could not).

Both kernels name faces the same way (`Base z+`, `Bore side`), so everything built on the names works on either. The CAD environment shows the part with the Implicit kernel while you edit (it is quick), and the node builds it with the kernel you chose when you apply.

Files. *File › Open* (and `afx.load`) opens a **`.step`** / **`.stp`** (AP203, AP214, AP242), **`.igs`** / **`.iges`**, **`.3mf`**, a SolidCraft **`.sc`** design, or an **`.afxcad`** document as a **CAD Model**: the file's solid becomes the base feature of a part, with its bytes kept inside the document, so you can add features to it (cut a hole through an imported bracket) and the part still opens on another computer. STL and OBJ keep their ordinary readers; to edit one, add it as a CAD Model from Python.

```
m = afx.cad_model("bracket.step")                  # a solid from a STEP file as a
CAD Model
m.cad.hole(diameter=6, depth=20, at=(10, 5, 8))    # cut into it
m.cad.save("bracket2.step")                        # exact STEP (faces and edges)
m.cad.save("bracket2.3mf"); m.cad.save("part.sc") # 3MF, a SolidCraft design (also
.stl, .obj)
m.cad.save("bracket2.igs")                         # IGES, exact (the exact kernel
only)
r = m.cad.save("lofted.step")                      # {"exact": False, "note":
"faceted: ..."} when

                                                   # only the Implicit kernel
builds the part
```

A part that only the Implicit kernel builds is written as STEP or STL from its tessellation: a valid solid of triangles whose header says *faceted B-rep* and why. IGES is written only for a part the exact kernel builds (its faces are true surfaces); for any other part the message says to write STEP instead.

The files auraFX writes were checked with an independent CAD reader: the same number of faces and the same volume, to a thousandth, for STEP and IGES (the checks are in the test suite and need that reader installed to run).

Edges, faces and measures

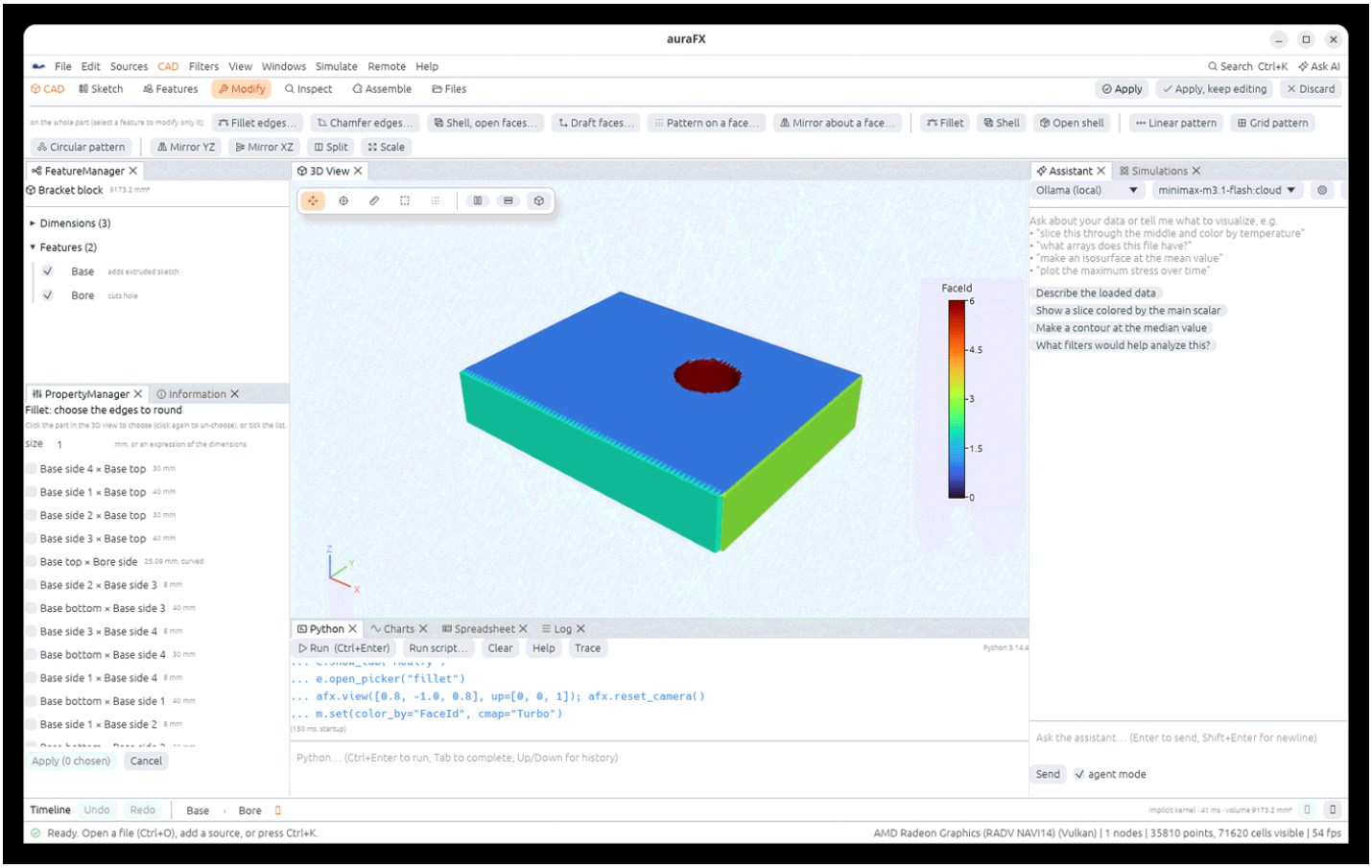
With the exact kernel the part has real edges, and you can work on them. In the **Modify** tab:

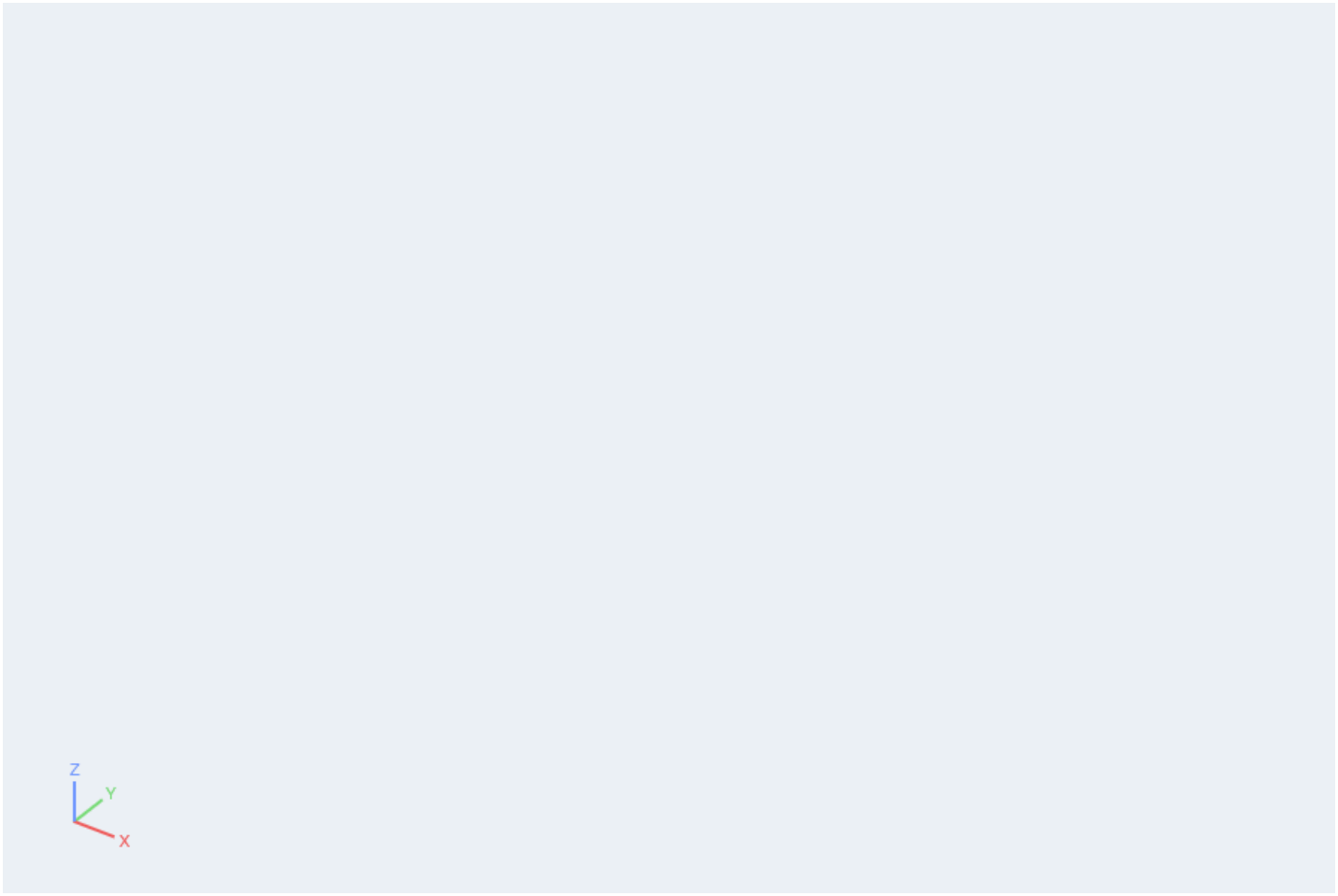
While one of these pickers is open you can also **click the part in the 3D view**: a click chooses the face under the pointer, or, for an edge, the edge of that face the click is nearest to (click again to un-choose). Chosen edges are drawn in orange and chosen faces are marked with a dot.

- **Fillet edges…** and **Chamfer edges…** open a picker in the PropertyManager that **lists the part's edges**, each with the two faces it lies between (`Base z+ × Bore side`), its length and whether it is curved. Tick the edges, set the size (a number or an expression of the dimensions) and press **Apply**: a *Fillet* or *Chamfer* feature is added after the features it modifies, so it follows them when you roll back or change a dimension.
- **Shell, open faces…** lists the faces; tick the ones to remove (none: a closed hollow body) and give the wall.
- **Draft faces…** lists the faces; tick the side faces to taper and give the angle. They stay fixed where the part meets the bottom plane and lean by that angle above it.
- **Pattern on a face…** lists the *flat* faces; pick one, choose **Holes** or **Bosses** and a **Row**, **Grid** or **Circle**, and give the size, the count, the spacing (or the radius of the circle). The first copy sits at the face's center and the row runs along the face's own x axis, so the pattern

follows a face that is not horizontal. A pattern on a face is one feature: the count and the spacing can be changed afterwards, and a hole's copies share its face names (`Hole side`).

- **Mirror about a face…** lists the flat faces; the reflection of the part about the chosen face is joined to it. Model half a part, mirror it, and the halves stay in step when a dimension changes.





These features need the **B-rep** kernel for a fillet, a chamfer and a draft. On the Implicit kernel a picked fillet or chamfer leaves the edges sharp and a draft leaves the faces vertical, and the result says so (`KernelNotes`); a picked-face shell is approximated. Patterns on a face and mirrors work on both kernels. **Fillet** and **Chamfer** without a picker round every edge of the selected feature or of the whole part.

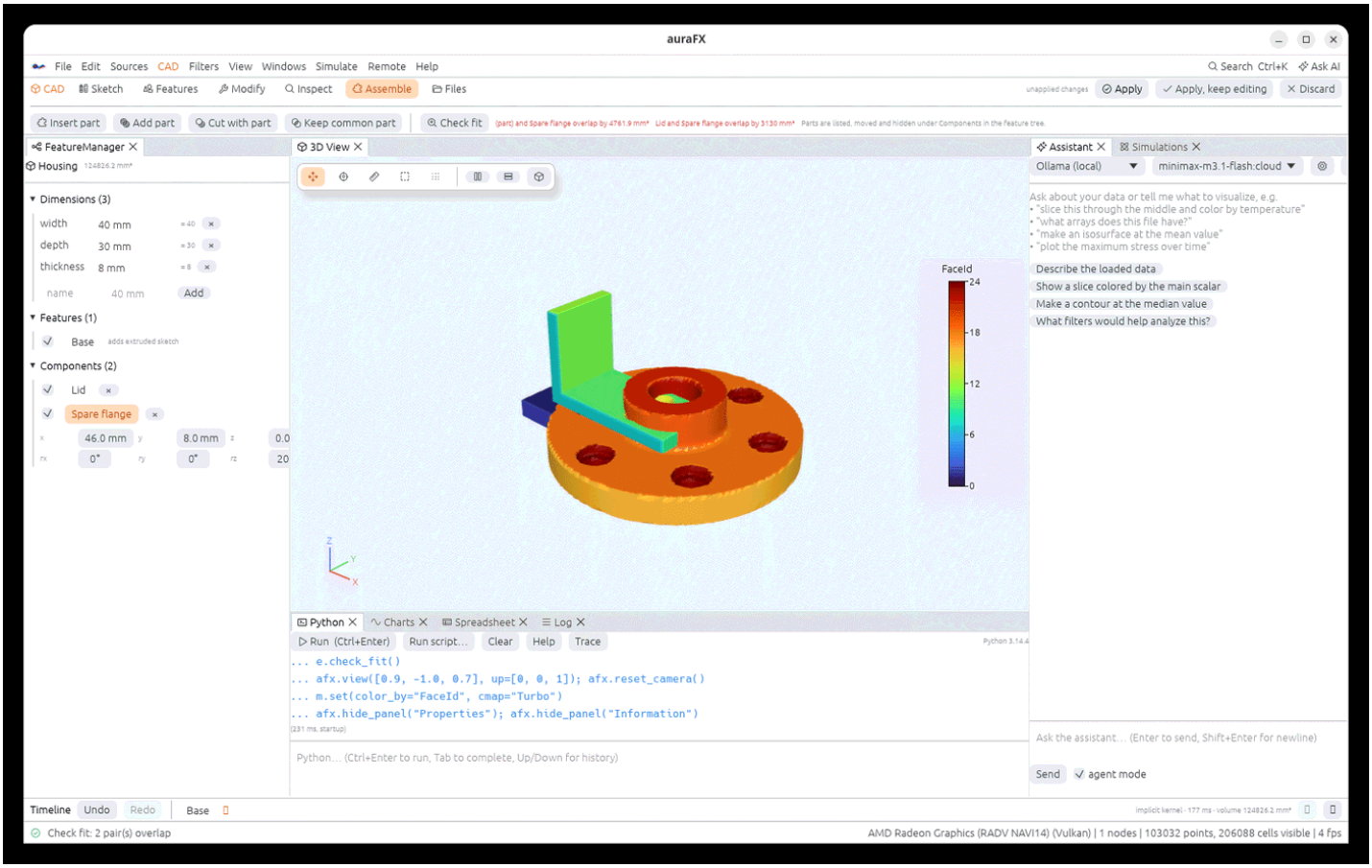
The **Inspect** tab and Python read the part: mass (volume, surface, center, bounds, mass for a density), the distance between two parallel flat faces and the angle between two faces, and the volume two parts overlap when one is placed against the other:

```
b = afx.cad_model()                                    # the default plate
b.cad.edges()                                          # [{'index': 0, 'point':
[...], 'length': 30.0,
                                                       #   'faces': ['Base bottom',
'Base side 1'], ...}, ...]
b.cad.fillet_edges(("Base z+", "Base side 2"), size=2)   # the edge between two
named faces
b.cad.chamfer_edges([0, 3, 5], size=1)                 # edges by index (or a point
on each)
b.cad.shell_faces(["Base z+"], thickness=2)            # an open box
b.cad.draft(["Base side 1"], angle=5)                  # taper a face, fixed at the
bottom plane
b.cad.mirror_about("Base x-", (1, 0, 0))               # the reflection about that
face, joined to the part
b.cad.edit("face_pattern", face="Base z+", layout="circular", count=6, radius=12,
diameter=4, depth=6)
b.cad.edit("combine", template="flange", op="subtract", at=(0, 0, 4))   # cut
another part from this one
b.cad.mass(7.85)                                       # steel
b.cad.measure_faces("Base x-", "Base x+")              # {'parallel': True,
'distance': 40.0, ...}
b.cad.interference(other, at=(12, 6, 0))               # overlap volume with another
CAD Model placed there
```

Assemblies

A CAD Model can hold **several parts side by side**. On the **Assemble** tab, **Insert part** places another part: a template, a blank part, or a CAD file (`.afxcad` , STEP, IGES, 3MF, `.sc`). The part keeps its own features and dimensions; the model shows all the parts together, as one surface with one body per part, and a part's faces are named `Part/Face` (`Lid/Base z+`), so the mesh tools and the downstream check refer to them as they do to any face.

In the feature tree a **Components** section lists the parts. Tick one off to leave it out, click a name to see where it sits: three positions (mm) and three turns (degrees, about x, y and then z), which you can drag or type; **×** removes it. **Check fit** lists the pairs of parts that overlap, with the overlap volume in mm³ (the parts are compared two at a time, on the exact solids when the exact kernel builds them), and says so when none do.



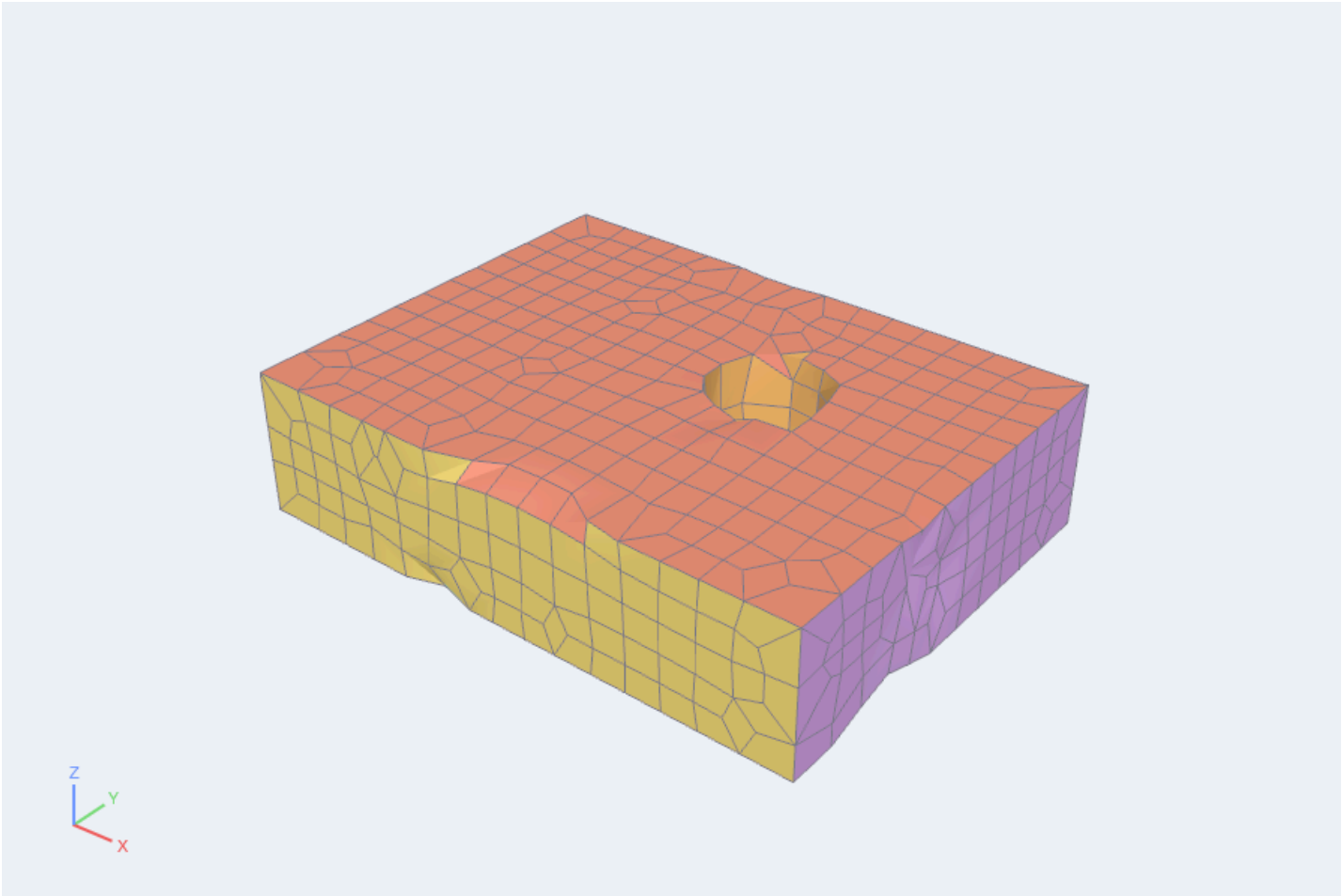
Add part, **Cut with part** and **Keep common part** (on the same tab) do something else: they make another part a *feature* of this one, which joins it, removes its volume or keeps only the common volume. Use components for parts that stay apart (a bolt in a bracket) and these for parts that become one solid.

```
m = afx.cad_model(name="Housing")
m.cad.component_add(name="Lid", template="bracket", at=(0, 0, 14))
m.cad.component_add(name="Bolt", file="bolt.step", at=(20, 0, 14), rotate=(0, 90,
0))
m.cad.component_op("Lid", "add", kind="cylinder", radius=3, height=6)   # edit a
part in place
m.cad.component_move("Bolt", by=(0, 5, 0))
m.cad.component_interference()                          # [{'a': 'Lid', 'b': 'Bolt',
'volume': 120.4}]
m.cad.components()                                      # name, position, turn,
visibility
```

From a CAD part to a mesh and a solver case

A CAD Model feeds the meshing tools directly, and **its faces come along as named side sets**:

- **Overlay Mesh** of a CAD Model fills it with hexahedra; the mesh has one side set per CAD face (`Bore side`, `Base z+`, or `plate_with_hole face 3` for a solid read from a STEP file), so a boundary condition can name the wall of a hole. **Extract Set** shows one.
- In geometry commands, `import 'part.step'` (also `.stp`, `.igs`, `.3mf`, `.sc`, `.afxcad`) brings the part in as a body whose faces keep their names, and `mesh body 1 size 0.5 scheme overlay` meshes it; the Hex mesh output carries the same side sets.
- **Tet Mesh** keeps the names too: one side set per CAD face.
- Assign Grains and the solver cases follow as for any mesh: give the set to hold and the set to pull by name (`fixed='plate_with_hole face 1'`, `loaded='plate_with_hole face 2'`).




```
part = afx.load("bracket.step")                    # a CAD Model
mesh = part.overlay_mesh(cell=1.0)                 # hexahedra; side sets named after
the faces
grains = mesh.assign_grains(grains=40, blocks=True, boundaries=True)
afx.simulation_case(grains, "moose", "case", fixed="bracket face 1",
loaded="bracket face 2")
```

Python

```
m = afx.cad_model()                                          # a plate; or
afx.cad_model("part.afxcad"),
                                                             #
afx.cad_model(template="flange")
m.cad.parameter("width", "60 mm")                            # a named dimension
m.cad.hole(diameter="width / 10", depth=20, name="Bore", at=(0, 0, 12))   # cuts
down from z = 12
m.cad.fillet(0.8)                                            # round the whole part
print(m.cad.volume(), [f["name"] for f in m.cad.faces()])
m.set(width=80)                                              # a dimension, without
editing the part
```

Any number may be an expression of the dimensions (`"width / 2"`): the field then follows the dimension. The calls:

Call	What it does
`m.cad.parameter(name, expr)`	names a dimension (units allowed: `"1.5 in"`)
`m.cad.box` , `sphere` , `cylinder` , `cone` , `torus`	add (`op="add"`), cut (`"cut"`) or intersect (`"intersect"`) a solid; `at=(x, y, z)` and `rotate=(rx, ry, rz)` place it
`m.cad.hole(diameter, depth, kind=...)`	a hole cut downward from `at` : `simple` , `drilled` , `counterbore` , `countersink` , `tapped`
`m.cad.thread(major_diameter, pitch, length)`	a helical thread
`s = m.cad.sketch(plane="XY")` , then `.rectangle(w, h, center=...)` , `.circle(r)` , `.polygon(points)` , `.regular(sides, r)`	a sketch with its dimensions
`m.cad.extrude(s, depth, end="blind")`	extrude a sketch (`"midplane"` , `"through_all"` ; `draft` , `top={"fillet": 1}`)
`m.cad.revolve(s, angle=360)`	revolve a sketch about its vertical axis (x is the radius)
`m.cad.fillet(r)` , `shell(t)` , `linear_pattern` , `grid_pattern` , `circular_pattern` , `mirror` , `split` , `scale`	modifiers on a feature (`feature="Base"`) or the whole part
`m.cad.mirror_about(face_or_point, normal)` , `draft(faces, angle)` , `edit("face_pattern",`	operations on faces and bodies

Call	What it does
`...)`, `edit("combine", ...)`	
`m.cad.component_add(...)`, `component_move`, `component_op`, `component_visible`, `component_remove`, `components()`, `component_interference()`	assemblies
`m.cad.suppress(name)`, `remove`, `rename`, `move`	edit the history
`m.cad.configuration("small", width=20)`, `activate("small")`	named variants
`m.cad.faces()`, `volume()`, `features()`, `parameters()`	read the result
`m.cad.save("part.afxcad")`, `m.cad.load(path)`	files

Every call is one undo step, and is recorded in the Python trace. A feature name can be used once (faces are named after features): adding a second `Base` is refused with that reason, and `m.cad.clear("Name")` empties a part to start from nothing.

Files

A part is saved as an `.afxcad` file: readable JSON with a format version, so a later auraFX opens an earlier file, and an earlier one says plainly that a file is from a newer version. The state files (`.afx`) hold the part inside, so a session reopens with it.

Meshing a part

A CAD model feeds the same tools as Geometry: **Overlay Mesh** turns its surface into hexahedra, **Tet Mesh** into tetrahedra, **Assign Grains** and the solver cases follow. Face names carry through: an extracted set or a boundary layer can name `Bore side`.

Limits

- The kernel used here describes the part as a distance field. It never fails on a hard combination of features, but edges are not exact lines: the faces it reports are labelled regions of the surface, and the same labels come back after every rebuild.

- Sketches start on the three principal planes or on a flat face whose normal is along x, y or z; a face at another angle can not be sketched on yet (a pattern on a face does follow any flat face).
- Components are placed rigidly, by position and turn; there are no mates or motion. A component is edited in place from Python (`component_op`); to model it with the sketch canvas, build it as its own CAD Model and insert the file.
- Check fit samples the two solids on a grid, so a thin overlap smaller than a cell is missed; the result is accurate to a few percent for overlaps that span several cells.

Settings and keyboard shortcuts

Settings

Edit › Settings… (Ctrl+,):

Setting	Meaning
Theme	dark, light or system; the 3D background per theme
Ask before adding a source	adding a source opens a window to insert it with its standard settings or change them first (on by default)
UI scale	enlarges the whole interface
Orientation axes, **Time label**	shown in the 3D views
Filter threads	cores used by the parallel filters (0 = all); takes effect at the next start; `AURAFX_THREADS` overrides it
Default color map	for new objects
AI	provider, model, server URL, API key, auto-run and agent mode
Plugins	the Plugins window (also **Edit › Plugins…**)
Lighting	in **View › Lighting**; the choices are saved with the settings

Settings live in `~/.config/aurafx/settings.toml` (Linux) or the app's config folder on other systems; the window layout in `~/.local/share/aurafx/app.ron`.

Keyboard shortcuts

Keys	Action
Ctrl+O	open a file
Ctrl+S	save state
Ctrl+Shift+S	save a screenshot
Ctrl+K, Ctrl+Space	command palette
Ctrl+Z, Ctrl+Y	undo, redo
Ctrl+Enter	run the code in the Python console

Keys	Action
Ctrl+,	settings
R	reset the camera
Esc, P, M, S, D	view modes: navigate, pick, ruler, select cells, select points
Delete	delete the selected object
Space	play / pause time
Double-click in the view	fit the data

Help › Keyboard shortcuts shows this list in the app.

Environment variables

Variable	Use
`AURAFX_THREADS`	thread limit for filters
`AURAFX_PLUGIN_PATH`	extra plugin folders
`AURAFX_SERVER`	path of `aurafx-server` on a host (in its setup lines)
`AURAFX_DATA_DIR`	where settings and state are kept
`OLLAMA_HOST`, `ANTHROPIC_API_KEY`	AI providers
`PYO3_PYTHON` (at build time)	which Python to embed